

Vulkan Tutorial

Alexander Overvoorde

August 2025

Contents

Introduction	6
About	6
E-book	7
Tutorial structure	7
License	9
Overview	10
Origin of Vulkan	10
What it takes to draw a triangle	11
Step 1 - Instance and physical device selection	11
Step 2 - Logical device and queue families	11
Step 3 - Window surface and swap chain	11
Step 4 - Image views and framebuffers	12
Step 5 - Render passes	12
Step 6 - Graphics pipeline	13
Step 7 - Command pools and command buffers	13
Step 8 - Main loop	13
Summary	14
API concepts	15
Coding conventions	15
Validation layers	15
Development environment	17
Windows	17
Vulkan SDK	17
GLFW	19
GLM	19
Setting up Visual Studio	20
Linux	27
Vulkan Packages	27
X Window System and XFree86-VidModeExtension	28
GLFW	29
GLM	29

Shader Compiler	29
Setting up a makefile project	30
MacOS	32
Vulkan SDK	33
GLFW	34
GLM	34
Setting up Xcode	35
Drawing a triangle	40
Setup	40
Base code	40
Instance	44
Validation layers	48
Physical devices and queue families	59
Logical device and queues	66
Presentation	69
Window surface	69
Swap chain	73
Image views	86
Graphics pipeline basics	88
Introduction	88
Shader modules	91
Fixed functions	101
Render passes	110
Conclusion	114
Drawing	117
Framebuffers	117
Command buffers	119
Rendering and presentation	125
Frames in flight	137
Swap chain recreation	140
Introduction	140
Recreating the swap chain	140
Suboptimal or out-of-date swap chain	143
Fixing a deadlock	144
Handling resizes explicitly	144
Handling minimization	146
Vertex buffers	147
Vertex input description	147
Introduction	147
Vertex shader	147
Vertex data	148
Binding descriptions	148
Attribute descriptions	149
Pipeline vertex input	151

Vertex buffer creation	151
Introduction	151
Buffer creation	151
Memory requirements	153
Memory allocation	155
Filling the vertex buffer	156
Binding the vertex buffer	157
Staging buffer	159
Introduction	159
Transfer queue	159
Abstracting buffer creation	160
Using a staging buffer	161
Conclusion	164
Index buffer	165
Introduction	165
Index buffer creation	166
Using an index buffer	168
Uniform buffers	170
Descriptor set layout and buffer	170
Introduction	170
Vertex shader	171
Descriptor set layout	172
Uniform buffer	174
Updating uniform data	176
Descriptor pool and sets	178
Introduction	178
Descriptor pool	178
Descriptor set	179
Using descriptor sets	182
Alignment requirements	183
Multiple descriptor sets	186
Texture mapping	187
Images	187
Introduction	187
Image library	188
Loading an image	189
Staging buffer	191
Texture Image	191
Layout transitions	196
Copying buffer to image	199
Preparing the texture image	200
Transition barrier masks	201
Cleanup	203
Image view and sampler	203

Texture image view	203
Samplers	205
Anisotropy device feature	210
Combined image sampler	211
Introduction	211
Updating the descriptors	211
Texture coordinates	214
Shaders	215
Depth buffering	220
Introduction	220
3D geometry	220
Depth image and view	223
Explicitly transitioning the depth image	227
Render pass	228
Framebuffer	230
Clear values	230
Depth and stencil state	231
Handling window resize	232
Loading models	234
Introduction	234
Library	234
Sample mesh	235
Loading vertices and indices	236
Vertex deduplication	240
Generating Mipmaps	243
Introduction	243
Image creation	244
Generating Mipmaps	245
Linear filtering support	250
Sampler	251
Multisampling	255
Introduction	255
Getting available sample count	257
Setting up a render target	258
Adding new attachments	260
Quality improvements	264
Conclusion	265
Compute Shader	266
Introduction	266
Advantages	266
The Vulkan pipeline	267

An example	267
Data manipulation	269
Shader storage buffer objects (SSBO)	269
Storage images	270
Compute queue families	271
The compute shader stage	272
Loading compute shaders	272
Preparing the shader storage buffers	273
Descriptors	274
Compute pipelines	277
Compute space	278
Compute shaders	279
Running compute commands	281
Dispatch	281
Submitting work	282
Synchronizing graphics and compute	282
Drawing the particle system	285
Conclusion	286
FAQ	287
I get an access violation error in the core validation layer	287
I don't see any messages from the validation layers / Validation layers are not available	287
vkCreateSwapchainKHR triggers an error in SteamOverlayVulkan- Layer64.dll	287
vkCreateInstance fails with VK_ERROR_INCOMPATIBLE_DRIVER	288

Introduction

About

This tutorial will teach you the basics of using the Vulkan graphics and compute API. Vulkan is a new API by the Khronos group (known for OpenGL) that provides a much better abstraction of modern graphics cards. This new interface allows you to better describe what your application intends to do, which can lead to better performance and less surprising driver behavior compared to existing APIs like OpenGL and Direct3D. The ideas behind Vulkan are similar to those of Direct3D 12 and Metal, but Vulkan has the advantage of being fully cross-platform and allows you to develop for Windows, Linux and Android at the same time.

However, the price you pay for these benefits is that you have to work with a significantly more verbose API. Every detail related to the graphics API needs to be set up from scratch by your application, including initial frame buffer creation and memory management for objects like buffers and texture images. The graphics driver will do a lot less hand holding, which means that you will have to do more work in your application to ensure correct behavior.

The takeaway message here is that Vulkan is not for everyone. It is targeted at programmers who are enthusiastic about high performance computer graphics, and are willing to put some work in. If you are more interested in game development, rather than computer graphics, then you may wish to stick to OpenGL or Direct3D, which will not be deprecated in favor of Vulkan anytime soon. Another alternative is to use an engine like Unreal Engine or Unity, which will be able to use Vulkan while exposing a much higher level API to you.

With that out of the way, let's cover some prerequisites for following this tutorial:

- A graphics card and driver compatible with Vulkan (NVIDIA, AMD, Intel, Apple Silicon (Or the Apple M1))
- Experience with C++ (familiarity with RAII, initializer lists)
- A compiler with decent support of C++17 features (Visual Studio 2017+, GCC 7+, Or Clang 5+)
- Some existing experience with 3D computer graphics

This tutorial will not assume knowledge of OpenGL or Direct3D concepts, but it does require you to know the basics of 3D computer graphics. It will not explain the math behind perspective projection, for example. See this online book for a great introduction of computer graphics concepts. Some other great computer graphics resources are:

- Ray tracing in one weekend
- Physically Based Rendering book
- Vulkan being used in a real engine in the open-source Quake and DOOM 3

You can use C instead of C++ if you want, but you will have to use a different linear algebra library and you will be on your own in terms of code structuring. We will use C++ features like classes and RAII to organize logic and resource lifetimes. There are also two alternative versions of this tutorial available for Rust developers: Vulkan based, Vulkanalia based.

To make it easier to follow along for developers using other programming languages, and to get some experience with the base API we'll be using the original C API to work with Vulkan. If you are using C++, however, you may prefer using the newer Vulkan-Hpp bindings that abstract some of the dirty work and help prevent certain classes of errors.

E-book

If you prefer to read this tutorial as an e-book, then you can download an EPUB or PDF version here:

- EPUB
- PDF

Tutorial structure

We'll start with an overview of how Vulkan works and the work we'll have to do to get the first triangle on the screen. The purpose of all the smaller steps will make more sense after you've understood their basic role in the whole picture. Next, we'll set up the development environment with the Vulkan SDK, the GLM library for linear algebra operations and GLFW for window creation. The tutorial will cover how to set these up on Windows with Visual Studio, and on Ubuntu Linux with GCC.

After that we'll implement all of the basic components of a Vulkan program that are necessary to render your first triangle. Each chapter will follow roughly the following structure:

- Introduce a new concept and its purpose
- Use all of the relevant API calls to integrate it into your program

- Abstract parts of it into helper functions

Although each chapter is written as a follow-up on the previous one, it is also possible to read the chapters as standalone articles introducing a certain Vulkan feature. That means that the site is also useful as a reference. All of the Vulkan functions and types are linked to the specification, so you can click them to learn more. Vulkan is a very new API, so there may be some shortcomings in the specification itself. You are encouraged to submit feedback to this Khronos repository.

As mentioned before, the Vulkan API has a rather verbose API with many parameters to give you maximum control over the graphics hardware. This causes basic operations like creating a texture to take a lot of steps that have to be repeated every time. Therefore we'll be creating our own collection of helper functions throughout the tutorial.

Every chapter will also conclude with a link to the full code listing up to that point. You can refer to it if you have any doubts about the structure of the code, or if you're dealing with a bug and want to compare. All of the code files have been tested on graphics cards from multiple vendors to verify correctness. Each chapter also has a comment section at the end where you can ask any questions that are relevant to the specific subject matter. Please specify your platform, driver version, source code, expected behavior and actual behavior to help us help you.

This tutorial is intended to be a community effort. Vulkan is still a very new API and best practices have not really been established yet. If you have any type of feedback on the tutorial and site itself, then please don't hesitate to submit an issue or pull request to the GitHub repository. You can *watch* the repository to be notified of updates to the tutorial.

After you've gone through the ritual of drawing your very first Vulkan powered triangle onscreen, we'll start expanding the program to include linear transformations, textures and 3D models.

If you've played with graphics APIs before, then you'll know that there can be a lot of steps until the first geometry shows up on the screen. There are many of these initial steps in Vulkan, but you'll see that each of the individual steps is easy to understand and does not feel redundant. It's also important to keep in mind that once you have that boring looking triangle, drawing fully textured 3D models does not take that much extra work, and each step beyond that point is much more rewarding.

If you encounter any problems while following the tutorial, then first check the FAQ to see if your problem and its solution is already listed there. If you are still stuck after that, then feel free to ask for help in the comment section of the closest related chapter.

Ready to dive into the future of high performance graphics APIs? Let's go!

License

Copyright (C) 2015-2023, Alexander Overvoorde

The contents are licensed under CC BY-SA 4.0, unless stated otherwise. By contributing, you agree to license your contributions to the public under that same license.

The code listings in the `code` directory in the source repository are licensed under CC0 1.0 Universal. By contributing to that directory, you agree to license your contributions to the public under that same public domain-like license.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Overview

This chapter will start off with an introduction of Vulkan and the problems it addresses. After that we're going to look at the ingredients that are required for the first triangle. This will give you a big picture to place each of the subsequent chapters in. We will conclude by covering the structure of the Vulkan API and the general usage patterns.

Origin of Vulkan

Just like the previous graphics APIs, Vulkan is designed as a cross-platform abstraction over GPUs. The problem with most of these APIs is that the era in which they were designed featured graphics hardware that was mostly limited to configurable fixed functionality. Programmers had to provide the vertex data in a standard format and were at the mercy of the GPU manufacturers with regards to lighting and shading options.

As graphics card architectures matured, they started offering more and more programmable functionality. All this new functionality had to be integrated with the existing APIs somehow. This resulted in less than ideal abstractions and a lot of guesswork on the graphics driver side to map the programmer's intent to the modern graphics architectures. That's why there are so many driver updates for improving the performance in games, sometimes by significant margins. Because of the complexity of these drivers, application developers also need to deal with inconsistencies between vendors, like the syntax that is accepted for shaders. Aside from these new features, the past decade also saw an influx of mobile devices with powerful graphics hardware. These mobile GPUs have different architectures based on their energy and space requirements. One such example is tiled rendering, which would benefit from improved performance by offering the programmer more control over this functionality. Another limitation originating from the age of these APIs is limited multi-threading support, which can result in a bottleneck on the CPU side.

Vulkan solves these problems by being designed from scratch for modern graphics architectures. It reduces driver overhead by allowing programmers to clearly specify their intent using a more verbose API, and allows multiple threads to

create and submit commands in parallel. It reduces inconsistencies in shader compilation by switching to a standardized byte code format with a single compiler. Lastly, it acknowledges the general purpose processing capabilities of modern graphics cards by unifying the graphics and compute functionality into a single API.

What it takes to draw a triangle

We'll now look at an overview of all the steps it takes to render a triangle in a well-behaved Vulkan program. All of the concepts introduced here will be elaborated on in the next chapters. This is just to give you a big picture to relate all of the individual components to.

Step 1 - Instance and physical device selection

A Vulkan application starts by setting up the Vulkan API through a **VkInstance**. An instance is created by describing your application and any API extensions you will be using. After creating the instance, you can query for Vulkan supported hardware and select one or more **VkPhysicalDevices** to use for operations. You can query for properties like VRAM size and device capabilities to select desired devices, for example to prefer using dedicated graphics cards.

Step 2 - Logical device and queue families

After selecting the right hardware device to use, you need to create a **VkDevice** (logical device), where you describe more specifically which **VkPhysicalDeviceFeatures** you will be using, like multi viewport rendering and 64 bit floats. You also need to specify which queue families you would like to use. Most operations performed with Vulkan, like draw commands and memory operations, are asynchronously executed by submitting them to a **VkQueue**. Queues are allocated from queue families, where each queue family supports a specific set of operations in its queues. For example, there could be separate queue families for graphics, compute and memory transfer operations. The availability of queue families could also be used as a distinguishing factor in physical device selection. It is possible for a device with Vulkan support to not offer any graphics functionality, however all graphics cards with Vulkan support today will generally support all queue operations that we're interested in.

Step 3 - Window surface and swap chain

Unless you're only interested in offscreen rendering, you will need to create a window to present rendered images to. Windows can be created with the native platform APIs or libraries like GLFW and SDL. We will be using GLFW in this tutorial, but more about that in the next chapter.

We need two more components to actually render to a window: a window surface (`VkSurfaceKHR`) and a swap chain (`VkSwapchainKHR`). Note the `KHR` postfix, which means that these objects are part of a Vulkan extension. The Vulkan API itself is completely platform agnostic, which is why we need to use the standardized WSI (Window System Interface) extension to interact with the window manager. The surface is a cross-platform abstraction over windows to render to and is generally instantiated by providing a reference to the native window handle, for example `HWND` on Windows. Luckily, the GLFW library has a built-in function to deal with the platform specific details of this.

The swap chain is a collection of render targets. Its basic purpose is to ensure that the image that we're currently rendering to is different from the one that is currently on the screen. This is important to make sure that only complete images are shown. Every time we want to draw a frame we have to ask the swap chain to provide us with an image to render to. When we've finished drawing a frame, the image is returned to the swap chain for it to be presented to the screen at some point. The number of render targets and conditions for presenting finished images to the screen depends on the present mode. Common present modes are double buffering (`vsync`) and triple buffering. We'll look into these in the swap chain creation chapter.

Some platforms allow you to render directly to a display without interacting with any window manager through the `VK_KHR_display` and `VK_KHR_display_swapchain` extensions. These allow you to create a surface that represents the entire screen and could be used to implement your own window manager, for example.

Step 4 - Image views and framebuffers

To draw to an image acquired from the swap chain, we have to wrap it into a `VkImageView` and `VkFramebuffer`. An image view references a specific part of an image to be used, and a framebuffer references image views that are to be used for color, depth and stencil targets. Because there could be many different images in the swap chain, we'll preemptively create an image view and framebuffer for each of them and select the right one at draw time.

Step 5 - Render passes

Render passes in Vulkan describe the type of images that are used during rendering operations, how they will be used, and how their contents should be treated. In our initial triangle rendering application, we'll tell Vulkan that we will use a single image as color target and that we want it to be cleared to a solid color right before the drawing operation. Whereas a render pass only describes the type of images, a `VkFramebuffer` actually binds specific images to these slots.

Step 6 - Graphics pipeline

The graphics pipeline in Vulkan is set up by creating a `VkPipeline` object. It describes the configurable state of the graphics card, like the viewport size and depth buffer operation and the programmable state using `VkShaderModule` objects. The `VkShaderModule` objects are created from shader byte code. The driver also needs to know which render targets will be used in the pipeline, which we specify by referencing the render pass.

One of the most distinctive features of Vulkan compared to existing APIs, is that almost all configuration of the graphics pipeline needs to be set in advance. That means that if you want to switch to a different shader or slightly change your vertex layout, then you need to entirely recreate the graphics pipeline. That means that you will have to create many `VkPipeline` objects in advance for all the different combinations you need for your rendering operations. Only some basic configuration, like viewport size and clear color, can be changed dynamically. All of the state also needs to be described explicitly, there is no default color blend state, for example.

The good news is that because you're doing the equivalent of ahead-of-time compilation versus just-in-time compilation, there are more optimization opportunities for the driver and runtime performance is more predictable, because large state changes like switching to a different graphics pipeline are made very explicit.

Step 7 - Command pools and command buffers

As mentioned earlier, many of the operations in Vulkan that we want to execute, like drawing operations, need to be submitted to a queue. These operations first need to be recorded into a `VkCommandBuffer` before they can be submitted. These command buffers are allocated from a `VkCommandPool` that is associated with a specific queue family. To draw a simple triangle, we need to record a command buffer with the following operations:

- Begin the render pass
- Bind the graphics pipeline
- Draw 3 vertices
- End the render pass

Because the image in the framebuffer depends on which specific image the swap chain will give us, we need to record a command buffer for each possible image and select the right one at draw time. The alternative would be to record the command buffer again every frame, which is not as efficient.

Step 8 - Main loop

Now that the drawing commands have been wrapped into a command buffer, the main loop is quite straightforward. We first acquire an image from the

swap chain with `vkAcquireNextImageKHR`. We can then select the appropriate command buffer for that image and execute it with `vkQueueSubmit`. Finally, we return the image to the swap chain for presentation to the screen with `vkQueuePresentKHR`.

Operations that are submitted to queues are executed asynchronously. Therefore we have to use synchronization objects like semaphores to ensure a correct order of execution. Execution of the draw command buffer must be set up to wait on image acquisition to finish, otherwise it may occur that we start rendering to an image that is still being read for presentation on the screen. The `vkQueuePresentKHR` call in turn needs to wait for rendering to be finished, for which we'll use a second semaphore that is signaled after rendering completes.

Summary

This whirlwind tour should give you a basic understanding of the work ahead for drawing the first triangle. A real-world program contains more steps, like allocating vertex buffers, creating uniform buffers and uploading texture images that will be covered in subsequent chapters, but we'll start simple because Vulkan has enough of a steep learning curve as it is. Note that we'll cheat a bit by initially embedding the vertex coordinates in the vertex shader instead of using a vertex buffer. That's because managing vertex buffers requires some familiarity with command buffers first.

So in short, to draw the first triangle we need to:

- Create a `VkInstance`
- Select a supported graphics card (`VkPhysicalDevice`)
- Create a `VkDevice` and `VkQueue` for drawing and presentation
- Create a window, window surface and swap chain
- Wrap the swap chain images into `VkImageView`
- Create a render pass that specifies the render targets and usage
- Create framebuffers for the render pass
- Set up the graphics pipeline
- Allocate and record a command buffer with the draw commands for every possible swap chain image
- Draw frames by acquiring images, submitting the right draw command buffer and returning the images back to the swap chain

It's a lot of steps, but the purpose of each individual step will be made very simple and clear in the upcoming chapters. If you're confused about the relation of a single step compared to the whole program, you should refer back to this chapter.

API concepts

This chapter will conclude with a short overview of how the Vulkan API is structured at a lower level.

Coding conventions

All of the Vulkan functions, enumerations and structs are defined in the `vulkan.h` header, which is included in the Vulkan SDK developed by LunarG. We'll look into installing this SDK in the next chapter.

Functions have a lower case `vk` prefix, types like enumerations and structs have a `Vk` prefix and enumeration values have a `VK_` prefix. The API heavily uses structs to provide parameters to functions. For example, object creation generally follows this pattern:

```
1 VkXXXCreateInfo createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_XXX_CREATE_INFO;
3 createInfo.pNext = nullptr;
4 createInfo.foo = ...;
5 createInfo.bar = ...;
6
7 VkXXX object;
8 if (vkCreateXXX(&createInfo, nullptr, &object) != VK_SUCCESS) {
9     std::cerr << "failed to create object" << std::endl;
10    return false;
11 }
```

Many structures in Vulkan require you to explicitly specify the type of structure in the `sType` member. The `pNext` member can point to an extension structure and will always be `nullptr` in this tutorial. Functions that create or destroy an object will have a `VkAllocationCallbacks` parameter that allows you to use a custom allocator for driver memory, which will also be left `nullptr` in this tutorial.

Almost all functions return a `VkResult` that is either `VK_SUCCESS` or an error code. The specification describes which error codes each function can return and what they mean.

Validation layers

As mentioned earlier, Vulkan is designed for high performance and low driver overhead. Therefore it will include very limited error checking and debugging capabilities by default. The driver will often crash instead of returning an error code if you do something wrong, or worse, it will appear to work on your graphics card and completely fail on others.

Vulkan allows you to enable extensive checks through a feature known as *validation layers*. Validation layers are pieces of code that can be inserted between the API and the graphics driver to do things like running extra checks on function parameters and tracking memory management problems. The nice thing is that you can enable them during development and then completely disable them when releasing your application for zero overhead. Anyone can write their own validation layers, but the Vulkan SDK by LunarG provides a standard set of validation layers that we'll be using in this tutorial. You also need to register a callback function to receive debug messages from the layers.

Because Vulkan is so explicit about every operation and the validation layers are so extensive, it can actually be a lot easier to find out why your screen is black compared to OpenGL and Direct3D!

There's only one more step before we'll start writing code and that's setting up the development environment.

Development environment

In this chapter we'll set up your environment for developing Vulkan applications and install some useful libraries. All of the tools we'll use, with the exception of the compiler, are compatible with Windows, Linux and MacOS, but the steps for installing them differ a bit, which is why they're described separately here.

Windows

If you're developing for Windows, then I will assume that you are using Visual Studio to compile your code. For complete C++17 support, you need to use either Visual Studio 2017 or 2019. The steps outlined below were written for VS 2017.

Vulkan SDK

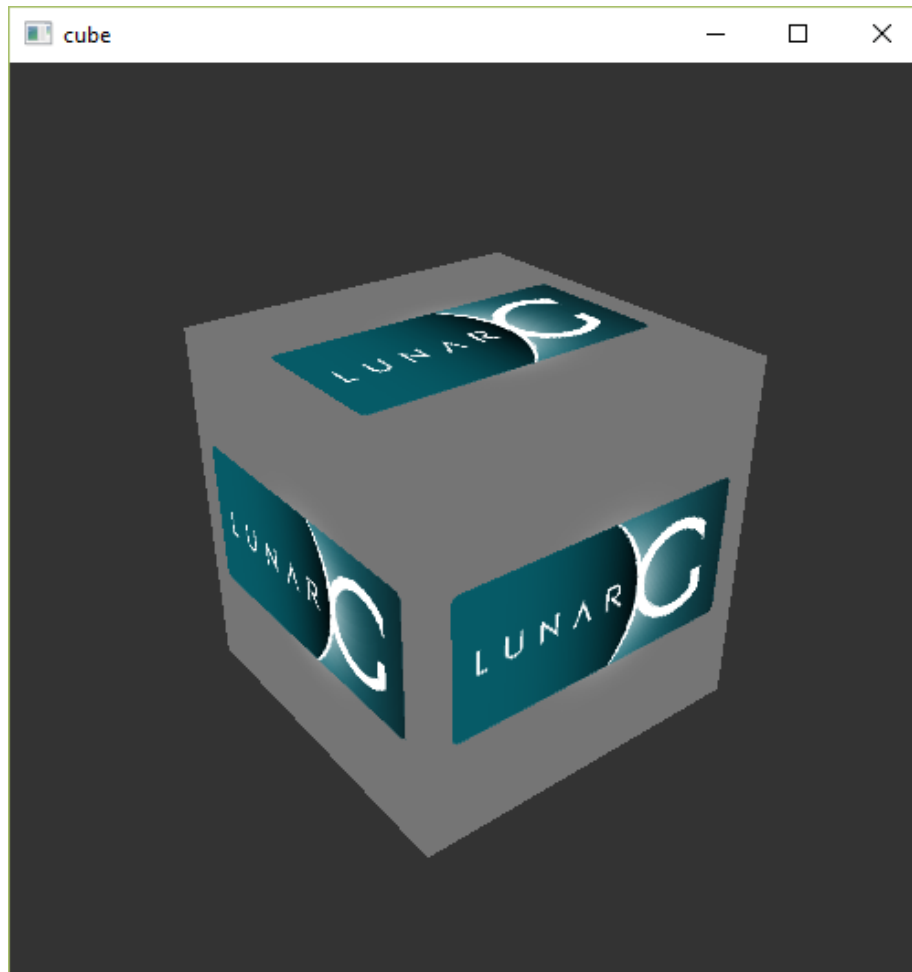
The most important component you'll need for developing Vulkan applications is the SDK. It includes the headers, standard validation layers, debugging tools and a loader for the Vulkan functions. The loader looks up the functions in the driver at runtime, similarly to GLEW for OpenGL - if you're familiar with that.

The SDK can be downloaded from the LunarG website using the buttons at the bottom of the page. You don't have to create an account, but it will give you access to some additional documentation that may be useful to you.



Proceed through the installation and pay attention to the install location of the SDK. The first thing we'll do is verify that your graphics card and driver properly support Vulkan. Go to the directory where you installed the SDK,

open the `Bin` directory and run the `vkcube.exe` demo. You should see the following:



If you receive an error message then ensure that your drivers are up-to-date, include the Vulkan runtime and that your graphics card is supported. See the introduction chapter for links to drivers from the major vendors.

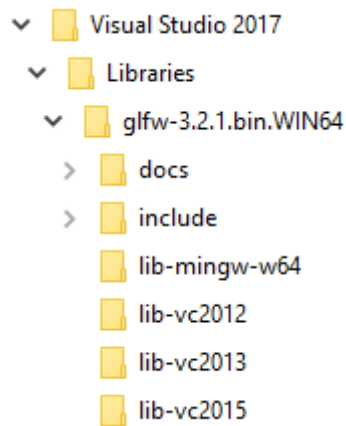
There is another program in this directory that will be useful for development. The `glslangValidator.exe` and `glslc.exe` programs will be used to compile shaders from the human-readable GLSL to bytecode. We'll cover this in depth in the shader modules chapter. The `Bin` directory also contains the binaries of the Vulkan loader and the validation layers, while the `Lib` directory contains the libraries.

Lastly, there's the `Include` directory that contains the Vulkan headers. Feel free to explore the other files, but we won't need them for this tutorial.

GLFW

As mentioned before, Vulkan by itself is a platform agnostic API and does not include tools for creating a window to display the rendered results. To benefit from the cross-platform advantages of Vulkan and to avoid the horrors of Win32, we'll use the GLFW library to create a window, which supports Windows, Linux and MacOS. There are other libraries available for this purpose, like SDL, but the advantage of GLFW is that it also abstracts away some of the other platform-specific things in Vulkan besides just window creation.

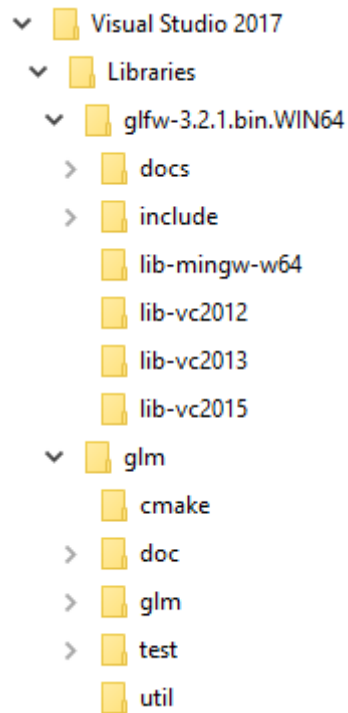
You can find the latest release of GLFW on the official website. In this tutorial we'll be using the 64-bit binaries, but you can of course also choose to build in 32 bit mode. In that case make sure to link with the Vulkan SDK binaries in the `Lib32` directory instead of `Lib`. After downloading it, extract the archive to a convenient location. I've chosen to create a `Libraries` directory in the Visual Studio directory under documents.



GLM

Unlike DirectX 12, Vulkan does not include a library for linear algebra operations, so we'll have to download one. GLM is a nice library that is designed for use with graphics APIs and is also commonly used with OpenGL.

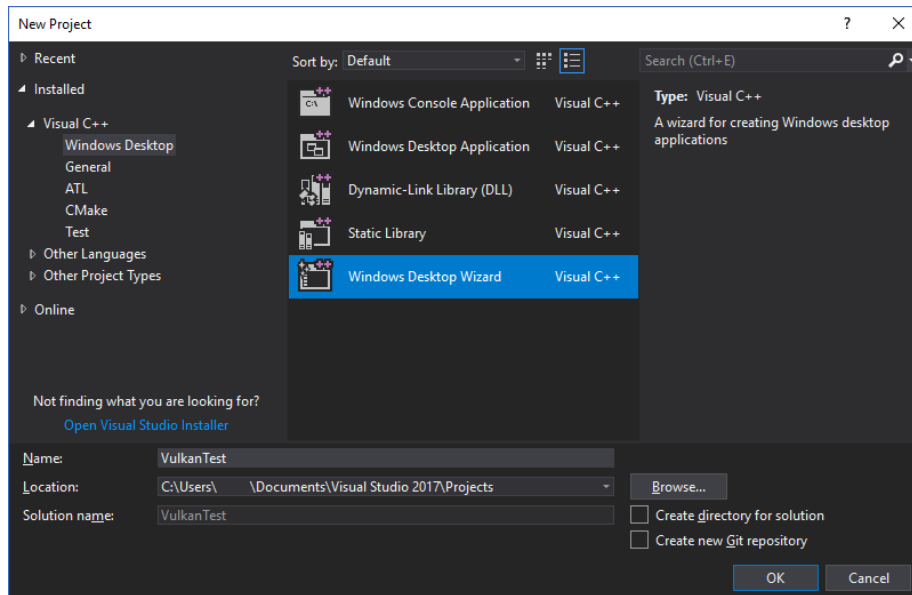
GLM is a header-only library, so just download the latest version and store it in a convenient location. You should have a directory structure similar to the following now:



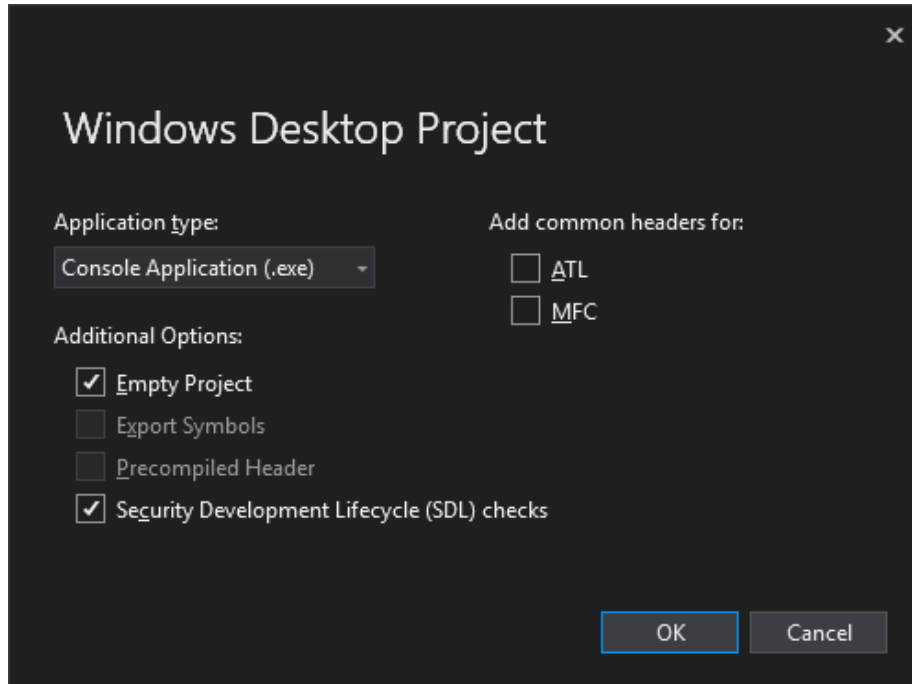
Setting up Visual Studio

Now that you've installed all of the dependencies we can set up a basic Visual Studio project for Vulkan and write a little bit of code to make sure that everything works.

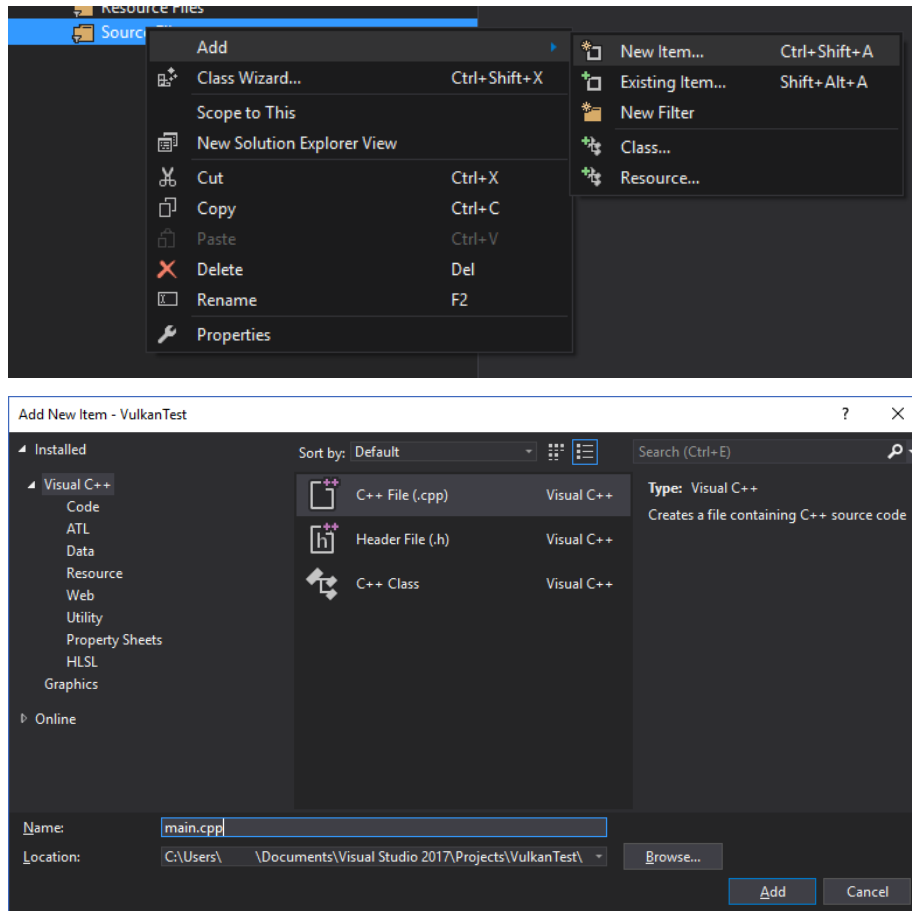
Start Visual Studio and create a new **Windows Desktop Wizard** project by entering a name and pressing **OK**.



Make sure that **Console Application (.exe)** is selected as application type so that we have a place to print debug messages to, and check **Empty Project** to prevent Visual Studio from adding boilerplate code.



Press OK to create the project and add a C++ source file. You should already know how to do that, but the steps are included here for completeness.



Now add the following code to the file. Don't worry about trying to understand it right now; we're just making sure that you can compile and run Vulkan applications. We'll start from scratch in the next chapter.

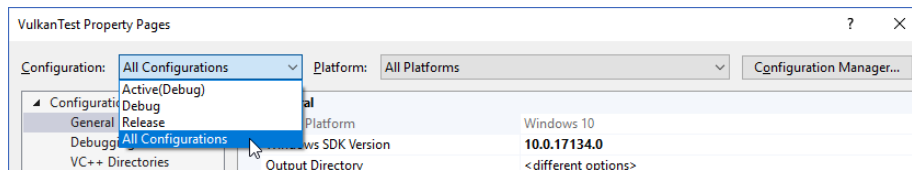
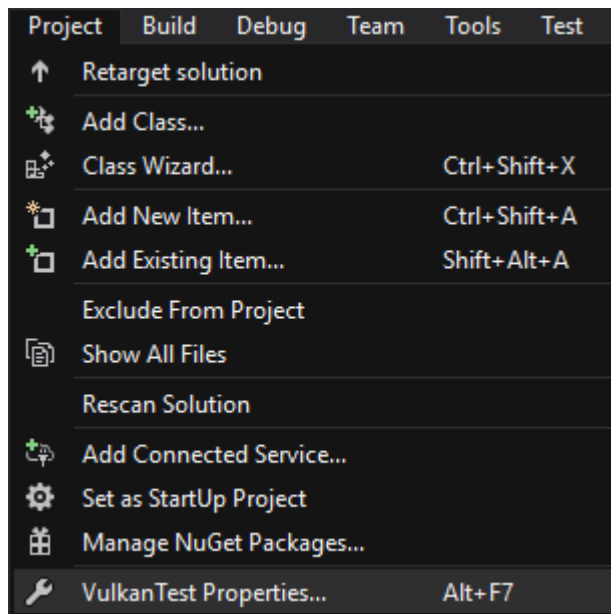
```
1 #define GLFW_INCLUDE_VULKAN
2 #include <GLFW/glfw3.h>
3
4 #define GLM_FORCE_RADIANS
5 #define GLM_FORCE_DEPTH_ZERO_TO_ONE
6 #include <glm/vec4.hpp>
7 #include <glm/mat4x4.hpp>
8
9 #include <iostream>
10
```

```

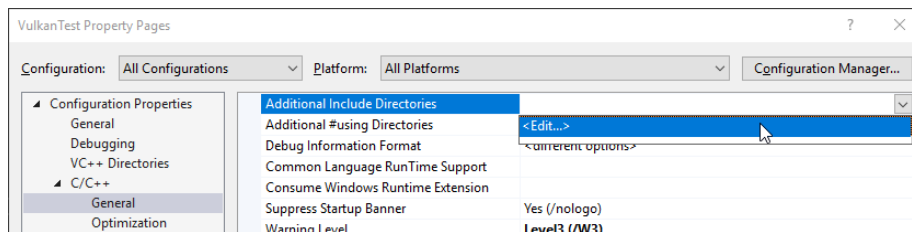
11 int main() {
12     glfwInit();
13
14     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
15     GLFWwindow* window = glfwCreateWindow(800, 600, "Vulkan window",
        nullptr, nullptr);
16
17     uint32_t extensionCount = 0;
18     vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,
        nullptr);
19
20     std::cout << extensionCount << " extensions supported\n";
21
22     glm::mat4 matrix;
23     glm::vec4 vec;
24     auto test = matrix * vec;
25
26     while(!glfwWindowShouldClose(window)) {
27         glfwPollEvents();
28     }
29
30     glfwDestroyWindow(window);
31
32     glfwTerminate();
33
34     return 0;
35 }

```

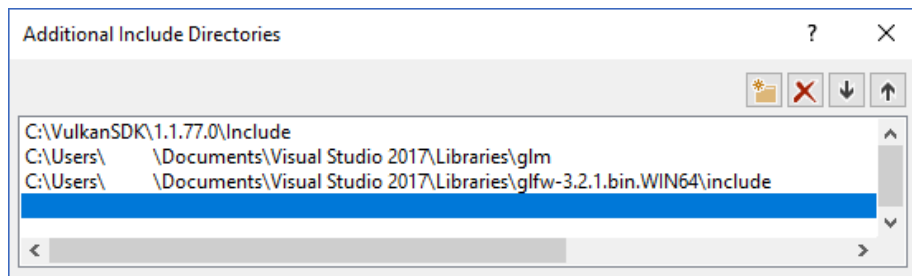
Let's now configure the project to get rid of the errors. Open the project properties dialog and ensure that **All Configurations** is selected, because most of the settings apply to both **Debug** and **Release** mode.



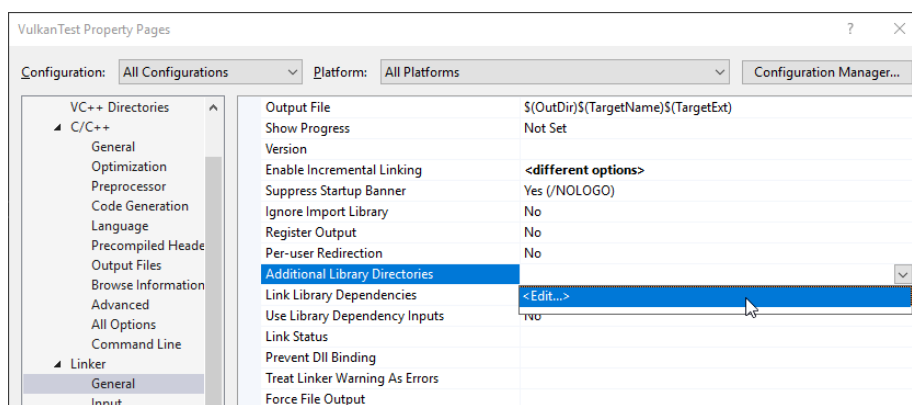
Go to C++ -> General -> Additional Include Directories and press <Edit...> in the dropdown box.



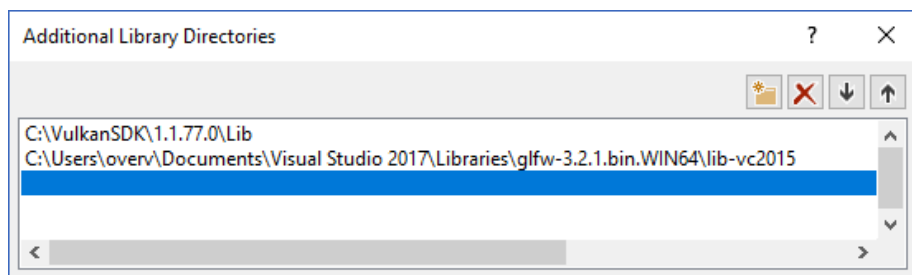
Add the header directories for Vulkan, GLFW and GLM:



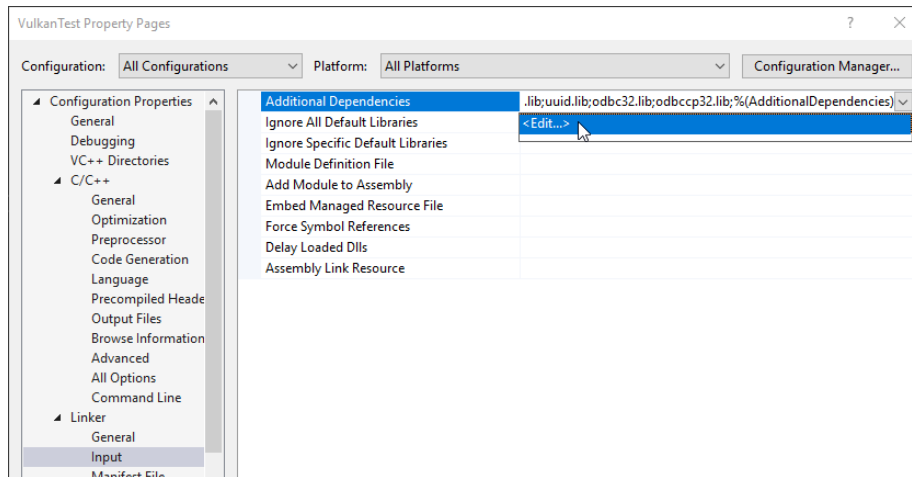
Next, open the editor for library directories under **Linker -> General**:



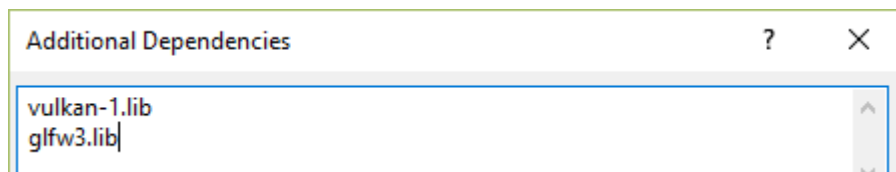
And add the locations of the object files for Vulkan and GLFW:



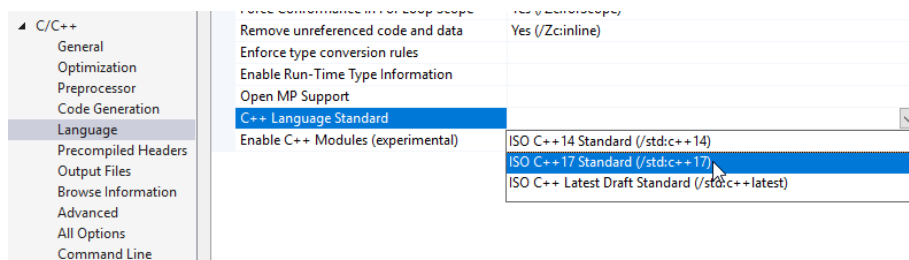
Go to **Linker -> Input** and press **<Edit...>** in the **Additional Dependencies** dropdown box.



Enter the names of the Vulkan and GLFW object files:

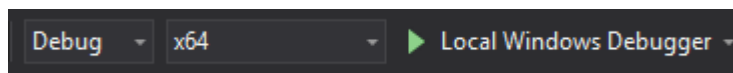


And finally change the compiler to support C++17 features:

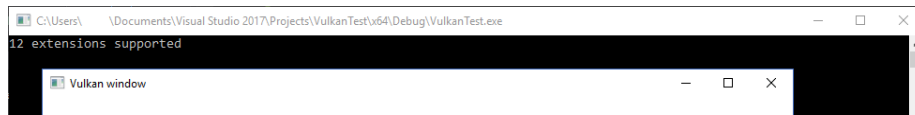


You can now close the project properties dialog. If you did everything right then you should no longer see any more errors being highlighted in the code.

Finally, ensure that you are actually compiling in 64 bit mode:



Press **F5** to compile and run the project and you should see a command prompt and a window pop up like this:



The number of extensions should be non-zero. Congratulations, you're all set for playing with Vulkan!

Linux

These instructions will be aimed at Ubuntu, Fedora and Arch Linux users, but you may be able to follow along by changing the package manager-specific commands to the ones that are appropriate for you. You should have a compiler that supports C++17 (GCC 7+ or Clang 5+). You'll also need **make**.

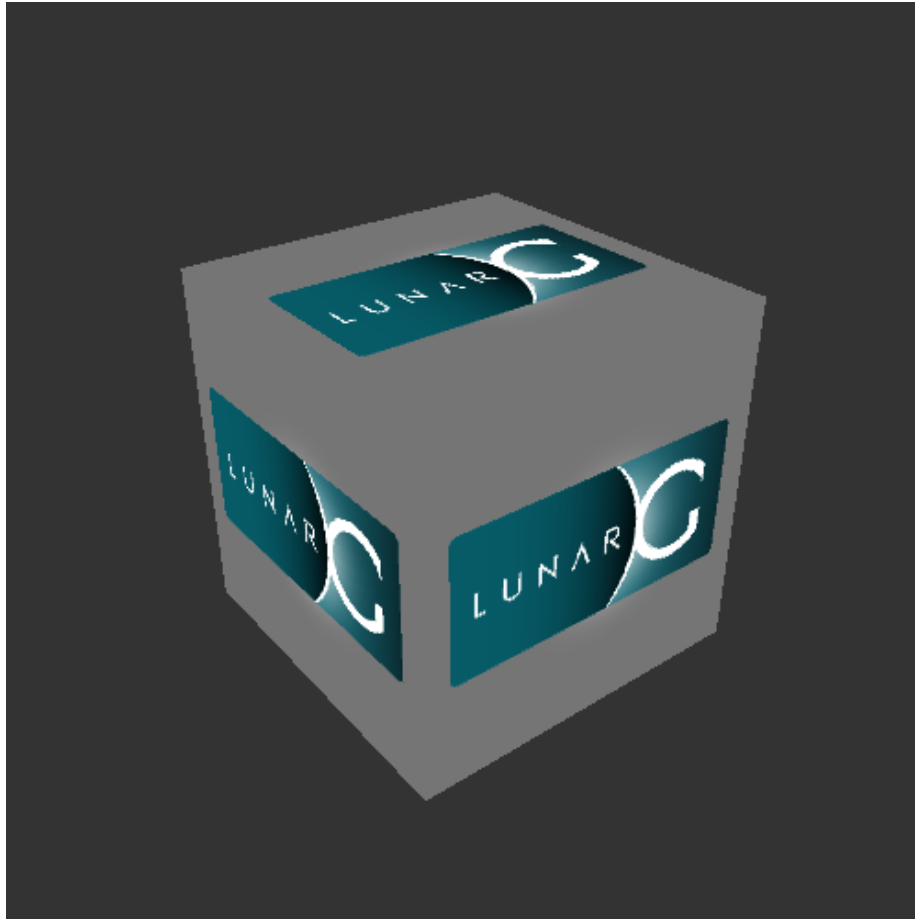
Vulkan Packages

The most important components you'll need for developing Vulkan applications on Linux are the Vulkan loader, validation layers, and a couple of command-line utilities to test whether your machine is Vulkan-capable:

- `sudo apt install vulkan-tools` or `sudo dnf install vulkan-tools`: Command-line utilities, most importantly `vulkaninfo` and `vkcube`. Run these to confirm your machine supports Vulkan.
- `sudo apt install libvulkan-dev` or `sudo dnf install vulkan-loader-devel`: Installs Vulkan loader. The loader looks up the functions in the driver at runtime, similarly to GLEW for OpenGL - if you're familiar with that.
- `sudo apt install vulkan-validationlayers spirv-tools` or `sudo dnf install mesa-vulkan-drivers vulkan-validation-layers-devel`: Installs the standard validation layers and required SPIR-V tools. These are crucial when debugging Vulkan applications, and we'll discuss them in the upcoming chapter.

On Arch Linux, you can run `sudo pacman -S vulkan-devel` to install all the required tools above.

If installation was successful, you should be all set with the Vulkan portion. Remember to run `vkcube` and ensure you see the following pop up in a window:



If you receive an error message then ensure that your drivers are up-to-date, include the Vulkan runtime and that your graphics card is supported. See the introduction chapter for links to drivers from the major vendors.

X Window System and XFree86-VidModeExtension

It is possible that these libraries are not on the system, if not, you can install them using the following commands: * `sudo apt install libxxf86vm-dev` or `dnf install libXxf86vm-devel`: Provides an interface to the XFree86-VidModeExtension. * `sudo apt install libxi-dev` or `dnf install libXi-devel`: Provides an X Window System client interface to the XINPUT extension.

GLFW

As mentioned before, Vulkan by itself is a platform agnostic API and does not include tools for creation a window to display the rendered results. To benefit from the cross-platform advantages of Vulkan and to avoid the horrors of X11, we'll use the GLFW library to create a window, which supports Windows, Linux and MacOS. There are other libraries available for this purpose, like SDL, but the advantage of GLFW is that it also abstracts away some of the other platform-specific things in Vulkan besides just window creation.

We'll be installing GLFW from the following command:

```
1 sudo apt install libglfw3-dev
```

or

```
1 sudo dnf install glfw-devel
```

or

```
1 sudo pacman -S glfw
```

GLM

Unlike DirectX 12, Vulkan does not include a library for linear algebra operations, so we'll have to download one. GLM is a nice library that is designed for use with graphics APIs and is also commonly used with OpenGL.

It is a header-only library that can be installed from the `libglm-dev` or `glm-devel` package:

```
1 sudo apt install libglm-dev
```

or

```
1 sudo dnf install glm-devel
```

or

```
1 sudo pacman -S glm
```

Shader Compiler

We have just about all we need, except we'll want a program to compile shaders from the human-readable GLSL to bytecode.

Two popular shader compilers are Khronos Group's `glslangValidator` and Google's `glslc`. The latter has a familiar GCC- and Clang-like usage, so we'll go with that: on Ubuntu, download Google's unofficial binaries and copy `glslc`

to your `/usr/local/bin`. Note you may need to `sudo` depending on your permissions. On Fedora use `sudo dnf install glslc`, while on Arch Linux run `sudo pacman -S shaderc`. To test, run `glslc` and it should rightfully complain we didn't pass any shaders to compile:

```
glslc: error: no input files
```

We'll cover `glslc` in depth in the shader modules chapter.

Setting up a makefile project

Now that you have installed all of the dependencies, we can set up a basic makefile project for Vulkan and write a little bit of code to make sure that everything works.

Create a new directory at a convenient location with a name like `VulkanTest`. Create a source file called `main.cpp` and insert the following code. Don't worry about trying to understand it right now; we're just making sure that you can compile and run Vulkan applications. We'll start from scratch in the next chapter.

```
1 #define GLFW_INCLUDE_VULKAN
2 #include <GLFW/glfw3.h>
3
4 #define GLM_FORCE_RADIANS
5 #define GLM_FORCE_DEPTH_ZERO_TO_ONE
6 #include <glm/vec4.hpp>
7 #include <glm/mat4x4.hpp>
8
9 #include <iostream>
10
11 int main() {
12     glfwInit();
13
14     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
15     GLFWwindow* window = glfwCreateWindow(800, 600, "Vulkan window",
16         nullptr, nullptr);
17
18     uint32_t extensionCount = 0;
19     vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,
20         nullptr);
21
22     std::cout << extensionCount << " extensions supported\n";
23
24     glm::mat4 matrix;
25     glm::vec4 vec;
26     auto test = matrix * vec;
```

```

26     while(!glfwWindowShouldClose(window)) {
27         glfwPollEvents();
28     }
29
30     glfwDestroyWindow(window);
31
32     glfwTerminate();
33
34     return 0;
35 }

```

Next, we'll write a makefile to compile and run this basic Vulkan code. Create a new empty file called **Makefile**. I will assume that you already have some basic experience with makefiles, like how variables and rules work. If not, you can get up to speed very quickly with this tutorial.

We'll first define a couple of variables to simplify the remainder of the file. Define a **CFLAGS** variable that will specify the basic compiler flags:

```
1 CFLAGS = -std=c++17 -O2
```

We're going to use modern C++ (**-std=c++17**), and we'll set optimization level to **O2**. We can remove **-O2** to compile programs faster, but we should remember to place it back for release builds.

Similarly, define the linker flags in a **LDFLAGS** variable:

```
1 LDFLAGS = -lglfw -lvulkan -ldl -lpthread -lX11 -lXxf86vm -lXrandr
          -lXi
```

The flag **-lglfw** is for GLFW, **-lvulkan** links with the Vulkan function loader and the remaining flags are low-level system libraries that GLFW needs. The remaining flags are dependencies of GLFW itself: the threading and window management.

It is possible that the **Xxf86vm** and **Xi** libraries are not yet installed on your system. You can find them in the following packages:

```
1 sudo apt install libxxf86vm-dev libxi-dev
```

or

```
1 sudo dnf install libXi-devel libXxf86vm-devel
```

or

```
1 sudo pacman -S libxi libxxf86vm
```

Specifying the rule to compile **VulkanTest** is straightforward now. Make sure to use tabs for indentation instead of spaces.

```

1 VulkanTest: main.cpp
2     g++ $(CFLAGS) -o VulkanTest main.cpp $(LDFLAGS)

```

Verify that this rule works by saving the makefile and running **make** in the directory with **main.cpp** and **Makefile**. This should result in a **VulkanTest** executable.

We'll now define two more rules, **test** and **clean**, where the former will run the executable and the latter will remove a built executable:

```

1 .PHONY: test clean
2
3 test: VulkanTest
4     ./VulkanTest
5
6 clean:
7     rm -f VulkanTest

```

Running **make test** should show the program running successfully, and displaying the number of Vulkan extensions. The application should exit with the success return code (0) when you close the empty window. You should now have a complete makefile that resembles the following:

```

1 CFLAGS = -std=c++17 -O2
2 LDFLAGS = -lglfw -lvulkan -ldl -lpthread -lX11 -lXxf86vm -lXrandr
3         -lXi
4 VulkanTest: main.cpp
5     g++ $(CFLAGS) -o VulkanTest main.cpp $(LDFLAGS)
6
7 .PHONY: test clean
8
9 test: VulkanTest
10     ./VulkanTest
11
12 clean:
13     rm -f VulkanTest

```

You can now use this directory as a template for your Vulkan projects. Make a copy, rename it to something like **HelloTriangle** and remove all of the code in **main.cpp**.

You are now all set for the real adventure.

MacOS

These instructions will assume you are using Xcode and the Homebrew package manager. Also, keep in mind that you will need at least MacOS version 10.11,

and your device needs to support the Metal API.

Vulkan SDK

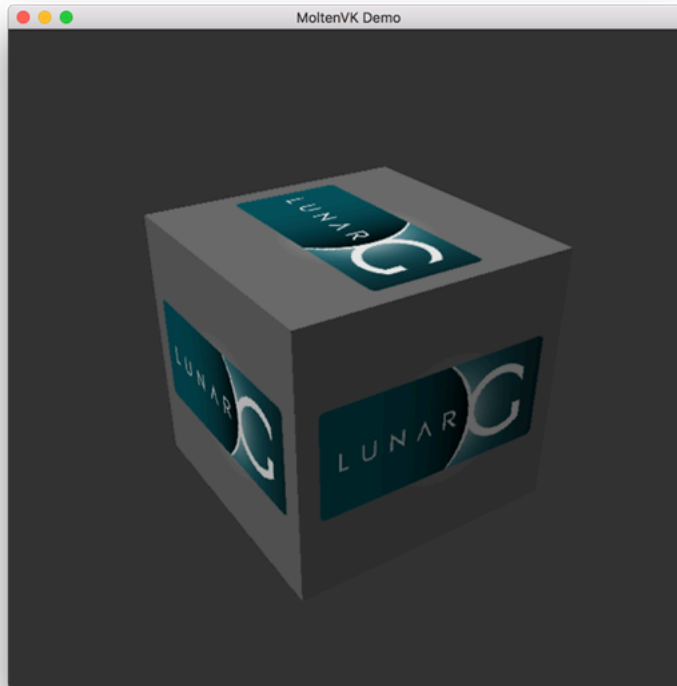
The most important component you'll need for developing Vulkan applications is the SDK. It includes the headers, standard validation layers, debugging tools and a loader for the Vulkan functions. The loader looks up the functions in the driver at runtime, similarly to GLEW for OpenGL - if you're familiar with that.

The SDK can be downloaded from the LunarG website using the buttons at the bottom of the page. You don't have to create an account, but it will give you access to some additional documentation that may be useful to you.



The SDK version for MacOS internally uses MoltenVK. There is no native support for Vulkan on MacOS, so what MoltenVK does is actually act as a layer that translates Vulkan API calls to Apple's Metal graphics framework. With this you can take advantage of debugging and performance benefits of Apple's Metal framework.

After downloading it, simply extract the contents to a folder of your choice (keep in mind you will need to reference it when creating your projects on Xcode). Inside the extracted folder, in the **Applications** folder you should have some executable files that will run a few demos using the SDK. Run the **vkcube** executable and you will see the following:



GLFW

As mentioned before, Vulkan by itself is a platform agnostic API and does not include tools for creation a window to display the rendered results. We'll use the GLFW library to create a window, which supports Windows, Linux and MacOS. There are other libraries available for this purpose, like SDL, but the advantage of GLFW is that it also abstracts away some of the other platform-specific things in Vulkan besides just window creation.

To install GLFW on MacOS we will use the Homebrew package manager to get the `glfw` package:

```
1 brew install glfw
```

GLM

Vulkan does not include a library for linear algebra operations, so we'll have to download one. GLM is a nice library that is designed for use with graphics APIs and is also commonly used with OpenGL.

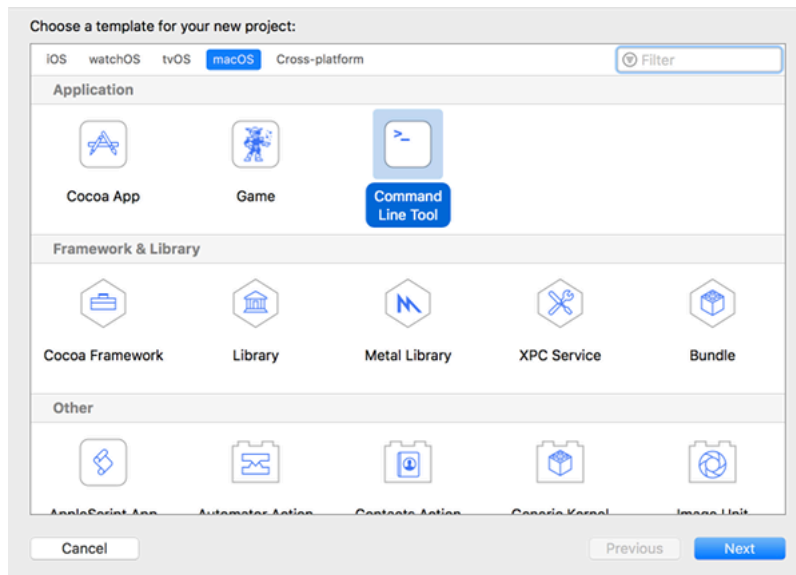
It is a header-only library that can be installed from the `glm` package:

```
1 brew install glm
```

Setting up Xcode

Now that all the dependencies are installed we can set up a basic Xcode project for Vulkan. Most of the instructions here are essentially a lot of “plumbing” so we can get all the dependencies linked to the project. Also, keep in mind that during the following instructions whenever we mention the folder `vulkansdk` we are referring to the folder where you extracted the Vulkan SDK.

Start Xcode and create a new Xcode project. On the window that will open select Application > Command Line Tool.



Select **Next**, write a name for the project and for **Language** select C++.

Choose options for your new project:

Product Name: VulkanTesting

Team: None

Organization Name: SomeNameHere

Organization Identifier: someorg

Bundle Identifier: someorg.VulkanTesting

Language: C++

Cancel Previous Next

Press Next and the project should have been created. Now, let's change the code in the generated `main.cpp` file to the following code:

```

1 #define GLFW_INCLUDE_VULKAN
2 #include <GLFW/glfw3.h>
3
4 #define GLM_FORCE_RADIANS
5 #define GLM_FORCE_DEPTH_ZERO_TO_ONE
6 #include <glm/vec4.hpp>
7 #include <glm/mat4x4.hpp>
8
9 #include <iostream>
10
11 int main() {
12     glfwInit();
13
14     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
15     GLFWwindow* window = glfwCreateWindow(800, 600, "Vulkan window",
16         nullptr, nullptr);
17
18     uint32_t extensionCount = 0;
19     vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,
20         nullptr);
21
22     std::cout << extensionCount << " extensions supported\n";
23
24     glm::mat4 matrix;

```

```

23     glm::vec4 vec;
24     auto test = matrix * vec;
25
26     while(!glfwWindowShouldClose(window)) {
27         glfwPollEvents();
28     }
29
30     glfwDestroyWindow(window);
31
32     glfwTerminate();
33
34     return 0;
35 }

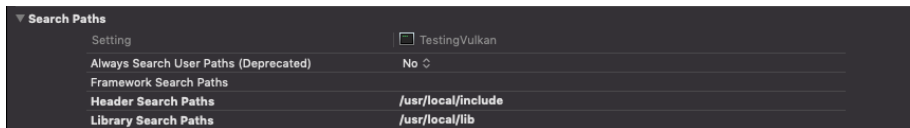
```

Keep in mind you are not required to understand all this code is doing yet, we are just setting up some API calls to make sure everything is working.

Xcode should already be showing some errors such as libraries it cannot find. We will now start configuring the project to get rid of those errors. On the *Project Navigator* panel select your project. Open the *Build Settings* tab and then:

- Find the **Header Search Paths** field and add a link to `/usr/local/include` (this is where Homebrew installs headers, so the glm and glfw3 header files should be there) and a link to `vulkansdk/macOS/include` for the Vulkan headers.
- Find the **Library Search Paths** field and add a link to `/usr/local/lib` (again, this is where Homebrew installs libraries, so the glm and glfw3 lib files should be there) and a link to `vulkansdk/macOS/lib`.

It should look like so (obviously, paths will be different depending on where you placed on your files):



Now, in the *Build Phases* tab, on **Link Binary With Libraries** we will add both the `glfw3` and the `vulkan` frameworks. To make things easier we will be adding the dynamic libraries in the project (you can check the documentation of these libraries if you want to use the static frameworks).

- For glfw open the folder `/usr/local/lib` and there you will find a file name like `libglfw.3.x.dylib` (“x” is the library’s version number, it might be different depending on when you downloaded the package from Homebrew). Simply drag that file to the Linked Frameworks and Libraries tab on Xcode.

- For vulkan, go to `vulkansdk/macOS/lib`. Do the same for the both files `libvulkan.1.dylib` and `libvulkan.1.x.xx.dylib` (where “x” will be the version number of the the SDK you downloaded).

After adding those libraries, in the same tab on **Copy Files** change **Destination** to “Frameworks”, clear the subpath and deselect “Copy only when installing”. Click on the “+” sign and add all those three frameworks here aswell.

Your Xcode configuration should look like:

The screenshot shows two panels from the Xcode interface. The top panel, titled "Link Binary With Libraries (3 items)", contains a table with the following data:

Name	Status
libglfw.3.3.dylib	Required
libvulkan.1.dylib	Required
libvulkan.1.1.73.dylib	Required

Below the table are plus and minus icons and the text "Drag to reorder frameworks". The bottom panel, titled "Copy Files (3 items)", has a "Destination" dropdown set to "Frameworks", an empty "Subpath" field, and the checkbox "Copy only when installing" is unchecked. It also has a "Code Sign On Copy" column. The table below contains the following data:

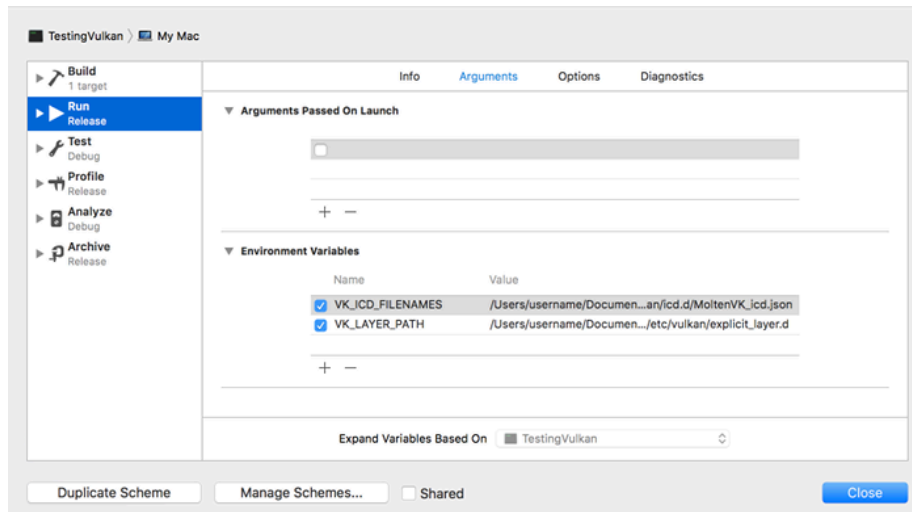
Name	Code Sign On Copy
libvulkan.1.1.73.dylib ...in ../././DevelopmentTools...	☐
libvulkan.1.dylib ...in ../././DevelopmentTools/vulk...	☐
libglfw.3.3.dylib ...in ../././././usr/local/lib	☐

Plus and minus icons are at the bottom of the table.

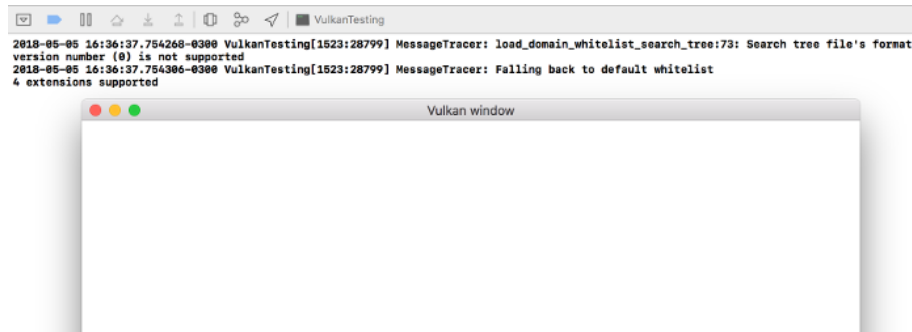
The last thing you need to setup are a couple of environment variables. On Xcode toolbar go to **Product > Scheme > Edit Scheme...**, and in the **Arguments** tab add the two following environment variables:

- `VK_ICD_FILENAMES = vulkansdk/macOS/share/vulkan/icd.d/MoltenVK_icd.json`
- `VK_LAYER_PATH = vulkansdk/macOS/share/vulkan/explicit_layer.d`

It should look like so:



Finally, you should be all set! Now if you run the project (remembering to setting the build configuration to Debug or Release depending on the configuration you chose) you should see the following:



The number of extensions should be non-zero. The other logs are from the libraries, you might get different messages from those depending on your configuration.

You are now all set for the real thing.

Drawing a triangle

Setup

Base code

General structure

In the previous chapter you've created a Vulkan project with all of the proper configuration and tested it with the sample code. In this chapter we're starting from scratch with the following code:

```
1 #include <vulkan/vulkan.h>
2
3 #include <iostream>
4 #include <stdexcept>
5 #include <cstdlib>
6
7 class HelloTriangleApplication {
8 public:
9     void run() {
10         initVulkan();
11         mainLoop();
12         cleanup();
13     }
14
15 private:
16     void initVulkan() {
17
18     }
19
20     void mainLoop() {
21
22     }
23
24     void cleanup() {
25
```

```

26     }
27 };
28
29 int main() {
30     HelloTriangleApplication app;
31
32     try {
33         app.run();
34     } catch (const std::exception& e) {
35         std::cerr << e.what() << std::endl;
36         return EXIT_FAILURE;
37     }
38
39     return EXIT_SUCCESS;
40 }

```

We first include the Vulkan header from the LunarG SDK, which provides the functions, structures and enumerations. The `stdexcept` and `iostream` headers are included for reporting and propagating errors. The `cstdlib` header provides the `EXIT_SUCCESS` and `EXIT_FAILURE` macros.

The program itself is wrapped into a class where we'll store the Vulkan objects as private class members and add functions to initiate each of them, which will be called from the `initVulkan` function. Once everything has been prepared, we enter the main loop to start rendering frames. We'll fill in the `mainLoop` function to include a loop that iterates until the window is closed in a moment. Once the window is closed and `mainLoop` returns, we'll make sure to deallocate the resources we've used in the `cleanup` function.

If any kind of fatal error occurs during execution then we'll throw a `std::runtime_error` exception with a descriptive message, which will propagate back to the `main` function and be printed to the command prompt. To handle a variety of standard exception types as well, we catch the more general `std::exception`. One example of an error that we will deal with soon is finding out that a certain required extension is not supported.

Roughly every chapter that follows after this one will add one new function that will be called from `initVulkan` and one or more new Vulkan objects to the private class members that need to be freed at the end in `cleanup`.

Resource management

Just like each chunk of memory allocated with `malloc` requires a call to `free`, every Vulkan object that we create needs to be explicitly destroyed when we no longer need it. In C++ it is possible to perform automatic resource management using RAII or smart pointers provided in the `<memory>` header. However, I've chosen to be explicit about allocation and deallocation of Vulkan objects in this

tutorial. After all, Vulkan’s niche is to be explicit about every operation to avoid mistakes, so it’s good to be explicit about the lifetime of objects to learn how the API works.

After following this tutorial, you could implement automatic resource management by writing C++ classes that acquire Vulkan objects in their constructor and release them in their destructor, or by providing a custom deleter to either `std::unique_ptr` or `std::shared_ptr`, depending on your ownership requirements. RAII is the recommended model for larger Vulkan programs, but for learning purposes it’s always good to know what’s going on behind the scenes.

Vulkan objects are either created directly with functions like `vkCreateXXX`, or allocated through another object with functions like `vkAllocateXXX`. After making sure that an object is no longer used anywhere, you need to destroy it with the counterparts `vkDestroyXXX` and `vkFreeXXX`. The parameters for these functions generally vary for different types of objects, but there is one parameter that they all share: `pAllocator`. This is an optional parameter that allows you to specify callbacks for a custom memory allocator. We will ignore this parameter in the tutorial and always pass `nullptr` as argument.

Integrating GLFW

Vulkan works perfectly fine without creating a window if you want to use it for off-screen rendering, but it’s a lot more exciting to actually show something! First replace the `#include <vulkan/vulkan.h>` line with

```
1 #define GLFW_INCLUDE_VULKAN
2 #include <GLFW/glfw3.h>
```

That way GLFW will include its own definitions and automatically load the Vulkan header with it. Add a `initWindow` function and add a call to it from the `run` function before the other calls. We’ll use that function to initialize GLFW and create a window.

```
1 void run() {
2     initWindow();
3     initVulkan();
4     mainLoop();
5     cleanup();
6 }
7
8 private:
9     void initWindow() {
10
11     }
```

The very first call in `initWindow` should be `glfwInit()`, which initializes the GLFW library. Because GLFW was originally designed to create an OpenGL

context, we need to tell it to not create an OpenGL context with a subsequent call:

```
1 glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
```

Because handling resized windows takes special care that we'll look into later, disable it for now with another window hint call:

```
1 glfwWindowHint(GLFW_RESIZABLE, GLFW_FALSE);
```

All that's left now is creating the actual window. Add a `GLFWwindow* window`; private class member to store a reference to it and initialize the window with:

```
1 window = glfwCreateWindow(800, 600, "Vulkan", nullptr, nullptr);
```

The first three parameters specify the width, height and title of the window. The fourth parameter allows you to optionally specify a monitor to open the window on and the last parameter is only relevant to OpenGL.

It's a good idea to use constants instead of hardcoded width and height numbers because we'll be referring to these values a couple of times in the future. I've added the following lines above the `HelloTriangleApplication` class definition:

```
1 const uint32_t WIDTH = 800;
2 const uint32_t HEIGHT = 600;
```

and replaced the window creation call with

```
1 window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr, nullptr);
```

You should now have a `initWindow` function that looks like this:

```
1 void initWindow() {
2     glfwInit();
3
4     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
5     glfwWindowHint(GLFW_RESIZABLE, GLFW_FALSE);
6
7     window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr,
8                             nullptr);
8 }
```

To keep the application running until either an error occurs or the window is closed, we need to add an event loop to the `mainLoop` function as follows:

```
1 void mainLoop() {
2     while (!glfwWindowShouldClose(window)) {
3         glfwPollEvents();
4     }
5 }
```

This code should be fairly self-explanatory. It loops and checks for events like pressing the X button until the window has been closed by the user. This is also the loop where we'll later call a function to render a single frame.

Once the window is closed, we need to clean up resources by destroying it and terminating GLFW itself. This will be our first **cleanup** code:

```
1 void cleanup() {
2     glfwDestroyWindow(window);
3
4     glfwTerminate();
5 }
```

When you run the program now you should see a window titled **Vulkan** show up until the application is terminated by closing the window. Now that we have the skeleton for the Vulkan application, let's create the first Vulkan object!

C++ code

Instance

Creating an instance

The very first thing you need to do is initialize the Vulkan library by creating an *instance*. The instance is the connection between your application and the Vulkan library and creating it involves specifying some details about your application to the driver.

Start by adding a `createInstance` function and invoking it in the `initVulkan` function.

```
1 void initVulkan() {
2     createInstance();
3 }
```

Additionally add a data member to hold the handle to the instance:

```
1 private:
2 VkInstance instance;
```

Now, to create an instance we'll first have to fill in a struct with some information about our application. This data is technically optional, but it may provide some useful information to the driver in order to optimize our specific application (e.g. because it uses a well-known graphics engine with certain special behavior). This struct is called `VkApplicationInfo`:

```
1 void createInstance() {
2     VkApplicationInfo appInfo{};
3     appInfo.sType = VK_STRUCTURE_TYPE_APPLICATION_INFO;
4     appInfo.pApplicationName = "Hello Triangle";
```

```

5     appInfo.applicationVersion = VK_MAKE_VERSION(1, 0, 0);
6     appInfo.pEngineName = "No Engine";
7     appInfo.engineVersion = VK_MAKE_VERSION(1, 0, 0);
8     appInfo.apiVersion = VK_API_VERSION_1_0;
9 }

```

As mentioned before, many structs in Vulkan require you to explicitly specify the type in the `sType` member. This is also one of the many structs with a `pNext` member that can point to extension information in the future. We're using value initialization here to leave it as `nullptr`.

A lot of information in Vulkan is passed through structs instead of function parameters and we'll have to fill in one more struct to provide sufficient information for creating an instance. This next struct is not optional and tells the Vulkan driver which global extensions and validation layers we want to use. Global here means that they apply to the entire program and not a specific device, which will become clear in the next few chapters.

```

1 VkInstanceCreateInfo createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
3 createInfo.pApplicationInfo = &appInfo;

```

The first two parameters are straightforward. The next two layers specify the desired global extensions. As mentioned in the overview chapter, Vulkan is a platform agnostic API, which means that you need an extension to interface with the window system. GLFW has a handy built-in function that returns the extension(s) it needs to do that which we can pass to the struct:

```

1 uint32_t glfwExtensionCount = 0;
2 const char** glfwExtensions;
3
4 glfwExtensions =
    glfwGetRequiredInstanceExtensions(&glfwExtensionCount);
5
6 createInfo.enabledExtensionCount = glfwExtensionCount;
7 createInfo.ppEnabledExtensionNames = glfwExtensions;

```

The last two members of the struct determine the global validation layers to enable. We'll talk about these more in-depth in the next chapter, so just leave these empty for now.

```

1 createInfo.enabledLayerCount = 0;

```

We've now specified everything Vulkan needs to create an instance and we can finally issue the `vkCreateInstance` call:

```

1 VkResult result = vkCreateInstance(&createInfo, nullptr, &instance);

```

As you'll see, the general pattern that object creation function parameters in Vulkan follow is:

- Pointer to struct with creation info
- Pointer to custom allocator callbacks, always `nullptr` in this tutorial
- Pointer to the variable that stores the handle to the new object

If everything went well then the handle to the instance was stored in the `VkInstance` class member. Nearly all Vulkan functions return a value of type `VkResult` that is either `VK_SUCCESS` or an error code. To check if the instance was created successfully, we don't need to store the result and can just use a check for the success value instead:

```
1 if (vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS)
2     {
3         throw std::runtime_error("failed to create instance!");
4     }
```

Now run the program to make sure that the instance is created successfully.

Encountered `VK_ERROR_INCOMPATIBLE_DRIVER`:

If using MacOS with the latest MoltenVK sdk, you may get `VK_ERROR_INCOMPATIBLE_DRIVER` returned from `vkCreateInstance`. According to the Getting Start Notes. Beginning with the 1.3.216 Vulkan SDK, the `VK_KHR_PORTABILITY_subset` extension is mandatory.

To get over this error, first add the `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR` bit to `VkInstanceCreateInfo` struct's flags, then add `VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME` to instance enabled extension list.

Typically the code could be like this:

```
1 ...
2
3 std::vector<const char*> requiredExtensions;
4
5 for(uint32_t i = 0; i < glfwExtensionCount; i++) {
6     requiredExtensions.emplace_back(glfwExtensions[i]);
7 }
8
9 requiredExtensions.emplace_back(VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME);
10
11 createInfo.flags |= VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR;
12
13 createInfo.enabledExtensionCount = (uint32_t)
14     requiredExtensions.size();
15 createInfo.ppEnabledExtensionNames = requiredExtensions.data();
```

```

16 if (vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS)
17     {
18         throw std::runtime_error("failed to create instance!");
19     }

```

Checking for extension support

If you look at the `vkCreateInstance` documentation then you'll see that one of the possible error codes is `VK_ERROR_EXTENSION_NOT_PRESENT`. We could simply specify the extensions we require and terminate if that error code comes back. That makes sense for essential extensions like the window system interface, but what if we want to check for optional functionality?

To retrieve a list of supported extensions before creating an instance, there's the `vkEnumerateInstanceExtensionProperties` function. It takes a pointer to a variable that stores the number of extensions and an array of `VkExtensionProperties` to store details of the extensions. It also takes an optional first parameter that allows us to filter extensions by a specific validation layer, which we'll ignore for now.

To allocate an array to hold the extension details we first need to know how many there are. You can request just the number of extensions by leaving the latter parameter empty:

```

1 uint32_t extensionCount = 0;
2 vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,
3     nullptr);

```

Now allocate an array to hold the extension details (include `<vector>`):

```

1 std::vector<VkExtensionProperties> extensions(extensionCount);

```

Finally we can query the extension details:

```

1 vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,
2     extensions.data());

```

Each `VkExtensionProperties` struct contains the name and version of an extension. We can list them with a simple for loop (`\t` is a tab for indentation):

```

1 std::cout << "available extensions:\n";
2
3 for (const auto& extension : extensions) {
4     std::cout << '\t' << extension.extensionName << '\n';
5 }

```

You can add this code to the `createInstance` function if you'd like to provide some details about the Vulkan support. As a challenge, try to create a function that checks if all of the extensions returned by

`glfwGetRequiredInstanceExtensions` are included in the supported extensions list.

Cleaning up

The `VkInstance` should be only destroyed right before the program exits. It can be destroyed in `cleanup` with the `vkDestroyInstance` function:

```
1 void cleanup() {  
2     vkDestroyInstance(instance, nullptr);  
3  
4     glfwDestroyWindow(window);  
5  
6     glfwTerminate();  
7 }
```

The parameters for the `vkDestroyInstance` function are straightforward. As mentioned in the previous chapter, the allocation and deallocation functions in Vulkan have an optional allocator callback that we'll ignore by passing `nullptr` to it. All of the other Vulkan resources that we'll create in the following chapters should be cleaned up before the instance is destroyed.

Before continuing with the more complex steps after instance creation, it's time to evaluate our debugging options by checking out validation layers.

C++ code

Validation layers

What are validation layers?

The Vulkan API is designed around the idea of minimal driver overhead and one of the manifestations of that goal is that there is very limited error checking in the API by default. Even mistakes as simple as setting enumerations to incorrect values or passing null pointers to required parameters are generally not explicitly handled and will simply result in crashes or undefined behavior. Because Vulkan requires you to be very explicit about everything you're doing, it's easy to make many small mistakes like using a new GPU feature and forgetting to request it at logical device creation time.

However, that doesn't mean that these checks can't be added to the API. Vulkan introduces an elegant system for this known as *validation layers*. Validation layers are optional components that hook into Vulkan function calls to apply additional operations. Common operations in validation layers are:

- Checking the values of parameters against the specification to detect misuse
- Tracking creation and destruction of objects to find resource leaks
- Checking thread safety by tracking the threads that calls originate from

- Logging every call and its parameters to the standard output
- Tracing Vulkan calls for profiling and replaying

Here's an example of what the implementation of a function in a diagnostics validation layer could look like:

```

1 VkResult vkCreateInstance(
2     const VkInstanceCreateInfo* pCreateInfo,
3     const VkAllocationCallbacks* pAllocator,
4     VkInstance* instance) {
5
6     if (pCreateInfo == nullptr || instance == nullptr) {
7         log("Null pointer passed to required parameter!");
8         return VK_ERROR_INITIALIZATION_FAILED;
9     }
10
11     return real_vkCreateInstance(pCreateInfo, pAllocator, instance);
12 }

```

These validation layers can be freely stacked to include all the debugging functionality that you're interested in. You can simply enable validation layers for debug builds and completely disable them for release builds, which gives you the best of both worlds!

Vulkan does not come with any validation layers built-in, but the LunarG Vulkan SDK provides a nice set of layers that check for common errors. They're also completely open source, so you can check which kind of mistakes they check for and contribute. Using the validation layers is the best way to avoid your application breaking on different drivers by accidentally relying on undefined behavior.

Validation layers can only be used if they have been installed onto the system. For example, the LunarG validation layers are only available on PCs with the Vulkan SDK installed.

There were formerly two different types of validation layers in Vulkan: instance and device specific. The idea was that instance layers would only check calls related to global Vulkan objects like instances, and device specific layers would only check calls related to a specific GPU. Device specific layers have now been deprecated, which means that instance validation layers apply to all Vulkan calls. The specification document still recommends that you enable validation layers at device level as well for compatibility, which is required by some implementations. We'll simply specify the same layers as the instance at logical device level, which we'll see later on.

Using validation layers

In this section we'll see how to enable the standard diagnostics layers provided by the Vulkan SDK. Just like extensions, validation layers need to be enabled

by specifying their name. All of the useful standard validation is bundled into a layer included in the SDK that is known as `VK_LAYER_KHRONOS_validation`.

Let's first add two configuration variables to the program to specify the layers to enable and whether to enable them or not. I've chosen to base that value on whether the program is being compiled in debug mode or not. The `NDEBUG` macro is part of the C++ standard and means "not debug".

```
1 const uint32_t WIDTH = 800;
2 const uint32_t HEIGHT = 600;
3
4 const std::vector<const char*> validationLayers = {
5     "VK_LAYER_KHRONOS_validation"
6 };
7
8 #ifdef NDEBUG
9     const bool enableValidationLayers = false;
10 #else
11     const bool enableValidationLayers = true;
12 #endif
```

We'll add a new function `checkValidationLayerSupport` that checks if all of the requested layers are available. First list all of the available layers using the `vkEnumerateInstanceLayerProperties` function. Its usage is identical to that of `vkEnumerateInstanceExtensionProperties` which was discussed in the instance creation chapter.

```
1 bool checkValidationLayerSupport() {
2     uint32_t layerCount;
3     vkEnumerateInstanceLayerProperties(&layerCount, nullptr);
4
5     std::vector<VkLayerProperties> availableLayers(layerCount);
6     vkEnumerateInstanceLayerProperties(&layerCount,
7         availableLayers.data());
8
9     return false;
10 }
```

Next, check if all of the layers in `validationLayers` exist in the `availableLayers` list. You may need to include `<cstring>` for `strcmp`.

```
1 for (const char* layerName : validationLayers) {
2     bool layerFound = false;
3
4     for (const auto& layerProperties : availableLayers) {
5         if (strcmp(layerName, layerProperties.layerName) == 0) {
6             layerFound = true;
7             break;
8         }
9     }
10 }
```

```

8         }
9     }
10
11     if (!layerFound) {
12         return false;
13     }
14 }
15
16 return true;

```

We can now use this function in `createInstance`:

```

1 void createInstance() {
2     if (enableValidationLayers && !checkValidationLayerSupport()) {
3         throw std::runtime_error("validation layers requested, but
                                     not available!");
4     }
5
6     ...
7 }

```

Now run the program in debug mode and ensure that the error does not occur. If it does, then have a look at the FAQ.

Finally, modify the `VkInstanceCreateInfo` struct instantiation to include the validation layer names if they are enabled:

```

1 if (enableValidationLayers) {
2     createInfo.enabledLayerCount =
3         static_cast<uint32_t>(validationLayers.size());
4     createInfo.ppEnabledLayerNames = validationLayers.data();
5 } else {
6     createInfo.enabledLayerCount = 0;
7 }

```

If the check was successful then `vkCreateInstance` should not ever return a `VK_ERROR_LAYER_NOT_PRESENT` error, but you should run the program to make sure.

Message callback

The validation layers will print debug messages to the standard output by default, but we can also handle them ourselves by providing an explicit callback in our program. This will also allow you to decide which kind of messages you would like to see, because not all are necessarily (fatal) errors. If you don't want to do that right now then you may skip to the last section in this chapter.

To set up a callback in the program to handle messages and the associated details, we have to set up a debug messenger with a callback using the `VK_EXT_debug_utils` extension.

We'll first create a `getRequiredExtensions` function that will return the required list of extensions based on whether validation layers are enabled or not:

```
1 std::vector<const char*> getRequiredExtensions() {
2     uint32_t glfwExtensionCount = 0;
3     const char** glfwExtensions;
4     glfwExtensions =
5         glfwGetRequiredInstanceExtensions(&glfwExtensionCount);
6
7     std::vector<const char*> extensions(glfwExtensions,
8         glfwExtensions + glfwExtensionCount);
9
10    if (enableValidationLayers) {
11        extensions.push_back(VK_EXT_DEBUG_UTILS_EXTENSION_NAME);
12    }
13
14    return extensions;
15 }
```

The extensions specified by GLFW are always required, but the debug messenger extension is conditionally added. Note that I've used the `VK_EXT_DEBUG_UTILS_EXTENSION_NAME` macro here which is equal to the literal string `"VK_EXT_debug_utils"`. Using this macro lets you avoid typos.

We can now use this function in `createInstance`:

```
1 auto extensions = getRequiredExtensions();
2 createInfo.enabledExtensionCount =
3     static_cast<uint32_t>(extensions.size());
4 createInfo.ppEnabledExtensionNames = extensions.data();
```

Run the program to make sure you don't receive a `VK_ERROR_EXTENSION_NOT_PRESENT` error. We don't really need to check for the existence of this extension, because it should be implied by the availability of the validation layers.

Now let's see what a debug callback function looks like. Add a new static member function called `debugCallback` with the `PFN_vkDebugUtilsMessengerCallbackEXT` prototype. The `VKAPI_ATTR` and `VKAPI_CALL` ensure that the function has the right signature for Vulkan to call it.

```
1 static VKAPI_ATTR VkBool32 VKAPI_CALL debugCallback(
2     VkDebugUtilsMessageSeverityFlagBitsEXT messageSeverity,
3     VkDebugUtilsMessageTypeFlagsEXT messageType,
4     const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData,
5     void* pUserData) {
```

```

6
7     std::cerr << "validation layer: " << pCallbackData->pMessage <<
        std::endl;
8
9     return VK_FALSE;
10 }

```

The first parameter specifies the severity of the message, which is one of the following flags:

- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT`: Diagnostic message
- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT`: Informational message like the creation of a resource
- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT`: Message about behavior that is not necessarily an error, but very likely a bug in your application
- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT`: Message about behavior that is invalid and may cause crashes

The values of this enumeration are set up in such a way that you can use a comparison operation to check if a message is equal or worse compared to some level of severity, for example:

```

1 if (messageSeverity >=
    VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT) {
2     // Message is important enough to show
3 }

```

The `messageType` parameter can have the following values:

- `VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT`: Some event has happened that is unrelated to the specification or performance
- `VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT`: Something has happened that violates the specification or indicates a possible mistake
- `VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT`: Potential non-optimal use of Vulkan

The `pCallbackData` parameter refers to a `VkDebugUtilsMessengerCallbackDataEXT` struct containing the details of the message itself, with the most important members being:

- `pMessage`: The debug message as a null-terminated string
- `pObjects`: Array of Vulkan object handles related to the message
- `objectCount`: Number of objects in array

Finally, the `pUserData` parameter contains a pointer that was specified during the setup of the callback and allows you to pass your own data to it.

The callback returns a boolean that indicates if the Vulkan call that triggered the validation layer message should be aborted. If the callback returns true, then the call is aborted with the `VK_ERROR_VALIDATION_FAILED_EXT` error. This is normally only used to test the validation layers themselves, so you should always return `VK_FALSE`.

All that remains now is telling Vulkan about the callback function. Perhaps somewhat surprisingly, even the debug callback in Vulkan is managed with a handle that needs to be explicitly created and destroyed. Such a callback is part of a *debug messenger* and you can have as many of them as you want. Add a class member for this handle right under `instance`:

```
1 VkDebugUtilsMessengerEXT debugMessenger;
```

Now add a function `setupDebugMessenger` to be called from `initVulkan` right after `createInstance`:

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4 }
5
6 void setupDebugMessenger() {
7     if (!enableValidationLayers) return;
8 }
9 }
```

We'll need to fill in a structure with details about the messenger and its callback:

```
1 VkDebugUtilsMessengerCreateInfoEXT createInfo{};
2 createInfo.sType =
3     VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT;
4 createInfo.messageSeverity =
5     VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT |
6     VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT |
7     VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
8 createInfo.messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT
9     | VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT |
10    VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT;
11 createInfo.pfnUserCallback = debugCallback;
12 createInfo.pUserData = nullptr; // Optional
```

The `messageSeverity` field allows you to specify all the types of severities you would like your callback to be called for. I've specified all types except for `VK_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT` here to receive notifications about possible problems while leaving out verbose general debug info.

Similarly the `messageType` field lets you filter which types of messages your

callback is notified about. I've simply enabled all types here. You can always disable some if they're not useful to you.

Finally, the `pfnUserCallback` field specifies the pointer to the callback function. You can optionally pass a pointer to the `pUserData` field which will be passed along to the callback function via the `pUserData` parameter. You could use this to pass a pointer to the `HelloTriangleApplication` class, for example.

Note that there are many more ways to configure validation layer messages and debug callbacks, but this is a good setup to get started with for this tutorial. See the extension specification for more info about the possibilities.

This struct should be passed to the `vkCreateDebugUtilsMessengerEXT` function to create the `VkDebugUtilsMessengerEXT` object. Unfortunately, because this function is an extension function, it is not automatically loaded. We have to look up its address ourselves using `vkGetInstanceProcAddr`. We're going to create our own proxy function that handles this in the background. I've added it right above the `HelloTriangleApplication` class definition.

```
1 VkResult CreateDebugUtilsMessengerEXT(VkInstance instance, const
    VkDebugUtilsMessengerCreateInfoEXT* pCreateInfo, const
    VkAllocationCallbacks* pAllocator, VkDebugUtilsMessengerEXT*
    pDebugMessenger) {
2     auto func = (PFN_vkCreateDebugUtilsMessengerEXT)
        vkGetInstanceProcAddr(instance,
            "vkCreateDebugUtilsMessengerEXT");
3     if (func != nullptr) {
4         return func(instance, pCreateInfo, pAllocator,
            pDebugMessenger);
5     } else {
6         return VK_ERROR_EXTENSION_NOT_PRESENT;
7     }
8 }
```

The `vkGetInstanceProcAddr` function will return `nullptr` if the function couldn't be loaded. We can now call this function to create the extension object if it's available:

```
1 if (CreateDebugUtilsMessengerEXT(instance, &createInfo, nullptr,
    &debugMessenger) != VK_SUCCESS) {
2     throw std::runtime_error("failed to set up debug messenger!");
3 }
```

The second to last parameter is again the optional allocator callback that we set to `nullptr`, other than that the parameters are fairly straightforward. Since the debug messenger is specific to our Vulkan instance and its layers, it needs to be explicitly specified as first argument. You will also see this pattern with other *child* objects later on.

The `VkDebugUtilsMessengerEXT` object also needs to be cleaned up with a call to `vkDestroyDebugUtilsMessengerEXT`. Similarly to `vkCreateDebugUtilsMessengerEXT` the function needs to be explicitly loaded.

Create another proxy function right below `CreateDebugUtilsMessengerEXT`:

```
1 void DestroyDebugUtilsMessengerEXT(VkInstance instance,
   VkDebugUtilsMessengerEXT debugMessenger, const
   VkAllocationCallbacks* pAllocator) {
2     auto func = (PFN_vkDestroyDebugUtilsMessengerEXT)
       vkGetInstanceProcAddr(instance,
         "vkDestroyDebugUtilsMessengerEXT");
3     if (func != nullptr) {
4         func(instance, debugMessenger, pAllocator);
5     }
6 }
```

Make sure that this function is either a static class function or a function outside the class. We can then call it in the `cleanup` function:

```
1 void cleanup() {
2     if (enableValidationLayers) {
3         DestroyDebugUtilsMessengerEXT(instance, debugMessenger,
           nullptr);
4     }
5
6     vkDestroyInstance(instance, nullptr);
7
8     glfwDestroyWindow(window);
9
10    glfwTerminate();
11 }
```

Debugging instance creation and destruction

Although we've now added debugging with validation layers to the program we're not covering everything quite yet. The `vkCreateDebugUtilsMessengerEXT` call requires a valid instance to have been created and `vkDestroyDebugUtilsMessengerEXT` must be called before the instance is destroyed. This currently leaves us unable to debug any issues in the `vkCreateInstance` and `vkDestroyInstance` calls.

However, if you closely read the extension documentation, you'll see that there is a way to create a separate debug utils messenger specifically for those two function calls. It requires you to simply pass a pointer to a `VkDebugUtilsMessengerCreateInfoEXT` struct in the `pNext` extension field of `VkInstanceCreateInfo`. First extract population of the messenger create info into a separate function:

```

1 void
  populateDebugMessengerCreateInfo(VkDebugUtilsMessengerCreateInfoEXT&
  createInfo) {
2   createInfo = {};
3   createInfo.sType =
      VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT;
4   createInfo.messageSeverity =
      VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT |
      VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT |
      VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
5   createInfo.messageType =
      VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
      VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT |
      VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT;
6   createInfo.pfnUserCallback = debugCallback;
7 }
8
9 ...
10
11 void setupDebugMessenger() {
12     if (!enableValidationLayers) return;
13
14     VkDebugUtilsMessengerCreateInfoEXT createInfo;
15     populateDebugMessengerCreateInfo(createInfo);
16
17     if (CreateDebugUtilsMessengerEXT(instance, &createInfo, nullptr,
18     &debugMessenger) != VK_SUCCESS) {
19         throw std::runtime_error("failed to set up debug
20         messenger!");
21     }
22 }

```

We can now re-use this in the `createInstance` function:

```

1 void createInstance() {
2     ...
3
4     VkInstanceCreateInfo createInfo{};
5     createInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
6     createInfo.pApplicationInfo = &appInfo;
7
8     ...
9
10    VkDebugUtilsMessengerCreateInfoEXT debugCreateInfo{};
11    if (enableValidationLayers) {
12        createInfo.enabledLayerCount =

```

```

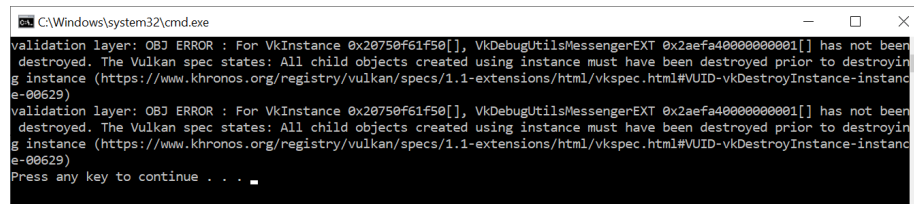
13         static_cast<uint32_t>(validationLayers.size());
14         createInfo.ppEnabledLayerNames = validationLayers.data();
15
16         populateDebugMessengerCreateInfo(debugCreateInfo);
17         createInfo.pNext = (VkDebugUtilsMessengerCreateInfoEXT*)
18             &debugCreateInfo;
19     } else {
20         createInfo.enabledLayerCount = 0;
21
22         createInfo.pNext = nullptr;
23     }
24
25     if (vkCreateInstance(&createInfo, nullptr, &instance) !=
26         VK_SUCCESS) {
27         throw std::runtime_error("failed to create instance!");
28     }
29 }

```

The `debugCreateInfo` variable is placed outside the if statement to ensure that it is not destroyed before the `vkCreateInstance` call. By creating an additional debug messenger this way it will automatically be used during `vkCreateInstance` and `vkDestroyInstance` and cleaned up after that.

Testing

Now let's intentionally make a mistake to see the validation layers in action. Temporarily remove the call to `DestroyDebugUtilsMessengerEXT` in the cleanup function and run your program. Once it exits you should see something like this:



The screenshot shows a Windows command prompt window titled "C:\Windows\system32\cmd.exe". It displays two identical Vulkan validation error messages. The first message is: "validation layer: OBJ ERROR : For VkInstance 0x20750f61f50[], VkDebugUtilsMessengerEXT 0x2aefa40000000001[] has not been destroyed. The Vulkan spec states: All child objects created using instance must have been destroyed prior to destroying instance (https://www.khronos.org/registry/vulkan/specs/1.1-extensions/html/vkspec.html#VUID-vkDestroyInstance-instance-00629)". The second message is identical. At the bottom of the window, it says "Press any key to continue . . .".

If you don't see any messages then check your installation.

If you want to see which call triggered a message, you can add a breakpoint to the message callback and look at the stack trace.

Configuration

There are a lot more settings for the behavior of validation layers than just the flags specified in the `VkDebugUtilsMessengerCreateInfoEXT` struct. Browse

to the Vulkan SDK and go to the `Config` directory. There you will find a `vk_layer_settings.txt` file that explains how to configure the layers.

To configure the layer settings for your own application, copy the file to the `Debug` and `Release` directories of your project and follow the instructions to set the desired behavior. However, for the remainder of this tutorial I'll assume that you're using the default settings.

Throughout this tutorial I'll be making a couple of intentional mistakes to show you how helpful the validation layers are with catching them and to teach you how important it is to know exactly what you're doing with Vulkan. Now it's time to look at Vulkan devices in the system.

C++ code

Physical devices and queue families

Selecting a physical device

After initializing the Vulkan library through a `VkInstance` we need to look for and select a graphics card in the system that supports the features we need. In fact we can select any number of graphics cards and use them simultaneously, but in this tutorial we'll stick to the first graphics card that suits our needs.

We'll add a function `pickPhysicalDevice` and add a call to it in the `initVulkan` function.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     pickPhysicalDevice();
5 }
6
7 void pickPhysicalDevice() {
8
9 }
```

The graphics card that we'll end up selecting will be stored in a `VkPhysicalDevice` handle that is added as a new class member. This object will be implicitly destroyed when the `VkInstance` is destroyed, so we won't need to do anything new in the `cleanup` function.

```
1 VkPhysicalDevice physicalDevice = VK_NULL_HANDLE;
```

Listing the graphics cards is very similar to listing extensions and starts with querying just the number.

```
1 uint32_t deviceCount = 0;
2 vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);
```

If there are 0 devices with Vulkan support then there is no point going further.

```
1 if (deviceCount == 0) {
2     throw std::runtime_error("failed to find GPUs with Vulkan
3     support!");
}
```

Otherwise we can now allocate an array to hold all of the `VkPhysicalDevice` handles.

```
1 std::vector<VkPhysicalDevice> devices(deviceCount);
2 vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());
```

Now we need to evaluate each of them and check if they are suitable for the operations we want to perform, because not all graphics cards are created equal. For that we'll introduce a new function:

```
1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     return true;
3 }
```

And we'll check if any of the physical devices meet the requirements that we'll add to that function.

```
1 for (const auto& device : devices) {
2     if (isDeviceSuitable(device)) {
3         physicalDevice = device;
4         break;
5     }
6 }
7
8 if (physicalDevice == VK_NULL_HANDLE) {
9     throw std::runtime_error("failed to find a suitable GPU!");
10 }
```

The next section will introduce the first requirements that we'll check for in the `isDeviceSuitable` function. As we'll start using more Vulkan features in the later chapters we will also extend this function to include more checks.

Base device suitability checks

To evaluate the suitability of a device we can start by querying for some details. Basic device properties like the name, type and supported Vulkan version can be queried using `vkGetPhysicalDeviceProperties`.

```
1 VkPhysicalDeviceProperties deviceProperties;
2 vkGetPhysicalDeviceProperties(device, &deviceProperties);
```

The support for optional features like texture compression, 64 bit floats and multi viewport rendering (useful for VR) can be queried using `vkGetPhysicalDeviceFeatures`:

```
1 VkPhysicalDeviceFeatures deviceFeatures;
2 vkGetPhysicalDeviceFeatures(device, &deviceFeatures);
```

There are more details that can be queried from devices that we'll discuss later concerning device memory and queue families (see the next section).

As an example, let's say we consider our application only usable for dedicated graphics cards that support geometry shaders. Then the `isDeviceSuitable` function would look like this:

```
1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     VkPhysicalDeviceProperties deviceProperties;
3     VkPhysicalDeviceFeatures deviceFeatures;
4     vkGetPhysicalDeviceProperties(device, &deviceProperties);
5     vkGetPhysicalDeviceFeatures(device, &deviceFeatures);
6
7     return deviceProperties.deviceType ==
8           VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU &&
9           deviceFeatures.geometryShader;
```

Instead of just checking if a device is suitable or not and going with the first one, you could also give each device a score and pick the highest one. That way you could favor a dedicated graphics card by giving it a higher score, but fall back to an integrated GPU if that's the only available one. You could implement something like that as follows:

```
1 #include <map>
2
3 ...
4
5 void pickPhysicalDevice() {
6     ...
7
8     // Use an ordered map to automatically sort candidates by
9     increasing score
10    std::multimap<int, VkPhysicalDevice> candidates;
11
12    for (const auto& device : devices) {
13        int score = rateDeviceSuitability(device);
14        candidates.insert(std::make_pair(score, device));
15    }
16
17    // Check if the best candidate is suitable at all
```

```

17     if (candidates.rbegin()->first > 0) {
18         physicalDevice = candidates.rbegin()->second;
19     } else {
20         throw std::runtime_error("failed to find a suitable GPU!");
21     }
22 }
23
24 int rateDeviceSuitability(VkPhysicalDevice device) {
25     ...
26
27     int score = 0;
28
29     // Discrete GPUs have a significant performance advantage
30     if (deviceProperties.deviceType ==
31         VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU) {
32         score += 1000;
33     }
34
35     // Maximum possible size of textures affects graphics quality
36     score += deviceProperties.limits.maxImageDimension2D;
37
38     // Application can't function without geometry shaders
39     if (!deviceFeatures.geometryShader) {
40         return 0;
41     }
42
43     return score;
44 }

```

You don't need to implement all that for this tutorial, but it's to give you an idea of how you could design your device selection process. Of course you can also just display the names of the choices and allow the user to select.

Because we're just starting out, Vulkan support is the only thing we need and therefore we'll settle for just any GPU:

```

1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     return true;
3 }

```

In the next section we'll discuss the first real required feature to check for.

Queue families

It has been briefly touched upon before that almost every operation in Vulkan, anything from drawing to uploading textures, requires commands to be submitted to a queue. There are different types of queues that originate from different

queue families and each family of queues allows only a subset of commands. For example, there could be a queue family that only allows processing of compute commands or one that only allows memory transfer related commands.

We need to check which queue families are supported by the device and which one of these supports the commands that we want to use. For that purpose we'll add a new function `findQueueFamilies` that looks for all the queue families we need.

Right now we are only going to look for a queue that supports graphics commands, so the function could look like this:

```
1 uint32_t findQueueFamilies(VkPhysicalDevice device) {
2     // Logic to find graphics queue family
3 }
```

However, in one of the next chapters we're already going to look for yet another queue, so it's better to prepare for that and bundle the indices into a struct:

```
1 struct QueueFamilyIndices {
2     uint32_t graphicsFamily;
3 };
4
5 QueueFamilyIndices findQueueFamilies(VkPhysicalDevice device) {
6     QueueFamilyIndices indices;
7     // Logic to find queue family indices to populate struct with
8     return indices;
9 }
```

But what if a queue family is not available? We could throw an exception in `findQueueFamilies`, but this function is not really the right place to make decisions about device suitability. For example, we may *prefer* devices with a dedicated transfer queue family, but not require it. Therefore we need some way of indicating whether a particular queue family was found.

It's not really possible to use a magic value to indicate the nonexistence of a queue family, since any value of `uint32_t` could in theory be a valid queue family index including 0. Luckily C++17 introduced a data structure to distinguish between the case of a value existing or not:

```
1 #include <optional>
2
3 ...
4
5 std::optional<uint32_t> graphicsFamily;
6
7 std::cout << std::boolalpha << graphicsFamily.has_value() <<
8     std::endl; // false
```

```

9 graphicsFamily = 0;
10
11 std::cout << std::boolalpha << graphicsFamily.has_value() <<
    std::endl; // true

```

`std::optional` is a wrapper that contains no value until you assign something to it. At any point you can query if it contains a value or not by calling its `has_value()` member function. That means that we can change the logic to:

```

1 #include <optional>
2
3 ...
4
5 struct QueueFamilyIndices {
6     std::optional<uint32_t> graphicsFamily;
7 };
8
9 QueueFamilyIndices findQueueFamilies(VkPhysicalDevice device) {
10     QueueFamilyIndices indices;
11     // Assign index to queue families that could be found
12     return indices;
13 }

```

We can now begin to actually implement `findQueueFamilies`:

```

1 QueueFamilyIndices findQueueFamilies(VkPhysicalDevice device) {
2     QueueFamilyIndices indices;
3
4     ...
5
6     return indices;
7 }

```

The process of retrieving the list of queue families is exactly what you expect and uses `vkGetPhysicalDeviceQueueFamilyProperties`:

```

1 uint32_t queueFamilyCount = 0;
2 vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount,
    nullptr);
3
4 std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
5 vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount,
    queueFamilies.data());

```

The `VkQueueFamilyProperties` struct contains some details about the queue family, including the type of operations that are supported and the number of queues that can be created based on that family. We need to find at least one queue family that supports `VK_QUEUE_GRAPHICS_BIT`.

```

1 int i = 0;
2 for (const auto& queueFamily : queueFamilies) {
3     if (queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT) {
4         indices.graphicsFamily = i;
5     }
6
7     i++;
8 }

```

Now that we have this fancy queue family lookup function, we can use it as a check in the `isDeviceSuitable` function to ensure that the device can process the commands we want to use:

```

1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     QueueFamilyIndices indices = findQueueFamilies(device);
3
4     return indices.graphicsFamily.has_value();
5 }

```

To make this a little bit more convenient, we'll also add a generic check to the struct itself:

```

1 struct QueueFamilyIndices {
2     std::optional<uint32_t> graphicsFamily;
3
4     bool isComplete() {
5         return graphicsFamily.has_value();
6     }
7 };
8
9 ...
10
11 bool isDeviceSuitable(VkPhysicalDevice device) {
12     QueueFamilyIndices indices = findQueueFamilies(device);
13
14     return indices.isComplete();
15 }

```

We can now also use this for an early exit from `findQueueFamilies`:

```

1 for (const auto& queueFamily : queueFamilies) {
2     ...
3
4     if (indices.isComplete()) {
5         break;
6     }
7 }

```

```

8     i++;
9 }

```

Great, that's all we need for now to find the right physical device! The next step is to create a logical device to interface with it.

C++ code

Logical device and queues

Introduction

After selecting a physical device to use we need to set up a *logical device* to interface with it. The logical device creation process is similar to the instance creation process and describes the features we want to use. We also need to specify which queues to create now that we've queried which queue families are available. You can even create multiple logical devices from the same physical device if you have varying requirements.

Start by adding a new class member to store the logical device handle in.

```

1 VkDevice device;

```

Next, add a `createLogicalDevice` function that is called from `initVulkan`.

```

1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     pickPhysicalDevice();
5     createLogicalDevice();
6 }
7
8 void createLogicalDevice() {
9
10 }

```

Specifying the queues to be created

The creation of a logical device involves specifying a bunch of details in structs again, of which the first one will be `VkDeviceQueueCreateInfo`. This structure describes the number of queues we want for a single queue family. Right now we're only interested in a queue with graphics capabilities.

```

1 QueueFamilyIndices indices = findQueueFamilies(physicalDevice);
2
3 VkDeviceQueueCreateInfo queueCreateInfo{};
4 queueCreateInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
5 queueCreateInfo.queueFamilyIndex = indices.graphicsFamily.value();
6 queueCreateInfo.queueCount = 1;

```

The currently available drivers will only allow you to create a small number of queues for each queue family and you don't really need more than one. That's because you can create all of the command buffers on multiple threads and then submit them all at once on the main thread with a single low-overhead call.

Vulkan lets you assign priorities to queues to influence the scheduling of command buffer execution using floating point numbers between 0.0 and 1.0. This is required even if there is only a single queue:

```
1 float queuePriority = 1.0f;
2 queueCreateInfo.pQueuePriorities = &queuePriority;
```

Specifying used device features

The next information to specify is the set of device features that we'll be using. These are the features that we queried support for with `vkGetPhysicalDeviceFeatures` in the previous chapter, like geometry shaders. Right now we don't need anything special, so we can simply define it and leave everything to `VK_FALSE`. We'll come back to this structure once we're about to start doing more interesting things with Vulkan.

```
1 VkPhysicalDeviceFeatures deviceFeatures{};
```

Creating the logical device

With the previous two structures in place, we can start filling in the main `VkDeviceCreateInfo` structure.

```
1 VkDeviceCreateInfo createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;
```

First add pointers to the queue creation info and device features structs:

```
1 createInfo.pQueueCreateInfos = &queueCreateInfo;
2 createInfo.queueCreateInfoCount = 1;
3
4 createInfo.pEnabledFeatures = &deviceFeatures;
```

The remainder of the information bears a resemblance to the `VkInstanceCreateInfo` struct and requires you to specify extensions and validation layers. The difference is that these are device specific this time.

An example of a device specific extension is `VK_KHR_swapchain`, which allows you to present rendered images from that device to windows. It is possible that there are Vulkan devices in the system that lack this ability, for example because they only support compute operations. We will come back to this extension in the swap chain chapter.

Previous implementations of Vulkan made a distinction between instance and device specific validation layers, but this is no longer the case. That means that the `enabledLayerCount` and `ppEnabledLayerNames` fields of `VkDeviceCreateInfo` are ignored by up-to-date implementations. However, it is still a good idea to set them anyway to be compatible with older implementations:

```
1 createInfo.enabledExtensionCount = 0;
2
3 if (enableValidationLayers) {
4     createInfo.enabledLayerCount =
5         static_cast<uint32_t>(validationLayers.size());
6     createInfo.ppEnabledLayerNames = validationLayers.data();
7 } else {
8     createInfo.enabledLayerCount = 0;
```

We won't need any device specific extensions for now.

That's it, we're now ready to instantiate the logical device with a call to the appropriately named `vkCreateDevice` function.

```
1 if (vkCreateDevice(physicalDevice, &createInfo, nullptr, &device) !=
2     VK_SUCCESS) {
3     throw std::runtime_error("failed to create logical device!");
4 }
```

The parameters are the physical device to interface with, the queue and usage info we just specified, the optional allocation callbacks pointer and a pointer to a variable to store the logical device handle in. Similarly to the instance creation function, this call can return errors based on enabling non-existent extensions or specifying the desired usage of unsupported features.

The device should be destroyed in `cleanup` with the `vkDestroyDevice` function:

```
1 void cleanup() {
2     vkDestroyDevice(device, nullptr);
3     ...
4 }
```

Logical devices don't interact directly with instances, which is why it's not included as a parameter.

Retrieving queue handles

The queues are automatically created along with the logical device, but we don't have a handle to interface with them yet. First add a class member to store a handle to the graphics queue:

```
1 VkQueue graphicsQueue;
```

Device queues are implicitly cleaned up when the device is destroyed, so we don't need to do anything in `cleanup`.

We can use the `vkGetDeviceQueue` function to retrieve queue handles for each queue family. The parameters are the logical device, queue family, queue index and a pointer to the variable to store the queue handle in. Because we're only creating a single queue from this family, we'll simply use index 0.

```
1 vkGetDeviceQueue(device, indices.graphicsFamily.value(), 0,
    &graphicsQueue);
```

With the logical device and queue handles we can now actually start using the graphics card to do things! In the next few chapters we'll set up the resources to present results to the window system.

C++ code

Presentation

Window surface

Since Vulkan is a platform agnostic API, it can not interface directly with the window system on its own. To establish the connection between Vulkan and the window system to present results to the screen, we need to use the WSI (Window System Integration) extensions. In this chapter we'll discuss the first one, which is `VK_KHR_surface`. It exposes a `VkSurfaceKHR` object that represents an abstract type of surface to present rendered images to. The surface in our program will be backed by the window that we've already opened with GLFW.

The `VK_KHR_surface` extension is an instance level extension and we've actually already enabled it, because it's included in the list returned by `glfwGetRequiredInstanceExtensions`. The list also includes some other WSI extensions that we'll use in the next couple of chapters.

The window surface needs to be created right after the instance creation, because it can actually influence the physical device selection. The reason we postponed this is because window surfaces are part of the larger topic of render targets and presentation for which the explanation would have cluttered the basic setup. It should also be noted that window surfaces are an entirely optional component in Vulkan, if you just need off-screen rendering. Vulkan allows you to do that without hacks like creating an invisible window (necessary for OpenGL).

Window surface creation

Start by adding a `surface` class member right below the debug callback.

```
1 VkSurfaceKHR surface;
```

Although the `VkSurfaceKHR` object and its usage is platform agnostic, its creation isn't because it depends on window system details. For example, it needs the `HWND` and `HMODULE` handles on Windows. Therefore there is a platform-specific addition to the extension, which on Windows is called `VK_KHR_win32_surface` and is also automatically included in the list from `glfwGetRequiredInstanceExtensions`.

I will demonstrate how this platform specific extension can be used to create a surface on Windows, but we won't actually use it in this tutorial. It doesn't make any sense to use a library like GLFW and then proceed to use platform-specific code anyway. GLFW actually has `glfwCreateWindowSurface` that handles the platform differences for us. Still, it's good to see what it does behind the scenes before we start relying on it.

To access native platform functions, you need to update the includes at the top:

```
1 #define VK_USE_PLATFORM_WIN32_KHR
2 #define GLFW_INCLUDE_VULKAN
3 #include <GLFW/glfw3.h>
4 #define GLFW_EXPOSE_NATIVE_WIN32
5 #include <GLFW/glfw3native.h>
```

Because a window surface is a Vulkan object, it comes with a `VkWin32SurfaceCreateInfoKHR` struct that needs to be filled in. It has two important parameters: `hwnd` and `hinstance`. These are the handles to the window and the process.

```
1 VkWin32SurfaceCreateInfoKHR createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_WIN32_SURFACE_CREATE_INFO_KHR;
3 createInfo.hwnd = glfwGetWin32Window(window);
4 createInfo.hinstance = GetModuleHandle(nullptr);
```

The `glfwGetWin32Window` function is used to get the raw `HWND` from the GLFW window object. The `GetModuleHandle` call returns the `HINSTANCE` handle of the current process.

After that the surface can be created with `vkCreateWin32SurfaceKHR`, which includes a parameter for the instance, surface creation details, custom allocators and the variable for the surface handle to be stored in. Technically this is a WSI extension function, but it is so commonly used that the standard Vulkan loader includes it, so unlike other extensions you don't need to explicitly load it.

```
1 if (vkCreateWin32SurfaceKHR(instance, &createInfo, nullptr,
2   &surface) != VK_SUCCESS) {
3   throw std::runtime_error("failed to create window surface!");
4 }
```

The process is similar for other platforms like Linux, where `vkCreateXcbSurfaceKHR` takes an XCB connection and window as creation details with X11.

The `glfwCreateWindowSurface` function performs exactly this operation with a different implementation for each platform. We'll now integrate it into our program. Add a function `createSurface` to be called from `initVulkan` right after instance creation and `setupDebugMessenger`.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7 }
8
9 void createSurface() {
10
11 }
```

The GLFW call takes simple parameters instead of a struct which makes the implementation of the function very straightforward:

```
1 void createSurface() {
2     if (glfwCreateWindowSurface(instance, window, nullptr, &surface)
3         != VK_SUCCESS) {
4         throw std::runtime_error("failed to create window surface!");
5     }
6 }
```

The parameters are the `VkInstance`, GLFW window pointer, custom allocators and pointer to `VkSurfaceKHR` variable. It simply passes through the `VkResult` from the relevant platform call. GLFW doesn't offer a special function for destroying a surface, but that can easily be done through the original API:

```
1 void cleanup() {
2     ...
3     vkDestroySurfaceKHR(instance, surface, nullptr);
4     vkDestroyInstance(instance, nullptr);
5     ...
6 }
```

Make sure that the surface is destroyed before the instance.

Querying for presentation support

Although the Vulkan implementation may support window system integration, that does not mean that every device in the system supports it. Therefore we need to extend `isDeviceSuitable` to ensure that a device can present images to the surface we created. Since the presentation is a queue-specific feature, the

problem is actually about finding a queue family that supports presenting to the surface we created.

It's actually possible that the queue families supporting drawing commands and the ones supporting presentation do not overlap. Therefore we have to take into account that there could be a distinct presentation queue by modifying the `QueueFamilyIndices` structure:

```
1 struct QueueFamilyIndices {
2     std::optional<uint32_t> graphicsFamily;
3     std::optional<uint32_t> presentFamily;
4
5     bool isComplete() {
6         return graphicsFamily.has_value() &&
7             presentFamily.has_value();
8     }
9 };
```

Next, we'll modify the `findQueueFamilies` function to look for a queue family that has the capability of presenting to our window surface. The function to check for that is `vkGetPhysicalDeviceSurfaceSupportKHR`, which takes the physical device, queue family index and surface as parameters. Add a call to it in the same loop as the `VK_QUEUE_GRAPHICS_BIT`:

```
1 VkBool32 presentSupport = false;
2 vkGetPhysicalDeviceSurfaceSupportKHR(device, i, surface,
3     &presentSupport);
```

Then simply check the value of the boolean and store the presentation family queue index:

```
1 if (presentSupport) {
2     indices.presentFamily = i;
3 }
```

Note that it's very likely that these end up being the same queue family after all, but throughout the program we will treat them as if they were separate queues for a uniform approach. Nevertheless, you could add logic to explicitly prefer a physical device that supports drawing and presentation in the same queue for improved performance.

Creating the presentation queue

The one thing that remains is modifying the logical device creation procedure to create the presentation queue and retrieve the `VkQueue` handle. Add a member variable for the handle:

```
1 VkQueue presentQueue;
```

Next, we need to have multiple `VkDeviceQueueCreateInfo` structs to create a queue from both families. An elegant way to do that is to create a set of all unique queue families that are necessary for the required queues:

```

1 #include <set>
2
3 ...
4
5 QueueFamilyIndices indices = findQueueFamilies(physicalDevice);
6
7 std::vector<VkDeviceQueueCreateInfo> queueCreateInfos;
8 std::set<uint32_t> uniqueQueueFamilies =
9     {indices.graphicsFamily.value(), indices.presentFamily.value()};
10
11 float queuePriority = 1.0f;
12 for (uint32_t queueFamily : uniqueQueueFamilies) {
13     VkDeviceQueueCreateInfo queueCreateInfo{};
14     queueCreateInfo.sType =
15         VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
16     queueCreateInfo.queueFamilyIndex = queueFamily;
17     queueCreateInfo.queueCount = 1;
18     queueCreateInfo.pQueuePriorities = &queuePriority;
19     queueCreateInfos.push_back(queueCreateInfo);
20 }

```

And modify `VkDeviceCreateInfo` to point to the vector:

```

1 createInfo.queueCreateInfoCount =
2     static_cast<uint32_t>(queueCreateInfos.size());
3 createInfo.pQueueCreateInfos = queueCreateInfos.data();

```

If the queue families are the same, then we only need to pass its index once. Finally, add a call to retrieve the queue handle:

```

1 vkGetDeviceQueue(device, indices.presentFamily.value(), 0,
2     &presentQueue);

```

In case the queue families are the same, the two handles will most likely have the same value now. In the next chapter we're going to look at swap chains and how they give us the ability to present images to the surface.

C++ code

Swap chain

Vulkan does not have the concept of a “default framebuffer”, hence it requires an infrastructure that will own the buffers we will render to before we visualize them on the screen. This infrastructure is known as the *swap chain* and must

be created explicitly in Vulkan. The swap chain is essentially a queue of images that are waiting to be presented to the screen. Our application will acquire such an image to draw to it, and then return it to the queue. How exactly the queue works and the conditions for presenting an image from the queue depend on how the swap chain is set up, but the general purpose of the swap chain is to synchronize the presentation of images with the refresh rate of the screen.

Checking for swap chain support

Not all graphics cards are capable of presenting images directly to a screen for various reasons, for example because they are designed for servers and don't have any display outputs. Secondly, since image presentation is heavily tied into the window system and the surfaces associated with windows, it is not actually part of the Vulkan core. You have to enable the `VK_KHR_swapchain` device extension after querying for its support.

For that purpose we'll first extend the `isDeviceSuitable` function to check if this extension is supported. We've previously seen how to list the extensions that are supported by a `VkPhysicalDevice`, so doing that should be fairly straightforward. Note that the Vulkan header file provides a nice macro `VK_KHR_SWAPCHAIN_EXTENSION_NAME` that is defined as `VK_KHR_swapchain`. The advantage of using this macro is that the compiler will catch misspellings.

First declare a list of required device extensions, similar to the list of validation layers to enable.

```
1 const std::vector<const char*> deviceExtensions = {
2     VK_KHR_SWAPCHAIN_EXTENSION_NAME
3 };
```

Next, create a new function `checkDeviceExtensionSupport` that is called from `isDeviceSuitable` as an additional check:

```
1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     QueueFamilyIndices indices = findQueueFamilies(device);
3
4     bool extensionsSupported = checkDeviceExtensionSupport(device);
5
6     return indices.isComplete() && extensionsSupported;
7 }
8
9 bool checkDeviceExtensionSupport(VkPhysicalDevice device) {
10     return true;
11 }
```

Modify the body of the function to enumerate the extensions and check if all of the required extensions are amongst them.

```

1 bool checkDeviceExtensionSupport(VkPhysicalDevice device) {
2     uint32_t extensionCount;
3     vkEnumerateDeviceExtensionProperties(device, nullptr,
4         &extensionCount, nullptr);
5
6     std::vector<VkExtensionProperties>
7         availableExtensions(extensionCount);
8     vkEnumerateDeviceExtensionProperties(device, nullptr,
9         &extensionCount, availableExtensions.data());
10
11     std::set<std::string>
12         requiredExtensions(deviceExtensions.begin(),
13             deviceExtensions.end());
14
15     for (const auto& extension : availableExtensions) {
16         requiredExtensions.erase(extension.extensionName);
17     }
18
19     return requiredExtensions.empty();
20 }

```

I've chosen to use a set of strings here to represent the unconfirmed required extensions. That way we can easily tick them off while enumerating the sequence of available extensions. Of course you can also use a nested loop like in `checkValidationLayerSupport`. The performance difference is irrelevant. Now run the code and verify that your graphics card is indeed capable of creating a swap chain. It should be noted that the availability of a presentation queue, as we checked in the previous chapter, implies that the swap chain extension must be supported. However, it's still good to be explicit about things, and the extension does have to be explicitly enabled.

Enabling device extensions

Using a swapchain requires enabling the `VK_KHR_swapchain` extension first. Enabling the extension just requires a small change to the logical device creation structure:

```

1 createInfo.enabledExtensionCount =
2     static_cast<uint32_t>(deviceExtensions.size());
3 createInfo.ppEnabledExtensionNames = deviceExtensions.data();

```

Make sure to replace the existing line `createInfo.enabledExtensionCount = 0;` when you do so.

Querying details of swap chain support

Just checking if a swap chain is available is not sufficient, because it may not actually be compatible with our window surface. Creating a swap chain also involves a lot more settings than instance and device creation, so we need to query for some more details before we're able to proceed.

There are basically three kinds of properties we need to check:

- Basic surface capabilities (min/max number of images in swap chain, min/-max width and height of images)
- Surface formats (pixel format, color space)
- Available presentation modes

Similar to `findQueueFamilies`, we'll use a struct to pass these details around once they've been queried. The three aforementioned types of properties come in the form of the following structs and lists of structs:

```
1 struct SwapChainSupportDetails {
2     VkSurfaceCapabilitiesKHR capabilities;
3     std::vector<VkSurfaceFormatKHR> formats;
4     std::vector<VkPresentModeKHR> presentModes;
5 };
```

We'll now create a new function `querySwapChainSupport` that will populate this struct.

```
1 SwapChainSupportDetails querySwapChainSupport(VkPhysicalDevice
2     device) {
3     SwapChainSupportDetails details;
4     return details;
5 }
```

This section covers how to query the structs that include this information. The meaning of these structs and exactly which data they contain is discussed in the next section.

Let's start with the basic surface capabilities. These properties are simple to query and are returned into a single `VkSurfaceCapabilitiesKHR` struct.

```
1 vkGetPhysicalDeviceSurfaceCapabilitiesKHR(device, surface,
2     &details.capabilities);
```

This function takes the specified `VkPhysicalDevice` and `VkSurfaceKHR` window surface into account when determining the supported capabilities. All of the support querying functions have these two as first parameters because they are the core components of the swap chain.

The next step is about querying the supported surface formats. Because this is a list of structs, it follows the familiar ritual of 2 function calls:

```

1 uint32_t formatCount;
2 vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface, &formatCount,
    nullptr);
3
4 if (formatCount != 0) {
5     details.formats.resize(formatCount);
6     vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface,
        &formatCount, details.formats.data());
7 }

```

Make sure that the vector is resized to hold all the available formats. And finally, querying the supported presentation modes works exactly the same way with `vkGetPhysicalDeviceSurfacePresentModesKHR`:

```

1 uint32_t presentModeCount;
2 vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface,
    &presentModeCount, nullptr);
3
4 if (presentModeCount != 0) {
5     details.presentModes.resize(presentModeCount);
6     vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface,
        &presentModeCount, details.presentModes.data());
7 }

```

All of the details are in the struct now, so let's extend `isDeviceSuitable` once more to utilize this function to verify that swap chain support is adequate. Swap chain support is sufficient for this tutorial if there is at least one supported image format and one supported presentation mode given the window surface we have.

```

1 bool swapChainAdequate = false;
2 if (extensionsSupported) {
3     SwapChainSupportDetails swapChainSupport =
        querySwapChainSupport(device);
4     swapChainAdequate = !swapChainSupport.formats.empty() &&
        !swapChainSupport.presentModes.empty();
5 }

```

It is important that we only try to query for swap chain support after verifying that the extension is available. The last line of the function changes to:

```

1 return indices.isComplete() && extensionsSupported &&
    swapChainAdequate;

```

Choosing the right settings for the swap chain

If the `swapChainAdequate` conditions were met then the support is definitely sufficient, but there may still be many different modes of varying optimality.

We'll now write a couple of functions to find the right settings for the best possible swap chain. There are three types of settings to determine:

- Surface format (color depth)
- Presentation mode (conditions for “swapping” images to the screen)
- Swap extent (resolution of images in swap chain)

For each of these settings we'll have an ideal value in mind that we'll go with if it's available and otherwise we'll create some logic to find the next best thing.

Surface format The function for this setting starts out like this. We'll later pass the `formats` member of the `SwapChainSupportDetails` struct as argument.

```
1 VkSurfaceFormatKHR chooseSwapSurfaceFormat(const
    std::vector<VkSurfaceFormatKHR>& availableFormats) {
2
3 }
```

Each `VkSurfaceFormatKHR` entry contains a `format` and a `colorSpace` member. The `format` member specifies the color channels and types. For example, `VK_FORMAT_B8G8R8A8_SRGB` means that we store the B, G, R and alpha channels in that order with an 8 bit unsigned integer for a total of 32 bits per pixel. The `colorSpace` member indicates if the SRGB color space is supported or not using the `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` flag. Note that this flag used to be called `VK_COLORSPACE_SRGB_NONLINEAR_KHR` in old versions of the specification.

For the color space we'll use SRGB if it is available, because it results in more accurate perceived colors. It is also pretty much the standard color space for images, like the textures we'll use later on. Because of that we should also use an SRGB color format, of which one of the most common ones is `VK_FORMAT_B8G8R8A8_SRGB`.

Let's go through the list and see if the preferred combination is available:

```
1 for (const auto& availableFormat : availableFormats) {
2     if (availableFormat.format == VK_FORMAT_B8G8R8A8_SRGB &&
        availableFormat.colorSpace ==
        VK_COLOR_SPACE_SRGB_NONLINEAR_KHR) {
3         return availableFormat;
4     }
5 }
```

If that also fails then we could start ranking the available formats based on how “good” they are, but in most cases it's okay to just settle with the first format that is specified.

```

1 VkSurfaceFormatKHR chooseSwapSurfaceFormat(const
    std::vector<VkSurfaceFormatKHR>& availableFormats) {
2     for (const auto& availableFormat : availableFormats) {
3         if (availableFormat.format == VK_FORMAT_B8G8R8A8_SRGB &&
            availableFormat.colorSpace ==
            VK_COLOR_SPACE_SRGB_NONLINEAR_KHR) {
4             return availableFormat;
5         }
6     }
7
8     return availableFormats[0];
9 }

```

Presentation mode The presentation mode is arguably the most important setting for the swap chain, because it represents the actual conditions for showing images to the screen. There are four possible modes available in Vulkan:

- **VK_PRESENT_MODE_IMMEDIATE_KHR**: Images submitted by your application are transferred to the screen right away, which may result in tearing.
- **VK_PRESENT_MODE_FIFO_KHR**: The swap chain is a queue where the display takes an image from the front of the queue when the display is refreshed and the program inserts rendered images at the back of the queue. If the queue is full then the program has to wait. This is most similar to vertical sync as found in modern games. The moment that the display is refreshed is known as “vertical blank”.
- **VK_PRESENT_MODE_FIFO_RELAXED_KHR**: This mode only differs from the previous one if the application is late and the queue was empty at the last vertical blank. Instead of waiting for the next vertical blank, the image is transferred right away when it finally arrives. This may result in visible tearing.
- **VK_PRESENT_MODE_MAILBOX_KHR**: This is another variation of the second mode. Instead of blocking the application when the queue is full, the images that are already queued are simply replaced with the newer ones. This mode can be used to render frames as fast as possible while still avoiding tearing, resulting in fewer latency issues than standard vertical sync. This is commonly known as “triple buffering”, although the existence of three buffers alone does not necessarily mean that the framerate is unlocked.

Only the **VK_PRESENT_MODE_FIFO_KHR** mode is guaranteed to be available, so we’ll again have to write a function that looks for the best mode that is available:

```

1 VkPresentModeKHR chooseSwapPresentMode(const
    std::vector<VkPresentModeKHR>& availablePresentModes) {
2     return VK_PRESENT_MODE_FIFO_KHR;
3 }

```

I personally think that `VK_PRESENT_MODE_MAILBOX_KHR` is a very nice trade-off if energy usage is not a concern. It allows us to avoid tearing while still maintaining a fairly low latency by rendering new images that are as up-to-date as possible right until the vertical blank. On mobile devices, where energy usage is more important, you will probably want to use `VK_PRESENT_MODE_FIFO_KHR` instead. Now, let's look through the list to see if `VK_PRESENT_MODE_MAILBOX_KHR` is available:

```

1 VkPresentModeKHR chooseSwapPresentMode(const
    std::vector<VkPresentModeKHR>& availablePresentModes) {
2     for (const auto& availablePresentMode : availablePresentModes) {
3         if (availablePresentMode == VK_PRESENT_MODE_MAILBOX_KHR) {
4             return availablePresentMode;
5         }
6     }
7
8     return VK_PRESENT_MODE_FIFO_KHR;
9 }

```

Swap extent That leaves only one major property, for which we'll add one last function:

```

1 VkExtent2D chooseSwapExtent(const VkSurfaceCapabilitiesKHR&
    capabilities) {
2
3 }

```

The swap extent is the resolution of the swap chain images and it's almost always exactly equal to the resolution of the window that we're drawing to *in pixels* (more on that in a moment). The range of the possible resolutions is defined in the `VkSurfaceCapabilitiesKHR` structure. Vulkan tells us to match the resolution of the window by setting the width and height in the `currentExtent` member. However, some window managers do allow us to differ here and this is indicated by setting the width and height in `currentExtent` to a special value: the maximum value of `uint32_t`. In that case we'll pick the resolution that best matches the window within the `minImageExtent` and `maxImageExtent` bounds. But we must specify the resolution in the correct unit.

GLFW uses two units when measuring sizes: pixels and screen coordinates. For example, the resolution `{WIDTH, HEIGHT}` that we specified earlier when creating the window is measured in screen coordinates. But Vulkan works with pixels, so the swap chain extent must be specified in pixels as well. Unfortunately, if you are using a high DPI display (like Apple's Retina display), screen coordinates don't correspond to pixels. Instead, due to the higher pixel density, the resolution of the window in pixel will be larger than the resolution in screen coordinates. So if Vulkan doesn't fix the swap extent for us, we can't just use the original `{WIDTH, HEIGHT}`. Instead, we must use `glfwGetFramebufferSize`

to query the resolution of the window in pixel before matching it against the minimum and maximum image extent.

```
1 #include <cstdint> // Necessary for uint32_t
2 #include <limits> // Necessary for std::numeric_limits
3 #include <algorithm> // Necessary for std::clamp
4
5 ...
6
7 VkExtent2D chooseSwapExtent(const VkSurfaceCapabilitiesKHR&
    capabilities) {
8     if (capabilities.currentExtent.width !=
        std::numeric_limits<uint32_t>::max()) {
9         return capabilities.currentExtent;
10    } else {
11        int width, height;
12        glfwGetFramebufferSize(window, &width, &height);
13
14        VkExtent2D actualExtent = {
15            static_cast<uint32_t>(width),
16            static_cast<uint32_t>(height)
17        };
18
19        actualExtent.width = std::clamp(actualExtent.width,
            capabilities.minImageExtent.width,
            capabilities.maxImageExtent.width);
20        actualExtent.height = std::clamp(actualExtent.height,
            capabilities.minImageExtent.height,
            capabilities.maxImageExtent.height);
21
22        return actualExtent;
23    }
24 }
```

The `clamp` function is used here to bound the values of `width` and `height` between the allowed minimum and maximum extents that are supported by the implementation.

Creating the swap chain

Now that we have all of these helper functions assisting us with the choices we have to make at runtime, we finally have all the information that is needed to create a working swap chain.

Create a `createSwapChain` function that starts out with the results of these calls and make sure to call it from `initVulkan` after logical device creation.

```
1 void initVulkan() {
```

```

2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8 }
9
10 void createSwapChain() {
11     SwapChainSupportDetails swapChainSupport =
12         querySwapChainSupport(physicalDevice);
13
14     VkSurfaceFormatKHR surfaceFormat =
15         chooseSwapSurfaceFormat(swapChainSupport.formats);
16     VkPresentModeKHR presentMode =
17         chooseSwapPresentMode(swapChainSupport.presentModes);
18     VkExtent2D extent =
19         chooseSwapExtent(swapChainSupport.capabilities);
20 }

```

Aside from these properties we also have to decide how many images we would like to have in the swap chain. The implementation specifies the minimum number that it requires to function:

```

1 uint32_t imageCount = swapChainSupport.capabilities.minImageCount;

```

However, simply sticking to this minimum means that we may sometimes have to wait on the driver to complete internal operations before we can acquire another image to render to. Therefore it is recommended to request at least one more image than the minimum:

```

1 uint32_t imageCount = swapChainSupport.capabilities.minImageCount +
2     1;

```

We should also make sure to not exceed the maximum number of images while doing this, where 0 is a special value that means that there is no maximum:

```

1 if (swapChainSupport.capabilities.maxImageCount > 0 && imageCount >
2     swapChainSupport.capabilities.maxImageCount) {
3     imageCount = swapChainSupport.capabilities.maxImageCount;
4 }

```

As is tradition with Vulkan objects, creating the swap chain object requires filling in a large structure. It starts out very familiarly:

```

1 VkSwapchainCreateInfoKHR createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR;
3 createInfo.surface = surface;

```

After specifying which surface the swap chain should be tied to, the details of the swap chain images are specified:

```
1 createInfo.minImageCount = imageCount;
2 createInfo.imageFormat = surfaceFormat.format;
3 createInfo.imageColorSpace = surfaceFormat.colorSpace;
4 createInfo.imageExtent = extent;
5 createInfo.imageArrayLayers = 1;
6 createInfo.imageUsage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT;
```

The `imageArrayLayers` specifies the amount of layers each image consists of. This is always 1 unless you are developing a stereoscopic 3D application. The `imageUsage` bit field specifies what kind of operations we'll use the images in the swap chain for. In this tutorial we're going to render directly to them, which means that they're used as color attachment. It is also possible that you'll render images to a separate image first to perform operations like post-processing. In that case you may use a value like `VK_IMAGE_USAGE_TRANSFER_DST_BIT` instead and use a memory operation to transfer the rendered image to a swap chain image.

```
1 QueueFamilyIndices indices = findQueueFamilies(physicalDevice);
2 uint32_t queueFamilyIndices[] = {indices.graphicsFamily.value(),
   indices.presentFamily.value()};
3
4 if (indices.graphicsFamily != indices.presentFamily) {
5     createInfo.imageSharingMode = VK_SHARING_MODE_CONCURRENT;
6     createInfo.queueFamilyIndexCount = 2;
7     createInfo.pQueueFamilyIndices = queueFamilyIndices;
8 } else {
9     createInfo.imageSharingMode = VK_SHARING_MODE_EXCLUSIVE;
10    createInfo.queueFamilyIndexCount = 0; // Optional
11    createInfo.pQueueFamilyIndices = nullptr; // Optional
12 }
```

Next, we need to specify how to handle swap chain images that will be used across multiple queue families. That will be the case in our application if the graphics queue family is different from the presentation queue. We'll be drawing on the images in the swap chain from the graphics queue and then submitting them on the presentation queue. There are two ways to handle images that are accessed from multiple queues:

- `VK_SHARING_MODE_EXCLUSIVE`: An image is owned by one queue family at a time and ownership must be explicitly transferred before using it in another queue family. This option offers the best performance.
- `VK_SHARING_MODE_CONCURRENT`: Images can be used across multiple queue families without explicit ownership transfers.

If the queue families differ, then we'll be using the concurrent mode in this

tutorial to avoid having to do the ownership chapters, because these involve some concepts that are better explained at a later time. Concurrent mode requires you to specify in advance between which queue families ownership will be shared using the `queueFamilyIndexCount` and `pQueueFamilyIndices` parameters. If the graphics queue family and presentation queue family are the same, which will be the case on most hardware, then we should stick to exclusive mode, because concurrent mode requires you to specify at least two distinct queue families.

```
1 createInfo.preTransform =
    swapChainSupport.capabilities.currentTransform;
```

We can specify that a certain transform should be applied to images in the swap chain if it is supported (`supportedTransforms` in `capabilities`), like a 90 degree clockwise rotation or horizontal flip. To specify that you do not want any transformation, simply specify the current transformation.

```
1 createInfo.compositeAlpha = VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR;
```

The `compositeAlpha` field specifies if the alpha channel should be used for blending with other windows in the window system. You'll almost always want to simply ignore the alpha channel, hence `VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR`.

```
1 createInfo.presentMode = presentMode;
2 createInfo.clipped = VK_TRUE;
```

The `presentMode` member speaks for itself. If the `clipped` member is set to `VK_TRUE` then that means that we don't care about the color of pixels that are obscured, for example because another window is in front of them. Unless you really need to be able to read these pixels back and get predictable results, you'll get the best performance by enabling clipping.

```
1 createInfo.oldSwapchain = VK_NULL_HANDLE;
```

That leaves one last field, `oldSwapchain`. With Vulkan it's possible that your swap chain becomes invalid or unoptimized while your application is running, for example because the window was resized. In that case the swap chain actually needs to be recreated from scratch and a reference to the old one must be specified in this field. This is a complex topic that we'll learn more about in a future chapter. For now we'll assume that we'll only ever create one swap chain.

Now add a class member to store the `VkSwapchainKHR` object:

```
1 VkSwapchainKHR swapChain;
```

Creating the swap chain is now as simple as calling `vkCreateSwapchainKHR`:

```
1 if (vkCreateSwapchainKHR(device, &createInfo, nullptr, &swapChain)
    != VK_SUCCESS) {
```

```

2     throw std::runtime_error("failed to create swap chain!");
3 }

```

The parameters are the logical device, swap chain creation info, optional custom allocators and a pointer to the variable to store the handle in. No surprises there. It should be cleaned up using `vkDestroySwapchainKHR` before the device:

```

1 void cleanup() {
2     vkDestroySwapchainKHR(device, swapChain, nullptr);
3     ...
4 }

```

Now run the application to ensure that the swap chain is created successfully! If at this point you get an access violation error in `vkCreateSwapchainKHR` or see a message like `Failed to find 'vkGetInstanceProcAddress' in layer SteamOverlayVulkanLayer.dll`, then see the FAQ entry about the Steam overlay layer.

Try removing the `createInfo.imageExtent = extent;` line with validation layers enabled. You'll see that one of the validation layers immediately catches the mistake and a helpful message is printed:

```

validation layer: vkCreateSwapchainKHR() called with pCreateInfo->imageExtent =
(0,0), which is not equal to the currentExtent = (800,600) returned by vkGetPhys
icalDeviceSurfaceCapabilitiesKHR().

```

Retrieving the swap chain images

The swap chain has been created now, so all that remains is retrieving the handles of the `VkImages` in it. We'll reference these during rendering operations in later chapters. Add a class member to store the handles:

```

1 std::vector<VkImage> swapChainImages;

```

The images were created by the implementation for the swap chain and they will be automatically cleaned up once the swap chain has been destroyed, therefore we don't need to add any cleanup code.

I'm adding the code to retrieve the handles to the end of the `createSwapChain` function, right after the `vkCreateSwapchainKHR` call. Retrieving them is very similar to the other times where we retrieved an array of objects from Vulkan. Remember that we only specified a minimum number of images in the swap chain, so the implementation is allowed to create a swap chain with more. That's why we'll first query the final number of images with `vkGetSwapchainImagesKHR`, then resize the container and finally call it again to retrieve the handles.

```

1 vkGetSwapchainImagesKHR(device, swapChain, &imageCount, nullptr);
2 swapChainImages.resize(imageCount);

```

```
3 vkGetSwapchainImagesKHR(device, swapChain, &imageCount,
    swapChainImages.data());
```

One last thing, store the format and extent we've chosen for the swap chain images in member variables. We'll need them in future chapters.

```
1 VkSwapchainKHR swapChain;
2 std::vector<VkImage> swapChainImages;
3 VkFormat swapChainImageFormat;
4 VkExtent2D swapChainExtent;
5
6 ...
7
8 swapChainImageFormat = surfaceFormat.format;
9 swapChainExtent = extent;
```

We now have a set of images that can be drawn onto and can be presented to the window. The next chapter will begin to cover how we can set up the images as render targets and then we start looking into the actual graphics pipeline and drawing commands!

C++ code

Image views

To use any `VkImage`, including those in the swap chain, in the render pipeline we have to create a `VkImageView` object. An image view is quite literally a view into an image. It describes how to access the image and which part of the image to access, for example if it should be treated as a 2D texture depth texture without any mipmapping levels.

In this chapter we'll write a `createImageViews` function that creates a basic image view for every image in the swap chain so that we can use them as color targets later on.

First add a class member to store the image views in:

```
1 std::vector<VkImageView> swapChainImageViews;
```

Create the `createImageViews` function and call it right after swap chain creation.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
```

```

8     createImageViews();
9 }
10
11 void createImageViews() {
12
13 }

```

The first thing we need to do is resize the list to fit all of the image views we'll be creating:

```

1 void createImageViews() {
2     swapChainImageViews.resize(swapChainImages.size());
3
4 }

```

Next, set up the loop that iterates over all of the swap chain images.

```

1 for (size_t i = 0; i < swapChainImages.size(); i++) {
2
3 }

```

The parameters for image view creation are specified in a `VkImageViewCreateInfo` structure. The first few parameters are straightforward.

```

1 VkImageViewCreateInfo createInfo{};
2 createInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
3 createInfo.image = swapChainImages[i];

```

The `viewType` and `format` fields specify how the image data should be interpreted. The `viewType` parameter allows you to treat images as 1D textures, 2D textures, 3D textures and cube maps.

```

1 createInfo.viewType = VK_IMAGE_VIEW_TYPE_2D;
2 createInfo.format = swapChainImageFormat;

```

The `components` field allows you to swizzle the color channels around. For example, you can map all of the channels to the red channel for a monochrome texture. You can also map constant values of 0 and 1 to a channel. In our case we'll stick to the default mapping.

```

1 createInfo.components.r = VK_COMPONENT_SWIZZLE_IDENTITY;
2 createInfo.components.g = VK_COMPONENT_SWIZZLE_IDENTITY;
3 createInfo.components.b = VK_COMPONENT_SWIZZLE_IDENTITY;
4 createInfo.components.a = VK_COMPONENT_SWIZZLE_IDENTITY;

```

The `subresourceRange` field describes what the image's purpose is and which part of the image should be accessed. Our images will be used as color targets without any mipmapping levels or multiple layers.

```

1 createInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
2 createInfo.subresourceRange.baseMipLevel = 0;
3 createInfo.subresourceRange.levelCount = 1;
4 createInfo.subresourceRange.baseArrayLayer = 0;
5 createInfo.subresourceRange.layerCount = 1;

```

If you were working on a stereographic 3D application, then you would create a swap chain with multiple layers. You could then create multiple image views for each image representing the views for the left and right eyes by accessing different layers.

Creating the image view is now a matter of calling `vkCreateImageView`:

```

1 if (vkCreateImageView(device, &createInfo, nullptr,
    &swapChainImageViews[i]) != VK_SUCCESS) {
2     throw std::runtime_error("failed to create image views!");
3 }

```

Unlike images, the image views were explicitly created by us, so we need to add a similar loop to destroy them again at the end of the program:

```

1 void cleanup() {
2     for (auto imageView : swapChainImageViews) {
3         vkDestroyImageView(device, imageView, nullptr);
4     }
5
6     ...
7 }

```

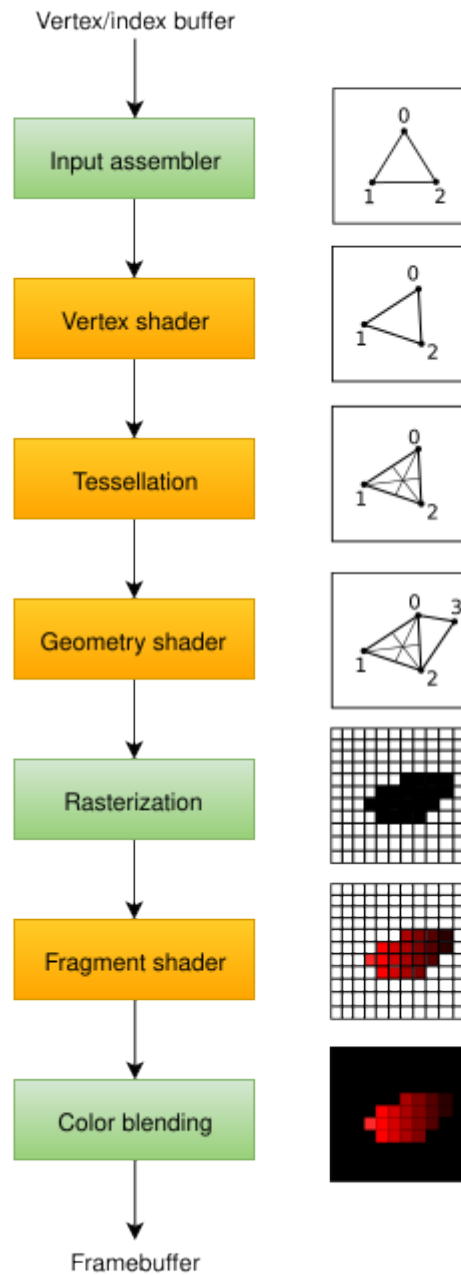
An image view is sufficient to start using an image as a texture, but it's not quite ready to be used as a render target just yet. That requires one more step of indirection, known as a framebuffer. But first we'll have to set up the graphics pipeline.

C++ code

Graphics pipeline basics

Introduction

Over the course of the next few chapters we'll be setting up a graphics pipeline that is configured to draw our first triangle. The graphics pipeline is the sequence of operations that take the vertices and textures of your meshes all the way to the pixels in the render targets. A simplified overview is displayed below:



The *input assembler* collects the raw vertex data from the buffers you specify and may also use an index buffer to repeat certain elements without having to duplicate the vertex data itself.

The *vertex shader* is run for every vertex and generally applies transformations

to turn vertex positions from model space to screen space. It also passes per-vertex data down the pipeline.

The *tessellation shaders* allow you to subdivide geometry based on certain rules to increase the mesh quality. This is often used to make surfaces like brick walls and staircases look less flat when they are nearby.

The *geometry shader* is run on every primitive (triangle, line, point) and can discard it or output more primitives than came in. This is similar to the tessellation shader, but much more flexible. However, it is not used much in today's applications because the performance is not that good on most graphics cards except for Intel's integrated GPUs.

The *rasterization stage* discretizes the primitives into *fragments*. These are the pixel elements that they fill on the framebuffer. Any fragments that fall outside the screen are discarded and the attributes outputted by the vertex shader are interpolated across the fragments, as shown in the figure. Usually the fragments that are behind other primitive fragments are also discarded here because of depth testing.

The *fragment shader* is invoked for every fragment that survives and determines which framebuffer(s) the fragments are written to and with which color and depth values. It can do this using the interpolated data from the vertex shader, which can include things like texture coordinates and normals for lighting.

The *color blending* stage applies operations to mix different fragments that map to the same pixel in the framebuffer. Fragments can simply overwrite each other, add up or be mixed based upon transparency.

Stages with a green color are known as *fixed-function* stages. These stages allow you to tweak their operations using parameters, but the way they work is predefined.

Stages with an orange color on the other hand are **programmable**, which means that you can upload your own code to the graphics card to apply exactly the operations you want. This allows you to use fragment shaders, for example, to implement anything from texturing and lighting to ray tracers. These programs run on many GPU cores simultaneously to process many objects, like vertices and fragments in parallel.

If you've used older APIs like OpenGL and Direct3D before, then you'll be used to being able to change any pipeline settings at will with calls like `glBlendFunc` and `OMSetBlendState`. The graphics pipeline in Vulkan is almost completely immutable, so you must recreate the pipeline from scratch if you want to change shaders, bind different framebuffers or change the blend function. The disadvantage is that you'll have to create a number of pipelines that represent all of the different combinations of states you want to use in your rendering operations. However, because all of the operations you'll be doing in the pipeline are known in advance, the driver can optimize for it much better.

Some of the programmable stages are optional based on what you intend to do. For example, the tessellation and geometry stages can be disabled if you are just drawing simple geometry. If you are only interested in depth values then you can disable the fragment shader stage, which is useful for shadow map generation.

In the next chapter we'll first create the two programmable stages required to put a triangle onto the screen: the vertex shader and fragment shader. The fixed-function configuration like blending mode, viewport, rasterization will be set up in the chapter after that. The final part of setting up the graphics pipeline in Vulkan involves the specification of input and output framebuffers.

Create a `createGraphicsPipeline` function that is called right after `createImageViews` in `initVulkan`. We'll work on this function throughout the following chapters.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8     createImageViews();
9     createGraphicsPipeline();
10 }
11
12 ...
13
14 void createGraphicsPipeline() {
15
16 }
```

C++ code

Shader modules

Unlike earlier APIs, shader code in Vulkan has to be specified in a bytecode format as opposed to human-readable syntax like GLSL and HLSL. This bytecode format is called SPIR-V and is designed to be used with both Vulkan and OpenCL (both Khronos APIs). It is a format that can be used to write graphics and compute shaders, but we will focus on shaders used in Vulkan's graphics pipelines in this tutorial.

The advantage of using a bytecode format is that the compilers written by GPU vendors to turn shader code into native code are significantly less complex. The past has shown that with human-readable syntax like GLSL, some GPU vendors were rather flexible with their interpretation of the standard. If you happen to

write non-trivial shaders with a GPU from one of these vendors, then you'd risk other vendor's drivers rejecting your code due to syntax errors, or worse, your shader running differently because of compiler bugs. With a straightforward bytecode format like SPIR-V that will hopefully be avoided.

However, that does not mean that we need to write this bytecode by hand. Khronos has released their own vendor-independent compiler that compiles GLSL to SPIR-V. This compiler is designed to verify that your shader code is fully standards compliant and produces one SPIR-V binary that you can ship with your program. You can also include this compiler as a library to produce SPIR-V at runtime, but we won't be doing that in this tutorial. Although we can use this compiler directly via `glslangValidator.exe`, we will be using `glslc.exe` by Google instead. The advantage of `glslc` is that it uses the same parameter format as well-known compilers like GCC and Clang and includes some extra functionality like *includes*. Both of them are already included in the Vulkan SDK, so you don't need to download anything extra.

GLSL is a shading language with a C-style syntax. Programs written in it have a `main` function that is invoked for every object. Instead of using parameters for input and a return value as output, GLSL uses global variables to handle input and output. The language includes many features to aid in graphics programming, like built-in vector and matrix primitives. Functions for operations like cross products, matrix-vector products and reflections around a vector are included. The vector type is called `vec` with a number indicating the amount of elements. For example, a 3D position would be stored in a `vec3`. It is possible to access single components through members like `.x`, but it's also possible to create a new vector from multiple components at the same time. For example, the expression `vec3(1.0, 2.0, 3.0).xy` would result in `vec2`. The constructors of vectors can also take combinations of vector objects and scalar values. For example, a `vec3` can be constructed with `vec3(vec2(1.0, 2.0), 3.0)`.

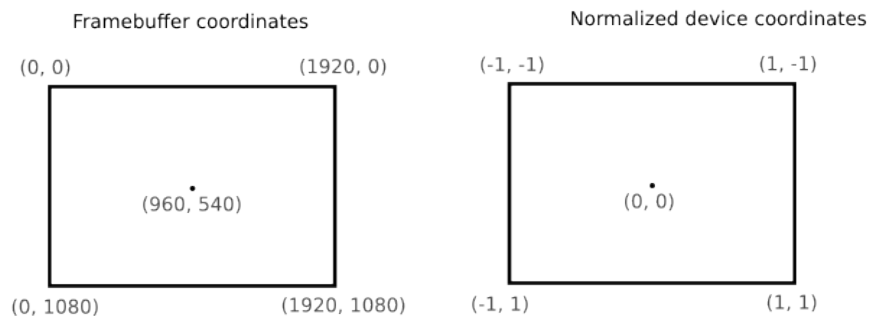
As the previous chapter mentioned, we need to write a vertex shader and a fragment shader to get a triangle on the screen. The next two sections will cover the GLSL code of each of those and after that I'll show you how to produce two SPIR-V binaries and load them into the program.

Vertex shader

The vertex shader processes each incoming vertex. It takes its attributes, like model space position, color, normal and texture coordinates as input. The output is the final position in clip coordinates and the attributes that need to be passed on to the fragment shader, like color and texture coordinates. These values will then be interpolated over the fragments by the rasterizer to produce a smooth gradient.

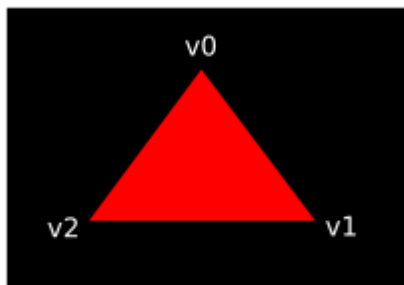
A *clip coordinate* is a four dimensional vector from the vertex shader that is subsequently turned into a *normalized device coordinate* by dividing the whole vector by its last component. These normalized device coordinates are homo-

geneous coordinates that map the framebuffer to a $[-1, 1]$ by $[-1, 1]$ coordinate system that looks like the following:



You should already be familiar with these if you have dabbled in computer graphics before. If you have used OpenGL before, then you'll notice that the sign of the Y coordinates is now flipped. The Z coordinate now uses the same range as it does in Direct3D, from 0 to 1.

For our first triangle we won't be applying any transformations, we'll just specify the positions of the three vertices directly as normalized device coordinates to create the following shape:



We can directly output normalized device coordinates by outputting them as clip coordinates from the vertex shader with the last component set to 1. That way the division to transform clip coordinates to normalized device coordinates will not change anything.

Normally these coordinates would be stored in a vertex buffer, but creating a vertex buffer in Vulkan and filling it with data is not trivial. Therefore I've

decided to postpone that until after we've had the satisfaction of seeing a triangle pop up on the screen. We're going to do something a little unorthodox in the meanwhile: include the coordinates directly inside the vertex shader. The code looks like this:

```
1 #version 450
2
3 vec2 positions[3] = vec2[] (
4     vec2(0.0, -0.5),
5     vec2(0.5, 0.5),
6     vec2(-0.5, 0.5)
7 );
8
9 void main() {
10     gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);
11 }
```

The `main` function is invoked for every vertex. The built-in `gl_VertexIndex` variable contains the index of the current vertex. This is usually an index into the vertex buffer, but in our case it will be an index into a hardcoded array of vertex data. The position of each vertex is accessed from the constant array in the shader and combined with dummy `z` and `w` components to produce a position in clip coordinates. The built-in variable `gl_Position` functions as the output.

Fragment shader

The triangle that is formed by the positions from the vertex shader fills an area on the screen with fragments. The fragment shader is invoked on these fragments to produce a color and depth for the framebuffer (or framebuffers). A simple fragment shader that outputs the color red for the entire triangle looks like this:

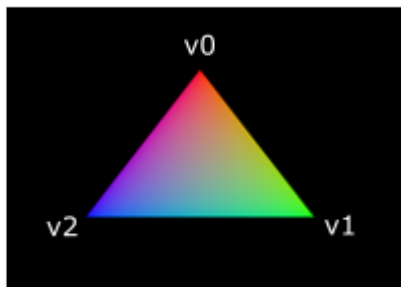
```
1 #version 450
2
3 layout(location = 0) out vec4 outColor;
4
5 void main() {
6     outColor = vec4(1.0, 0.0, 0.0, 1.0);
7 }
```

The `main` function is called for every fragment just like the vertex shader `main` function is called for every vertex. Colors in GLSL are 4-component vectors with the R, G, B and alpha channels within the `[0, 1]` range. Unlike `gl_Position` in the vertex shader, there is no built-in variable to output a color for the current fragment. You have to specify your own output variable for each framebuffer

where the `layout(location = 0)` modifier specifies the index of the framebuffer. The color red is written to this `outColor` variable that is linked to the first (and only) framebuffer at index 0.

Per-vertex colors

Making the entire triangle red is not very interesting, wouldn't something like the following look a lot nicer?



We have to make a couple of changes to both shaders to accomplish this. First off, we need to specify a distinct color for each of the three vertices. The vertex shader should now include an array with colors just like it does for positions:

```
1 vec3 colors[3] = vec3[3] (  
2     vec3(1.0, 0.0, 0.0),  
3     vec3(0.0, 1.0, 0.0),  
4     vec3(0.0, 0.0, 1.0)  
5 );
```

Now we just need to pass these per-vertex colors to the fragment shader so it can output their interpolated values to the framebuffer. Add an output for color to the vertex shader and write to it in the `main` function:

```
1 layout(location = 0) out vec3 fragColor;  
2  
3 void main() {  
4     gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);  
5     fragColor = colors[gl_VertexIndex];  
6 }
```

Next, we need to add a matching input in the fragment shader:

```
1 layout(location = 0) in vec3 fragColor;  
2
```

```

3 void main() {
4     outColor = vec4(fragColor, 1.0);
5 }

```

The input variable does not necessarily have to use the same name, they will be linked together using the indexes specified by the `location` directives. The `main` function has been modified to output the color along with an alpha value. As shown in the image above, the values for `fragColor` will be automatically interpolated for the fragments between the three vertices, resulting in a smooth gradient.

Compiling the shaders

Create a directory called `shaders` in the root directory of your project and store the vertex shader in a file called `shader.vert` and the fragment shader in a file called `shader.frag` in that directory. GLSL shaders don't have an official extension, but these two are commonly used to distinguish them.

The contents of `shader.vert` should be:

```

1 #version 450
2
3 layout(location = 0) out vec3 fragColor;
4
5 vec2 positions[3] = vec2[](
6     vec2(0.0, -0.5),
7     vec2(0.5, 0.5),
8     vec2(-0.5, 0.5)
9 );
10
11 vec3 colors[3] = vec3[](
12     vec3(1.0, 0.0, 0.0),
13     vec3(0.0, 1.0, 0.0),
14     vec3(0.0, 0.0, 1.0)
15 );
16
17 void main() {
18     gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);
19     fragColor = colors[gl_VertexIndex];
20 }

```

And the contents of `shader.frag` should be:

```

1 #version 450
2
3 layout(location = 0) in vec3 fragColor;
4
5 layout(location = 0) out vec4 outColor;

```

```

6
7 void main() {
8     outColor = vec4(fragColor, 1.0);
9 }

```

We're now going to compile these into SPIR-V bytecode using the `glslc` program.

Windows

Create a `compile.bat` file with the following contents:

```

1 C:/VulkanSDK/x.x.x.x/Bin/glslc.exe shader.vert -o vert.spv
2 C:/VulkanSDK/x.x.x.x/Bin/glslc.exe shader.frag -o frag.spv
3 pause

```

Replace the path to `glslc.exe` with the path to where you installed the Vulkan SDK. Double click the file to run it.

Linux

Create a `compile.sh` file with the following contents:

```

1 /home/user/VulkanSDK/x.x.x.x/x86_64/bin/glslc shader.vert -o vert.spv
2 /home/user/VulkanSDK/x.x.x.x/x86_64/bin/glslc shader.frag -o frag.spv

```

Replace the path to `glslc` with the path to where you installed the Vulkan SDK. Make the script executable with `chmod +x compile.sh` and run it.

End of platform-specific instructions

These two commands tell the compiler to read the GLSL source file and output a SPIR-V bytecode file using the `-o` (output) flag.

If your shader contains a syntax error then the compiler will tell you the line number and problem, as you would expect. Try leaving out a semicolon for example and run the compile script again. Also try running the compiler without any arguments to see what kinds of flags it supports. It can, for example, also output the bytecode into a human-readable format so you can see exactly what your shader is doing and any optimizations that have been applied at this stage.

Compiling shaders on the commandline is one of the most straightforward options and it's the one that we'll use in this tutorial, but it's also possible to compile shaders directly from your own code. The Vulkan SDK includes `libshaderc`, which is a library to compile GLSL code to SPIR-V from within your program.

Loading a shader

Now that we have a way of producing SPIR-V shaders, it's time to load them into our program to plug them into the graphics pipeline at some point. We'll first write a simple helper function to load the binary data from the files.

```

1 #include <fstream>
2
3 ...
4
5 static std::vector<char> readFile(const std::string& filename) {
6     std::ifstream file(filename, std::ios::ate | std::ios::binary);
7
8     if (!file.is_open()) {
9         throw std::runtime_error("failed to open file!");
10    }
11 }

```

The `readFile` function will read all of the bytes from the specified file and return them in a byte array managed by `std::vector`. We start by opening the file with two flags:

- `ate`: Start reading at the end of the file
- `binary`: Read the file as binary file (avoid text transformations)

The advantage of starting to read at the end of the file is that we can use the read position to determine the size of the file and allocate a buffer:

```

1 size_t fileSize = (size_t) file.tellg();
2 std::vector<char> buffer(fileSize);

```

After that, we can seek back to the beginning of the file and read all of the bytes at once:

```

1 file.seekg(0);
2 file.read(buffer.data(), fileSize);

```

And finally close the file and return the bytes:

```

1 file.close();
2
3 return buffer;

```

We'll now call this function from `createGraphicsPipeline` to load the bytecode of the two shaders:

```

1 void createGraphicsPipeline() {
2     auto vertShaderCode = readFile("shaders/vert.spv");
3     auto fragShaderCode = readFile("shaders/frag.spv");
4 }

```

Make sure that the shaders are loaded correctly by printing the size of the buffers and checking if they match the actual file size in bytes. Note that the code doesn't need to be null terminated since it's binary code and we will later be explicit about its size.

Creating shader modules

Before we can pass the code to the pipeline, we have to wrap it in a `VkShaderModule` object. Let's create a helper function `createShaderModule` to do that.

```
1 VkShaderModule createShaderModule(const std::vector<char>& code) {  
2  
3 }
```

The function will take a buffer with the bytecode as parameter and create a `VkShaderModule` from it.

Creating a shader module is simple, we only need to specify a pointer to the buffer with the bytecode and the length of it. This information is specified in a `VkShaderModuleCreateInfo` structure. The one catch is that the size of the bytecode is specified in bytes, but the bytecode pointer is a `uint32_t` pointer rather than a `char` pointer. Therefore we will need to cast the pointer with `reinterpret_cast` as shown below. When you perform a cast like this, you also need to ensure that the data satisfies the alignment requirements of `uint32_t`. Lucky for us, the data is stored in an `std::vector` where the default allocator already ensures that the data satisfies the worst case alignment requirements.

```
1 VkShaderModuleCreateInfo createInfo{};  
2 createInfo.sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO;  
3 createInfo.codeSize = code.size();  
4 createInfo.pCode = reinterpret_cast<const uint32_t*>(code.data());
```

The `VkShaderModule` can then be created with a call to `vkCreateShaderModule`:

```
1 VkShaderModule shaderModule;  
2 if (vkCreateShaderModule(device, &createInfo, nullptr,  
    &shaderModule) != VK_SUCCESS) {  
3     throw std::runtime_error("failed to create shader module!");  
4 }
```

The parameters are the same as those in previous object creation functions: the logical device, pointer to create info structure, optional pointer to custom allocators and handle output variable. The buffer with the code can be freed immediately after creating the shader module. Don't forget to return the created shader module:

```
1 return shaderModule;
```

Shader modules are just a thin wrapper around the shader bytecode that we've previously loaded from a file and the functions defined in it. The compilation and linking of the SPIR-V bytecode to machine code for execution by the GPU doesn't happen until the graphics pipeline is created. That means that we're allowed to destroy the shader modules again as soon as pipeline creation is finished,

which is why we'll make them local variables in the `createGraphicsPipeline` function instead of class members:

```
1 void createGraphicsPipeline() {
2     auto vertShaderCode = readFile("shaders/vert.spv");
3     auto fragShaderCode = readFile("shaders/frag.spv");
4
5     VkShaderModule vertShaderModule =
        createShaderModule(vertShaderCode);
6     VkShaderModule fragShaderModule =
        createShaderModule(fragShaderCode);
```

The cleanup should then happen at the end of the function by adding two calls to `vkDestroyShaderModule`. All of the remaining code in this chapter will be inserted before these lines.

```
1     ...
2     vkDestroyShaderModule(device, fragShaderModule, nullptr);
3     vkDestroyShaderModule(device, vertShaderModule, nullptr);
4 }
```

Shader stage creation

To actually use the shaders we'll need to assign them to a specific pipeline stage through `VkPipelineShaderStageCreateInfo` structures as part of the actual pipeline creation process.

We'll start by filling in the structure for the vertex shader, again in the `createGraphicsPipeline` function.

```
1 VkPipelineShaderStageCreateInfo vertShaderStageInfo{};
2 vertShaderStageInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
3 vertShaderStageInfo.stage = VK_SHADER_STAGE_VERTEX_BIT;
```

The first step, besides the obligatory `sType` member, is telling Vulkan in which pipeline stage the shader is going to be used. There is an enum value for each of the programmable stages described in the previous chapter.

```
1 vertShaderStageInfo.module = vertShaderModule;
2 vertShaderStageInfo.pName = "main";
```

The next two members specify the shader module containing the code, and the function to invoke, known as the *entrypoint*. That means that it's possible to combine multiple fragment shaders into a single shader module and use different entry points to differentiate between their behaviors. In this case we'll stick to the standard `main`, however.

There is one more (optional) member, `pSpecializationInfo`, which we won't be using here, but is worth discussing. It allows you to specify values for shader constants. You can use a single shader module where its behavior can be configured at pipeline creation by specifying different values for the constants used in it. This is more efficient than configuring the shader using variables at render time, because the compiler can do optimizations like eliminating `if` statements that depend on these values. If you don't have any constants like that, then you can set the member to `nullptr`, which our struct initialization does automatically.

Modifying the structure to suit the fragment shader is easy:

```
1 VkPipelineShaderStageCreateInfo fragShaderStageInfo{};
2 fragShaderStageInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
3 fragShaderStageInfo.stage = VK_SHADER_STAGE_FRAGMENT_BIT;
4 fragShaderStageInfo.module = fragShaderModule;
5 fragShaderStageInfo.pName = "main";
```

Finish by defining an array that contains these two structs, which we'll later use to reference them in the actual pipeline creation step.

```
1 VkPipelineShaderStageCreateInfo shaderStages[] =
    {vertShaderStageInfo, fragShaderStageInfo};
```

That's all there is to describing the programmable stages of the pipeline. In the next chapter we'll look at the fixed-function stages.

C++ code / Vertex shader / Fragment shader

Fixed functions

The older graphics APIs provided default state for most of the stages of the graphics pipeline. In Vulkan you have to be explicit about most pipeline states as it'll be baked into an immutable pipeline state object. In this chapter we'll fill in all of the structures to configure these fixed-function operations.

Dynamic state

While *most* of the pipeline state needs to be baked into the pipeline state, a limited amount of the state *can* actually be changed without recreating the pipeline at draw time. Examples are the size of the viewport, line width and blend constants. If you want to use dynamic state and keep these properties out, then you'll have to fill in a `VkPipelineDynamicStateCreateInfo` structure like this:

```
1 std::vector<VkDynamicState> dynamicStates = {
2     VK_DYNAMIC_STATE_VIEWPORT,
3     VK_DYNAMIC_STATE_SCISSOR
```

```

4 };
5
6 VkPipelineDynamicStateCreateInfo dynamicState{};
7 dynamicState.sType =
    VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO;
8 dynamicState.dynamicStateCount =
    static_cast<uint32_t>(dynamicStates.size());
9 dynamicState.pDynamicStates = dynamicStates.data();

```

This will cause the configuration of these values to be ignored and you will be able (and required) to specify the data at drawing time. This results in a more flexible setup and is very common for things like viewport and scissor state, which would result in a more complex setup when being baked into the pipeline state.

Vertex input

The `VkPipelineVertexInputStateCreateInfo` structure describes the format of the vertex data that will be passed to the vertex shader. It describes this in roughly two ways:

- Bindings: spacing between data and whether the data is per-vertex or per-instance (see instancing)
- Attribute descriptions: type of the attributes passed to the vertex shader, which binding to load them from and at which offset

Because we're hard coding the vertex data directly in the vertex shader, we'll fill in this structure to specify that there is no vertex data to load for now. We'll get back to it in the vertex buffer chapter.

```

1 VkPipelineVertexInputStateCreateInfo vertexInputInfo{};
2 vertexInputInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
3 vertexInputInfo.vertexBindingDescriptionCount = 0;
4 vertexInputInfo.pVertexBindingDescriptions = nullptr; // Optional
5 vertexInputInfo.vertexAttributeDescriptionCount = 0;
6 vertexInputInfo.pVertexAttributeDescriptions = nullptr; // Optional

```

The `pVertexBindingDescriptions` and `pVertexAttributeDescriptions` members point to an array of structs that describe the aforementioned details for loading vertex data. Add this structure to the `createGraphicsPipeline` function right after the `shaderStages` array.

Input assembly

The `VkPipelineInputAssemblyStateCreateInfo` struct describes two things: what kind of geometry will be drawn from the vertices and if primitive restart

should be enabled. The former is specified in the `topology` member and can have values like:

- `VK_PRIMITIVE_TOPOLOGY_POINT_LIST`: points from vertices
- `VK_PRIMITIVE_TOPOLOGY_LINE_LIST`: line from every 2 vertices without reuse
- `VK_PRIMITIVE_TOPOLOGY_LINE_STRIP`: the end vertex of every line is used as start vertex for the next line
- `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`: triangle from every 3 vertices without reuse
- `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP`: the second and third vertex of every triangle are used as first two vertices of the next triangle

Normally, the vertices are loaded from the vertex buffer by index in sequential order, but with an *element buffer* you can specify the indices to use yourself. This allows you to perform optimizations like reusing vertices. If you set the `primitiveRestartEnable` member to `VK_TRUE`, then it's possible to break up lines and triangles in the `_STRIP` topology modes by using a special index of `0xFFFF` or `0xFFFFFFFF`.

We intend to draw triangles throughout this tutorial, so we'll stick to the following data for the structure:

```
1 VkPipelineInputAssemblyStateCreateInfo inputAssembly{};
2 inputAssembly.sType =
    VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO;
3 inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
4 inputAssembly.primitiveRestartEnable = VK_FALSE;
```

Viewports and scissors

A viewport basically describes the region of the framebuffer that the output will be rendered to. This will almost always be (0, 0) to (`width`, `height`) and in this tutorial that will also be the case.

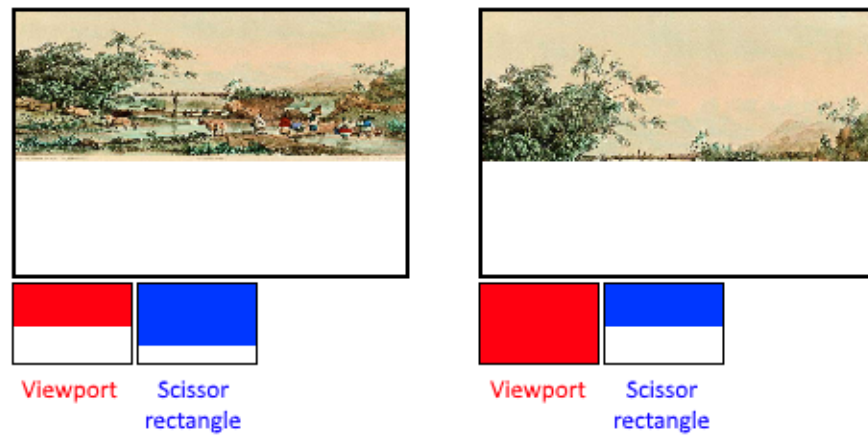
```
1 VkViewport viewport{};
2 viewport.x = 0.0f;
3 viewport.y = 0.0f;
4 viewport.width = (float) swapChainExtent.width;
5 viewport.height = (float) swapChainExtent.height;
6 viewport.minDepth = 0.0f;
7 viewport.maxDepth = 1.0f;
```

Remember that the size of the swap chain and its images may differ from the `WIDTH` and `HEIGHT` of the window. The swap chain images will be used as framebuffers later on, so we should stick to their size.

The `minDepth` and `maxDepth` values specify the range of depth values to use for the framebuffer. These values must be within the `[0.0f, 1.0f]` range, but

`minDepth` may be higher than `maxDepth`. If you aren't doing anything special, then you should stick to the standard values of `0.0f` and `1.0f`.

While viewports define the transformation from the image to the framebuffer, scissor rectangles define in which regions pixels will actually be stored. Any pixels outside the scissor rectangles will be discarded by the rasterizer. They function like a filter rather than a transformation. The difference is illustrated below. Note that the left scissor rectangle is just one of the many possibilities that would result in that image, as long as it's larger than the viewport.



So if we wanted to draw to the entire framebuffer, we would specify a scissor rectangle that covers it entirely:

```
1 VkRect2D scissor{};
2 scissor.offset = {0, 0};
3 scissor.extent = swapChainExtent;
```

Viewport(s) and scissor rectangle(s) can either be specified as a static part of the pipeline or as a dynamic state set in the command buffer. While the former is more in line with the other states it's often convenient to make viewport and scissor state dynamic as it gives you a lot more flexibility. This is very common and all implementations can handle this dynamic state without a performance penalty.

When opting for dynamic viewport(s) and scissor rectangle(s) you need to enable the respective dynamic states for the pipeline:

```
1 std::vector<VkDynamicState> dynamicStates = {
2     VK_DYNAMIC_STATE_VIEWPORT,
3     VK_DYNAMIC_STATE_SCISSOR
4 };
5
```

```

6 VkPipelineDynamicStateCreateInfo dynamicState{};
7 dynamicState.sType =
    VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO;
8 dynamicState.dynamicStateCount =
    static_cast<uint32_t>(dynamicStates.size());
9 dynamicState.pDynamicStates = dynamicStates.data();

```

And then you only need to specify their count at pipeline creation time:

```

1 VkPipelineViewportStateCreateInfo viewportState{};
2 viewportState.sType =
    VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
3 viewportState.viewportCount = 1;
4 viewportState.scissorCount = 1;

```

The actual viewport(s) and scissor rectangle(s) will then later be set up at drawing time.

With dynamic state it's even possible to specify different viewports and or scissor rectangles within a single command buffer.

Without dynamic state, the viewport and scissor rectangle need to be set in the pipeline using the `VkPipelineViewportStateCreateInfo` struct. This makes the viewport and scissor rectangle for this pipeline immutable. Any changes required to these values would require a new pipeline to be created with the new values.

```

1 VkPipelineViewportStateCreateInfo viewportState{};
2 viewportState.sType =
    VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
3 viewportState.viewportCount = 1;
4 viewportState.pViewports = &viewport;
5 viewportState.scissorCount = 1;
6 viewportState.pScissors = &scissor;

```

Independent of how you set them, it's possible to use multiple viewports and scissor rectangles on some graphics cards, so the structure members reference an array of them. Using multiple requires enabling a GPU feature (see logical device creation).

Rasterizer

The rasterizer takes the geometry that is shaped by the vertices from the vertex shader and turns it into fragments to be colored by the fragment shader. It also performs depth testing, face culling and the scissor test, and it can be configured to output fragments that fill entire polygons or just the edges (wireframe rendering). All this is configured using the `VkPipelineRasterizationStateCreateInfo` structure.

```

1 VkPipelineRasterizationStateCreateInfo rasterizer{};
2 rasterizer.sType =
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
3 rasterizer.depthClampEnable = VK_FALSE;

```

If `depthClampEnable` is set to `VK_TRUE`, then fragments that are beyond the near and far planes are clamped to them as opposed to discarding them. This is useful in some special cases like shadow maps. Using this requires enabling a GPU feature.

```

1 rasterizer.rasterizerDiscardEnable = VK_FALSE;

```

If `rasterizerDiscardEnable` is set to `VK_TRUE`, then geometry never passes through the rasterizer stage. This basically disables any output to the framebuffer.

```

1 rasterizer.polygonMode = VK_POLYGON_MODE_FILL;

```

The `polygonMode` determines how fragments are generated for geometry. The following modes are available:

- `VK_POLYGON_MODE_FILL`: fill the area of the polygon with fragments
- `VK_POLYGON_MODE_LINE`: polygon edges are drawn as lines
- `VK_POLYGON_MODE_POINT`: polygon vertices are drawn as points

Using any mode other than fill requires enabling a GPU feature.

```

1 rasterizer.lineWidth = 1.0f;

```

The `lineWidth` member is straightforward, it describes the thickness of lines in terms of number of fragments. The maximum line width that is supported depends on the hardware and any line thicker than `1.0f` requires you to enable the `wideLines` GPU feature.

```

1 rasterizer.cullMode = VK_CULL_MODE_BACK_BIT;
2 rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;

```

The `cullMode` variable determines the type of face culling to use. You can disable culling, cull the front faces, cull the back faces or both. The `frontFace` variable specifies the vertex order for faces to be considered front-facing and can be clockwise or counterclockwise.

```

1 rasterizer.depthBiasEnable = VK_FALSE;
2 rasterizer.depthBiasConstantFactor = 0.0f; // Optional
3 rasterizer.depthBiasClamp = 0.0f; // Optional
4 rasterizer.depthBiasSlopeFactor = 0.0f; // Optional

```

The rasterizer can alter the depth values by adding a constant value or biasing them based on a fragment's slope. This is sometimes used for shadow mapping, but we won't be using it. Just set `depthBiasEnable` to `VK_FALSE`.

Multisampling

The `VkPipelineMultisampleStateCreateInfo` struct configures multisampling, which is one of the ways to perform anti-aliasing. It works by combining the fragment shader results of multiple polygons that rasterize to the same pixel. This mainly occurs along edges, which is also where the most noticeable aliasing artifacts occur. Because it doesn't need to run the fragment shader multiple times if only one polygon maps to a pixel, it is significantly less expensive than simply rendering to a higher resolution and then downscaling. Enabling it requires enabling a GPU feature.

```
1 VkPipelineMultisampleStateCreateInfo multisampling{};
2 multisampling.sType =
    VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO;
3 multisampling.sampleShadingEnable = VK_FALSE;
4 multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
5 multisampling.minSampleShading = 1.0f; // Optional
6 multisampling.pSampleMask = nullptr; // Optional
7 multisampling.alphaToCoverageEnable = VK_FALSE; // Optional
8 multisampling.alphaToOneEnable = VK_FALSE; // Optional
```

We'll revisit multisampling in later chapter, for now let's keep it disabled.

Depth and stencil testing

If you are using a depth and/or stencil buffer, then you also need to configure the depth and stencil tests using `VkPipelineDepthStencilStateCreateInfo`. We don't have one right now, so we can simply pass a `nullptr` instead of a pointer to such a struct. We'll get back to it in the depth buffering chapter.

Color blending

After a fragment shader has returned a color, it needs to be combined with the color that is already in the framebuffer. This transformation is known as color blending and there are two ways to do it:

- Mix the old and new value to produce a final color
- Combine the old and new value using a bitwise operation

There are two types of structs to configure color blending. The first struct, `VkPipelineColorBlendAttachmentState` contains the configuration per attached framebuffer and the second struct, `VkPipelineColorBlendStateCreateInfo` contains the *global* color blending settings. In our case we only have one framebuffer:

```
1 VkPipelineColorBlendAttachmentState colorBlendAttachment{};
2 colorBlendAttachment.colorWriteMask = VK_COLOR_COMPONENT_R_BIT |
    VK_COLOR_COMPONENT_G_BIT | VK_COLOR_COMPONENT_B_BIT |
    VK_COLOR_COMPONENT_A_BIT;
```

```

3 colorBlendAttachment.blendEnable = VK_FALSE;
4 colorBlendAttachment.srcColorBlendFactor = VK_BLEND_FACTOR_ONE; //
  Optional
5 colorBlendAttachment.dstColorBlendFactor = VK_BLEND_FACTOR_ZERO; //
  Optional
6 colorBlendAttachment.colorBlendOp = VK_BLEND_OP_ADD; // Optional
7 colorBlendAttachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE; //
  Optional
8 colorBlendAttachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ZERO; //
  Optional
9 colorBlendAttachment.alphaBlendOp = VK_BLEND_OP_ADD; // Optional

```

This per-framebuffer struct allows you to configure the first way of color blending. The operations that will be performed are best demonstrated using the following pseudocode:

```

1 if (blendEnable) {
2     finalColor.rgb = (srcColorBlendFactor * newColor.rgb)
        <colorBlendOp> (dstColorBlendFactor * oldColor.rgb);
3     finalColor.a = (srcAlphaBlendFactor * newColor.a) <alphaBlendOp>
        (dstAlphaBlendFactor * oldColor.a);
4 } else {
5     finalColor = newColor;
6 }
7
8 finalColor = finalColor & colorWriteMask;

```

If `blendEnable` is set to `VK_FALSE`, then the new color from the fragment shader is passed through unmodified. Otherwise, the two mixing operations are performed to compute a new color. The resulting color is AND'd with the `colorWriteMask` to determine which channels are actually passed through.

The most common way to use color blending is to implement alpha blending, where we want the new color to be blended with the old color based on its opacity. The `finalColor` should then be computed as follows:

```

1 finalColor.rgb = newAlpha * newColor + (1 - newAlpha) * oldColor;
2 finalColor.a = newAlpha.a;

```

This can be accomplished with the following parameters:

```

1 colorBlendAttachment.blendEnable = VK_TRUE;
2 colorBlendAttachment.srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA;
3 colorBlendAttachment.dstColorBlendFactor =
    VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
4 colorBlendAttachment.colorBlendOp = VK_BLEND_OP_ADD;
5 colorBlendAttachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
6 colorBlendAttachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ZERO;

```

```
7 colorBlendAttachment.alphaBlendOp = VK_BLEND_OP_ADD;
```

You can find all of the possible operations in the `VkBlendFactor` and `VkBlendOp` enumerations in the specification.

The second structure references the array of structures for all of the framebuffers and allows you to set blend constants that you can use as blend factors in the aforementioned calculations.

```
1 VkPipelineColorBlendStateCreateInfo colorBlending{};
2 colorBlending.sType =
    VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO;
3 colorBlending.logicOpEnable = VK_FALSE;
4 colorBlending.logicOp = VK_LOGIC_OP_COPY; // Optional
5 colorBlending.attachmentCount = 1;
6 colorBlending.pAttachments = &colorBlendAttachment;
7 colorBlending.blendConstants[0] = 0.0f; // Optional
8 colorBlending.blendConstants[1] = 0.0f; // Optional
9 colorBlending.blendConstants[2] = 0.0f; // Optional
10 colorBlending.blendConstants[3] = 0.0f; // Optional
```

If you want to use the second method of blending (bitwise combination), then you should set `logicOpEnable` to `VK_TRUE`. The bitwise operation can then be specified in the `logicOp` field. Note that this will automatically disable the first method, as if you had set `blendEnable` to `VK_FALSE` for every attached framebuffer! The `colorWriteMask` will also be used in this mode to determine which channels in the framebuffer will actually be affected. It is also possible to disable both modes, as we've done here, in which case the fragment colors will be written to the framebuffer unmodified.

Pipeline layout

You can use **uniform** values in shaders, which are globals similar to dynamic state variables that can be changed at drawing time to alter the behavior of your shaders without having to recreate them. They are commonly used to pass the transformation matrix to the vertex shader, or to create texture samplers in the fragment shader.

These uniform values need to be specified during pipeline creation by creating a `VkPipelineLayout` object. Even though we won't be using them until a future chapter, we are still required to create an empty pipeline layout.

Create a class member to hold this object, because we'll refer to it from other functions at a later point in time:

```
1 VkPipelineLayout pipelineLayout;
```

And then create the object in the `createGraphicsPipeline` function:

```

1 VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
2 pipelineLayoutInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
3 pipelineLayoutInfo.setLayoutCount = 0; // Optional
4 pipelineLayoutInfo.pSetLayouts = nullptr; // Optional
5 pipelineLayoutInfo.pushConstantRangeCount = 0; // Optional
6 pipelineLayoutInfo.pPushConstantRanges = nullptr; // Optional
7
8 if (vkCreatePipelineLayout(device, &pipelineLayoutInfo, nullptr,
    &pipelineLayout) != VK_SUCCESS) {
9     throw std::runtime_error("failed to create pipeline layout!");
10 }

```

The structure also specifies *push constants*, which are another way of passing dynamic values to shaders that we may get into in a future chapter. The pipeline layout will be referenced throughout the program's lifetime, so it should be destroyed at the end:

```

1 void cleanup() {
2     vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
3     ...
4 }

```

Conclusion

That's it for all of the fixed-function state! It's a lot of work to set all of this up from scratch, but the advantage is that we're now nearly fully aware of everything that is going on in the graphics pipeline! This reduces the chance of running into unexpected behavior because the default state of certain components is not what you expect.

There is however one more object to create before we can finally create the graphics pipeline and that is a render pass.

C++ code / Vertex shader / Fragment shader

Render passes

Setup

Before we can finish creating the pipeline, we need to tell Vulkan about the framebuffer attachments that will be used while rendering. We need to specify how many color and depth buffers there will be, how many samples to use for each of them and how their contents should be handled throughout the rendering operations. All of this information is wrapped in a *render pass* object, for which we'll create a new `createRenderPass` function. Call this function from `initVulkan` before `createGraphicsPipeline`.

```

1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8     createImageViews();
9     createRenderPass();
10    createGraphicsPipeline();
11 }
12
13 ...
14
15 void createRenderPass() {
16
17 }

```

Attachment description

In our case we'll have just a single color buffer attachment represented by one of the images from the swap chain.

```

1 void createRenderPass() {
2     VkAttachmentDescription colorAttachment{};
3     colorAttachment.format = swapChainImageFormat;
4     colorAttachment.samples = VK_SAMPLE_COUNT_1_BIT;
5 }

```

The `format` of the color attachment should match the format of the swap chain images, and we're not doing anything with multisampling yet, so we'll stick to 1 sample.

```

1 colorAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;
2 colorAttachment.storeOp = VK_ATTACHMENT_STORE_OP_STORE;

```

The `loadOp` and `storeOp` determine what to do with the data in the attachment before rendering and after rendering. We have the following choices for `loadOp`:

- `VK_ATTACHMENT_LOAD_OP_LOAD`: Preserve the existing contents of the attachment
- `VK_ATTACHMENT_LOAD_OP_CLEAR`: Clear the values to a constant at the start
- `VK_ATTACHMENT_LOAD_OP_DONT_CARE`: Existing contents are undefined; we don't care about them

In our case we're going to use the clear operation to clear the framebuffer to black before drawing a new frame. There are only two possibilities for the

`storeOp`:

- `VK_ATTACHMENT_STORE_OP_STORE`: Rendered contents will be stored in memory and can be read later
- `VK_ATTACHMENT_STORE_OP_DONT_CARE`: Contents of the framebuffer will be undefined after the rendering operation

We're interested in seeing the rendered triangle on the screen, so we're going with the store operation here.

```
1 colorAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;  
2 colorAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
```

The `loadOp` and `storeOp` apply to color and depth data, and `stencilLoadOp` / `stencilStoreOp` apply to stencil data. Our application won't do anything with the stencil buffer, so the results of loading and storing are irrelevant.

```
1 colorAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;  
2 colorAttachment.finalLayout = VK_IMAGE_LAYOUT_PRESENT_SRC_KHR;
```

Textures and framebuffers in Vulkan are represented by `VkImage` objects with a certain pixel format, however the layout of the pixels in memory can change based on what you're trying to do with an image.

Some of the most common layouts are:

- `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`: Images used as color attachment
- `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`: Images to be presented in the swap chain
- `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`: Images to be used as destination for a memory copy operation

We'll discuss this topic in more depth in the texturing chapter, but what's important to know right now is that images need to be transitioned to specific layouts that are suitable for the operation that they're going to be involved in next.

The `initialLayout` specifies which layout the image will have before the render pass begins. The `finalLayout` specifies the layout to automatically transition to when the render pass finishes. Using `VK_IMAGE_LAYOUT_UNDEFINED` for `initialLayout` means that we don't care what previous layout the image was in. The caveat of this special value is that the contents of the image are not guaranteed to be preserved, but that doesn't matter since we're going to clear it anyway. We want the image to be ready for presentation using the swap chain after rendering, which is why we use `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR` as `finalLayout`.

Subpasses and attachment references

A single render pass can consist of multiple subpasses. Subpasses are subsequent rendering operations that depend on the contents of framebuffers in previous passes, for example a sequence of post-processing effects that are applied one after another. If you group these rendering operations into one render pass, then Vulkan is able to reorder the operations and conserve memory bandwidth for possibly better performance. For our very first triangle, however, we'll stick to a single subpass.

Every subpass references one or more of the attachments that we've described using the structure in the previous sections. These references are themselves `VkAttachmentReference` structs that look like this:

```
1 VkAttachmentReference colorAttachmentRef{};
2 colorAttachmentRef.attachment = 0;
3 colorAttachmentRef.layout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
```

The `attachment` parameter specifies which attachment to reference by its index in the attachment descriptions array. Our array consists of a single `VkAttachmentDescription`, so its index is 0. The `layout` specifies which layout we would like the attachment to have during a subpass that uses this reference. Vulkan will automatically transition the attachment to this layout when the subpass is started. We intend to use the attachment to function as a color buffer and the `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` layout will give us the best performance, as its name implies.

The subpass is described using a `VkSubpassDescription` structure:

```
1 VkSubpassDescription subpass{};
2 subpass.pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS;
```

Vulkan may also support compute subpasses in the future, so we have to be explicit about this being a graphics subpass. Next, we specify the reference to the color attachment:

```
1 subpass.colorAttachmentCount = 1;
2 subpass.pColorAttachments = &colorAttachmentRef;
```

The index of the attachment in this array is directly referenced from the fragment shader with the `layout(location = 0)out vec4 outColor` directive!

The following other types of attachments can be referenced by a subpass:

- `pInputAttachments`: Attachments that are read from a shader
- `pResolveAttachments`: Attachments used for multisampling color attachments
- `pDepthStencilAttachment`: Attachment for depth and stencil data
- `pPreserveAttachments`: Attachments that are not used by this subpass, but for which the data must be preserved

Render pass

Now that the attachment and a basic subpass referencing it have been described, we can create the render pass itself. Create a new class member variable to hold the `VkRenderPass` object right above the `pipelineLayout` variable:

```
1 VkRenderPass renderPass;
2 VkPipelineLayout pipelineLayout;
```

The render pass object can then be created by filling in the `VkRenderPassCreateInfo` structure with an array of attachments and subpasses. The `VkAttachmentReference` objects reference attachments using the indices of this array.

```
1 VkRenderPassCreateInfo renderPassInfo{};
2 renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO;
3 renderPassInfo.attachmentCount = 1;
4 renderPassInfo.pAttachments = &colorAttachment;
5 renderPassInfo.subpassCount = 1;
6 renderPassInfo.pSubpasses = &subpass;
7
8 if (vkCreateRenderPass(device, &renderPassInfo, nullptr,
9   &renderPass) != VK_SUCCESS) {
10   throw std::runtime_error("failed to create render pass!");
11 }
```

Just like the pipeline layout, the render pass will be referenced throughout the program, so it should only be cleaned up at the end:

```
1 void cleanup() {
2   vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
3   vkDestroyRenderPass(device, renderPass, nullptr);
4   ...
5 }
```

That was a lot of work, but in the next chapter it all comes together to finally create the graphics pipeline object!

C++ code / Vertex shader / Fragment shader

Conclusion

We can now combine all of the structures and objects from the previous chapters to create the graphics pipeline! Here's the types of objects we have now, as a quick recap:

- Shader stages: the shader modules that define the functionality of the programmable stages of the graphics pipeline
- Fixed-function state: all of the structures that define the fixed-function stages of the pipeline, like input assembly, rasterizer, viewport and color blending

- Pipeline layout: the uniform and push values referenced by the shader that can be updated at draw time
- Render pass: the attachments referenced by the pipeline stages and their usage

All of these combined fully define the functionality of the graphics pipeline, so we can now begin filling in the `VkGraphicsPipelineCreateInfo` structure at the end of the `createGraphicsPipeline` function. But before the calls to `vkDestroyShaderModule` because these are still to be used during the creation.

```
1 VkGraphicsPipelineCreateInfo pipelineInfo{};
2 pipelineInfo.sType = VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO;
3 pipelineInfo.stageCount = 2;
4 pipelineInfo.pStages = shaderStages;
```

We start by referencing the array of `VkPipelineShaderStageCreateInfo` structs.

```
1 pipelineInfo.pVertexInputState = &vertexInputInfo;
2 pipelineInfo.pInputAssemblyState = &inputAssembly;
3 pipelineInfo.pViewportState = &viewportState;
4 pipelineInfo.pRasterizationState = &rasterizer;
5 pipelineInfo.pMultisampleState = &multisampling;
6 pipelineInfo.pDepthStencilState = nullptr; // Optional
7 pipelineInfo.pColorBlendState = &colorBlending;
8 pipelineInfo.pDynamicState = &dynamicState;
```

Then we reference all of the structures describing the fixed-function stage.

```
1 pipelineInfo.layout = pipelineLayout;
```

After that comes the pipeline layout, which is a Vulkan handle rather than a struct pointer.

```
1 pipelineInfo.renderPass = renderPass;
2 pipelineInfo.subpass = 0;
```

And finally we have the reference to the render pass and the index of the sub pass where this graphics pipeline will be used. It is also possible to use other render passes with this pipeline instead of this specific instance, but they have to be *compatible* with `renderPass`. The requirements for compatibility are described here, but we won't be using that feature in this tutorial.

```
1 pipelineInfo.basePipelineHandle = VK_NULL_HANDLE; // Optional
2 pipelineInfo.basePipelineIndex = -1; // Optional
```

There are actually two more parameters: `basePipelineHandle` and `basePipelineIndex`. Vulkan allows you to create a new graphics pipeline by deriving from an existing pipeline. The idea of pipeline derivatives is that

it is less expensive to set up pipelines when they have much functionality in common with an existing pipeline and switching between pipelines from the same parent can also be done quicker. You can either specify the handle of an existing pipeline with `basePipelineHandle` or reference another pipeline that is about to be created by index with `basePipelineIndex`. Right now there is only a single pipeline, so we'll simply specify a null handle and an invalid index. These values are only used if the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag is also specified in the `flags` field of `VkGraphicsPipelineCreateInfo`.

Now prepare for the final step by creating a class member to hold the `VkPipeline` object:

```
1 VkPipeline graphicsPipeline;
```

And finally create the graphics pipeline:

```
1 if (vkCreateGraphicsPipelines(device, VK_NULL_HANDLE, 1,
2   &pipelineInfo, nullptr, &graphicsPipeline) != VK_SUCCESS) {
3   throw std::runtime_error("failed to create graphics pipeline!");
4 }
```

The `vkCreateGraphicsPipelines` function actually has more parameters than the usual object creation functions in Vulkan. It is designed to take multiple `VkGraphicsPipelineCreateInfo` objects and create multiple `VkPipeline` objects in a single call.

The second parameter, for which we've passed the `VK_NULL_HANDLE` argument, references an optional `VkPipelineCache` object. A pipeline cache can be used to store and reuse data relevant to pipeline creation across multiple calls to `vkCreateGraphicsPipelines` and even across program executions if the cache is stored to a file. This makes it possible to significantly speed up pipeline creation at a later time. We'll get into this in the pipeline cache chapter.

The graphics pipeline is required for all common drawing operations, so it should also only be destroyed at the end of the program:

```
1 void cleanup() {
2   vkDestroyPipeline(device, graphicsPipeline, nullptr);
3   vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
4   ...
5 }
```

Now run your program to confirm that all this hard work has resulted in a successful pipeline creation! We are already getting quite close to seeing something pop up on the screen. In the next couple of chapters we'll set up the actual framebuffers from the swap chain images and prepare the drawing commands.

C++ code / Vertex shader / Fragment shader

Drawing

Framebuffers

We've talked a lot about framebuffers in the past few chapters and we've set up the render pass to expect a single framebuffer with the same format as the swap chain images, but we haven't actually created any yet.

The attachments specified during render pass creation are bound by wrapping them into a `VkFramebuffer` object. A framebuffer object references all of the `VkImageView` objects that represent the attachments. In our case that will be only a single one: the color attachment. However, the image that we have to use for the attachment depends on which image the swap chain returns when we retrieve one for presentation. That means that we have to create a framebuffer for all of the images in the swap chain and use the one that corresponds to the retrieved image at drawing time.

To that end, create another `std::vector` class member to hold the framebuffers:

```
1 std::vector<VkFramebuffer> swapChainFramebuffers;
```

We'll create the objects for this array in a new function `createFramebuffers` that is called from `initVulkan` right after creating the graphics pipeline:

```
1 void initVulkan() {  
2     createInstance();  
3     setupDebugMessenger();  
4     createSurface();  
5     pickPhysicalDevice();  
6     createLogicalDevice();  
7     createSwapChain();  
8     createImageViews();  
9     createRenderPass();  
10    createGraphicsPipeline();  
11    createFramebuffers();  
12 }  
13  
14 ...  
15  
16 void createFramebuffers() {  
17  
18 }
```

Start by resizing the container to hold all of the framebuffers:

```
1 void createFramebuffers() {  
2     swapChainFramebuffers.resize(swapChainImageViews.size());  
3 }
```

We'll then iterate through the image views and create framebuffers from them:

```
1 for (size_t i = 0; i < swapChainImageViews.size(); i++) {
2     VkImageView attachments[] = {
3         swapChainImageViews[i]
4     };
5
6     VkFramebufferCreateInfo framebufferInfo{};
7     framebufferInfo.sType =
8         VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO;
9     framebufferInfo.renderPass = renderPass;
10    framebufferInfo.attachmentCount = 1;
11    framebufferInfo.pAttachments = attachments;
12    framebufferInfo.width = swapChainExtent.width;
13    framebufferInfo.height = swapChainExtent.height;
14    framebufferInfo.layers = 1;
15
16    if (vkCreateFramebuffer(device, &framebufferInfo, nullptr,
17        &swapChainFramebuffers[i]) != VK_SUCCESS) {
18        throw std::runtime_error("failed to create framebuffer!");
19    }
20 }
```

As you can see, creation of framebuffers is quite straightforward. We first need to specify with which `renderPass` the framebuffer needs to be compatible. You can only use a framebuffer with the render passes that it is compatible with, which roughly means that they use the same number and type of attachments.

The `attachmentCount` and `pAttachments` parameters specify the `VkImageView` objects that should be bound to the respective attachment descriptions in the render pass `pAttachment` array.

The `width` and `height` parameters are self-explanatory and `layers` refers to the number of layers in image arrays. Our swap chain images are single images, so the number of layers is 1.

We should delete the framebuffers before the image views and render pass that they are based on, but only after we've finished rendering:

```
1 void cleanup() {
2     for (auto framebuffer : swapChainFramebuffers) {
3         vkDestroyFramebuffer(device, framebuffer, nullptr);
4     }
5
6     ...
7 }
```

We've now reached the milestone where we have all of the objects that are

required for rendering. In the next chapter we're going to write the first actual drawing commands.

C++ code / Vertex shader / Fragment shader

Command buffers

Commands in Vulkan, like drawing operations and memory transfers, are not executed directly using function calls. You have to record all of the operations you want to perform in command buffer objects. The advantage of this is that when we are ready to tell the Vulkan what we want to do, all of the commands are submitted together and Vulkan can more efficiently process the commands since all of them are available together. In addition, this allows command recording to happen in multiple threads if so desired.

Command pools

We have to create a command pool before we can create command buffers. Command pools manage the memory that is used to store the buffers and command buffers are allocated from them. Add a new class member to store a `VkCommandPool`:

```
1 VkCommandPool commandPool;
```

Then create a new function `createCommandPool` and call it from `initVulkan` after the framebuffers were created.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8     createImageViews();
9     createRenderPass();
10    createGraphicsPipeline();
11    createFramebuffers();
12    createCommandPool();
13 }
14
15 ...
16
17 void createCommandPool() {
18
19 }
```

Command pool creation only takes two parameters:

```

1 QueueFamilyIndices queueFamilyIndices =
    findQueueFamilies(physicalDevice);
2
3 VkCommandPoolCreateInfo poolInfo{};
4 poolInfo.sType = VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO;
5 poolInfo.flags = VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT;
6 poolInfo.queueFamilyIndex =
    queueFamilyIndices.graphicsFamily.value();

```

There are two possible flags for command pools:

- `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT`: Hint that command buffers are rerecorded with new commands very often (may change memory allocation behavior)
- `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`: Allow command buffers to be rerecorded individually, without this flag they all have to be reset together

We will be recording a command buffer every frame, so we want to be able to reset and rerecord over it. Thus, we need to set the `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` flag bit for our command pool.

Command buffers are executed by submitting them on one of the device queues, like the graphics and presentation queues we retrieved. Each command pool can only allocate command buffers that are submitted on a single type of queue. We're going to record commands for drawing, which is why we've chosen the graphics queue family.

```

1 if (vkCreateCommandPool(device, &poolInfo, nullptr, &commandPool) !=
    VK_SUCCESS) {
2     throw std::runtime_error("failed to create command pool!");
3 }

```

Finish creating the command pool using the `vkCreateCommandPool` function. It doesn't have any special parameters. Commands will be used throughout the program to draw things on the screen, so the pool should only be destroyed at the end:

```

1 void cleanup() {
2     vkDestroyCommandPool(device, commandPool, nullptr);
3
4     ...
5 }

```

Command buffer allocation

We can now start allocating command buffers.

Create a `VkCommandBuffer` object as a class member. Command buffers will be automatically freed when their command pool is destroyed, so we don't need explicit cleanup.

```
1 VkCommandBuffer commandBuffer;
```

We'll now start working on a `createCommandBuffer` function to allocate a single command buffer from the command pool.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8     createImageViews();
9     createRenderPass();
10    createGraphicsPipeline();
11    createFramebuffers();
12    createCommandPool();
13    createCommandBuffer();
14 }
15
16 ...
17
18 void createCommandBuffer() {
19
20 }
```

Command buffers are allocated with the `vkAllocateCommandBuffers` function, which takes a `VkCommandBufferAllocateInfo` struct as parameter that specifies the command pool and number of buffers to allocate:

```
1 VkCommandBufferAllocateInfo allocInfo{};
2 allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
3 allocInfo.commandPool = commandPool;
4 allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
5 allocInfo.commandBufferCount = 1;
6
7 if (vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer) !=
8     VK_SUCCESS) {
9     throw std::runtime_error("failed to allocate command buffers!");
10 }
```

The `level` parameter specifies if the allocated command buffers are primary or secondary command buffers.

- `VK_COMMAND_BUFFER_LEVEL_PRIMARY`: Can be submitted to a queue for execution, but cannot be called from other command buffers.
- `VK_COMMAND_BUFFER_LEVEL_SECONDARY`: Cannot be submitted directly, but can be called from primary command buffers.

We won't make use of the secondary command buffer functionality here, but you can imagine that it's helpful to reuse common operations from primary command buffers.

Since we are only allocating one command buffer, the `commandBufferCount` parameter is just one.

Command buffer recording

We'll now start working on the `recordCommandBuffer` function that writes the commands we want to execute into a command buffer. The `VkCommandBuffer` used will be passed in as a parameter, as well as the index of the current swapchain image we want to write to.

```
1 void recordCommandBuffer(VkCommandBuffer commandBuffer, uint32_t
   imageIndex) {
2
3 }
```

We always begin recording a command buffer by calling `vkBeginCommandBuffer` with a small `VkCommandBufferBeginInfo` structure as argument that specifies some details about the usage of this specific command buffer.

```
1 VkCommandBufferBeginInfo beginInfo{};
2 beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
3 beginInfo.flags = 0; // Optional
4 beginInfo.pInheritanceInfo = nullptr; // Optional
5
6 if (vkBeginCommandBuffer(commandBuffer, &beginInfo) != VK_SUCCESS) {
7     throw std::runtime_error("failed to begin recording command
   buffer!");
8 }
```

The `flags` parameter specifies how we're going to use the command buffer. The following values are available:

- `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT`: The command buffer will be rerecorded right after executing it once.
- `VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT`: This is a secondary command buffer that will be entirely within a single render pass.
- `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`: The command buffer can be resubmitted while it is also already pending execution.

None of these flags are applicable for us right now.

The `pInheritanceInfo` parameter is only relevant for secondary command buffers. It specifies which state to inherit from the calling primary command buffers.

If the command buffer was already recorded once, then a call to `vkBeginCommandBuffer` will implicitly reset it. It's not possible to append commands to a buffer at a later time.

Starting a render pass

Drawing starts by beginning the render pass with `vkCmdBeginRenderPass`. The render pass is configured using some parameters in a `VkRenderPassBeginInfo` struct.

```
1 VkRenderPassBeginInfo renderPassInfo{};
2 renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO;
3 renderPassInfo.renderPass = renderPass;
4 renderPassInfo.framebuffer = swapChainFramebuffers[imageIndex];
```

The first parameters are the render pass itself and the attachments to bind. We created a framebuffer for each swap chain image where it is specified as a color attachment. Thus we need to bind the framebuffer for the swapchain image we want to draw to. Using the `imageIndex` parameter which was passed in, we can pick the right framebuffer for the current swapchain image.

```
1 renderPassInfo.renderArea.offset = {0, 0};
2 renderPassInfo.renderArea.extent = swapChainExtent;
```

The next two parameters define the size of the render area. The render area defines where shader loads and stores will take place. The pixels outside this region will have undefined values. It should match the size of the attachments for best performance.

```
1 VkClearColorValue clearColor = {{{0.0f, 0.0f, 0.0f, 1.0f}}};
2 renderPassInfo.clearValueCount = 1;
3 renderPassInfo.pClearValues = &clearColor;
```

The last two parameters define the clear values to use for `VK_ATTACHMENT_LOAD_OP_CLEAR`, which we used as load operation for the color attachment. I've defined the clear color to simply be black with 100% opacity.

```
1 vkCmdBeginRenderPass(commandBuffer, &renderPassInfo,
   VK_SUBPASS_CONTENTS_INLINE);
```

The render pass can now begin. All of the functions that record commands can be recognized by their `vkCmd` prefix. They all return `void`, so there will be no error handling until we've finished recording.

The first parameter for every command is always the command buffer to record the command to. The second parameter specifies the details of the render pass

we've just provided. The final parameter controls how the drawing commands within the render pass will be provided. It can have one of two values:

- `VK_SUBPASS_CONTENTS_INLINE`: The render pass commands will be embedded in the primary command buffer itself and no secondary command buffers will be executed.
- `VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS`: The render pass commands will be executed from secondary command buffers.

We will not be using secondary command buffers, so we'll go with the first option.

Basic drawing commands

We can now bind the graphics pipeline:

```
1 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS,
   graphicsPipeline);
```

The second parameter specifies if the pipeline object is a graphics or compute pipeline. We've now told Vulkan which operations to execute in the graphics pipeline and which attachment to use in the fragment shader.

As noted in the fixed functions chapter, we did specify viewport and scissor state for this pipeline to be dynamic. So we need to set them in the command buffer before issuing our draw command:

```
1 VkViewport viewport{};
2 viewport.x = 0.0f;
3 viewport.y = 0.0f;
4 viewport.width = static_cast<float>(swapChainExtent.width);
5 viewport.height = static_cast<float>(swapChainExtent.height);
6 viewport.minDepth = 0.0f;
7 viewport.maxDepth = 1.0f;
8 vkCmdSetViewport(commandBuffer, 0, 1, &viewport);
9
10 VkRect2D scissor{};
11 scissor.offset = {0, 0};
12 scissor.extent = swapChainExtent;
13 vkCmdSetScissor(commandBuffer, 0, 1, &scissor);
```

Now we are ready to issue the draw command for the triangle:

```
1 vkCmdDraw(commandBuffer, 3, 1, 0, 0);
```

The actual `vkCmdDraw` function is a bit anticlimactic, but it's so simple because of all the information we specified in advance. It has the following parameters, aside from the command buffer:

- **vertexCount**: Even though we don't have a vertex buffer, we technically still have 3 vertices to draw.
- **instanceCount**: Used for instanced rendering, use 1 if you're not doing that.
- **firstVertex**: Used as an offset into the vertex buffer, defines the lowest value of `gl_VertexIndex`.
- **firstInstance**: Used as an offset for instanced rendering, defines the lowest value of `gl_InstanceIndex`.

Finishing up

The render pass can now be ended:

```
1 vkCmdEndRenderPass(commandBuffer);
```

And we've finished recording the command buffer:

```
1 if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
2     throw std::runtime_error("failed to record command buffer!");
3 }
```

In the next chapter we'll write the code for the main loop, which will acquire an image from the swap chain, record and execute a command buffer, then return the finished image to the swap chain.

C++ code / Vertex shader / Fragment shader

Rendering and presentation

This is the chapter where everything is going to come together. We're going to write the `drawFrame` function that will be called from the main loop to put the triangle on the screen. Let's start by creating the function and call it from `mainLoop`:

```
1 void mainLoop() {
2     while (!glfwWindowShouldClose(window)) {
3         glfwPollEvents();
4         drawFrame();
5     }
6 }
7
8 ...
9
10 void drawFrame() {
11
12 }
```

Outline of a frame

At a high level, rendering a frame in Vulkan consists of a common set of steps:

- Wait for the previous frame to finish
- Acquire an image from the swap chain
- Record a command buffer which draws the scene onto that image
- Submit the recorded command buffer
- Present the swap chain image

While we will expand the drawing function in later chapters, for now this is the core of our render loop.

Synchronization

A core design philosophy in Vulkan is that synchronization of execution on the GPU is explicit. The order of operations is up to us to define using various synchronization primitives which tell the driver the order we want things to run in. This means that many Vulkan API calls which start executing work on the GPU are asynchronous, the functions will return before the operation has finished.

In this chapter there are a number of events that we need to order explicitly because they happen on the GPU, such as:

- Acquire an image from the swap chain
- Execute commands that draw onto the acquired image
- Present that image to the screen for presentation, returning it to the swapchain

Each of these events is set in motion using a single function call, but are all executed asynchronously. The function calls will return before the operations are actually finished and the order of execution is also undefined. That is unfortunate, because each of the operations depends on the previous one finishing. Thus we need to explore which primitives we can use to achieve the desired ordering.

Semaphores A semaphore is used to add order between queue operations. Queue operations refer to the work we submit to a queue, either in a command buffer or from within a function as we will see later. Examples of queues are the graphics queue and the presentation queue. Semaphores are used both to order work inside the same queue and between different queues.

There happens to be two kinds of semaphores in Vulkan, binary and timeline. Because only binary semaphores will be used in this tutorial, we will not discuss timeline semaphores. Further mention of the term semaphore exclusively refers to binary semaphores.

A semaphore is either unsignaled or signaled. It begins life as unsignaled. The way we use a semaphore to order queue operations is by providing the

same semaphore as a ‘signal’ semaphore in one queue operation and as a ‘wait’ semaphore in another queue operation. For example, lets say we have semaphore S and queue operations A and B that we want to execute in order. What we tell Vulkan is that operation A will ‘signal’ semaphore S when it finishes executing, and operation B will ‘wait’ on semaphore S before it begins executing. When operation A finishes, semaphore S will be signaled, while operation B wont start until S is signaled. After operation B begins executing, semaphore S is automatically reset back to being unsignaled, allowing it to be used again.

Pseudo-code of what was just described:

```
1 VkCommandBuffer A, B = ... // record command buffers
2 VkSemaphore S = ... // create a semaphore
3
4 // enqueue A, signal S when done - starts executing immediately
5 vkQueueSubmit(work: A, signal: S, wait: None)
6
7 // enqueue B, wait on S to start
8 vkQueueSubmit(work: B, signal: None, wait: S)
```

Note that in this code snippet, both calls to `vkQueueSubmit()` return immediately - the waiting only happens on the GPU. The CPU continues running without blocking. To make the CPU wait, we need a different synchronization primitive, which we will now describe.

Fences A fence has a similar purpose, in that it is used to synchronize execution, but it is for ordering the execution on the CPU, otherwise known as the host. Simply put, if the host needs to know when the GPU has finished something, we use a fence.

Similar to semaphores, fences are either in a signaled or unsignaled state. Whenever we submit work to execute, we can attach a fence to that work. When the work is finished, the fence will be signaled. Then we can make the host wait for the fence to be signaled, guaranteeing that the work has finished before the host continues.

A concrete example is taking a screenshot. Say we have already done the necessary work on the GPU. Now need to transfer the image from the GPU over to the host and then save the memory to a file. We have command buffer A which executes the transfer and fence F. We submit command buffer A with fence F, then immediately tell the host to wait for F to signal. This causes the host to block until command buffer A finishes execution. Thus we are safe to let the host save the file to disk, as the memory transfer has completed.

Pseudo-code for what was described:

```
1 VkCommandBuffer A = ... // record command buffer with the transfer
2 VkFence F = ... // create the fence
3
```

```

4 // enqueue A, start work immediately, signal F when done
5 vkQueueSubmit(work: A, fence: F)
6
7 vkWaitForFence(F) // blocks execution until A has finished executing
8
9 save_screenshot_to_disk() // can't run until the transfer has
    finished

```

Unlike the semaphore example, this example *does* block host execution. This means the host won't do anything except wait until execution has finished. For this case, we had to make sure the transfer was complete before we could save the screenshot to disk.

In general, it is preferable to not block the host unless necessary. We want to feed the GPU and the host with useful work to do. Waiting on fences to signal is not useful work. Thus we prefer semaphores, or other synchronization primitives not yet covered, to synchronize our work.

Fences must be reset manually to put them back into the unsignaled state. This is because fences are used to control the execution of the host, and so the host gets to decide when to reset the fence. Contrast this to semaphores which are used to order work on the GPU without the host being involved.

In summary, semaphores are used to specify the execution order of operations on the GPU while fences are used to keep the CPU and GPU in sync with each-other.

What to choose? We have two synchronization primitives to use and conveniently two places to apply synchronization: Swapchain operations and waiting for the previous frame to finish. We want to use semaphores for swapchain operations because they happen on the GPU, thus we don't want to make the host wait around if we can help it. For waiting on the previous frame to finish, we want to use fences for the opposite reason, because we need the host to wait. This is so we don't draw more than one frame at a time. Because we re-record the command buffer every frame, we cannot record the next frame's work to the command buffer until the current frame has finished executing, as we don't want to overwrite the current contents of the command buffer while the GPU is using it.

Creating the synchronization objects

We'll need one semaphore to signal that an image has been acquired from the swapchain and is ready for rendering, another one to signal that rendering has finished and presentation can happen, and a fence to make sure only one frame is rendering at a time.

Create three class members to store these semaphore objects and fence object:

```

1 VkSemaphore imageAvailableSemaphore;
2 VkSemaphore renderFinishedSemaphore;
3 VkFence inFlightFence;

```

To create the semaphores, we'll add the last `create` function for this part of the tutorial: `createSyncObjects`:

```

1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
8     createImageViews();
9     createRenderPass();
10    createGraphicsPipeline();
11    createFramebuffers();
12    createCommandPool();
13    createCommandBuffer();
14    createSyncObjects();
15 }
16
17 ...
18
19 void createSyncObjects() {
20
21 }

```

Creating semaphores requires filling in the `VkSemaphoreCreateInfo`, but in the current version of the API it doesn't actually have any required fields besides `sType`:

```

1 void createSyncObjects() {
2     VkSemaphoreCreateInfo semaphoreInfo{};
3     semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;
4 }

```

Future versions of the Vulkan API or extensions may add functionality for the `flags` and `pNext` parameters like it does for the other structures.

Creating a fence requires filling in the `VkFenceCreateInfo`:

```

1 VkFenceCreateInfo fenceInfo{};
2 fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;

```

Creating the semaphores and fence follows the familiar pattern with `vkCreateSemaphore` & `vkCreateFence`:

```

1 if (vkCreateSemaphore(device, &semaphoreInfo, nullptr,
    &imageAvailableSemaphore) != VK_SUCCESS ||
2     vkCreateSemaphore(device, &semaphoreInfo, nullptr,
    &renderFinishedSemaphore) != VK_SUCCESS ||
3     vkCreateFence(device, &fenceInfo, nullptr, &inFlightFence) !=
    VK_SUCCESS) {
4     throw std::runtime_error("failed to create semaphores!");
5 }

```

The semaphores and fence should be cleaned up at the end of the program, when all commands have finished and no more synchronization is necessary:

```

1 void cleanup() {
2     vkDestroySemaphore(device, imageAvailableSemaphore, nullptr);
3     vkDestroySemaphore(device, renderFinishedSemaphore, nullptr);
4     vkDestroyFence(device, inFlightFence, nullptr);

```

Onto the main drawing function!

Waiting for the previous frame

At the start of the frame, we want to wait until the previous frame has finished, so that the command buffer and semaphores are available to use. To do that, we call `vkWaitForFences`:

```

1 void drawFrame() {
2     vkWaitForFences(device, 1, &inFlightFence, VK_TRUE, UINT64_MAX);
3 }

```

The `vkWaitForFences` function takes an array of fences and waits on the host for either any or all of the fences to be signaled before returning. The `VK_TRUE` we pass here indicates that we want to wait for all fences, but in the case of a single one it doesn't matter. This function also has a timeout parameter that we set to the maximum value of a 64 bit unsigned integer, `UINT64_MAX`, which effectively disables the timeout.

After waiting, we need to manually reset the fence to the unsignaled state with the `vkResetFences` call:

```

1     vkResetFences(device, 1, &inFlightFence);

```

Before we can proceed, there is a slight hiccup in our design. On the first frame we call `drawFrame()`, which immediately waits on `inFlightFence` to be signaled. `inFlightFence` is only signaled after a frame has finished rendering, yet since this is the first frame, there are no previous frames in which to signal the fence! Thus `vkWaitForFences()` blocks indefinitely, waiting on something which will never happen.

Of the many solutions to this dilemma, there is a clever workaround built into the API. Create the fence in the signaled state, so that the first call to `vkWaitForFences()` returns immediately since the fence is already signaled.

To do this, we add the `VK_FENCE_CREATE_SIGNALED_BIT` flag to the `VkFenceCreateInfo`:

```
1 void createSyncObjects() {
2     ...
3
4     VkFenceCreateInfo fenceInfo{};
5     fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
6     fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;
7
8     ...
9 }
```

Acquiring an image from the swap chain

The next thing we need to do in the `drawFrame` function is acquire an image from the swap chain. Recall that the swap chain is an extension feature, so we must use a function with the `vk*KHR` naming convention:

```
1 void drawFrame() {
2     ...
3
4     uint32_t imageIndex;
5     vkAcquireNextImageKHR(device, swapChain, UINT64_MAX,
6         imageAvailableSemaphore, VK_NULL_HANDLE, &imageIndex);
7 }
```

The first two parameters of `vkAcquireNextImageKHR` are the logical device and the swap chain from which we wish to acquire an image. The third parameter specifies a timeout in nanoseconds for an image to become available. Using the maximum value of a 64 bit unsigned integer means we effectively disable the timeout.

The next two parameters specify synchronization objects that are to be signaled when the presentation engine is finished using the image. That's the point in time where we can start drawing to it. It is possible to specify a semaphore, fence or both. We're going to use our `imageAvailableSemaphore` for that purpose here.

The last parameter specifies a variable to output the index of the swap chain image that has become available. The index refers to the `VkImage` in our `swapChainImages` array. We're going to use that index to pick the `VkFramebuffer`.

Recording the command buffer

With the `imageIndex` specifying the swap chain image to use in hand, we can now record the command buffer. First, we call `vkResetCommandBuffer` on the command buffer to make sure it is able to be recorded.

```
1 vkResetCommandBuffer(commandBuffer, 0);
```

The second parameter of `vkResetCommandBuffer` is a `VkCommandBufferResetFlagBits` flag. Since we don't want to do anything special, we leave it as 0.

Now call the function `recordCommandBuffer` to record the commands we want.

```
1 recordCommandBuffer(commandBuffer, imageIndex);
```

With a fully recorded command buffer, we can now submit it.

Submitting the command buffer

Queue submission and synchronization is configured through parameters in the `VkSubmitInfo` structure.

```
1 VkSubmitInfo submitInfo{};
2 submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
3
4 VkSemaphore waitSemaphores[] = {imageAvailableSemaphore};
5 VkPipelineStageFlags waitStages[] =
    {VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT};
6 submitInfo.waitSemaphoreCount = 1;
7 submitInfo.pWaitSemaphores = waitSemaphores;
8 submitInfo.pWaitDstStageMask = waitStages;
```

The first three parameters specify which semaphores to wait on before execution begins and in which stage(s) of the pipeline to wait. We want to wait with writing colors to the image until it's available, so we're specifying the stage of the graphics pipeline that writes to the color attachment. That means that theoretically the implementation can already start executing our vertex shader and such while the image is not yet available. Each entry in the `waitStages` array corresponds to the semaphore with the same index in `pWaitSemaphores`.

```
1 submitInfo.commandBufferCount = 1;
2 submitInfo.pCommandBuffers = &commandBuffer;
```

The next two parameters specify which command buffers to actually submit for execution. We simply submit the single command buffer we have.

```
1 VkSemaphore signalSemaphores[] = {renderFinishedSemaphore};
2 submitInfo.signalSemaphoreCount = 1;
3 submitInfo.pSignalSemaphores = signalSemaphores;
```

The `signalSemaphoreCount` and `pSignalSemaphores` parameters specify which semaphores to signal once the command buffer(s) have finished execution. In our case we're using the `renderFinishedSemaphore` for that purpose.

```
1 if (vkQueueSubmit(graphicsQueue, 1, &submitInfo, inFlightFence) !=
    VK_SUCCESS) {
2     throw std::runtime_error("failed to submit draw command
        buffer!");
3 }
```

We can now submit the command buffer to the graphics queue using `vkQueueSubmit`. The function takes an array of `VkSubmitInfo` structures as argument for efficiency when the workload is much larger. The last parameter references an optional fence that will be signaled when the command buffers finish execution. This allows us to know when it is safe for the command buffer to be reused, thus we want to give it `inFlightFence`. Now on the next frame, the CPU will wait for this command buffer to finish executing before it records new commands into it.

Subpass dependencies

Remember that the subpasses in a render pass automatically take care of image layout transitions. These transitions are controlled by *subpass dependencies*, which specify memory and execution dependencies between subpasses. We have only a single subpass right now, but the operations right before and right after this subpass also count as implicit “subpasses”.

There are two built-in dependencies that take care of the transition at the start of the render pass and at the end of the render pass, but the former does not occur at the right time. It assumes that the transition occurs at the start of the pipeline, but we haven't acquired the image yet at that point! There are two ways to deal with this problem. We could change the `waitStages` for the `imageAvailableSemaphore` to `VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT` to ensure that the render passes don't begin until the image is available, or we can make the render pass wait for the `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT` stage. I've decided to go with the second option here, because it's a good excuse to have a look at subpass dependencies and how they work.

Subpass dependencies are specified in `VkSubpassDependency` structs. Go to the `createRenderPass` function and add one:

```
1 VkSubpassDependency dependency{};
2 dependency.srcSubpass = VK_SUBPASS_EXTERNAL;
3 dependency.dstSubpass = 0;
```

The first two fields specify the indices of the dependency and the dependent subpass. The special value `VK_SUBPASS_EXTERNAL` refers to the implicit subpass before or after the render pass depending on whether it is specified in `srcSubpass`

or `dstSubpass`. The index 0 refers to our subpass, which is the first and only one. The `dstSubpass` must always be higher than `srcSubpass` to prevent cycles in the dependency graph (unless one of the subpasses is `VK_SUBPASS_EXTERNAL`).

```
1 dependency.srcStageMask =
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT;
2 dependency.srcAccessMask = 0;
```

The next two fields specify the operations to wait on and the stages in which these operations occur. We need to wait for the swap chain to finish reading from the image before we can access it. This can be accomplished by waiting on the color attachment output stage itself.

```
1 dependency.dstStageMask =
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT;
2 dependency.dstAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT;
```

The operations that should wait on this are in the color attachment stage and involve the writing of the color attachment. These settings will prevent the transition from happening until it's actually necessary (and allowed): when we want to start writing colors to it.

```
1 renderPassInfo.dependencyCount = 1;
2 renderPassInfo.pDependencies = &dependency;
```

The `VkRenderPassCreateInfo` struct has two fields to specify an array of dependencies.

Presentation

The last step of drawing a frame is submitting the result back to the swap chain to have it eventually show up on the screen. Presentation is configured through a `VkPresentInfoKHR` structure at the end of the `drawFrame` function.

```
1 VkPresentInfoKHR presentInfo{};
2 presentInfo.sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR;
3
4 presentInfo.waitSemaphoreCount = 1;
5 presentInfo.pWaitSemaphores = signalSemaphores;
```

The first two parameters specify which semaphores to wait on before presentation can happen, just like `VkSubmitInfo`. Since we want to wait on the command buffer to finish execution, thus our triangle being drawn, we take the semaphores which will be signalled and wait on them, thus we use `signalSemaphores`.

```
1 VkSwapchainKHR swapChains[] = {swapChain};
2 presentInfo.swapchainCount = 1;
3 presentInfo.pSwapchains = swapChains;
4 presentInfo.pImageIndices = &imageIndex;
```

The next two parameters specify the swap chains to present images to and the index of the image for each swap chain. This will almost always be a single one.

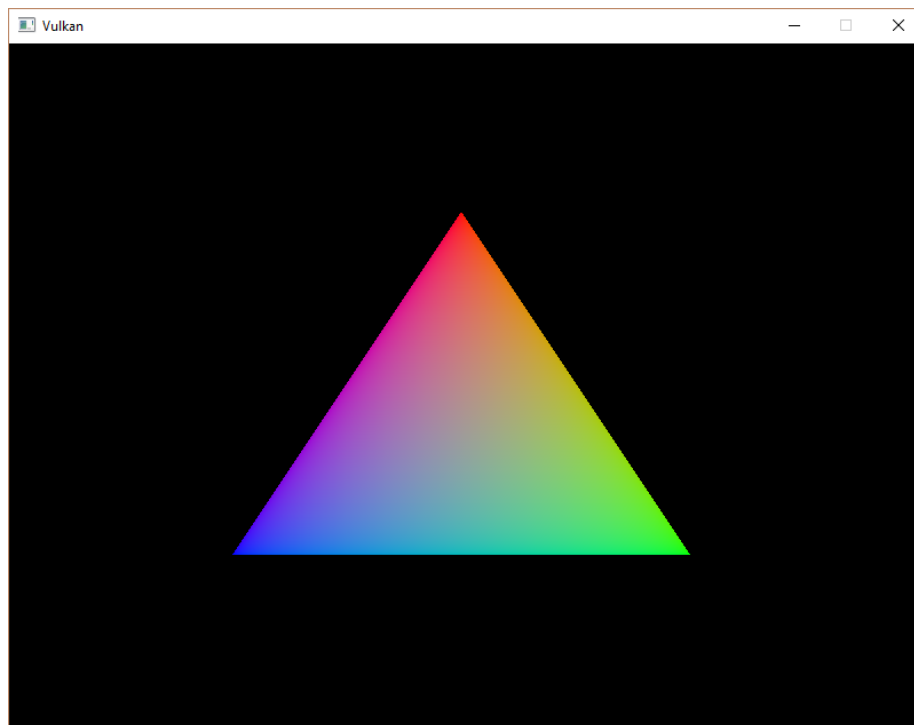
```
1 presentInfo.pResults = nullptr; // Optional
```

There is one last optional parameter called **pResults**. It allows you to specify an array of **VkResult** values to check for every individual swap chain if presentation was successful. It's not necessary if you're only using a single swap chain, because you can simply use the return value of the present function.

```
1 vkQueuePresentKHR(presentQueue, &presentInfo);
```

The **vkQueuePresentKHR** function submits the request to present an image to the swap chain. We'll add error handling for both **vkAcquireNextImageKHR** and **vkQueuePresentKHR** in the next chapter, because their failure does not necessarily mean that the program should terminate, unlike the functions we've seen so far.

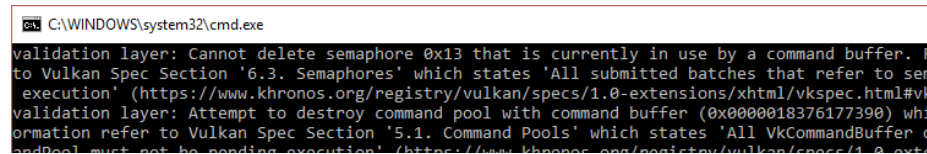
If you did everything correctly up to this point, then you should now see something resembling the following when you run your program:



This colored triangle may look a bit different from the one you're used to seeing in graphics tutorials. That's because this tutorial lets the shader interpolate in linear color space and converts to sRGB

color space afterwards. See this blog post for a discussion of the difference.

Yay! Unfortunately, you'll see that when validation layers are enabled, the program crashes as soon as you close it. The messages printed to the terminal from `debugCallback` tell us why:



```
C:\WINDOWS\system32\cmd.exe
Validation layer: Cannot delete semaphore 0x13 that is currently in use by a command buffer. Refer to Vulkan Spec Section '6.3. Semaphores' which states 'All submitted batches that refer to semaphore must be pending execution' (https://www.khronos.org/registry/vulkan/specs/1.0-extensions/xhtml/vkspec.html#VkSemaphore)
Validation layer: Attempt to destroy command pool with command buffer (0x0000018376177390) which is not in the pool. Information refer to Vulkan Spec Section '5.1. Command Pools' which states 'All VkCommandBuffer objects must be pending execution' (https://www.khronos.org/registry/vulkan/specs/1.0-extensions/xhtml/vkspec.html#VkCommandBuffer)
```

Remember that all of the operations in `drawFrame` are asynchronous. That means that when we exit the loop in `mainLoop`, drawing and presentation operations may still be going on. Cleaning up resources while that is happening is a bad idea.

To fix that problem, we should wait for the logical device to finish operations before exiting `mainLoop` and destroying the window:

```
1 void mainLoop() {
2     while (!glfwWindowShouldClose(window)) {
3         glfwPollEvents();
4         drawFrame();
5     }
6
7     vkDeviceWaitIdle(device);
8 }
```

You can also wait for operations in a specific command queue to be finished with `vkQueueWaitIdle`. These functions can be used as a very rudimentary way to perform synchronization. You'll see that the program now exits without problems when closing the window.

Conclusion

A little over 900 lines of code later, we've finally gotten to the stage of seeing something pop up on the screen! Bootstrapping a Vulkan program is definitely a lot of work, but the take-away message is that Vulkan gives you an immense amount of control through its explicitness. I recommend you to take some time now to reread the code and build a mental model of the purpose of all of the Vulkan objects in the program and how they relate to each other. We'll be building on top of that knowledge to extend the functionality of the program from this point on.

The next chapter will expand the render loop to handle multiple frames in flight.

C++ code / Vertex shader / Fragment shader

Frames in flight

Frames in flight

Right now our render loop has one glaring flaw. We are required to wait on the previous frame to finish before we can start rendering the next which results in unnecessary idling of the host.

The way to fix this is to allow multiple frames to be *in-flight* at once, that is to say, allow the rendering of one frame to not interfere with the recording of the next. How do we do this? Any resource that is accessed and modified during rendering must be duplicated. Thus, we need multiple command buffers, semaphores, and fences. In later chapters we will also add multiple instances of other resources, so we will see this concept reappear.

Start by adding a constant at the top of the program that defines how many frames should be processed concurrently:

```
1 const int MAX_FRAMES_IN_FLIGHT = 2;
```

We choose the number 2 because we don't want the CPU to get *too* far ahead of the GPU. With 2 frames in flight, the CPU and the GPU can be working on their own tasks at the same time. If the CPU finishes early, it will wait till the GPU finishes rendering before submitting more work. With 3 or more frames in flight, the CPU could get ahead of the GPU, adding frames of latency. Generally, extra latency isn't desired. But giving the application control over the number of frames in flight is another example of Vulkan being explicit.

Each frame should have its own command buffer, set of semaphores, and fence. Rename and then change them to be `std::vector`s of the objects:

```
1 std::vector<VkCommandBuffer> commandBuffers;
2
3 ...
4
5 std::vector<VkSemaphore> imageAvailableSemaphores;
6 std::vector<VkSemaphore> renderFinishedSemaphores;
7 std::vector<VkFence> inFlightFences;
```

Then we need to create multiple command buffers. Rename `createCommandBuffer` to `createCommandBuffers`. Next we need to resize the command buffers vector to the size of `MAX_FRAMES_IN_FLIGHT`, alter the `VkCommandBufferAllocateInfo` to contain that many command buffers, and then change the destination to our vector of command buffers:

```
1 void createCommandBuffers() {
2     commandBuffers.resize(MAX_FRAMES_IN_FLIGHT);
3     ...
4     allocInfo.commandBufferCount = (uint32_t) commandBuffers.size();
5 }
```

```

6     if (vkAllocateCommandBuffers(device, &allocInfo,
7         commandBuffers.data()) != VK_SUCCESS) {
8         throw std::runtime_error("failed to allocate command
9         buffers!");
10    }
11 }

```

The createSyncObjects function should be changed to create all of the objects:

```

1 void createSyncObjects() {
2     imageAvailableSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
3     renderFinishedSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
4     inFlightFences.resize(MAX_FRAMES_IN_FLIGHT);
5
6     VkSemaphoreCreateInfo semaphoreInfo{};
7     semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;
8
9     VkFenceCreateInfo fenceInfo{};
10    fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
11    fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;
12
13    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
14        if (vkCreateSemaphore(device, &semaphoreInfo, nullptr,
15            &imageAvailableSemaphores[i]) != VK_SUCCESS ||
16            vkCreateSemaphore(device, &semaphoreInfo, nullptr,
17            &renderFinishedSemaphores[i]) != VK_SUCCESS ||
18            vkCreateFence(device, &fenceInfo, nullptr,
19            &inFlightFences[i]) != VK_SUCCESS) {
20
21            throw std::runtime_error("failed to create
22            synchronization objects for a frame!");
23        }
24    }
25 }

```

Similarly, they should also all be cleaned up:

```

1 void cleanup() {
2     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
3         vkDestroySemaphore(device, renderFinishedSemaphores[i],
4             nullptr);
5         vkDestroySemaphore(device, imageAvailableSemaphores[i],
6             nullptr);
7         vkDestroyFence(device, inFlightFences[i], nullptr);
8     }
9
10    ...

```

```
9 }
```

Remember, because command buffers are freed for us when we free the command pool, there is nothing extra to do for command buffer cleanup.

To use the right objects every frame, we need to keep track of the current frame. We will use a frame index for that purpose:

```
1 uint32_t currentFrame = 0;
```

The `drawFrame` function can now be modified to use the right objects:

```
1 void drawFrame() {
2     vkWaitForFences(device, 1, &inFlightFences[currentFrame],
3                     VK_TRUE, UINT64_MAX);
4     vkResetFences(device, 1, &inFlightFences[currentFrame]);
5     vkAcquireNextImageKHR(device, swapChain, UINT64_MAX,
6                           imageAvailableSemaphores[currentFrame], VK_NULL_HANDLE,
7                           &imageIndex);
8     ...
9     vkResetCommandBuffer(commandBuffers[currentFrame], 0);
10    recordCommandBuffer(commandBuffers[currentFrame], imageIndex);
11    ...
12    submitInfo.pCommandBuffers = &commandBuffers[currentFrame];
13    ...
14    VkSemaphore waitSemaphores[] =
15        {imageAvailableSemaphores[currentFrame]};
16    ...
17    VkSemaphore signalSemaphores[] =
18        {renderFinishedSemaphores[currentFrame]};
19    ...
20    if (vkQueueSubmit(graphicsQueue, 1, &submitInfo,
21                     inFlightFences[currentFrame]) != VK_SUCCESS) {
22    }
```

Of course, we shouldn't forget to advance to the next frame every time:

```

1 void drawFrame() {
2     ...
3
4     currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
5 }

```

By using the modulo (%) operator, we ensure that the frame index loops around after every `MAX_FRAMES_IN_FLIGHT` enqueued frames.

We’ve now implemented all the needed synchronization to ensure that there are no more than `MAX_FRAMES_IN_FLIGHT` frames of work enqueued and that these frames are not stepping over each other. Note that it is fine for other parts of the code, like the final cleanup, to rely on more rough synchronization like `vkDeviceWaitIdle`. You should decide on which approach to use based on performance requirements.

To learn more about synchronization through examples, have a look at this extensive overview by Khronos.

In the next chapter we’ll deal with one more small thing that is required for a well-behaved Vulkan program.

C++ code / Vertex shader / Fragment shader

Swap chain recreation

Introduction

The application we have now successfully draws a triangle, but there are some circumstances that it isn’t handling properly yet. It is possible for the window surface to change such that the swap chain is no longer compatible with it. One of the reasons that could cause this to happen is the size of the window changing. We have to catch these events and recreate the swap chain.

Recreating the swap chain

Create a new `recreateSwapChain` function that calls `createSwapChain` and all of the creation functions for the objects that depend on the swap chain or the window size.

```

1 void recreateSwapChain() {
2     vkDeviceWaitIdle(device);
3
4     createSwapChain();
5     createImageViews();
6     createFramebuffers();
7 }

```

We first call `vkDeviceWaitIdle`, because just like in the last chapter, we shouldn't touch resources that may still be in use. Obviously, we'll have to recreate the swap chain itself. The image views need to be recreated because they are based directly on the swap chain images. Finally, the framebuffers directly depend on the swap chain images, and thus must be recreated as well.

To make sure that the old versions of these objects are cleaned up before recreating them, we should move some of the cleanup code to a separate function that we can call from the `recreateSwapChain` function. Let's call it `cleanupSwapChain`:

```
1 void cleanupSwapChain() {
2
3 }
4
5 void recreateSwapChain() {
6     vkDeviceWaitIdle(device);
7
8     cleanupSwapChain();
9
10    createSwapChain();
11    createImageViews();
12    createFramebuffers();
13 }
```

Note that we don't recreate the renderpass here for simplicity. In theory it can be possible for the swap chain image format to change during an application's lifetime, e.g. when moving a window from a standard range to a high dynamic range monitor. This may require the application to recreate the renderpass to make sure the change between dynamic ranges is properly reflected.

We'll move the cleanup code of all objects that are recreated as part of a swap chain refresh from `cleanup` to `cleanupSwapChain`:

```
1 void cleanupSwapChain() {
2     for (auto framebuffer : swapChainFramebuffers) {
3         vkDestroyFramebuffer(device, framebuffer, nullptr);
4     }
5
6     for (auto imageView : swapChainImageViews) {
7         vkDestroyImageView(device, imageView, nullptr);
8     }
9
10    vkDestroySwapchainKHR(device, swapChain, nullptr);
11 }
12
13 void cleanup() {
14     cleanupSwapChain();
15 }
```

```

15
16     vkDestroyPipeline(device, graphicsPipeline, nullptr);
17     vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
18
19     vkDestroyRenderPass(device, renderPass, nullptr);
20
21     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
22         vkDestroySemaphore(device, renderFinishedSemaphores[i],
23                             nullptr);
24         vkDestroySemaphore(device, imageAvailableSemaphores[i],
25                             nullptr);
26         vkDestroyFence(device, inFlightFences[i], nullptr);
27     }
28
29     vkDestroyCommandPool(device, commandPool, nullptr);
30
31     vkDestroyDevice(device, nullptr);
32
33     if (enableValidationLayers) {
34         DestroyDebugUtilsMessengerEXT(instance, debugMessenger,
35                                     nullptr);
36     }
37
38     vkDestroySurfaceKHR(instance, surface, nullptr);
39     vkDestroyInstance(instance, nullptr);
40
41     glfwDestroyWindow(window);
42
43     glfwTerminate();
44 }

```

Note that in `chooseSwapExtent` we already query the new window resolution to make sure that the swap chain images have the (new) right size, so there's no need to modify `chooseSwapExtent` (remember that we already had to use `glfwGetFramebufferSize` to get the resolution of the surface in pixels when creating the swap chain).

That's all it takes to recreate the swap chain! However, the disadvantage of this approach is that we need to stop all rendering before creating the new swap chain. It is possible to create a new swap chain while drawing commands on an image from the old swap chain are still in-flight. You need to pass the previous swap chain to the `oldSwapChain` field in the `VkSwapchainCreateInfoKHR` struct and destroy the old swap chain as soon as you've finished using it.

Suboptimal or out-of-date swap chain

Now we just need to figure out when swap chain recreation is necessary and call our new `recreateSwapChain` function. Luckily, Vulkan will usually just tell us that the swap chain is no longer adequate during presentation. The `vkAcquireNextImageKHR` and `vkQueuePresentKHR` functions can return the following special values to indicate this.

- `VK_ERROR_OUT_OF_DATE_KHR`: The swap chain has become incompatible with the surface and can no longer be used for rendering. Usually happens after a window resize.
- `VK_SUBOPTIMAL_KHR`: The swap chain can still be used to successfully present to the surface, but the surface properties are no longer matched exactly.

```
1 VkResult result = vkAcquireNextImageKHR(device, swapChain,
    UINT64_MAX, imageAvailableSemaphores[currentFrame],
    VK_NULL_HANDLE, &imageIndex);
2
3 if (result == VK_ERROR_OUT_OF_DATE_KHR) {
4     recreateSwapChain();
5     return;
6 } else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
7     throw std::runtime_error("failed to acquire swap chain image!");
8 }
```

If the swap chain turns out to be out of date when attempting to acquire an image, then it is no longer possible to present to it. Therefore we should immediately recreate the swap chain and try again in the next `drawFrame` call.

You could also decide to do that if the swap chain is suboptimal, but I've chosen to proceed anyway in that case because we've already acquired an image. Both `VK_SUCCESS` and `VK_SUBOPTIMAL_KHR` are considered "success" return codes.

```
1 result = vkQueuePresentKHR(presentQueue, &presentInfo);
2
3 if (result == VK_ERROR_OUT_OF_DATE_KHR || result ==
    VK_SUBOPTIMAL_KHR) {
4     recreateSwapChain();
5 } else if (result != VK_SUCCESS) {
6     throw std::runtime_error("failed to present swap chain image!");
7 }
8
9 currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
```

The `vkQueuePresentKHR` function returns the same values with the same meaning. In this case we will also recreate the swap chain if it is suboptimal, because we want the best possible result.

Fixing a deadlock

If we try to run the code now, it is possible to encounter a deadlock. Debugging the code, we find that the application reaches `vkWaitForFences` but never continues past it. This is because when `vkAcquireNextImageKHR` returns `VK_ERROR_OUT_OF_DATE_KHR`, we recreate the swapchain and then return from `drawFrame`. But before that happens, the current frame's fence was waited upon and reset. Since we return immediately, no work is submitted for execution and the fence will never be signaled, causing `vkWaitForFences` to halt forever.

There is a simple fix thankfully. Delay resetting the fence until after we know for sure we will be submitting work with it. Thus, if we return early, the fence is still signaled and `vkWaitForFences` won't deadlock the next time we use the same fence object.

The beginning of `drawFrame` should now look like this:

```
1 vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE,
    UINT64_MAX);
2
3 uint32_t imageIndex;
4 VkResult result = vkAcquireNextImageKHR(device, swapChain,
    UINT64_MAX, imageAvailableSemaphores[currentFrame],
    VK_NULL_HANDLE, &imageIndex);
5
6 if (result == VK_ERROR_OUT_OF_DATE_KHR) {
7     recreateSwapChain();
8     return;
9 } else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
10     throw std::runtime_error("failed to acquire swap chain image!");
11 }
12
13 // Only reset the fence if we are submitting work
14 vkResetFences(device, 1, &inFlightFences[currentFrame]);
```

Handling resizes explicitly

Although many drivers and platforms trigger `VK_ERROR_OUT_OF_DATE_KHR` automatically after a window resize, it is not guaranteed to happen. That's why we'll add some extra code to also handle resizes explicitly. First add a new member variable that flags that a resize has happened:

```
1 std::vector<VkFence> inFlightFences;
2
3 bool framebufferResized = false;
```

The `drawFrame` function should then be modified to also check for this flag:

```

1 if (result == VK_ERROR_OUT_OF_DATE_KHR || result ==
    VK_SUBOPTIMAL_KHR || framebufferResized) {
2     framebufferResized = false;
3     recreateSwapChain();
4 } else if (result != VK_SUCCESS) {
5     ...
6 }

```

It is important to do this after `vkQueuePresentKHR` to ensure that the semaphores are in a consistent state, otherwise a signaled semaphore may never be properly waited upon. Now to actually detect resizes we can use the `glfwSetFramebufferSizeCallback` function in the GLFW framework to set up a callback:

```

1 void initWindow() {
2     glfwInit();
3
4     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
5
6     window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr,
7                               nullptr);
8     glfwSetFramebufferSizeCallback(window,
9                                     framebufferResizeCallback);
10 }
11
12 static void framebufferResizeCallback(GLFWwindow* window, int width,
13                                       int height) {
14 }

```

The reason that we're creating a `static` function as a callback is because GLFW does not know how to properly call a member function with the right `this` pointer to our `HelloTriangleApplication` instance.

However, we do get a reference to the `GLFWwindow` in the callback and there is another GLFW function that allows you to store an arbitrary pointer inside of it: `glfwSetWindowUserPointer`:

```

1 window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr, nullptr);
2 glfwSetWindowUserPointer(window, this);
3 glfwSetFramebufferSizeCallback(window, framebufferResizeCallback);

```

This value can now be retrieved from within the callback with `glfwGetWindowUserPointer` to properly set the flag:

```

1 static void framebufferResizeCallback(GLFWwindow* window, int width,
2                                       int height) {

```

```

2   auto app =
      reinterpret_cast<HelloTriangleApplication*>(glfwGetWindowUserPointer(window));
3   app->framebufferResized = true;
4 }

```

Now try to run the program and resize the window to see if the framebuffer is indeed resized properly with the window.

Handling minimization

There is another case where a swap chain may become out of date and that is a special kind of window resizing: window minimization. This case is special because it will result in a frame buffer size of 0. In this tutorial we will handle that by pausing until the window is in the foreground again by extending the `recreateSwapChain` function:

```

1 void recreateSwapChain() {
2     int width = 0, height = 0;
3     glfwGetFramebufferSize(window, &width, &height);
4     while (width == 0 || height == 0) {
5         glfwGetFramebufferSize(window, &width, &height);
6         glfwWaitEvents();
7     }
8
9     vkDeviceWaitIdle(device);
10
11     ...
12 }

```

The initial call to `glfwGetFramebufferSize` handles the case where the size is already correct and `glfwWaitEvents` would have nothing to wait on.

Congratulations, you've now finished your very first well-behaved Vulkan program! In the next chapter we're going to get rid of the hardcoded vertices in the vertex shader and actually use a vertex buffer.

C++ code / Vertex shader / Fragment shader

Vertex buffers

Vertex input description

Introduction

In the next few chapters, we're going to replace the hardcoded vertex data in the vertex shader with a vertex buffer in memory. We'll start with the easiest approach of creating a CPU visible buffer and using `memcpy` to copy the vertex data into it directly, and after that we'll see how to use a staging buffer to copy the vertex data to high performance memory.

Vertex shader

First change the vertex shader to no longer include the vertex data in the shader code itself. The vertex shader takes input from a vertex buffer using the `in` keyword.

```
1 #version 450
2
3 layout(location = 0) in vec2 inPosition;
4 layout(location = 1) in vec3 inColor;
5
6 layout(location = 0) out vec3 fragColor;
7
8 void main() {
9     gl_Position = vec4(inPosition, 0.0, 1.0);
10    fragColor = inColor;
11 }
```

The `inPosition` and `inColor` variables are *vertex attributes*. They're properties that are specified per-vertex in the vertex buffer, just like we manually specified a position and color per vertex using the two arrays. Make sure to recompile the vertex shader!

Just like `fragColor`, the `layout(location = x)` annotations assign indices to the inputs that we can later use to reference them. It is important to know that

some types, like `dvec3` 64 bit vectors, use multiple *slots*. That means that the index after it must be at least 2 higher:

```
1 layout(location = 0) in dvec3 inPosition;
2 layout(location = 2) in vec3 inColor;
```

You can find more info about the layout qualifier in the OpenGL wiki.

Vertex data

We're moving the vertex data from the shader code to an array in the code of our program. Start by including the GLM library, which provides us with linear algebra related types like vectors and matrices. We're going to use these types to specify the position and color vectors.

```
1 #include <glm/glm.hpp>
```

Create a new structure called **Vertex** with the two attributes that we're going to use in the vertex shader inside it:

```
1 struct Vertex {
2     glm::vec2 pos;
3     glm::vec3 color;
4 };
```

GLM conveniently provides us with C++ types that exactly match the vector types used in the shader language.

```
1 const std::vector<Vertex> vertices = {
2     {{0.0f, -0.5f}, {1.0f, 0.0f, 0.0f}},
3     {{0.5f, 0.5f}, {0.0f, 1.0f, 0.0f}},
4     {{-0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}}
5 };
```

Now use the **Vertex** structure to specify an array of vertex data. We're using exactly the same position and color values as before, but now they're combined into one array of vertices. This is known as *interleaving* vertex attributes.

Binding descriptions

The next step is to tell Vulkan how to pass this data format to the vertex shader once it's been uploaded into GPU memory. There are two types of structures needed to convey this information.

The first structure is **VkVertexInputBindingDescription** and we'll add a member function to the **Vertex** struct to populate it with the right data.

```
1 struct Vertex {
2     glm::vec2 pos;
```

```

3     glm::vec3 color;
4
5     static VkVertexInputBindingDescription getBindingDescription() {
6         VkVertexInputBindingDescription bindingDescription{};
7
8         return bindingDescription;
9     }
10 };

```

A vertex binding describes at which rate to load data from memory throughout the vertices. It specifies the number of bytes between data entries and whether to move to the next data entry after each vertex or after each instance.

```

1 VkVertexInputBindingDescription bindingDescription{};
2 bindingDescription.binding = 0;
3 bindingDescription.stride = sizeof(Vertex);
4 bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;

```

All of our per-vertex data is packed together in one array, so we're only going to have one binding. The `binding` parameter specifies the index of the binding in the array of bindings. The `stride` parameter specifies the number of bytes from one entry to the next, and the `inputRate` parameter can have one of the following values:

- `VK_VERTEX_INPUT_RATE_VERTEX`: Move to the next data entry after each vertex
- `VK_VERTEX_INPUT_RATE_INSTANCE`: Move to the next data entry after each instance

We're not going to use instanced rendering, so we'll stick to per-vertex data.

Attribute descriptions

The second structure that describes how to handle vertex input is `VkVertexInputAttributeDescription`. We're going to add another helper function to `Vertex` to fill in these structs.

```

1 #include <array>
2
3 ...
4
5 static std::array<VkVertexInputAttributeDescription, 2>
6     getAttributeDescriptions() {
7     std::array<VkVertexInputAttributeDescription, 2>
8         attributeDescriptions{};
9
10    return attributeDescriptions;
11 }

```

As the function prototype indicates, there are going to be two of these structures. An attribute description struct describes how to extract a vertex attribute from a chunk of vertex data originating from a binding description. We have two attributes, position and color, so we need two attribute description structs.

```
1 attributeDescriptions[0].binding = 0;
2 attributeDescriptions[0].location = 0;
3 attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
4 attributeDescriptions[0].offset = offsetof(Vertex, pos);
```

The **binding** parameter tells Vulkan from which binding the per-vertex data comes. The **location** parameter references the **location** directive of the input in the vertex shader. The input in the vertex shader with location 0 is the position, which has two 32-bit float components.

The **format** parameter describes the type of data for the attribute. A bit confusingly, the formats are specified using the same enumeration as color formats. The following shader types and formats are commonly used together:

- float: VK_FORMAT_R32_SFLOAT
- vec2: VK_FORMAT_R32G32_SFLOAT
- vec3: VK_FORMAT_R32G32B32_SFLOAT
- vec4: VK_FORMAT_R32G32B32A32_SFLOAT

As you can see, you should use the format where the amount of color channels matches the number of components in the shader data type. It is allowed to use more channels than the number of components in the shader, but they will be silently discarded. If the number of channels is lower than the number of components, then the BGA components will use default values of (0, 0, 1). The color type (SFLOAT, UINT, SINT) and bit width should also match the type of the shader input. See the following examples:

- ivec2: VK_FORMAT_R32G32_SINT, a 2-component vector of 32-bit signed integers
- uvec4: VK_FORMAT_R32G32B32A32_UINT, a 4-component vector of 32-bit unsigned integers
- double: VK_FORMAT_R64_SFLOAT, a double-precision (64-bit) float

The **format** parameter implicitly defines the byte size of attribute data and the **offset** parameter specifies the number of bytes since the start of the per-vertex data to read from. The binding is loading one **Vertex** at a time and the position attribute (**pos**) is at an offset of 0 bytes from the beginning of this struct. This is automatically calculated using the **offsetof** macro.

```
1 attributeDescriptions[1].binding = 0;
2 attributeDescriptions[1].location = 1;
3 attributeDescriptions[1].format = VK_FORMAT_R32G32B32_SFLOAT;
4 attributeDescriptions[1].offset = offsetof(Vertex, color);
```

The color attribute is described in much the same way.

Pipeline vertex input

We now need to set up the graphics pipeline to accept vertex data in this format by referencing the structures in `createGraphicsPipeline`. Find the `vertexInputInfo` struct and modify it to reference the two descriptions:

```
1 auto bindingDescription = Vertex::getBindingDescription();
2 auto attributeDescriptions = Vertex::getAttributeDescriptions();
3
4 vertexInputInfo.vertexBindingDescriptionCount = 1;
5 vertexInputInfo.vertexAttributeDescriptionCount =
    static_cast<uint32_t>(attributeDescriptions.size());
6 vertexInputInfo.pVertexBindingDescriptions = &bindingDescription;
7 vertexInputInfo.pVertexAttributeDescriptions =
    attributeDescriptions.data();
```

The pipeline is now ready to accept vertex data in the format of the `vertices` container and pass it on to our vertex shader. If you run the program now with validation layers enabled, you'll see that it complains that there is no vertex buffer bound to the binding. The next step is to create a vertex buffer and move the vertex data to it so the GPU is able to access it.

C++ code / Vertex shader / Fragment shader

Vertex buffer creation

Introduction

Buffers in Vulkan are regions of memory used for storing arbitrary data that can be read by the graphics card. They can be used to store vertex data, which we'll do in this chapter, but they can also be used for many other purposes that we'll explore in future chapters. Unlike the Vulkan objects we've been dealing with so far, buffers do not automatically allocate memory for themselves. The work from the previous chapters has shown that the Vulkan API puts the programmer in control of almost everything and memory management is one of those things.

Buffer creation

Create a new function `createVertexBuffer` and call it from `initVulkan` right before `createCommandBuffers`.

```
1 void initVulkan() {
2     createInstance();
3     setupDebugMessenger();
4     createSurface();
5     pickPhysicalDevice();
6     createLogicalDevice();
7     createSwapChain();
```

```

8     createImageViews();
9     createRenderPass();
10    createGraphicsPipeline();
11    createFramebuffers();
12    createCommandPool();
13    createVertexBuffer();
14    createCommandBuffers();
15    createSyncObjects();
16 }
17
18 ...
19
20 void createVertexBuffer() {
21
22 }

```

Creating a buffer requires us to fill a `VkBufferCreateInfo` structure.

```

1 VkBufferCreateInfo bufferInfo{};
2 bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
3 bufferInfo.size = sizeof(vertices[0]) * vertices.size();

```

The first field of the struct is `size`, which specifies the size of the buffer in bytes. Calculating the byte size of the vertex data is straightforward with `sizeof`.

```

1 bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;

```

The second field is `usage`, which indicates for which purposes the data in the buffer is going to be used. It is possible to specify multiple purposes using a bitwise or. Our use case will be a vertex buffer, we'll look at other types of usage in future chapters.

```

1 bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;

```

Just like the images in the swap chain, buffers can also be owned by a specific queue family or be shared between multiple at the same time. The buffer will only be used from the graphics queue, so we can stick to exclusive access.

The `flags` parameter is used to configure sparse buffer memory, which is not relevant right now. We'll leave it at the default value of 0.

We can now create the buffer with `vkCreateBuffer`. Define a class member to hold the buffer handle and call it `vertexBuffer`.

```

1 VkBuffer vertexBuffer;
2
3 ...
4
5 void createVertexBuffer() {

```

```

6     VkBufferCreateInfo bufferInfo{};
7     bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
8     bufferInfo.size = sizeof(vertices[0]) * vertices.size();
9     bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;
10    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
11
12    if (vkCreateBuffer(device, &bufferInfo, nullptr, &vertexBuffer)
13        != VK_SUCCESS) {
14        throw std::runtime_error("failed to create vertex buffer!");
15    }

```

The buffer should be available for use in rendering commands until the end of the program and it does not depend on the swap chain, so we'll clean it up in the original `cleanup` function:

```

1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyBuffer(device, vertexBuffer, nullptr);
5
6     ...
7 }

```

Memory requirements

The buffer has been created, but it doesn't actually have any memory assigned to it yet. The first step of allocating memory for the buffer is to query its memory requirements using the aptly named `vkGetBufferMemoryRequirements` function.

```

1 VkMemoryRequirements memRequirements;
2 vkGetBufferMemoryRequirements(device, vertexBuffer,
3                               &memRequirements);

```

The `VkMemoryRequirements` struct has three fields:

- **size:** The size of the required amount of memory in bytes, may differ from `bufferInfo.size`.
- **alignment:** The offset in bytes where the buffer begins in the allocated region of memory, depends on `bufferInfo.usage` and `bufferInfo.flags`.
- **memoryTypeBits:** Bit field of the memory types that are suitable for the buffer.

Graphics cards can offer different types of memory to allocate from. Each type of memory varies in terms of allowed operations and performance characteristics. We need to combine the requirements of the buffer and our own application

requirements to find the right type of memory to use. Let's create a new function `findMemoryType` for this purpose.

```
1 uint32_t findMemoryType(uint32_t typeFilter, VkMemoryPropertyFlags
    properties) {
2
3 }
```

First we need to query info about the available types of memory using `vkGetPhysicalDeviceMemoryProperties`.

```
1 VkPhysicalDeviceMemoryProperties memProperties;
2 vkGetPhysicalDeviceMemoryProperties(physicalDevice, &memProperties);
```

The `VkPhysicalDeviceMemoryProperties` structure has two arrays `memoryTypes` and `memoryHeaps`. Memory heaps are distinct memory resources like dedicated VRAM and swap space in RAM for when VRAM runs out. The different types of memory exist within these heaps. Right now we'll only concern ourselves with the type of memory and not the heap it comes from, but you can imagine that this can affect performance.

Let's first find a memory type that is suitable for the buffer itself:

```
1 for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
2     if (typeFilter & (1 << i)) {
3         return i;
4     }
5 }
6
7 throw std::runtime_error("failed to find suitable memory type!");
```

The `typeFilter` parameter will be used to specify the bit field of memory types that are suitable. That means that we can find the index of a suitable memory type by simply iterating over them and checking if the corresponding bit is set to 1.

However, we're not just interested in a memory type that is suitable for the vertex buffer. We also need to be able to write our vertex data to that memory. The `memoryTypes` array consists of `VkMemoryType` structs that specify the heap and properties of each type of memory. The properties define special features of the memory, like being able to map it so we can write to it from the CPU. This property is indicated with `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`, but we also need to use the `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` property. We'll see why when we map the memory.

We can now modify the loop to also check for the support of this property:

```
1 for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
```

```

2     if ((typeFilter & (1 << i)) &&
        (memProperties.memoryTypes[i].propertyFlags & properties) ==
        properties) {
3         return i;
4     }
5 }

```

We may have more than one desirable property, so we should check if the result of the bitwise AND is not just non-zero, but equal to the desired properties bit field. If there is a memory type suitable for the buffer that also has all of the properties we need, then we return its index, otherwise we throw an exception.

Memory allocation

We now have a way to determine the right memory type, so we can actually allocate the memory by filling in the `VkMemoryAllocateInfo` structure.

```

1 VkMemoryAllocateInfo allocInfo{};
2 allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
3 allocInfo.allocationSize = memRequirements.size;
4 allocInfo.memoryTypeIndex =
    findMemoryType(memRequirements.memoryTypeBits,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
    VK_MEMORY_PROPERTY_HOST_COHERENT_BIT);

```

Memory allocation is now as simple as specifying the size and type, both of which are derived from the memory requirements of the vertex buffer and the desired property. Create a class member to store the handle to the memory and allocate it with `vkAllocateMemory`.

```

1 VkBuffer vertexBuffer;
2 VkDeviceMemory vertexBufferMemory;
3
4 ...
5
6 if (vkAllocateMemory(device, &allocInfo, nullptr,
    &vertexBufferMemory) != VK_SUCCESS) {
7     throw std::runtime_error("failed to allocate vertex buffer
        memory!");
8 }

```

If memory allocation was successful, then we can now associate this memory with the buffer using `vkBindBufferMemory`:

```

1 vkBindBufferMemory(device, vertexBuffer, vertexBufferMemory, 0);

```

The first three parameters are self-explanatory and the fourth parameter is the offset within the region of memory. Since this memory is allocated specifically

for this the vertex buffer, the offset is simply 0. If the offset is non-zero, then it is required to be divisible by `memRequirements.alignment`.

Of course, just like dynamic memory allocation in C++, the memory should be freed at some point. Memory that is bound to a buffer object may be freed once the buffer is no longer used, so let's free it after the buffer has been destroyed:

```
1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyBuffer(device, vertexBuffer, nullptr);
5     vkFreeMemory(device, vertexBufferMemory, nullptr);
}
```

Filling the vertex buffer

It is now time to copy the vertex data to the buffer. This is done by mapping the buffer memory into CPU accessible memory with `vkMapMemory`.

```
1 void* data;
2 vkMapMemory(device, vertexBufferMemory, 0, bufferInfo.size, 0,
3             &data);
```

This function allows us to access a region of the specified memory resource defined by an offset and size. The offset and size here are 0 and `bufferInfo.size`, respectively. It is also possible to specify the special value `VK_WHOLE_SIZE` to map all of the memory. The second to last parameter can be used to specify flags, but there aren't any available yet in the current API. It must be set to the value 0. The last parameter specifies the output for the pointer to the mapped memory.

```
1 void* data;
2 vkMapMemory(device, vertexBufferMemory, 0, bufferInfo.size, 0,
3             &data);
4 memcpy(data, vertices.data(), (size_t) bufferInfo.size);
5 vkUnmapMemory(device, vertexBufferMemory);
```

You can now simply `memcpy` the vertex data to the mapped memory and unmap it again using `vkUnmapMemory`. Unfortunately the driver may not immediately copy the data into the buffer memory, for example because of caching. It is also possible that writes to the buffer are not visible in the mapped memory yet. There are two ways to deal with that problem:

- Use a memory heap that is host coherent, indicated with `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT`
- Call `vkFlushMappedMemoryRanges` after writing to the mapped memory, and call `vkInvalidateMappedMemoryRanges` before reading from the mapped memory

We went for the first approach, which ensures that the mapped memory always matches the contents of the allocated memory. Do keep in mind that this may

lead to slightly worse performance than explicit flushing, but we'll see why that doesn't matter in the next chapter.

Flushing memory ranges or using a coherent memory heap means that the driver will be aware of our writes to the buffer, but it doesn't mean that they are actually visible on the GPU yet. The transfer of data to the GPU is an operation that happens in the background and the specification simply tells us that it is guaranteed to be complete as of the next call to `vkQueueSubmit`.

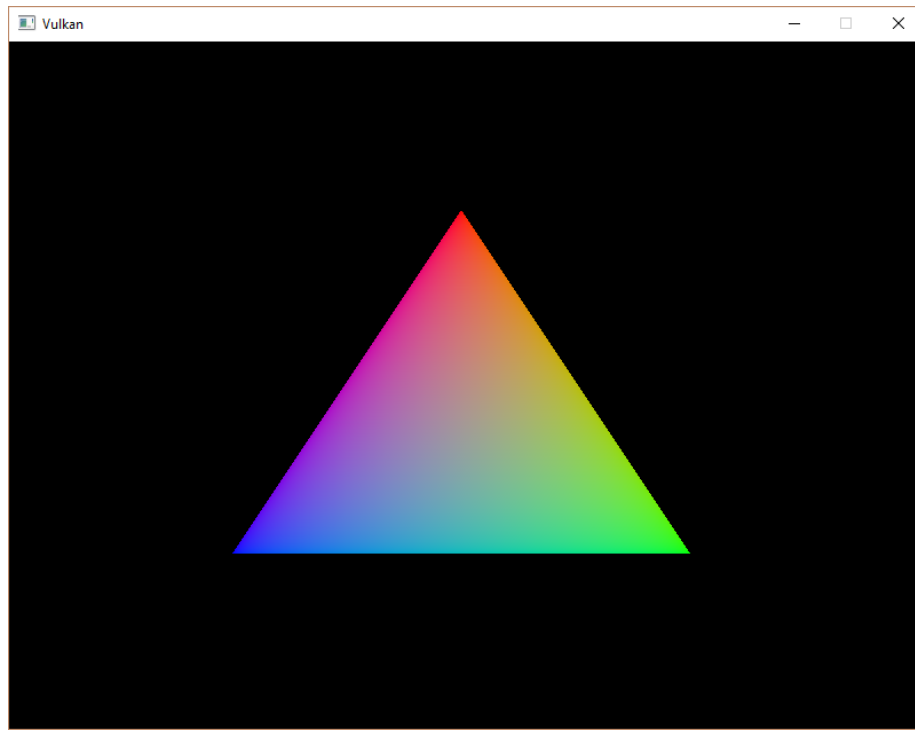
Binding the vertex buffer

All that remains now is binding the vertex buffer during rendering operations. We're going to extend the `recordCommandBuffer` function to do that.

```
1 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS,  
   graphicsPipeline);  
2  
3 VkBuffer vertexBuffers[] = {vertexBuffer};  
4 VkDeviceSize offsets[] = {0};  
5 vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);  
6  
7 vkCmdDraw(commandBuffer, static_cast<uint32_t>(vertices.size()), 1,  
   0, 0);
```

The `vkCmdBindVertexBuffers` function is used to bind vertex buffers to bindings, like the one we set up in the previous chapter. The first two parameters, besides the command buffer, specify the offset and number of bindings we're going to specify vertex buffers for. The last two parameters specify the array of vertex buffers to bind and the byte offsets to start reading vertex data from. You should also change the call to `vkCmdDraw` to pass the number of vertices in the buffer as opposed to the hardcoded number 3.

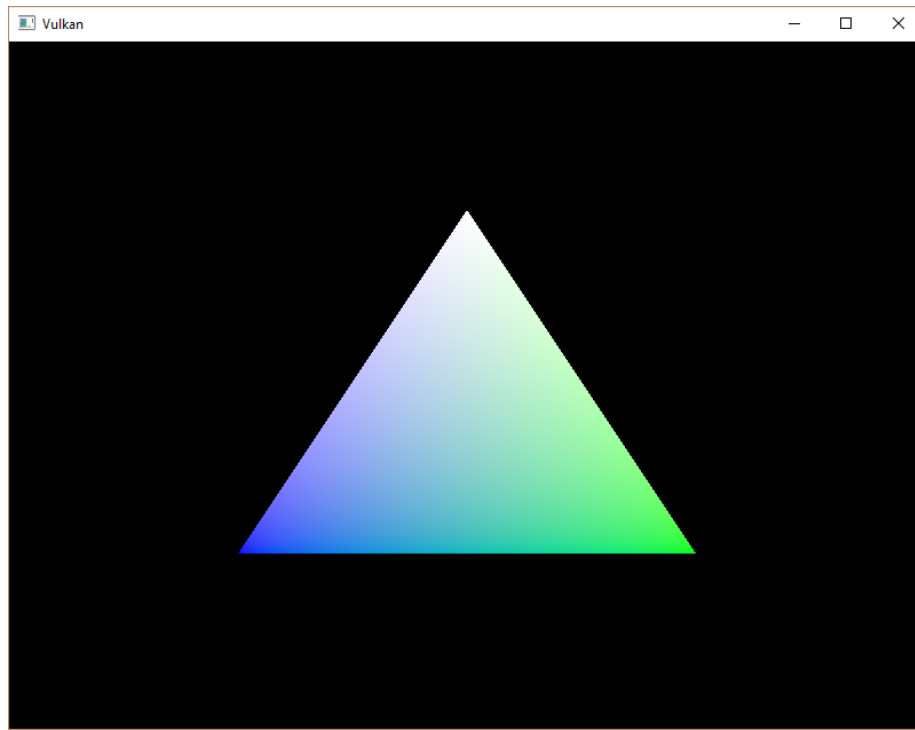
Now run the program and you should see the familiar triangle again:



Try changing the color of the top vertex to white by modifying the `vertices` array:

```
1 const std::vector<Vertex> vertices = {  
2     {{0.0f, -0.5f}, {1.0f, 1.0f, 1.0f}},  
3     {{0.5f, 0.5f}, {0.0f, 1.0f, 0.0f}},  
4     {{-0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}}  
5 };
```

Run the program again and you should see the following:



In the next chapter we'll look at a different way to copy vertex data to a vertex buffer that results in better performance, but takes some more work.

[C++ code](#) / [Vertex shader](#) / [Fragment shader](#)

Staging buffer

Introduction

The vertex buffer we have right now works correctly, but the memory type that allows us to access it from the CPU may not be the most optimal memory type for the graphics card itself to read from. The most optimal memory has the `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` flag and is usually not accessible by the CPU on dedicated graphics cards. In this chapter we're going to create two vertex buffers. One *staging buffer* in CPU accessible memory to upload the data from the vertex array to, and the final vertex buffer in device local memory. We'll then use a buffer copy command to move the data from the staging buffer to the actual vertex buffer.

Transfer queue

The buffer copy command requires a queue family that supports transfer operations, which is indicated using `VK_QUEUE_TRANSFER_BIT`. The good news is that

any queue family with `VK_QUEUE_GRAPHICS_BIT` or `VK_QUEUE_COMPUTE_BIT` capabilities already implicitly support `VK_QUEUE_TRANSFER_BIT` operations. The implementation is not required to explicitly list it in `queueFlags` in those cases.

If you like a challenge, then you can still try to use a different queue family specifically for transfer operations. It will require you to make the following modifications to your program:

- Modify `QueueFamilyIndices` and `findQueueFamilies` to explicitly look for a queue family with the `VK_QUEUE_TRANSFER_BIT` bit, but not the `VK_QUEUE_GRAPHICS_BIT`.
- Modify `createLogicalDevice` to request a handle to the transfer queue
- Create a second command pool for command buffers that are submitted on the transfer queue family
- Change the `sharingMode` of resources to be `VK_SHARING_MODE_CONCURRENT` and specify both the graphics and transfer queue families
- Submit any transfer commands like `vkCmdCopyBuffer` (which we'll be using in this chapter) to the transfer queue instead of the graphics queue

It's a bit of work, but it'll teach you a lot about how resources are shared between queue families.

Abstracting buffer creation

Because we're going to create multiple buffers in this chapter, it's a good idea to move buffer creation to a helper function. Create a new function `createBuffer` and move the code in `createVertexBuffer` (except mapping) to it.

```
1 void createBuffer(VkDeviceSize size, VkBufferUsageFlags usage,
2   VkMemoryPropertyFlags properties, VkBuffer& buffer,
3   VkDeviceMemory& bufferMemory) {
4   VkBufferCreateInfo bufferInfo{};
5   bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
6   bufferInfo.size = size;
7   bufferInfo.usage = usage;
8   bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
9
10  if (vkCreateBuffer(device, &bufferInfo, nullptr, &buffer) !=
11      VK_SUCCESS) {
12      throw std::runtime_error("failed to create buffer!");
13  }
14
15  VkMemoryRequirements memRequirements;
16  vkGetBufferMemoryRequirements(device, buffer, &memRequirements);
17
18  VkMemoryAllocateInfo allocInfo{};
19  allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
20  allocInfo.allocationSize = memRequirements.size;
```

```

18     allocInfo.memoryTypeIndex =
        findMemoryType(memRequirements.memoryTypeBits, properties);
19
20     if (vkAllocateMemory(device, &allocInfo, nullptr, &bufferMemory)
        != VK_SUCCESS) {
21         throw std::runtime_error("failed to allocate buffer
            memory!");
22     }
23
24     vkBindBufferMemory(device, buffer, bufferMemory, 0);
25 }

```

Make sure to add parameters for the buffer size, memory properties and usage so that we can use this function to create many different types of buffers. The last two parameters are output variables to write the handles to.

You can now remove the buffer creation and memory allocation code from `createVertexBuffer` and just call `createBuffer` instead:

```

1 void createVertexBuffer() {
2     VkDeviceSize bufferSize = sizeof(vertices[0]) * vertices.size();
3     createBuffer(bufferSize, VK_BUFFER_USAGE_VERTEX_BUFFER_BIT,
        VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
        VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, vertexBuffer,
        vertexBufferMemory);
4
5     void* data;
6     vkMapMemory(device, vertexBufferMemory, 0, bufferSize, 0, &data);
7     memcpy(data, vertices.data(), (size_t) bufferSize);
8     vkUnmapMemory(device, vertexBufferMemory);
9 }

```

Run your program to make sure that the vertex buffer still works properly.

Using a staging buffer

We're now going to change `createVertexBuffer` to only use a host visible buffer as temporary buffer and use a device local one as actual vertex buffer.

```

1 void createVertexBuffer() {
2     VkDeviceSize bufferSize = sizeof(vertices[0]) * vertices.size();
3
4     VkBuffer stagingBuffer;
5     VkDeviceMemory stagingBufferMemory;
6     createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
        VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
        VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer,
        stagingBufferMemory);

```

```

7
8     void* data;
9     vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0,
10                 &data);
11     memcpy(data, vertices.data(), (size_t) bufferSize);
12     vkUnmapMemory(device, stagingBufferMemory);
13
14     createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT |
15                     VK_BUFFER_USAGE_VERTEX_BUFFER_BIT,
16                     VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, vertexBuffer,
17                     vertexBufferMemory);
18 }

```

We're now using a new `stagingBuffer` with `stagingBufferMemory` for mapping and copying the vertex data. In this chapter we're going to use two new buffer usage flags:

- `VK_BUFFER_USAGE_TRANSFER_SRC_BIT`: Buffer can be used as source in a memory transfer operation.
- `VK_BUFFER_USAGE_TRANSFER_DST_BIT`: Buffer can be used as destination in a memory transfer operation.

The `vertexBuffer` is now allocated from a memory type that is device local, which generally means that we're not able to use `vkMapMemory`. However, we can copy data from the `stagingBuffer` to the `vertexBuffer`. We have to indicate that we intend to do that by specifying the transfer source flag for the `stagingBuffer` and the transfer destination flag for the `vertexBuffer`, along with the vertex buffer usage flag.

We're now going to write a function to copy the contents from one buffer to another, called `copyBuffer`.

```

1 void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize
2                 size) {
3 }

```

Memory transfer operations are executed using command buffers, just like drawing commands. Therefore we must first allocate a temporary command buffer. You may wish to create a separate command pool for these kinds of short-lived buffers, because the implementation may be able to apply memory allocation optimizations. You should use the `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT` flag during command pool generation in that case.

```

1 void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize
2                 size) {
3     VkCommandBufferAllocateInfo allocInfo{};
4     allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;

```

```

4     allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
5     allocInfo.commandPool = commandPool;
6     allocInfo.commandBufferCount = 1;
7
8     VkCommandBuffer commandBuffer;
9     vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer);
10 }

```

And immediately start recording the command buffer:

```

1 VkCommandBufferBeginInfo beginInfo{};
2 beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
3 beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;
4
5 vkBeginCommandBuffer(commandBuffer, &beginInfo);

```

We're only going to use the command buffer once and wait with returning from the function until the copy operation has finished executing. It's good practice to tell the driver about our intent using `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT`.

```

1 VkBufferCopy copyRegion{};
2 copyRegion.srcOffset = 0; // Optional
3 copyRegion.dstOffset = 0; // Optional
4 copyRegion.size = size;
5 vkCmdCopyBuffer(commandBuffer, srcBuffer, dstBuffer, 1, &copyRegion);

```

Contents of buffers are transferred using the `vkCmdCopyBuffer` command. It takes the source and destination buffers as arguments, and an array of regions to copy. The regions are defined in `VkBufferCopy` structs and consist of a source buffer offset, destination buffer offset and size. It is not possible to specify `VK_WHOLE_SIZE` here, unlike the `vkMapMemory` command.

```

1 vkEndCommandBuffer(commandBuffer);

```

This command buffer only contains the copy command, so we can stop recording right after that. Now execute the command buffer to complete the transfer:

```

1 VkSubmitInfo submitInfo{};
2 submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
3 submitInfo.commandBufferCount = 1;
4 submitInfo.pCommandBuffers = &commandBuffer;
5
6 vkQueueSubmit(graphicsQueue, 1, &submitInfo, VK_NULL_HANDLE);
7 vkQueueWaitIdle(graphicsQueue);

```

Unlike the draw commands, there are no events we need to wait on this time. We just want to execute the transfer on the buffers immediately. There are

again two possible ways to wait on this transfer to complete. We could use a fence and wait with `vkWaitForFences`, or simply wait for the transfer queue to become idle with `vkQueueWaitIdle`. A fence would allow you to schedule multiple transfers simultaneously and wait for all of them complete, instead of executing one at a time. That may give the driver more opportunities to optimize.

```
1 vkFreeCommandBuffers(device, commandPool, 1, &commandBuffer);
```

Don't forget to clean up the command buffer used for the transfer operation.

We can now call `copyBuffer` from the `createVertexBuffer` function to move the vertex data to the device local buffer:

```
1 createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT |  
    VK_BUFFER_USAGE_VERTEX_BUFFER_BIT,  
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, vertexBuffer,  
    vertexBufferMemory);  
2  
3 copyBuffer(stagingBuffer, vertexBuffer, bufferSize);
```

After copying the data from the staging buffer to the device buffer, we should clean it up:

```
1     ...  
2  
3     copyBuffer(stagingBuffer, vertexBuffer, bufferSize);  
4  
5     vkDestroyBuffer(device, stagingBuffer, nullptr);  
6     vkFreeMemory(device, stagingBufferMemory, nullptr);  
7 }
```

Run your program to verify that you're seeing the familiar triangle again. The improvement may not be visible right now, but its vertex data is now being loaded from high performance memory. This will matter when we're going to start rendering more complex geometry.

Conclusion

It should be noted that in a real world application, you're not supposed to actually call `vkAllocateMemory` for every individual buffer. The maximum number of simultaneous memory allocations is limited by the `maxMemoryAllocationCount` physical device limit, which may be as low as 4096 even on high end hardware like an NVIDIA GTX 1080. The right way to allocate memory for a large number of objects at the same time is to create a custom allocator that splits up a single allocation among many different objects by using the `offset` parameters that we've seen in many functions.

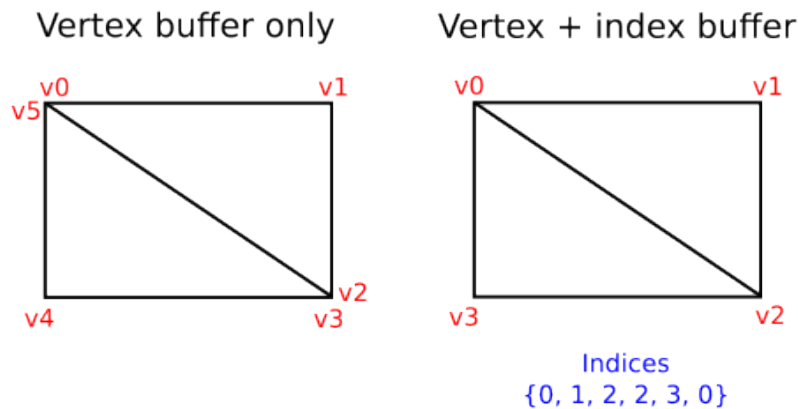
You can either implement such an allocator yourself, or use the VulkanMemoryAllocator library provided by the GPUOpen initiative. However, for this tutorial it's okay to use a separate allocation for every resource, because we won't come close to hitting any of these limits for now.

[C++ code](#) / [Vertex shader](#) / [Fragment shader](#)

Index buffer

Introduction

The 3D meshes you'll be rendering in a real world application will often share vertices between multiple triangles. This already happens even with something simple like drawing a rectangle:



Drawing a rectangle takes two triangles, which means that we need a vertex buffer with 6 vertices. The problem is that the data of two vertices needs to be duplicated resulting in 50% redundancy. It only gets worse with more complex meshes, where vertices are reused in an average number of 3 triangles. The solution to this problem is to use an *index buffer*.

An index buffer is essentially an array of pointers into the vertex buffer. It allows you to reorder the vertex data, and reuse existing data for multiple vertices. The illustration above demonstrates what the index buffer would look like for the rectangle if we have a vertex buffer containing each of the four unique vertices. The first three indices define the upper-right triangle and the last three indices define the vertices for the bottom-left triangle.

Index buffer creation

In this chapter we’re going to modify the vertex data and add index data to draw a rectangle like the one in the illustration. Modify the vertex data to represent the four corners:

```
1 const std::vector<Vertex> vertices = {
2     {{-0.5f, -0.5f}, {1.0f, 0.0f, 0.0f}},
3     {{0.5f, -0.5f}, {0.0f, 1.0f, 0.0f}},
4     {{0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}},
5     {{-0.5f, 0.5f}, {1.0f, 1.0f, 1.0f}}
6 };
```

The top-left corner is red, top-right is green, bottom-right is blue and the bottom-left is white. We’ll add a new array `indices` to represent the contents of the index buffer. It should match the indices in the illustration to draw the upper-right triangle and bottom-left triangle.

```
1 const std::vector<uint16_t> indices = {
2     0, 1, 2, 2, 3, 0
3 };
```

It is possible to use either `uint16_t` or `uint32_t` for your index buffer depending on the number of entries in `vertices`. We can stick to `uint16_t` for now because we’re using less than 65535 unique vertices.

Just like the vertex data, the indices need to be uploaded into a `VkBuffer` for the GPU to be able to access them. Define two new class members to hold the resources for the index buffer:

```
1 VkBuffer vertexBuffer;
2 VkDeviceMemory vertexBufferMemory;
3 VkBuffer indexBuffer;
4 VkDeviceMemory indexBufferMemory;
```

The `createIndexBuffer` function that we’ll add now is almost identical to `createVertexBuffer`:

```
1 void initVulkan() {
2     ...
3     createVertexBuffer();
4     createIndexBuffer();
5     ...
6 }
7
8 void createIndexBuffer() {
9     VkDeviceSize bufferSize = sizeof(indices[0]) * indices.size();
10
11     VkBuffer stagingBuffer;
```

```

12     VkDeviceMemory stagingBufferMemory;
13     createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
        VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
        VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer,
        stagingBufferMemory);
14
15     void* data;
16     vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0,
        &data);
17     memcpy(data, indices.data(), (size_t) bufferSize);
18     vkUnmapMemory(device, stagingBufferMemory);
19
20     createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT |
        VK_BUFFER_USAGE_INDEX_BUFFER_BIT,
        VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, indexBuffer,
        indexBufferMemory);
21
22     copyBuffer(stagingBuffer, indexBuffer, bufferSize);
23
24     vkDestroyBuffer(device, stagingBuffer, nullptr);
25     vkFreeMemory(device, stagingBufferMemory, nullptr);
26 }

```

There are only two notable differences. The `bufferSize` is now equal to the number of indices times the size of the index type, either `uint16_t` or `uint32_t`. The usage of the `indexBuffer` should be `VK_BUFFER_USAGE_INDEX_BUFFER_BIT` instead of `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT`, which makes sense. Other than that, the process is exactly the same. We create a staging buffer to copy the contents of `indices` to and then copy it to the final device local index buffer.

The index buffer should be cleaned up at the end of the program, just like the vertex buffer:

```

1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyBuffer(device, indexBuffer, nullptr);
5     vkFreeMemory(device, indexBufferMemory, nullptr);
6
7     vkDestroyBuffer(device, vertexBuffer, nullptr);
8     vkFreeMemory(device, vertexBufferMemory, nullptr);
9
10    ...
11 }

```

Using an index buffer

Using an index buffer for drawing involves two changes to `recordCommandBuffer`. We first need to bind the index buffer, just like we did for the vertex buffer. The difference is that you can only have a single index buffer. It's unfortunately not possible to use different indices for each vertex attribute, so we do still have to completely duplicate vertex data even if just one attribute varies.

```
1 vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);
2
3 vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0,
    VK_INDEX_TYPE_UINT16);
```

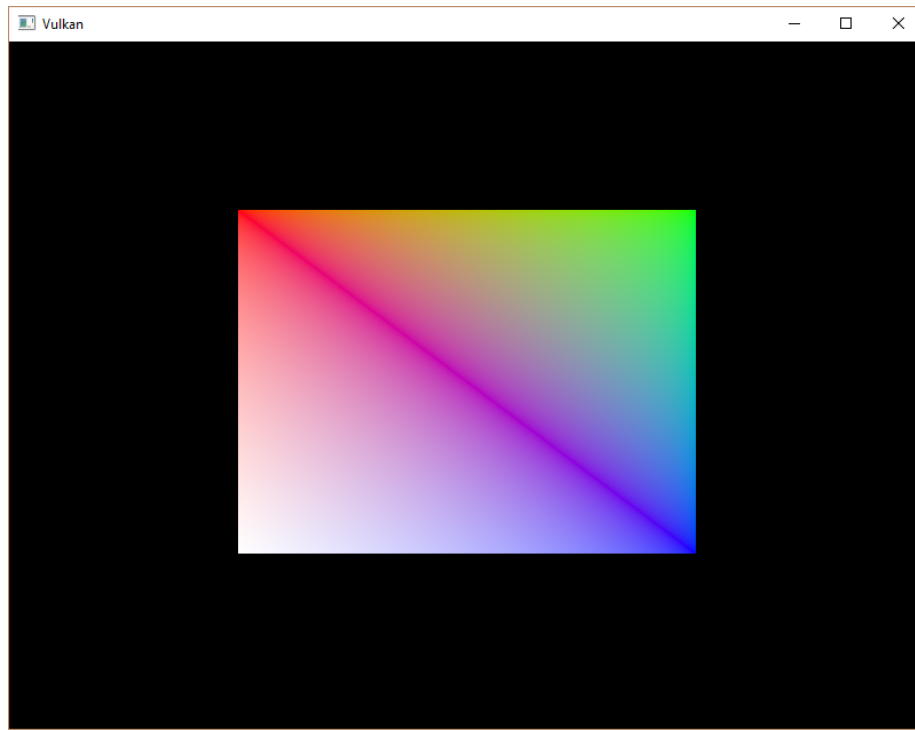
An index buffer is bound with `vkCmdBindIndexBuffer` which has the index buffer, a byte offset into it, and the type of index data as parameters. As mentioned before, the possible types are `VK_INDEX_TYPE_UINT16` and `VK_INDEX_TYPE_UINT32`.

Just binding an index buffer doesn't change anything yet, we also need to change the drawing command to tell Vulkan to use the index buffer. Remove the `vkCmdDraw` line and replace it with `vkCmdDrawIndexed`:

```
1 vkCmdDrawIndexed(commandBuffer,
    static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);
```

A call to this function is very similar to `vkCmdDraw`. The first two parameters specify the number of indices and the number of instances. We're not using instancing, so just specify 1 instance. The number of indices represents the number of vertices that will be passed to the vertex shader. The next parameter specifies an offset into the index buffer, using a value of 1 would cause the graphics card to start reading at the second index. The second to last parameter specifies an offset to add to the indices in the index buffer. The final parameter specifies an offset for instancing, which we're not using.

Now run your program and you should see the following:



You now know how to save memory by reusing vertices with index buffers. This will become especially important in a future chapter where we're going to load complex 3D models.

The previous chapter already mentioned that you should allocate multiple resources like buffers from a single memory allocation, but in fact you should go a step further. Driver developers recommend that you also store multiple buffers, like the vertex and index buffer, into a single `VkBuffer` and use offsets in commands like `vkCmdBindVertexBuffers`. The advantage is that your data is more cache friendly in that case, because it's closer together. It is even possible to reuse the same chunk of memory for multiple resources if they are not used during the same render operations, provided that their data is refreshed, of course. This is known as *aliasing* and some Vulkan functions have explicit flags to specify that you want to do this.

C++ code / Vertex shader / Fragment shader

Uniform buffers

Descriptor set layout and buffer

Introduction

We're now able to pass arbitrary attributes to the vertex shader for each vertex, but what about global variables? We're going to move on to 3D graphics from this chapter on and that requires a model-view-projection matrix. We could include it as vertex data, but that's a waste of memory and it would require us to update the vertex buffer whenever the transformation changes. The transformation could easily change every single frame.

The right way to tackle this in Vulkan is to use *resource descriptors*. A descriptor is a way for shaders to freely access resources like buffers and images. We're going to set up a buffer that contains the transformation matrices and have the vertex shader access them through a descriptor. Usage of descriptors consists of three parts:

- Specify a descriptor set layout during pipeline creation
- Allocate a descriptor set from a descriptor pool
- Bind the descriptor set during rendering

The *descriptor set layout* specifies the types of resources that are going to be accessed by the pipeline, just like a render pass specifies the types of attachments that will be accessed. A *descriptor set* specifies the actual buffer or image resources that will be bound to the descriptors, just like a framebuffer specifies the actual image views to bind to render pass attachments. The descriptor set is then bound for the drawing commands just like the vertex buffers and framebuffer.

There are many types of descriptors, but in this chapter we'll work with uniform buffer objects (UBO). We'll look at other types of descriptors in future chapters, but the basic process is the same. Let's say we have the data we want the vertex shader to have in a C struct like this:

```
1 struct UniformBufferObject {  
2     glm::mat4 model;
```

```

3     glm::mat4 view;
4     glm::mat4 proj;
5 };

```

Then we can copy the data to a `VkBuffer` and access it through a uniform buffer object descriptor from the vertex shader like this:

```

1 layout(binding = 0) uniform UniformBufferObject {
2     mat4 model;
3     mat4 view;
4     mat4 proj;
5 } ubo;
6
7 void main() {
8     gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition,
9         0.0, 1.0);
9     fragColor = inColor;
10 }

```

We're going to update the model, view and projection matrices every frame to make the rectangle from the previous chapter spin around in 3D.

Vertex shader

Modify the vertex shader to include the uniform buffer object like it was specified above. I will assume that you are familiar with MVP transformations. If you're not, see the resource mentioned in the first chapter.

```

1 #version 450
2
3 layout(binding = 0) uniform UniformBufferObject {
4     mat4 model;
5     mat4 view;
6     mat4 proj;
7 } ubo;
8
9 layout(location = 0) in vec2 inPosition;
10 layout(location = 1) in vec3 inColor;
11
12 layout(location = 0) out vec3 fragColor;
13
14 void main() {
15     gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition,
16         0.0, 1.0);
16     fragColor = inColor;
17 }

```

Note that the order of the `uniform`, `in` and `out` declarations doesn't matter. The `binding` directive is similar to the `location` directive for attributes. We're going to reference this binding in the descriptor set layout. The line with `gl_Position` is changed to use the transformations to compute the final position in clip coordinates. Unlike the 2D triangles, the last component of the clip coordinates may not be 1, which will result in a division when converted to the final normalized device coordinates on the screen. This is used in perspective projection as the *perspective division* and is essential for making closer objects look larger than objects that are further away.

Descriptor set layout

The next step is to define the UBO on the C++ side and to tell Vulkan about this descriptor in the vertex shader.

```
1 struct UniformBufferObject {
2     glm::mat4 model;
3     glm::mat4 view;
4     glm::mat4 proj;
5 };
```

We can exactly match the definition in the shader using data types in GLM. The data in the matrices is binary compatible with the way the shader expects it, so we can later just `memcpy` a `UniformBufferObject` to a `VkBuffer`.

We need to provide details about every descriptor binding used in the shaders for pipeline creation, just like we had to do for every vertex attribute and its `location` index. We'll set up a new function to define all of this information called `createDescriptorSetLayout`. It should be called right before pipeline creation, because we're going to need it there.

```
1 void initVulkan() {
2     ...
3     createDescriptorSetLayout();
4     createGraphicsPipeline();
5     ...
6 }
7
8 ...
9
10 void createDescriptorSetLayout() {
11
12 }
```

Every binding needs to be described through a `VkDescriptorSetLayoutBinding` struct.

```
1 void createDescriptorSetLayout() {
```

```

2   VkDescriptorSetLayoutBinding uboLayoutBinding{};
3   uboLayoutBinding.binding = 0;
4   uboLayoutBinding.descriptorType =
        VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
5   uboLayoutBinding.descriptorCount = 1;
6 }

```

The first two fields specify the `binding` used in the shader and the type of descriptor, which is a uniform buffer object. It is possible for the shader variable to represent an array of uniform buffer objects, and `descriptorCount` specifies the number of values in the array. This could be used to specify a transformation for each of the bones in a skeleton for skeletal animation, for example. Our MVP transformation is in a single uniform buffer object, so we're using a `descriptorCount` of 1.

```

1 uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;

```

We also need to specify in which shader stages the descriptor is going to be referenced. The `stageFlags` field can be a combination of `VkShaderStageFlagBits` values or the value `VK_SHADER_STAGE_ALL_GRAPHICS`. In our case, we're only referencing the descriptor from the vertex shader.

```

1 uboLayoutBinding.pImmutableSamplers = nullptr; // Optional

```

The `pImmutableSamplers` field is only relevant for image sampling related descriptors, which we'll look at later. You can leave this to its default value.

All of the descriptor bindings are combined into a single `VkDescriptorSetLayout` object. Define a new class member above `pipelineLayout`:

```

1 VkDescriptorSetLayout descriptorSetLayout;
2 VkPipelineLayout pipelineLayout;

```

We can then create it using `vkCreateDescriptorSetLayout`. This function accepts a simple `VkDescriptorSetLayoutCreateInfo` with the array of bindings:

```

1 VkDescriptorSetLayoutCreateInfo layoutInfo{};
2 layoutInfo.sType =
        VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
3 layoutInfo.bindingCount = 1;
4 layoutInfo.pBindings = &uboLayoutBinding;
5
6 if (vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr,
        &descriptorSetLayout) != VK_SUCCESS) {
7     throw std::runtime_error("failed to create descriptor set
        layout!");
8 }

```

We need to specify the descriptor set layout during pipeline creation to tell Vulkan which descriptors the shaders will be using. Descriptor set layouts are specified in the pipeline layout object. Modify the `VkPipelineLayoutCreateInfo` to reference the layout object:

```
1 VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
2 pipelineLayoutInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
3 pipelineLayoutInfo.setLayoutCount = 1;
4 pipelineLayoutInfo.pSetLayouts = &descriptorSetLayout;
```

You may be wondering why it's possible to specify multiple descriptor set layouts here, because a single one already includes all of the bindings. We'll get back to that in the next chapter, where we'll look into descriptor pools and descriptor sets.

The descriptor set layout should stick around while we may create new graphics pipelines i.e. until the program ends:

```
1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyDescriptorSetLayout(device, descriptorSetLayout,
        nullptr);
5
6     ...
7 }
```

Uniform buffer

In the next chapter we'll specify the buffer that contains the UBO data for the shader, but we need to create this buffer first. We're going to copy new data to the uniform buffer every frame, so it doesn't really make any sense to have a staging buffer. It would just add extra overhead in this case and likely degrade performance instead of improving it.

We should have multiple buffers, because multiple frames may be in flight at the same time and we don't want to update the buffer in preparation of the next frame while a previous one is still reading from it! Thus, we need to have as many uniform buffers as we have frames in flight, and write to a uniform buffer that is not currently being read by the GPU.

To that end, add new class members for `uniformBuffers`, and `uniformBuffersMemory`:

```
1 VkBuffer indexBuffer;
2 VkDeviceMemory indexBufferMemory;
3
4 std::vector<VkBuffer> uniformBuffers;
```

```

5 std::vector<VkDeviceMemory> uniformBuffersMemory;
6 std::vector<void*> uniformBuffersMapped;

```

Similarly, create a new function `createUniformBuffers` that is called after `createIndexBuffer` and allocates the buffers:

```

1 void initVulkan() {
2     ...
3     createVertexBuffer();
4     createIndexBuffer();
5     createUniformBuffers();
6     ...
7 }
8
9 ...
10
11 void createUniformBuffers() {
12     VkDeviceSize bufferSize = sizeof(UniformBufferObject);
13
14     uniformBuffers.resize(MAX_FRAMES_IN_FLIGHT);
15     uniformBuffersMemory.resize(MAX_FRAMES_IN_FLIGHT);
16     uniformBuffersMapped.resize(MAX_FRAMES_IN_FLIGHT);
17
18     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
19         createBuffer(bufferSize, VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT,
20                     VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
21                     VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, uniformBuffers[i],
22                     uniformBuffersMemory[i]);
23
24         vkMapMemory(device, uniformBuffersMemory[i], 0, bufferSize,
25                     0, &uniformBuffersMapped[i]);
26     }
27 }

```

We map the buffer right after creation using `vkMapMemory` to get a pointer to which we can write the data later on. The buffer stays mapped to this pointer for the application's whole lifetime. This technique is called “**persistent mapping**” and works on all Vulkan implementations. Not having to map the buffer every time we need to update it increases performances, as mapping is not free.

The uniform data will be used for all draw calls, so the buffer containing it should only be destroyed when we stop rendering.

```

1 void cleanup() {
2     ...
3

```

```

4     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
5         vkDestroyBuffer(device, uniformBuffers[i], nullptr);
6         vkFreeMemory(device, uniformBuffersMemory[i], nullptr);
7     }
8
9     vkDestroyDescriptorSetLayout(device, descriptorSetLayout,
10        nullptr);
11
12     ...
13 }

```

Updating uniform data

Create a new function `updateUniformBuffer` and add a call to it from the `drawFrame` function before submitting the next frame:

```

1 void drawFrame() {
2     ...
3
4     updateUniformBuffer(currentFrame);
5
6     ...
7
8     VkSubmitInfo submitInfo{};
9     submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
10
11     ...
12 }
13
14 ...
15
16 void updateUniformBuffer(uint32_t currentImage) {
17
18 }

```

This function will generate a new transformation every frame to make the geometry spin around. We need to include two new headers to implement this functionality:

```

1 #define GLM_FORCE_RADIANS
2 #include <glm/glm.hpp>
3 #include <glm/gtc/matrix_transform.hpp>
4
5 #include <chrono>

```

The `glm/gtc/matrix_transform.hpp` header exposes functions that can be used to generate model transformations like `glm::rotate`, view transformations like `glm::lookAt` and projection transformations like `glm::perspective`. The `GLM_FORCE_RADIANS` definition is necessary to make sure that functions like `glm::rotate` use radians as arguments, to avoid any possible confusion.

The `chrono` standard library header exposes functions to do precise timekeeping. We'll use this to make sure that the geometry rotates 90 degrees per second regardless of frame rate.

```
1 void updateUniformBuffer(uint32_t currentImage) {
2     static auto startTime =
3         std::chrono::high_resolution_clock::now();
4
5     auto currentTime = std::chrono::high_resolution_clock::now();
6     float time = std::chrono::duration<float,
7         std::chrono::seconds::period>(currentTime -
8         startTime).count();
9 }
```

The `updateUniformBuffer` function will start out with some logic to calculate the time in seconds since rendering has started with floating point accuracy.

We will now define the model, view and projection transformations in the uniform buffer object. The model rotation will be a simple rotation around the Z-axis using the `time` variable:

```
1 UniformBufferObject ubo{};
2 ubo.model = glm::rotate(glm::mat4(1.0f), time * glm::radians(90.0f),
3     glm::vec3(0.0f, 0.0f, 1.0f));
```

The `glm::rotate` function takes an existing transformation, rotation angle and rotation axis as parameters. The `glm::mat4(1.0f)` constructor returns an identity matrix. Using a rotation angle of `time * glm::radians(90.0f)` accomplishes the purpose of rotation 90 degrees per second.

```
1 ubo.view = glm::lookAt(glm::vec3(2.0f, 2.0f, 2.0f), glm::vec3(0.0f,
2     0.0f, 0.0f), glm::vec3(0.0f, 0.0f, 1.0f));
```

For the view transformation I've decided to look at the geometry from above at a 45 degree angle. The `glm::lookAt` function takes the eye position, center position and up axis as parameters.

```
1 ubo.proj = glm::perspective(glm::radians(45.0f),
2     swapChainExtent.width / (float) swapChainExtent.height, 0.1f,
3     10.0f);
```

I've chosen to use a perspective projection with a 45 degree vertical field-of-view. The other parameters are the aspect ratio, near and far view planes. It

is important to use the current swap chain extent to calculate the aspect ratio to take into account the new width and height of the window after a resize.

```
1 ubo.proj[1][1] *= -1;
```

GLM was originally designed for OpenGL, where the Y coordinate of the clip coordinates is inverted. The easiest way to compensate for that is to flip the sign on the scaling factor of the Y axis in the projection matrix. If you don't do this, then the image will be rendered upside down.

All of the transformations are defined now, so we can copy the data in the uniform buffer object to the current uniform buffer. This happens in exactly the same way as we did for vertex buffers, except without a staging buffer. As noted earlier, we only map the uniform buffer once, so we can directly write to it without having to map again:

```
1 memcpy(uniformBuffersMapped[currentImage], &ubo, sizeof(ubo));
```

Using a UBO this way is not the most efficient way to pass frequently changing values to the shader. A more efficient way to pass a small buffer of data to shaders are *push constants*. We may look at these in a future chapter.

In the next chapter we'll look at descriptor sets, which will actually bind the `VkBuffers` to the uniform buffer descriptors so that the shader can access this transformation data.

C++ code / Vertex shader / Fragment shader

Descriptor pool and sets

Introduction

The descriptor set layout from the previous chapter describes the type of descriptors that can be bound. In this chapter we're going to create a descriptor set for each `VkBuffer` resource to bind it to the uniform buffer descriptor.

Descriptor pool

Descriptor sets can't be created directly, they must be allocated from a pool like command buffers. The equivalent for descriptor sets is unsurprisingly called a *descriptor pool*. We'll write a new function `createDescriptorPool` to set it up.

```
1 void initVulkan() {
2     ...
3     createUniformBuffers();
4     createDescriptorPool();
5     ...
6 }
7
```

```

8 ...
9
10 void createDescriptorPool() {
11
12 }

```

We first need to describe which descriptor types our descriptor sets are going to contain and how many of them, using `VkDescriptorPoolSize` structures.

```

1 VkDescriptorPoolSize poolSize{};
2 poolSize.type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
3 poolSize.descriptorCount =
    static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);

```

We will allocate one of these descriptors for every frame. This pool size structure is referenced by the main `VkDescriptorPoolCreateInfo`:

```

1 VkDescriptorPoolCreateInfo poolInfo{};
2 poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
3 poolInfo.poolSizeCount = 1;
4 poolInfo.pPoolSizes = &poolSize;

```

Aside from the maximum number of individual descriptors that are available, we also need to specify the maximum number of descriptor sets that may be allocated:

```

1 poolInfo.maxSets = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);

```

The structure has an optional flag similar to command pools that determines if individual descriptor sets can be freed or not: `VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT`. We're not going to touch the descriptor set after creating it, so we don't need this flag. You can leave `flags` to its default value of 0.

```

1 VkDescriptorPool descriptorPool;
2
3 ...
4
5 if (vkCreateDescriptorPool(device, &poolInfo, nullptr,
    &descriptorPool) != VK_SUCCESS) {
6     throw std::runtime_error("failed to create descriptor pool!");
7 }

```

Add a new class member to store the handle of the descriptor pool and call `vkCreateDescriptorPool` to create it.

Descriptor set

We can now allocate the descriptor sets themselves. Add a `createDescriptorSets` function for that purpose:

```

1 void initVulkan() {
2     ...
3     createDescriptorPool();
4     createDescriptorSets();
5     ...
6 }
7
8 ...
9
10 void createDescriptorSets() {
11
12 }

```

A descriptor set allocation is described with a `VkDescriptorSetAllocateInfo` struct. You need to specify the descriptor pool to allocate from, the number of descriptor sets to allocate, and the descriptor set layout to base them on:

```

1 std::vector<VkDescriptorSetLayout> layouts(MAX_FRAMES_IN_FLIGHT,
2     descriptorSetLayout);
3 VkDescriptorSetAllocateInfo allocInfo{};
4 allocInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO;
5 allocInfo.descriptorPool = descriptorPool;
6 allocInfo.descriptorSetCount =
7     static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
8 allocInfo.pSetLayouts = layouts.data();

```

In our case we will create one descriptor set for each frame in flight, all with the same layout. Unfortunately we do need all the copies of the layout because the next function expects an array matching the number of sets.

Add a class member to hold the descriptor set handles and allocate them with `vkAllocateDescriptorSets`:

```

1 VkDescriptorPool descriptorPool;
2 std::vector<VkDescriptorSet> descriptorSets;
3
4 ...
5
6 descriptorSets.resize(MAX_FRAMES_IN_FLIGHT);
7 if (vkAllocateDescriptorSets(device, &allocInfo,
8     descriptorSets.data()) != VK_SUCCESS) {
9     throw std::runtime_error("failed to allocate descriptor sets!");
10 }

```

You don't need to explicitly clean up descriptor sets, because they will be automatically freed when the descriptor pool is destroyed. The call to `vkAllocateDescriptorSets` will allocate descriptor sets, each with one uniform buffer descriptor.

```

1 void cleanup() {
2     ...
3     vkDestroyDescriptorPool(device, descriptorPool, nullptr);
4
5     vkDestroyDescriptorSetLayout(device, descriptorSetLayout,
6         nullptr);
7     ...
8 }

```

The descriptor sets have been allocated now, but the descriptors within still need to be configured. We'll now add a loop to populate every descriptor:

```

1 for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
2
3 }

```

Descriptors that refer to buffers, like our uniform buffer descriptor, are configured with a `VkDescriptorBufferInfo` struct. This structure specifies the buffer and the region within it that contains the data for the descriptor.

```

1 for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
2     VkDescriptorBufferInfo bufferInfo{};
3     bufferInfo.buffer = uniformBuffers[i];
4     bufferInfo.offset = 0;
5     bufferInfo.range = sizeof(UniformBufferObject);
6 }

```

If you're overwriting the whole buffer, like we are in this case, then it is also possible to use the `VK_WHOLE_SIZE` value for the range. The configuration of descriptors is updated using the `vkUpdateDescriptorSets` function, which takes an array of `VkWriteDescriptorSet` structs as parameter.

```

1 VkWriteDescriptorSet descriptorWrite{};
2 descriptorWrite.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
3 descriptorWrite.dstSet = descriptorSets[i];
4 descriptorWrite.dstBinding = 0;
5 descriptorWrite.dstArrayElement = 0;

```

The first two fields specify the descriptor set to update and the binding. We gave our uniform buffer binding index 0. Remember that descriptors can be arrays, so we also need to specify the first index in the array that we want to update. We're not using an array, so the index is simply 0.

```

1 descriptorWrite.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
2 descriptorWrite.descriptorCount = 1;

```

We need to specify the type of descriptor again. It's possible to update multiple descriptors at once in an array, starting at index `dstArrayElement`. The `descriptorCount` field specifies how many array elements you want to update.

```

1 descriptorWrite.pBufferInfo = &bufferInfo;
2 descriptorWrite.pImageInfo = nullptr; // Optional
3 descriptorWrite.pTexelBufferView = nullptr; // Optional

```

The last field references an array with `descriptorCount` structs that actually configure the descriptors. It depends on the type of descriptor which one of the three you actually need to use. The `pBufferInfo` field is used for descriptors that refer to buffer data, `pImageInfo` is used for descriptors that refer to image data, and `pTexelBufferView` is used for descriptors that refer to buffer views. Our descriptor is based on buffers, so we're using `pBufferInfo`.

```

1 vkUpdateDescriptorSets(device, 1, &descriptorWrite, 0, nullptr);

```

The updates are applied using `vkUpdateDescriptorSets`. It accepts two kinds of arrays as parameters: an array of `VkWriteDescriptorSet` and an array of `VkCopyDescriptorSet`. The latter can be used to copy descriptors to each other, as its name implies.

Using descriptor sets

We now need to update the `recordCommandBuffer` function to actually bind the right descriptor set for each frame to the descriptors in the shader with `vkCmdBindDescriptorSets`. This needs to be done before the `vkCmdDrawIndexed` call:

```

1 vkCmdBindDescriptorSets(commandBuffer,
    VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1,
    &descriptorSets[currentFrame], 0, nullptr);
2 vkCmdDrawIndexed(commandBuffer,
    static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);

```

Unlike vertex and index buffers, descriptor sets are not unique to graphics pipelines. Therefore we need to specify if we want to bind descriptor sets to the graphics or compute pipeline. The next parameter is the layout that the descriptors are based on. The next three parameters specify the index of the first descriptor set, the number of sets to bind, and the array of sets to bind. We'll get back to this in a moment. The last two parameters specify an array of offsets that are used for dynamic descriptors. We'll look at these in a future chapter.

If you run your program now, then you'll notice that unfortunately nothing is visible. The problem is that because of the Y-flip we did in the projection matrix, the vertices are now being drawn in counter-clockwise order instead of clockwise order. This causes backface culling to kick in and prevents any geometry from being drawn. Go to the `createGraphicsPipeline` function and modify the `frontFace` in `VkPipelineRasterizationStateCreateInfo` to correct this:

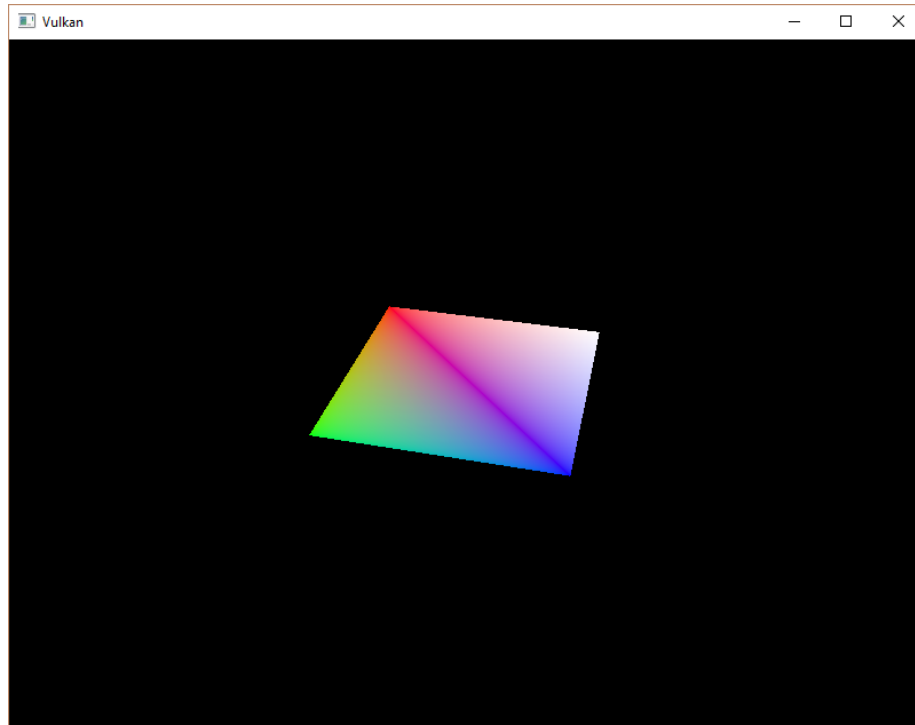
```

1 rasterizer.cullMode = VK_CULL_MODE_BACK_BIT;

```

```
2 rasterizer.frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE;
```

Run your program again and you should now see the following:



The rectangle has changed into a square because the projection matrix now corrects for aspect ratio. The `updateUniformBuffer` takes care of screen resizing, so we don't need to recreate the descriptor set in `recreateSwapChain`.

Alignment requirements

One thing we've glossed over so far is how exactly the data in the C++ structure should match with the uniform definition in the shader. It seems obvious enough to simply use the same types in both:

```
1 struct UniformBufferObject {
2     glm::mat4 model;
3     glm::mat4 view;
4     glm::mat4 proj;
5 };
6
7 layout(binding = 0) uniform UniformBufferObject {
8     mat4 model;
9     mat4 view;
```

```

10     mat4 proj;
11 } ubo;

```

However, that's not all there is to it. For example, try modifying the struct and shader to look like this:

```

1 struct UniformBufferObject {
2     glm::vec2 foo;
3     glm::mat4 model;
4     glm::mat4 view;
5     glm::mat4 proj;
6 };
7
8 layout(binding = 0) uniform UniformBufferObject {
9     vec2 foo;
10    mat4 model;
11    mat4 view;
12    mat4 proj;
13 } ubo;

```

Recompile your shader and your program and run it and you'll find that the colorful square you worked so far has disappeared! That's because we haven't taken into account the *alignment requirements*.

Vulkan expects the data in your structure to be aligned in memory in a specific way, for example:

- Scalars have to be aligned by N (= 4 bytes given 32 bit floats).
- A `vec2` must be aligned by 2N (= 8 bytes)
- A `vec3` or `vec4` must be aligned by 4N (= 16 bytes)
- A nested structure must be aligned by the base alignment of its members rounded up to a multiple of 16.
- A `mat4` matrix must have the same alignment as a `vec4`.

You can find the full list of alignment requirements in the specification.

Our original shader with just three `mat4` fields already met the alignment requirements. As each `mat4` is $4 \times 4 \times 4 = 64$ bytes in size, `model` has an offset of 0, `view` has an offset of 64 and `proj` has an offset of 128. All of these are multiples of 16 and that's why it worked fine.

The new structure starts with a `vec2` which is only 8 bytes in size and therefore throws off all of the offsets. Now `model` has an offset of 8, `view` an offset of 72 and `proj` an offset of 136, none of which are multiples of 16. To fix this problem we can use the `alignas` specifier introduced in C++11:

```

1 struct UniformBufferObject {
2     glm::vec2 foo;
3     alignas(16) glm::mat4 model;

```

```

4     glm::mat4 view;
5     glm::mat4 proj;
6 };

```

If you now compile and run your program again you should see that the shader correctly receives its matrix values once again.

Luckily there is a way to not have to think about these alignment requirements *most* of the time. We can define `GLM_FORCE_DEFAULT_ALIGNED_GENTYPES` right before including GLM:

```

1 #define GLM_FORCE_RADIANS
2 #define GLM_FORCE_DEFAULT_ALIGNED_GENTYPES
3 #include <glm/glm.hpp>

```

This will force GLM to use a version of `vec2` and `mat4` that has the alignment requirements already specified for us. If you add this definition then you can remove the `alignas` specifier and your program should still work.

Unfortunately this method can break down if you start using nested structures. Consider the following definition in the C++ code:

```

1 struct Foo {
2     glm::vec2 v;
3 };
4
5 struct UniformBufferObject {
6     Foo f1;
7     Foo f2;
8 };

```

And the following shader definition:

```

1 struct Foo {
2     vec2 v;
3 };
4
5 layout(binding = 0) uniform UniformBufferObject {
6     Foo f1;
7     Foo f2;
8 } ubo;

```

In this case `f2` will have an offset of 8 whereas it should have an offset of 16 since it is a nested structure. In this case you must specify the alignment yourself:

```

1 struct UniformBufferObject {
2     Foo f1;
3     alignas(16) Foo f2;
4 };

```

These gotchas are a good reason to always be explicit about alignment. That way you won't be caught offguard by the strange symptoms of alignment errors.

```
1 struct UniformBufferObject {  
2     alignas(16) glm::mat4 model;  
3     alignas(16) glm::mat4 view;  
4     alignas(16) glm::mat4 proj;  
5 };
```

Don't forget to recompile your shader after removing the `foo` field.

Multiple descriptor sets

As some of the structures and function calls hinted at, it is actually possible to bind multiple descriptor sets simultaneously. You need to specify a descriptor set layout for each descriptor set when creating the pipeline layout. Shaders can then reference specific descriptor sets like this:

```
1 layout(set = 0, binding = 0) uniform UniformBufferObject { ... }
```

You can use this feature to put descriptors that vary per-object and descriptors that are shared into separate descriptor sets. In that case you avoid rebinding most of the descriptors across draw calls which is potentially more efficient.

C++ code / Vertex shader / Fragment shader

Texture mapping

Images

Introduction

The geometry has been colored using per-vertex colors so far, which is a rather limited approach. In this part of the tutorial we're going to implement texture mapping to make the geometry look more interesting. This will also allow us to load and draw basic 3D models in a future chapter.

Adding a texture to our application will involve the following steps:

- Create an image object backed by device memory
- Fill it with pixels from an image file
- Create an image sampler
- Add a combined image sampler descriptor to sample colors from the texture

We've already worked with image objects before, but those were automatically created by the swap chain extension. This time we'll have to create one by ourselves. Creating an image and filling it with data is similar to vertex buffer creation. We'll start by creating a staging resource and filling it with pixel data and then we copy this to the final image object that we'll use for rendering. Although it is possible to create a staging image for this purpose, Vulkan also allows you to copy pixels from a `VkBuffer` to an image and the API for this is actually faster on some hardware. We'll first create this buffer and fill it with pixel values, and then we'll create an image to copy the pixels to. Creating an image is not very different from creating buffers. It involves querying the memory requirements, allocating device memory and binding it, just like we've seen before.

However, there is something extra that we'll have to take care of when working with images. Images can have different *layouts* that affect how the pixels are organized in memory. Due to the way graphics hardware works, simply storing the pixels row by row may not lead to the best performance, for example. When performing any operation on images, you must make sure that they have the

layout that is optimal for use in that operation. We've actually already seen some of these layouts when we specified the render pass:

- `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`: Optimal for presentation
- `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`: Optimal as attachment for writing colors from the fragment shader
- `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL`: Optimal as source in a transfer operation, like `vkCmdCopyImageToBuffer`
- `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`: Optimal as destination in a transfer operation, like `vkCmdCopyBufferToImage`
- `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`: Optimal for sampling from a shader

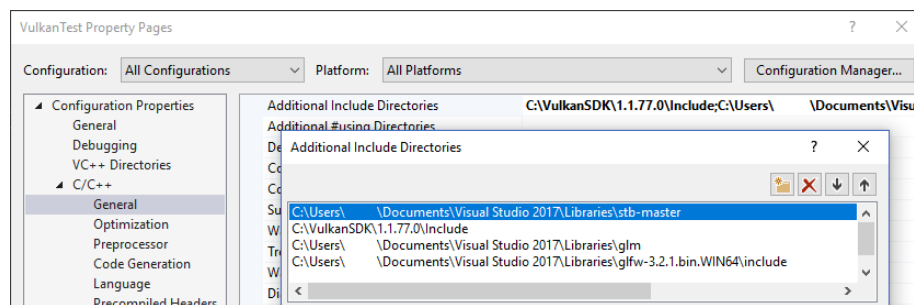
One of the most common ways to transition the layout of an image is a *pipeline barrier*. Pipeline barriers are primarily used for synchronizing access to resources, like making sure that an image was written to before it is read, but they can also be used to transition layouts. In this chapter we'll see how pipeline barriers are used for this purpose. Barriers can additionally be used to transfer queue family ownership when using `VK_SHARING_MODE_EXCLUSIVE`.

Image library

There are many libraries available for loading images, and you can even write your own code to load simple formats like BMP and PPM. In this tutorial we'll be using the `stb_image` library from the `stb` collection. The advantage of it is that all of the code is in a single file, so it doesn't require any tricky build configuration. Download `stb_image.h` and store it in a convenient location, like the directory where you saved GLFW and GLM. Add the location to your include path.

Visual Studio

Add the directory with `stb_image.h` in it to the **Additional Include Directories** paths.



Makefile

Add the directory with `stb_image.h` to the include directories for GCC:

```

1 VULKAN_SDK_PATH = /home/user/VulkanSDK/x.x.x.x/x86_64
2 STB_INCLUDE_PATH = /home/user/libraries/stb
3
4 ...
5
6 CFLAGS = -std=c++17 -I$(VULKAN_SDK_PATH)/include
        -I$(STB_INCLUDE_PATH)

```

Loading an image

Include the image library like this:

```

1 #define STB_IMAGE_IMPLEMENTATION
2 #include <stb_image.h>

```

The header only defines the prototypes of the functions by default. One code file needs to include the header with the `STB_IMAGE_IMPLEMENTATION` definition to include the function bodies, otherwise we'll get linking errors.

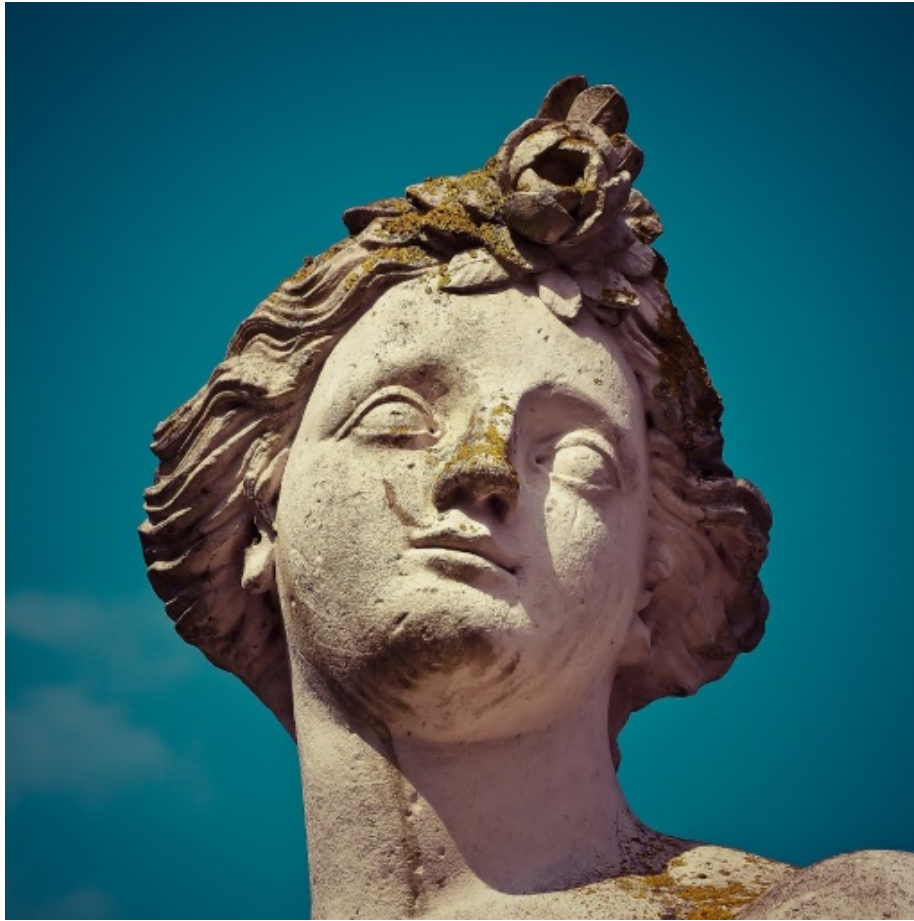
```

1 void initVulkan() {
2     ...
3     createCommandPool();
4     createTextureImage();
5     createVertexBuffer();
6     ...
7 }
8
9 ...
10
11 void createTextureImage() {
12
13 }

```

Create a new function `createTextureImage` where we'll load an image and upload it into a Vulkan image object. We're going to use command buffers, so it should be called after `createCommandPool`.

Create a new directory `textures` next to the `shaders` directory to store texture images in. We're going to load an image called `texture.jpg` from that directory. I've chosen to use the following CC0 licensed image resized to 512 x 512 pixels, but feel free to pick any image you want. The library supports most common image file formats, like JPEG, PNG, BMP and GIF.



Loading an image with this library is really easy:

```
1 void createTextureImage() {  
2     int texWidth, texHeight, texChannels;  
3     stbi_uc* pixels = stbi_load("textures/texture.jpg", &texWidth,  
4                               &texHeight, &texChannels, STBI_rgb_alpha);  
5     VkDeviceSize imageSize = texWidth * texHeight * 4;  
6     if (!pixels) {  
7         throw std::runtime_error("failed to load texture image!");  
8     }  
9 }
```

The `stbi_load` function takes the file path and number of channels to load as arguments. The `STBI_rgb_alpha` value forces the image to be loaded with an alpha channel, even if it doesn't have one, which is nice for consistency with other textures in the future. The middle three parameters are outputs for the

width, height and actual number of channels in the image. The pointer that is returned is the first element in an array of pixel values. The pixels are laid out row by row with 4 bytes per pixel in the case of `STBI_rgb_alpha` for a total of `texWidth * texHeight * 4` values.

Staging buffer

We're now going to create a buffer in host visible memory so that we can use `vkMapMemory` and copy the pixels to it. Add variables for this temporary buffer to the `createTextureImage` function:

```
1 VkBuffer stagingBuffer;
2 VkDeviceMemory stagingBufferMemory;
```

The buffer should be in host visible memory so that we can map it and it should be usable as a transfer source so that we can copy it to an image later on:

```
1 createBuffer(imageSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
    VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer,
    stagingBufferMemory);
```

We can then directly copy the pixel values that we got from the image loading library to the buffer:

```
1 void* data;
2 vkMapMemory(device, stagingBufferMemory, 0, imageSize, 0, &data);
3 memcpy(data, pixels, static_cast<size_t>(imageSize));
4 vkUnmapMemory(device, stagingBufferMemory);
```

Don't forget to clean up the original pixel array now:

```
1 stbi_image_free(pixels);
```

Texture Image

Although we could set up the shader to access the pixel values in the buffer, it's better to use image objects in Vulkan for this purpose. Image objects will make it easier and faster to retrieve colors by allowing us to use 2D coordinates, for one. Pixels within an image object are known as texels and we'll use that name from this point on. Add the following new class members:

```
1 VkImage textureImage;
2 VkDeviceMemory textureImageMemory;
```

The parameters for an image are specified in a `VkImageCreateInfo` struct:

```

1 VkImageCreateInfo imageInfo{};
2 imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
3 imageInfo.imageType = VK_IMAGE_TYPE_2D;
4 imageInfo.extent.width = static_cast<uint32_t>(texWidth);
5 imageInfo.extent.height = static_cast<uint32_t>(texHeight);
6 imageInfo.extent.depth = 1;
7 imageInfo.mipLevels = 1;
8 imageInfo.arrayLayers = 1;

```

The image type, specified in the `imageType` field, tells Vulkan with what kind of coordinate system the texels in the image are going to be addressed. It is possible to create 1D, 2D and 3D images. One dimensional images can be used to store an array of data or gradient, two dimensional images are mainly used for textures, and three dimensional images can be used to store voxel volumes, for example. The `extent` field specifies the dimensions of the image, basically how many texels there are on each axis. That's why `depth` must be 1 instead of 0. Our texture will not be an array and we won't be using mipmapping for now.

```

1 imageInfo.format = VK_FORMAT_R8G8B8A8_SRGB;

```

Vulkan supports many possible image formats, but we should use the same format for the texels as the pixels in the buffer, otherwise the copy operation will fail.

```

1 imageInfo.tiling = VK_IMAGE_TILING_OPTIMAL;

```

The `tiling` field can have one of two values:

- `VK_IMAGE_TILING_LINEAR`: Texels are laid out in row-major order like our `pixels` array
- `VK_IMAGE_TILING_OPTIMAL`: Texels are laid out in an implementation defined order for optimal access

Unlike the layout of an image, the tiling mode cannot be changed at a later time. If you want to be able to directly access texels in the memory of the image, then you must use `VK_IMAGE_TILING_LINEAR`. We will be using a staging buffer instead of a staging image, so this won't be necessary. We will be using `VK_IMAGE_TILING_OPTIMAL` for efficient access from the shader.

```

1 imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;

```

There are only two possible values for the `initialLayout` of an image:

- `VK_IMAGE_LAYOUT_UNDEFINED`: Not usable by the GPU and the very first transition will discard the texels.
- `VK_IMAGE_LAYOUT_PREINITIALIZED`: Not usable by the GPU, but the first transition will preserve the texels.

There are few situations where it is necessary for the texels to be preserved during the first transition. One example, however, would be if you wanted to use an image as a staging image in combination with the `VK_IMAGE_TILING_LINEAR` layout. In that case, you'd want to upload the texel data to it and then transition the image to be a transfer source without losing the data. In our case, however, we're first going to transition the image to be a transfer destination and then copy texel data to it from a buffer object, so we don't need this property and can safely use `VK_IMAGE_LAYOUT_UNDEFINED`.

```
1 imageInfo.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT |
    VK_IMAGE_USAGE_SAMPLED_BIT;
```

The `usage` field has the same semantics as the one during buffer creation. The image is going to be used as destination for the buffer copy, so it should be set up as a transfer destination. We also want to be able to access the image from the shader to color our mesh, so the usage should include `VK_IMAGE_USAGE_SAMPLED_BIT`.

```
1 imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
```

The image will only be used by one queue family: the one that supports graphics (and therefore also) transfer operations.

```
1 imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
2 imageInfo.flags = 0; // Optional
```

The `samples` flag is related to multisampling. This is only relevant for images that will be used as attachments, so stick to one sample. There are some optional flags for images that are related to sparse images. Sparse images are images where only certain regions are actually backed by memory. If you were using a 3D texture for a voxel terrain, for example, then you could use this to avoid allocating memory to store large volumes of “air” values. We won't be using it in this tutorial, so leave it to its default value of 0.

```
1 if (vkCreateImage(device, &imageInfo, nullptr, &textureImage) !=
    VK_SUCCESS) {
2     throw std::runtime_error("failed to create image!");
3 }
```

The image is created using `vkCreateImage`, which doesn't have any particularly noteworthy parameters. It is possible that the `VK_FORMAT_R8G8B8A8_SRGB` format is not supported by the graphics hardware. You should have a list of acceptable alternatives and go with the best one that is supported. However, support for this particular format is so widespread that we'll skip this step. Using different formats would also require annoying conversions. We will get back to this in the depth buffer chapter, where we'll implement such a system.

```
1 VkMemoryRequirements memRequirements;
```

```

2 vkGetImageMemoryRequirements(device, textureImage, &memRequirements);
3
4 VkMemoryAllocateInfo allocInfo{};
5 allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
6 allocInfo.allocationSize = memRequirements.size;
7 allocInfo.memoryTypeIndex =
    findMemoryType(memRequirements.memoryTypeBits,
        VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
8
9 if (vkAllocateMemory(device, &allocInfo, nullptr,
    &textureImageMemory) != VK_SUCCESS) {
10     throw std::runtime_error("failed to allocate image memory!");
11 }
12
13 vkBindImageMemory(device, textureImage, textureImageMemory, 0);

```

Allocating memory for an image works in exactly the same way as allocating memory for a buffer. Use `vkGetImageMemoryRequirements` instead of `vkGetBufferMemoryRequirements`, and use `vkBindImageMemory` instead of `vkBindBufferMemory`.

This function is already getting quite large and there'll be a need to create more images in later chapters, so we should abstract image creation into a `createImage` function, like we did for buffers. Create the function and move the image object creation and memory allocation to it:

```

1 void createImage(uint32_t width, uint32_t height, VkFormat format,
    VkImageTiling tiling, VkImageUsageFlags usage,
    VkMemoryPropertyFlags properties, VkImage& image,
    VkDeviceMemory& imageMemory) {
2     VkImageCreateInfo imageInfo{};
3     imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
4     imageInfo.imageType = VK_IMAGE_TYPE_2D;
5     imageInfo.extent.width = width;
6     imageInfo.extent.height = height;
7     imageInfo.extent.depth = 1;
8     imageInfo.mipLevels = 1;
9     imageInfo.arrayLayers = 1;
10    imageInfo.format = format;
11    imageInfo.tiling = tiling;
12    imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
13    imageInfo.usage = usage;
14    imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
15    imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
16
17    if (vkCreateImage(device, &imageInfo, nullptr, &image) !=
        VK_SUCCESS) {

```

```

18     throw std::runtime_error("failed to create image!");
19 }
20
21 VkMemoryRequirements memRequirements;
22 vkGetImageMemoryRequirements(device, image, &memRequirements);
23
24 VkMemoryAllocateInfo allocInfo{};
25 allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
26 allocInfo.allocationSize = memRequirements.size;
27 allocInfo.memoryTypeIndex =
    findMemoryType(memRequirements.memoryTypeBits, properties);
28
29 if (vkAllocateMemory(device, &allocInfo, nullptr, &imageMemory)
    != VK_SUCCESS) {
30     throw std::runtime_error("failed to allocate image memory!");
31 }
32
33 vkBindImageMemory(device, image, imageMemory, 0);
34 }

```

I've made the width, height, format, tiling mode, usage, and memory properties parameters, because these will all vary between the images we'll be creating throughout this tutorial.

The `createTextureImage` function can now be simplified to:

```

1 void createTextureImage() {
2     int texWidth, texHeight, texChannels;
3     stbi_uc* pixels = stbi_load("textures/texture.jpg", &texWidth,
4                               &texHeight, &texChannels, STBI_rgb_alpha);
5     VkDeviceSize imageSize = texWidth * texHeight * 4;
6
7     if (!pixels) {
8         throw std::runtime_error("failed to load texture image!");
9     }
10
11     VkBuffer stagingBuffer;
12     VkDeviceMemory stagingBufferMemory;
13     createBuffer(imageSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
14                 VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
15                 VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer,
16                 stagingBufferMemory);
17
18     void* data;
19     vkMapMemory(device, stagingBufferMemory, 0, imageSize, 0, &data);
20     memcpy(data, pixels, static_cast<size_t>(imageSize));
21     vkUnmapMemory(device, stagingBufferMemory);
22 }

```

```

18     stbi_image_free(pixels);
19
20
21     createImage(texWidth, texHeight, VK_FORMAT_R8G8B8A8_SRGB,
                VK_IMAGE_TILING_OPTIMAL, VK_IMAGE_USAGE_TRANSFER_DST_BIT |
                VK_IMAGE_USAGE_SAMPLED_BIT,
                VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, textureImage,
                textureImageMemory);
22 }

```

Layout transitions

The function we're going to write now involves recording and executing a command buffer again, so now's a good time to move that logic into a helper function or two:

```

1  VkCommandBuffer beginSingleTimeCommands() {
2      VkCommandBufferAllocateInfo allocInfo{};
3      allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
4      allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
5      allocInfo.commandPool = commandPool;
6      allocInfo.commandBufferCount = 1;
7
8      VkCommandBuffer commandBuffer;
9      vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer);
10
11     VkCommandBufferBeginInfo beginInfo{};
12     beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
13     beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;
14
15     vkBeginCommandBuffer(commandBuffer, &beginInfo);
16
17     return commandBuffer;
18 }
19
20 void endSingleTimeCommands(VkCommandBuffer commandBuffer) {
21     vkEndCommandBuffer(commandBuffer);
22
23     VkSubmitInfo submitInfo{};
24     submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
25     submitInfo.commandBufferCount = 1;
26     submitInfo.pCommandBuffers = &commandBuffer;
27
28     vkQueueSubmit(graphicsQueue, 1, &submitInfo, VK_NULL_HANDLE);
29     vkQueueWaitIdle(graphicsQueue);
30 }

```

```

31     vkFreeCommandBuffers(device, commandPool, 1, &commandBuffer);
32 }

```

The code for these functions is based on the existing code in `copyBuffer`. You can now simplify that function to:

```

1 void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize
   size) {
2     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
3
4     VkBufferCopy copyRegion{};
5     copyRegion.size = size;
6     vkCmdCopyBuffer(commandBuffer, srcBuffer, dstBuffer, 1,
       &copyRegion);
7
8     endSingleTimeCommands(commandBuffer);
9 }

```

If we were still using buffers, then we could now write a function to record and execute `vkCmdCopyBufferToImage` to finish the job, but this command requires the image to be in the right layout first. Create a new function to handle layout transitions:

```

1 void transitionImageLayout(VkImage image, VkFormat format,
   VkImageLayout oldLayout, VkImageLayout newLayout) {
2     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
3
4     endSingleTimeCommands(commandBuffer);
5 }

```

One of the most common ways to perform layout transitions is using an *image memory barrier*. A pipeline barrier like that is generally used to synchronize access to resources, like ensuring that a write to a buffer completes before reading from it, but it can also be used to transition image layouts and transfer queue family ownership when `VK_SHARING_MODE_EXCLUSIVE` is used. There is an equivalent *buffer memory barrier* to do this for buffers.

```

1 VkImageMemoryBarrier barrier{};
2 barrier.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER;
3 barrier.oldLayout = oldLayout;
4 barrier.newLayout = newLayout;

```

The first two fields specify layout transition. It is possible to use `VK_IMAGE_LAYOUT_UNDEFINED` as `oldLayout` if you don't care about the existing contents of the image.

```

1 barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
2 barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;

```

If you are using the barrier to transfer queue family ownership, then these two fields should be the indices of the queue families. They must be set to `VK_QUEUE_FAMILY_IGNORED` if you don't want to do this (not the default value!).

```
1 barrier.image = image;
2 barrier.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
3 barrier.subresourceRange.baseMipLevel = 0;
4 barrier.subresourceRange.levelCount = 1;
5 barrier.subresourceRange.baseArrayLayer = 0;
6 barrier.subresourceRange.layerCount = 1;
```

The `image` and `subresourceRange` specify the image that is affected and the specific part of the image. Our image is not an array and does not have mipmapping levels, so only one level and layer are specified.

```
1 barrier.srcAccessMask = 0; // TODO
2 barrier.dstAccessMask = 0; // TODO
```

Barriers are primarily used for synchronization purposes, so you must specify which types of operations that involve the resource must happen before the barrier, and which operations that involve the resource must wait on the barrier. We need to do that despite already using `vkQueueWaitIdle` to manually synchronize. The right values depend on the old and new layout, so we'll get back to this once we've figured out which transitions we're going to use.

```
1 vkCmdPipelineBarrier(
2     commandBuffer,
3     0 /* TODO */, 0 /* TODO */,
4     0,
5     0, nullptr,
6     0, nullptr,
7     1, &barrier
8 );
```

All types of pipeline barriers are submitted using the same function. The first parameter after the command buffer specifies in which pipeline stage the operations occur that should happen before the barrier. The second parameter specifies the pipeline stage in which operations will wait on the barrier. The pipeline stages that you are allowed to specify before and after the barrier depend on how you use the resource before and after the barrier. The allowed values are listed in this table of the specification. For example, if you're going to read from a uniform after the barrier, you would specify a usage of `VK_ACCESS_UNIFORM_READ_BIT` and the earliest shader that will read from the uniform as pipeline stage, for example `VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT`. It would not make sense to specify a non-shader pipeline stage for this type of usage and the validation layers will warn you when you specify a pipeline stage that does not match the type of usage.

The third parameter is either 0 or `VK_DEPENDENCY_BY_REGION_BIT`. The latter turns the barrier into a per-region condition. That means that the implementation is allowed to already begin reading from the parts of a resource that were written so far, for example.

The last three pairs of parameters reference arrays of pipeline barriers of the three available types: memory barriers, buffer memory barriers, and image memory barriers like the one we're using here. Note that we're not using the `VkFormat` parameter yet, but we'll be using that one for special transitions in the depth buffer chapter.

Copying buffer to image

Before we get back to `createTextureImage`, we're going to write one more helper function: `copyBufferToImage`:

```
1 void copyBufferToImage(VkBuffer buffer, VkImage image, uint32_t
   width, uint32_t height) {
2     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
3
4     endSingleTimeCommands(commandBuffer);
5 }
```

Just like with buffer copies, you need to specify which part of the buffer is going to be copied to which part of the image. This happens through `VkBufferImageCopy` structs:

```
1 VkBufferImageCopy region{};
2 region.bufferOffset = 0;
3 region.bufferRowLength = 0;
4 region.bufferImageHeight = 0;
5
6 region.imageSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
7 region.imageSubresource.mipLevel = 0;
8 region.imageSubresource.baseArrayLayer = 0;
9 region.imageSubresource.layerCount = 1;
10
11 region.imageOffset = {0, 0, 0};
12 region.imageExtent = {
13     width,
14     height,
15     1
16 };
```

Most of these fields are self-explanatory. The `bufferOffset` specifies the byte offset in the buffer at which the pixel values start. The `bufferRowLength` and `bufferImageHeight` fields specify how the pixels are laid out in memory. For

example, you could have some padding bytes between rows of the image. Specifying 0 for both indicates that the pixels are simply tightly packed like they are in our case. The `imageSubresource`, `imageOffset` and `imageExtent` fields indicate to which part of the image we want to copy the pixels.

Buffer to image copy operations are enqueued using the `vkCmdCopyBufferToImage` function:

```
1 vkCmdCopyBufferToImage(  
2     commandBuffer,  
3     buffer,  
4     image,  
5     VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,  
6     1,  
7     &region  
8 );
```

The fourth parameter indicates which layout the image is currently using. I'm assuming here that the image has already been transitioned to the layout that is optimal for copying pixels to. Right now we're only copying one chunk of pixels to the whole image, but it's possible to specify an array of `VkBufferImageCopy` to perform many different copies from this buffer to the image in one operation.

Preparing the texture image

We now have all of the tools we need to finish setting up the texture image, so we're going back to the `createTextureImage` function. The last thing we did there was creating the texture image. The next step is to copy the staging buffer to the texture image. This involves two steps:

- Transition the texture image to `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`
- Execute the buffer to image copy operation

This is easy to do with the functions we just created:

```
1 transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,  
    VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL);  
2 copyBufferToImage(stagingBuffer, textureImage,  
    static_cast<uint32_t>(texWidth),  
    static_cast<uint32_t>(texHeight));
```

The image was created with the `VK_IMAGE_LAYOUT_UNDEFINED` layout, so that one should be specified as old layout when transitioning `textureImage`. Remember that we can do this because we don't care about its contents before performing the copy operation.

To be able to start sampling from the texture image in the shader, we need one last transition to prepare it for shader access:

```

1 transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
    VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL);

```

Transition barrier masks

If you run your application with validation layers enabled now, then you'll see that it complains about the access masks and pipeline stages in `transitionImageLayout` being invalid. We still need to set those based on the layouts in the transition.

There are two transitions we need to handle:

- Undefined → transfer destination: transfer writes that don't need to wait on anything
- Transfer destination → shader reading: shader reads should wait on transfer writes, specifically the shader reads in the fragment shader, because that's where we're going to use the texture

These rules are specified using the following access masks and pipeline stages:

```

1 VkPipelineStageFlags sourceStage;
2 VkPipelineStageFlags destinationStage;
3
4 if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED && newLayout ==
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL) {
5     barrier.srcAccessMask = 0;
6     barrier.dstAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
7
8     sourceStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;
9     destinationStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
10 } else if (oldLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL &&
    newLayout == VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL) {
11     barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
12     barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
13
14     sourceStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
15     destinationStage = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT;
16 } else {
17     throw std::invalid_argument("unsupported layout transition!");
18 }
19
20 vkCmdPipelineBarrier(
21     commandBuffer,
22     sourceStage, destinationStage,
23     0,
24     0, nullptr,

```

```

25     0, nullptr,
26     1, &barrier
27 );

```

As you can see in the aforementioned table, transfer writes must occur in the pipeline transfer stage. Since the writes don't have to wait on anything, you may specify an empty access mask and the earliest possible pipeline stage `VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT` for the pre-barrier operations. It should be noted that `VK_PIPELINE_STAGE_TRANSFER_BIT` is not a *real* stage within the graphics and compute pipelines. It is more of a pseudo-stage where transfers happen. See the documentation for more information and other examples of pseudo-stages.

The image will be written in the same pipeline stage and subsequently read by the fragment shader, which is why we specify shader reading access in the fragment shader pipeline stage.

If we need to do more transitions in the future, then we'll extend the function. The application should now run successfully, although there are of course no visual changes yet.

One thing to note is that command buffer submission results in implicit `VK_ACCESS_HOST_WRITE_BIT` synchronization at the beginning. Since the `transitionImageLayout` function executes a command buffer with only a single command, you could use this implicit synchronization and set `srcAccessMask` to 0 if you ever needed a `VK_ACCESS_HOST_WRITE_BIT` dependency in a layout transition. It's up to you if you want to be explicit about it or not, but I'm personally not a fan of relying on these OpenGL-like "hidden" operations.

There is actually a special type of image layout that supports all operations, `VK_IMAGE_LAYOUT_GENERAL`. The problem with it, of course, is that it doesn't necessarily offer the best performance for any operation. It is required for some special cases, like using an image as both input and output, or for reading an image after it has left the preinitialized layout.

All of the helper functions that submit commands so far have been set up to execute synchronously by waiting for the queue to become idle. For practical applications it is recommended to combine these operations in a single command buffer and execute them asynchronously for higher throughput, especially the transitions and copy in the `createTextureImage` function. Try to experiment with this by creating a `setupCommandBuffer` that the helper functions record commands into, and add a `flushSetupCommands` to execute the commands that have been recorded so far. It's best to do this after the texture mapping works to check if the texture resources are still set up correctly.

Cleanup

Finish the `createTextureImage` function by cleaning up the staging buffer and its memory at the end:

```
1     transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
2                             VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
3                             VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL);
4     vkDestroyBuffer(device, stagingBuffer, nullptr);
5     vkFreeMemory(device, stagingBufferMemory, nullptr);
6 }
```

The main texture image is used until the end of the program:

```
1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyImage(device, textureImage, nullptr);
5     vkFreeMemory(device, textureImageMemory, nullptr);
6
7     ...
8 }
```

The image now contains the texture, but we still need a way to access it from the graphics pipeline. We'll work on that in the next chapter.

C++ code / Vertex shader / Fragment shader

Image view and sampler

In this chapter we're going to create two more resources that are needed for the graphics pipeline to sample an image. The first resource is one that we've already seen before while working with the swap chain images, but the second one is new - it relates to how the shader will read texels from the image.

Texture image view

We've seen before, with the swap chain images and the framebuffer, that images are accessed through image views rather than directly. We will also need to create such an image view for the texture image.

Add a class member to hold a `VkImageView` for the texture image and create a new function `createTextureImageView` where we'll create it:

```
1 VkImageView textureImageView;
2
3 ...
```

```

4
5 void initVulkan() {
6     ...
7     createTextureImage();
8     createTextureImageView();
9     createVertexBuffer();
10    ...
11 }
12
13 ...
14
15 void createTextureImageView() {
16
17 }

```

The code for this function can be based directly on `createImageViews`. The only two changes you have to make are the `format` and the `image`:

```

1 VkImageViewCreateInfo viewInfo{};
2 viewInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
3 viewInfo.image = textureImage;
4 viewInfo.viewType = VK_IMAGE_VIEW_TYPE_2D;
5 viewInfo.format = VK_FORMAT_R8G8B8A8_SRGB;
6 viewInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
7 viewInfo.subresourceRange.baseMipLevel = 0;
8 viewInfo.subresourceRange.levelCount = 1;
9 viewInfo.subresourceRange.baseArrayLayer = 0;
10 viewInfo.subresourceRange.layerCount = 1;

```

I've left out the explicit `viewInfo.components` initialization, because `VK_COMPONENT_SWIZZLE_IDENTITY` is defined as 0 anyway. Finish creating the image view by calling `vkCreateImageView`:

```

1 if (vkCreateImageView(device, &viewInfo, nullptr, &textureImageView)
2     != VK_SUCCESS) {
3     throw std::runtime_error("failed to create texture image view!");
4 }

```

Because so much of the logic is duplicated from `createImageViews`, you may wish to abstract it into a new `createImageView` function:

```

1 VkImageView createImageView(VkImage image, VkFormat format) {
2     VkImageViewCreateInfo viewInfo{};
3     viewInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
4     viewInfo.image = image;
5     viewInfo.viewType = VK_IMAGE_VIEW_TYPE_2D;
6     viewInfo.format = format;

```

```

7     viewInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
8     viewInfo.subresourceRange.baseMipLevel = 0;
9     viewInfo.subresourceRange.levelCount = 1;
10    viewInfo.subresourceRange.baseArrayLayer = 0;
11    viewInfo.subresourceRange.layerCount = 1;
12
13    VkImageView imageView;
14    if (vkCreateImageView(device, &viewInfo, nullptr, &imageView) !=
        VK_SUCCESS) {
15        throw std::runtime_error("failed to create image view!");
16    }
17
18    return imageView;
19 }

```

The createTextureImageView function can now be simplified to:

```

1 void createTextureImageView() {
2     textureImageView = createImageView(textureImage,
        VK_FORMAT_R8G8B8A8_SRGB);
3 }

```

And createImageViews can be simplified to:

```

1 void createImageViews() {
2     swapChainImageViews.resize(swapChainImages.size());
3
4     for (uint32_t i = 0; i < swapChainImages.size(); i++) {
5         swapChainImageViews[i] = createImageView(swapChainImages[i],
            swapChainImageFormat);
6     }
7 }

```

Make sure to destroy the image view at the end of the program, right before destroying the image itself:

```

1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroyImageView(device, textureImageView, nullptr);
5
6     vkDestroyImage(device, textureImage, nullptr);
7     vkFreeMemory(device, textureImageMemory, nullptr);

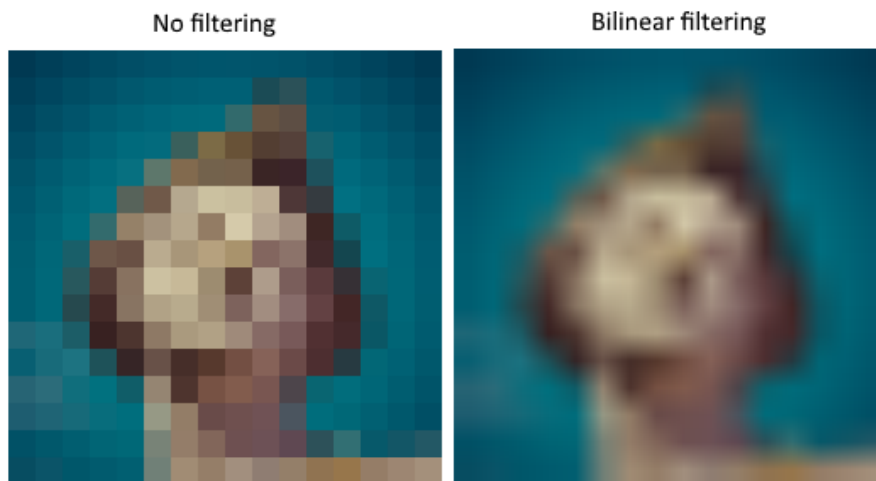
```

Samplers

It is possible for shaders to read texels directly from images, but that is not very common when they are used as textures. Textures are usually accessed through

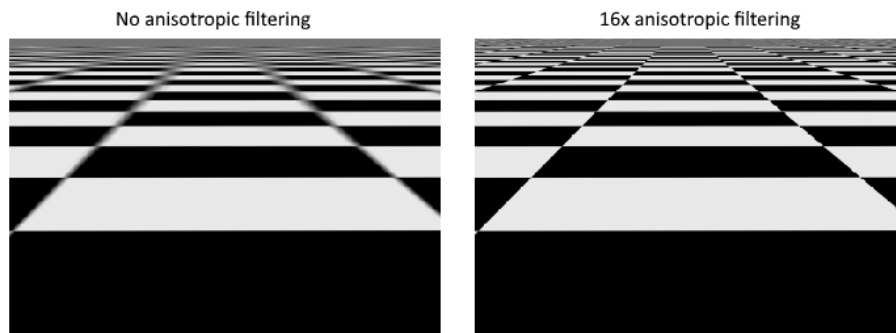
samplers, which will apply filtering and transformations to compute the final color that is retrieved.

These filters are helpful to deal with problems like oversampling. Consider a texture that is mapped to geometry with more fragments than texels. If you simply took the closest texel for the texture coordinate in each fragment, then you would get a result like the first image:



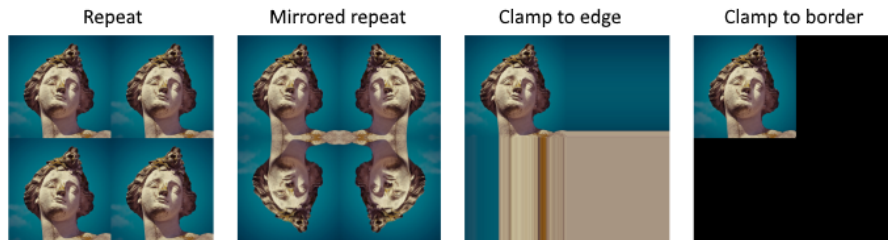
If you combined the 4 closest texels through linear interpolation, then you would get a smoother result like the one on the right. Of course your application may have art style requirements that fit the left style more (think Minecraft), but the right is preferred in conventional graphics applications. A sampler object automatically applies this filtering for you when reading a color from the texture.

Undersampling is the opposite problem, where you have more texels than fragments. This will lead to artifacts when sampling high frequency patterns like a checkerboard texture at a sharp angle:



As shown in the left image, the texture turns into a blurry mess in the distance. The solution to this is anisotropic filtering, which can also be applied automatically by a sampler.

Aside from these filters, a sampler can also take care of transformations. It determines what happens when you try to read texels outside the image through its *addressing mode*. The image below displays some of the possibilities:



We will now create a function `createTextureSampler` to set up such a sampler object. We'll be using that sampler to read colors from the texture in the shader later on.

```
1 void initVulkan() {
2     ...
3     createTextureImage();
4     createTextureImageView();
5     createTextureSampler();
6     ...
7 }
8
9 ...
10
11 void createTextureSampler() {
12
13 }
```

Samplers are configured through a `VkSamplerCreateInfo` structure, which specifies all filters and transformations that it should apply.

```
1 VkSamplerCreateInfo samplerInfo{};
2 samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
3 samplerInfo.magFilter = VK_FILTER_LINEAR;
4 samplerInfo.minFilter = VK_FILTER_LINEAR;
```

The `magFilter` and `minFilter` fields specify how to interpolate texels that are magnified or minified. Magnification concerns the oversampling problem described above, and minification concerns undersampling. The choices are `VK_FILTER_NEAREST` and `VK_FILTER_LINEAR`, corresponding to the modes demonstrated in the images above.

```

1 samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
2 samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
3 samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;

```

The addressing mode can be specified per axis using the `addressMode` fields. The available values are listed below. Most of these are demonstrated in the image above. Note that the axes are called U, V and W instead of X, Y and Z. This is a convention for texture space coordinates.

- `VK_SAMPLER_ADDRESS_MODE_REPEAT`: Repeat the texture when going beyond the image dimensions.
- `VK_SAMPLER_ADDRESS_MODE_MIRRORED_REPEAT`: Like repeat, but inverts the coordinates to mirror the image when going beyond the dimensions.
- `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE`: Take the color of the edge closest to the coordinate beyond the image dimensions.
- `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE`: Like clamp to edge, but instead uses the edge opposite to the closest edge.
- `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER`: Return a solid color when sampling beyond the dimensions of the image.

It doesn't really matter which addressing mode we use here, because we're not going to sample outside of the image in this tutorial. However, the repeat mode is probably the most common mode, because it can be used to tile textures like floors and walls.

```

1 samplerInfo.anisotropyEnable = VK_TRUE;
2 samplerInfo.maxAnisotropy = ???;

```

These two fields specify if anisotropic filtering should be used. There is no reason not to use this unless performance is a concern. The `maxAnisotropy` field limits the amount of texel samples that can be used to calculate the final color. A lower value results in better performance, but lower quality results. To figure out which value we can use, we need to retrieve the properties of the physical device like so:

```

1 VkPhysicalDeviceProperties properties{};
2 vkGetPhysicalDeviceProperties(physicalDevice, &properties);

```

If you look at the documentation for the `VkPhysicalDeviceProperties` structure, you'll see that it contains a `VkPhysicalDeviceLimits` member named `limits`. This struct in turn has a member called `maxSamplerAnisotropy` and this is the maximum value we can specify for `maxAnisotropy`. If we want to go for maximum quality, we can simply use that value directly:

```

1 samplerInfo.maxAnisotropy = properties.limits.maxSamplerAnisotropy;

```

You can either query the properties at the beginning of your program and pass them around to the functions that need them, or query them in the `createTextureSampler` function itself.

```
1 samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
```

The `borderColor` field specifies which color is returned when sampling beyond the image with clamp to border addressing mode. It is possible to return black, white or transparent in either float or int formats. You cannot specify an arbitrary color.

```
1 samplerInfo.unnormalizedCoordinates = VK_FALSE;
```

The `unnormalizedCoordinates` field specifies which coordinate system you want to use to address texels in an image. If this field is `VK_TRUE`, then you can simply use coordinates within the `[0, texWidth)` and `[0, texHeight)` range. If it is `VK_FALSE`, then the texels are addressed using the `[0, 1)` range on all axes. Real-world applications almost always use normalized coordinates, because then it's possible to use textures of varying resolutions with the exact same coordinates.

```
1 samplerInfo.compareEnable = VK_FALSE;
2 samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;
```

If a comparison function is enabled, then texels will first be compared to a value, and the result of that comparison is used in filtering operations. This is mainly used for percentage-closer filtering on shadow maps. We'll look at this in a future chapter.

```
1 samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
2 samplerInfo.mipLodBias = 0.0f;
3 samplerInfo.minLod = 0.0f;
4 samplerInfo.maxLod = 0.0f;
```

All of these fields apply to mipmapping. We will look at mipmapping in a later chapter, but basically it's another type of filter that can be applied.

The functioning of the sampler is now fully defined. Add a class member to hold the handle of the sampler object and create the sampler with `vkCreateSampler`:

```
1 VkImageView textureImageView;
2 VkSampler textureSampler;
3
4 ...
5
6 void createTextureSampler() {
7     ...
8
9     if (vkCreateSampler(device, &samplerInfo, nullptr,
10         &textureSampler) != VK_SUCCESS) {
11         throw std::runtime_error("failed to create texture
12             sampler!");
13     }
14 }
```

```

11     }
12 }

```

Note the sampler does not reference a `VkImage` anywhere. The sampler is a distinct object that provides an interface to extract colors from a texture. It can be applied to any image you want, whether it is 1D, 2D or 3D. This is different from many older APIs, which combined texture images and filtering into a single state.

Destroy the sampler at the end of the program when we'll no longer be accessing the image:

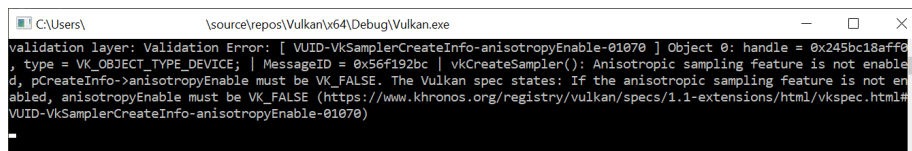
```

1 void cleanup() {
2     cleanupSwapChain();
3
4     vkDestroySampler(device, textureSampler, nullptr);
5     vkDestroyImageView(device, textureImageView, nullptr);
6
7     ...
8 }

```

Anisotropy device feature

If you run your program right now, you'll see a validation layer message like this:



That's because anisotropic filtering is actually an optional device feature. We need to update the `createLogicalDevice` function to request it:

```

1 VkPhysicalDeviceFeatures deviceFeatures{};
2 deviceFeatures.samplerAnisotropy = VK_TRUE;

```

And even though it is very unlikely that a modern graphics card will not support it, we should update `isDeviceSuitable` to check if it is available:

```

1 bool isDeviceSuitable(VkPhysicalDevice device) {
2     ...
3
4     VkPhysicalDeviceFeatures supportedFeatures;
5     vkGetPhysicalDeviceFeatures(device, &supportedFeatures);
6
7     return indices.isComplete() && extensionsSupported &&
           swapChainAdequate && supportedFeatures.samplerAnisotropy;
}

```

```
8 }
```

The `vkGetPhysicalDeviceFeatures` repurposes the `VkPhysicalDeviceFeatures` struct to indicate which features are supported rather than requested by setting the boolean values.

Instead of enforcing the availability of anisotropic filtering, it's also possible to simply not use it by conditionally setting:

```
1 samplerInfo.anisotropyEnable = VK_FALSE;
2 samplerInfo.maxAnisotropy = 1.0f;
```

In the next chapter we will expose the image and sampler objects to the shaders to draw the texture onto the square.

C++ code / Vertex shader / Fragment shader

Combined image sampler

Introduction

We looked at descriptors for the first time in the uniform buffers part of the tutorial. In this chapter we will look at a new type of descriptor: *combined image sampler*. This descriptor makes it possible for shaders to access an image resource through a sampler object like the one we created in the previous chapter.

We'll start by modifying the descriptor set layout, descriptor pool and descriptor set to include such a combined image sampler descriptor. After that, we're going to add texture coordinates to `Vertex` and modify the fragment shader to read colors from the texture instead of just interpolating the vertex colors.

Updating the descriptors

Browse to the `createDescriptorSetLayout` function and add a `VkDescriptorSetLayoutBinding` for a combined image sampler descriptor. We'll simply put it in the binding after the uniform buffer:

```
1 VkDescriptorSetLayoutBinding samplerLayoutBinding{};
2 samplerLayoutBinding.binding = 1;
3 samplerLayoutBinding.descriptorCount = 1;
4 samplerLayoutBinding.descriptorType =
    VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
5 samplerLayoutBinding.pImmutableSamplers = nullptr;
6 samplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
7
8 std::array<VkDescriptorSetLayoutBinding, 2> bindings =
    {uboLayoutBinding, samplerLayoutBinding};
```

```

9 VkDescriptorSetLayoutCreateInfo layoutInfo{};
10 layoutInfo.sType =
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
11 layoutInfo.bindingCount = static_cast<uint32_t>(bindings.size());
12 layoutInfo.pBindings = bindings.data();

```

Make sure to set the `stageFlags` to indicate that we intend to use the combined image sampler descriptor in the fragment shader. That's where the color of the fragment is going to be determined. It is possible to use texture sampling in the vertex shader, for example to dynamically deform a grid of vertices by a heightmap.

We must also create a larger descriptor pool to make room for the allocation of the combined image sampler by adding another `VkPoolSize` of type `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` to the `VkDescriptorPoolCreateInfo`. Go to the `createDescriptorPool` function and modify it to include a `VkDescriptorPoolSize` for this descriptor:

```

1 std::array<VkDescriptorPoolSize, 2> poolSizes{};
2 poolSizes[0].type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
3 poolSizes[0].descriptorCount =
    static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
4 poolSizes[1].type = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
5 poolSizes[1].descriptorCount =
    static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
6
7 VkDescriptorPoolCreateInfo poolInfo{};
8 poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
9 poolInfo.poolSizeCount = static_cast<uint32_t>(poolSizes.size());
10 poolInfo.pPoolSizes = poolSizes.data();
11 poolInfo.maxSets = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);

```

Inadequate descriptor pools are a good example of a problem that the validation layers will not catch: As of Vulkan 1.1, `vkAllocateDescriptorSets` may fail with the error code `VK_ERROR_POOL_OUT_OF_MEMORY` if the pool is not sufficiently large, but the driver may also try to solve the problem internally. This means that sometimes (depending on hardware, pool size and allocation size) the driver will let us get away with an allocation that exceeds the limits of our descriptor pool. Other times, `vkAllocateDescriptorSets` will fail and return `VK_ERROR_POOL_OUT_OF_MEMORY`. This can be particularly frustrating if the allocation succeeds on some machines, but fails on others.

Since Vulkan shifts the responsibility for the allocation to the driver, it is no longer a strict requirement to only allocate as many descriptors of a certain type (`VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, etc.) as specified by the corresponding `descriptorCount` members for the creation of the descriptor pool. However, it remains best practise to do so, and in the future,

VK_LAYER_KHRONOS_validation will warn about this type of problem if you enable Best Practice Validation.

The final step is to bind the actual image and sampler resources to the descriptors in the descriptor set. Go to the `createDescriptorSets` function.

```
1 for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
2     VkDescriptorBufferInfo bufferInfo{};
3     bufferInfo.buffer = uniformBuffers[i];
4     bufferInfo.offset = 0;
5     bufferInfo.range = sizeof(UniformBufferObject);
6
7     VkDescriptorImageInfo imageInfo{};
8     imageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
9     imageInfo.imageView = textureImageView;
10    imageInfo.sampler = textureSampler;
11
12    ...
13 }
```

The resources for a combined image sampler structure must be specified in a `VkDescriptorImageInfo` struct, just like the buffer resource for a uniform buffer descriptor is specified in a `VkDescriptorBufferInfo` struct. This is where the objects from the previous chapter come together.

```
1 std::array<VkWriteDescriptorSet, 2> descriptorWrites{};
2
3 descriptorWrites[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
4 descriptorWrites[0].dstSet = descriptorSets[i];
5 descriptorWrites[0].dstBinding = 0;
6 descriptorWrites[0].dstArrayElement = 0;
7 descriptorWrites[0].descriptorType =
8     VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
9 descriptorWrites[0].descriptorCount = 1;
10 descriptorWrites[0].pBufferInfo = &bufferInfo;
11
12 descriptorWrites[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
13 descriptorWrites[1].dstSet = descriptorSets[i];
14 descriptorWrites[1].dstBinding = 1;
15 descriptorWrites[1].dstArrayElement = 0;
16 descriptorWrites[1].descriptorType =
17     VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
18 descriptorWrites[1].descriptorCount = 1;
19 descriptorWrites[1].pImageInfo = &imageInfo;
20
21 vkUpdateDescriptorSets(device,
22     static_cast<uint32_t>(descriptorWrites.size()),
23     descriptorWrites.data(), 0, nullptr);
```

The descriptors must be updated with this image info, just like the buffer. This time we're using the `pImageInfo` array instead of `pBufferInfo`. The descriptors are now ready to be used by the shaders!

Texture coordinates

There is one important ingredient for texture mapping that is still missing, and that's the actual texture coordinates for each vertex. The texture coordinates determine how the image is actually mapped to the geometry.

```
1 struct Vertex {
2     glm::vec2 pos;
3     glm::vec3 color;
4     glm::vec2 texCoord;
5
6     static VkVertexInputBindingDescription getBindingDescription() {
7         VkVertexInputBindingDescription bindingDescription{};
8         bindingDescription.binding = 0;
9         bindingDescription.stride = sizeof(Vertex);
10        bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
11
12        return bindingDescription;
13    }
14
15    static std::array<VkVertexInputAttributeDescription, 3>
16        getAttributeDescriptions() {
17        std::array<VkVertexInputAttributeDescription, 3>
18            attributeDescriptions{};
19
20        attributeDescriptions[0].binding = 0;
21        attributeDescriptions[0].location = 0;
22        attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
23        attributeDescriptions[0].offset = offsetof(Vertex, pos);
24
25        attributeDescriptions[1].binding = 0;
26        attributeDescriptions[1].location = 1;
27        attributeDescriptions[1].format = VK_FORMAT_R32G32B32_SFLOAT;
28        attributeDescriptions[1].offset = offsetof(Vertex, color);
29
30        attributeDescriptions[2].binding = 0;
31        attributeDescriptions[2].location = 2;
32        attributeDescriptions[2].format = VK_FORMAT_R32G32_SFLOAT;
33        attributeDescriptions[2].offset = offsetof(Vertex, texCoord);
34
35        return attributeDescriptions;
36    }
```

```
35 };
```

Modify the `Vertex` struct to include a `vec2` for texture coordinates. Make sure to also add a `VkVertexInputAttributeDescription` so that we can use access texture coordinates as input in the vertex shader. That is necessary to be able to pass them to the fragment shader for interpolation across the surface of the square.

```
1 const std::vector<Vertex> vertices = {
2     {{-0.5f, -0.5f}, {1.0f, 0.0f, 0.0f}, {1.0f, 0.0f}},
3     {{0.5f, -0.5f}, {0.0f, 1.0f, 0.0f}, {0.0f, 0.0f}},
4     {{0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}, {0.0f, 1.0f}},
5     {{-0.5f, 0.5f}, {1.0f, 1.0f, 1.0f}, {1.0f, 1.0f}}
6 };
```

In this tutorial, I will simply fill the square with the texture by using coordinates from 0, 0 in the top-left corner to 1, 1 in the bottom-right corner. Feel free to experiment with different coordinates. Try using coordinates below 0 or above 1 to see the addressing modes in action!

Shaders

The final step is modifying the shaders to sample colors from the texture. We first need to modify the vertex shader to pass through the texture coordinates to the fragment shader:

```
1 layout(location = 0) in vec2 inPosition;
2 layout(location = 1) in vec3 inColor;
3 layout(location = 2) in vec2 inTexCoord;
4
5 layout(location = 0) out vec3 fragColor;
6 layout(location = 1) out vec2 fragTexCoord;
7
8 void main() {
9     gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition,
10         0.0, 1.0);
11     fragColor = inColor;
12     fragTexCoord = inTexCoord;
13 }
```

Just like the per vertex colors, the `fragTexCoord` values will be smoothly interpolated across the area of the square by the rasterizer. We can visualize this by having the fragment shader output the texture coordinates as colors:

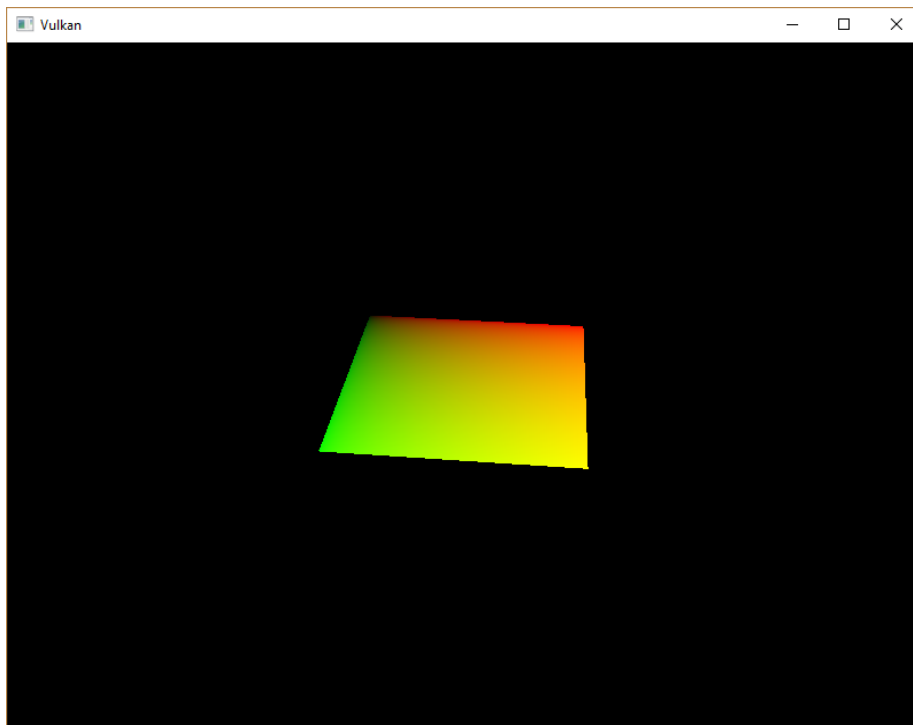
```
1 #version 450
2
3 layout(location = 0) in vec3 fragColor;
```

```

4 layout(location = 1) in vec2 fragTexCoord;
5
6 layout(location = 0) out vec4 outColor;
7
8 void main() {
9     outColor = vec4(fragTexCoord, 0.0, 1.0);
10 }

```

You should see something like the image below. Don't forget to recompile the shaders!



The green channel represents the horizontal coordinates and the red channel the vertical coordinates. The black and yellow corners confirm that the texture coordinates are correctly interpolated from 0, 0 to 1, 1 across the square. Visualizing data using colors is the shader programming equivalent of `printf` debugging, for lack of a better option!

A combined image sampler descriptor is represented in GLSL by a `sampler` uniform. Add a reference to it in the fragment shader:

```

1 layout(binding = 1) uniform sampler2D texSampler;

```

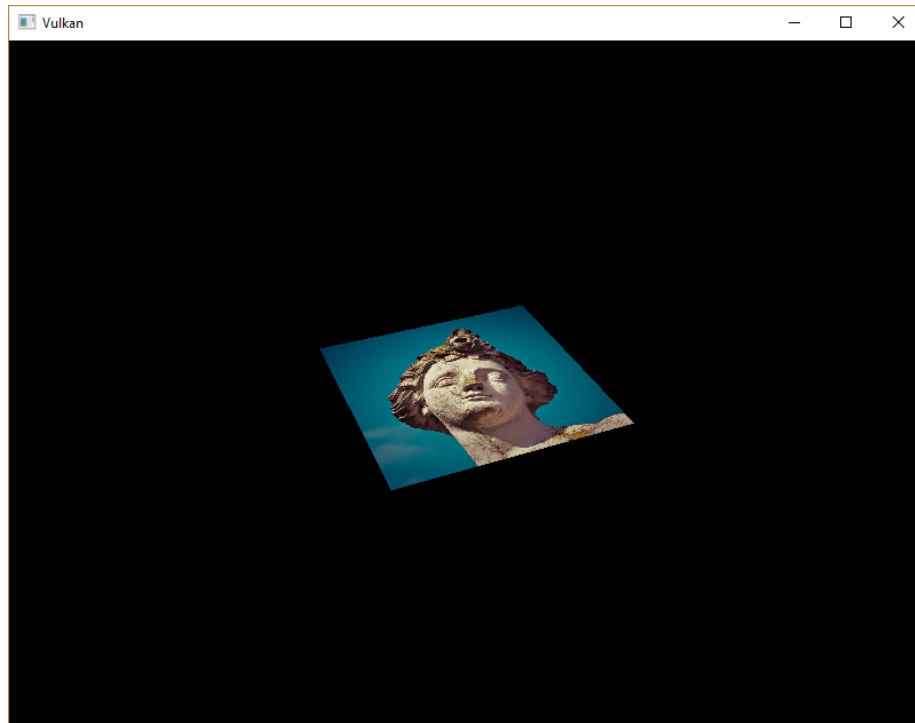
There are equivalent `sampler1D` and `sampler3D` types for other types of images. Make sure to use the correct binding here.

```

1 void main() {
2     outColor = texture(texSampler, fragTexCoord);
3 }

```

Textures are sampled using the built-in `texture` function. It takes a `sampler` and coordinate as arguments. The sampler automatically takes care of the filtering and transformations in the background. You should now see the texture on the square when you run the application:

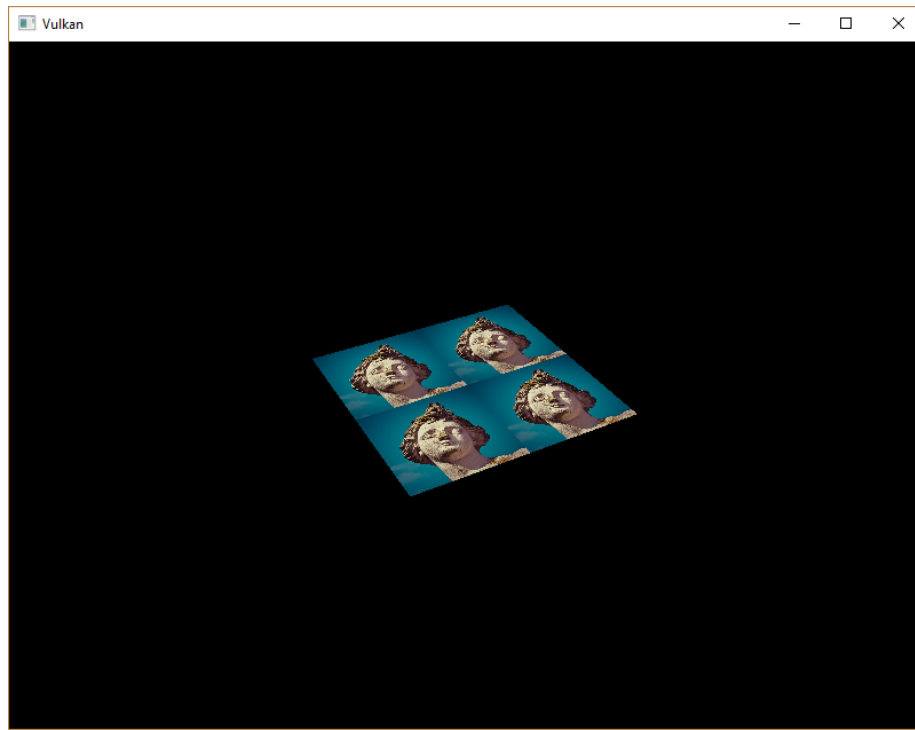


Try experimenting with the addressing modes by scaling the texture coordinates to values higher than 1. For example, the following fragment shader produces the result in the image below when using `VK_SAMPLER_ADDRESS_MODE_REPEAT`:

```

1 void main() {
2     outColor = texture(texSampler, fragTexCoord * 2.0);
3 }

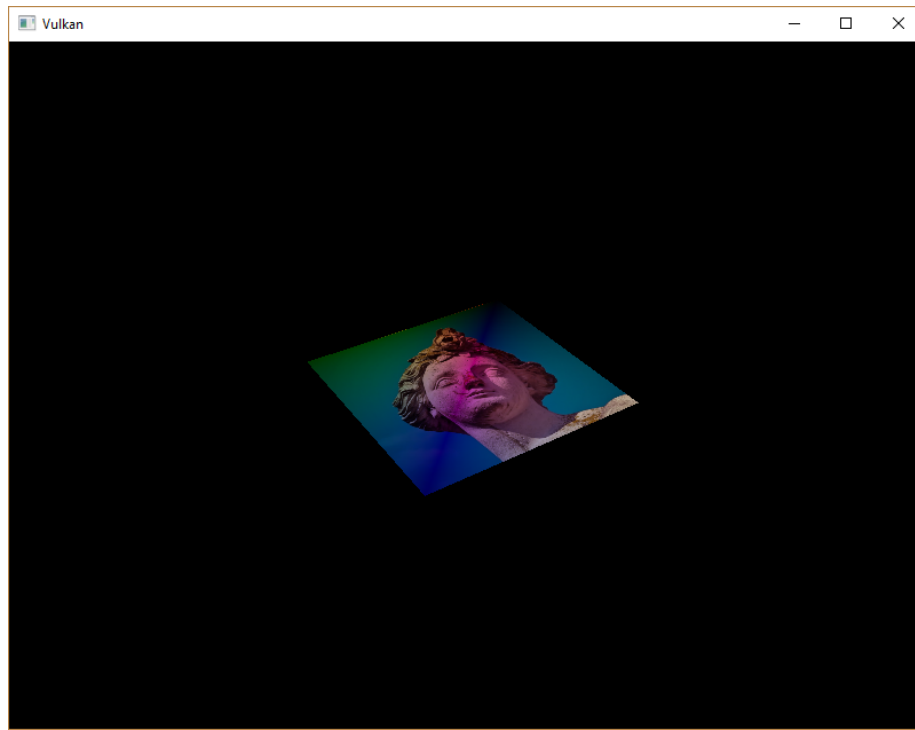
```



You can also manipulate the texture colors using the vertex colors:

```
1 void main() {  
2     outColor = vec4(fragColor * texture(texSampler,  
3         fragTexCoord).rgb, 1.0);  
}
```

I've separated the RGB and alpha channels here to not scale the alpha channel.



You now know how to access images in shaders! This is a very powerful technique when combined with images that are also written to in framebuffers. You can use these images as inputs to implement cool effects like post-processing and camera displays within the 3D world.

C++ code / Vertex shader / Fragment shader

Depth buffering

Introduction

The geometry we've worked with so far is projected into 3D, but it's still completely flat. In this chapter we're going to add a Z coordinate to the position to prepare for 3D meshes. We'll use this third coordinate to place a square over the current square to see a problem that arises when geometry is not sorted by depth.

3D geometry

Change the `Vertex` struct to use a 3D vector for the position, and update the format in the corresponding `VkVertexInputAttributeDescription`:

```
1 struct Vertex {
2     glm::vec3 pos;
3     glm::vec3 color;
4     glm::vec2 texCoord;
5
6     ...
7
8     static std::array<VkVertexInputAttributeDescription, 3>
        getAttributeDescriptions() {
9         std::array<VkVertexInputAttributeDescription, 3>
            attributeDescriptions{};
10
11         attributeDescriptions[0].binding = 0;
12         attributeDescriptions[0].location = 0;
13         attributeDescriptions[0].format = VK_FORMAT_R32G32B32_SFLOAT;
14         attributeDescriptions[0].offset = offsetof(Vertex, pos);
15
16         ...
17     }
18 };
```

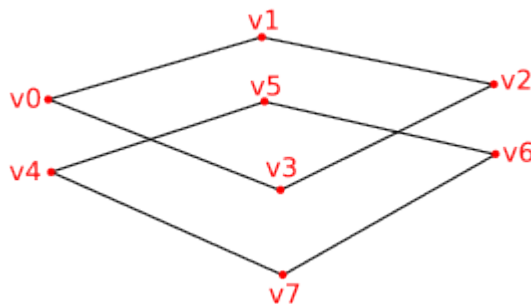
Next, update the vertex shader to accept and transform 3D coordinates as input. Don't forget to recompile it afterwards!

```
1 layout(location = 0) in vec3 inPosition;
2
3 ...
4
5 void main() {
6     gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition,
7         1.0);
8     fragColor = inColor;
9     fragTexCoord = inTexCoord;
10 }
```

Lastly, update the `vertices` container to include Z coordinates:

```
1 const std::vector<Vertex> vertices = {
2     {{-0.5f, -0.5f, 0.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 0.0f}},
3     {{0.5f, -0.5f, 0.0f}, {0.0f, 1.0f, 0.0f}, {1.0f, 0.0f}},
4     {{0.5f, 0.5f, 0.0f}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f}},
5     {{-0.5f, 0.5f, 0.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 1.0f}}
6 };
```

If you run your application now, then you should see exactly the same result as before. It's time to add some extra geometry to make the scene more interesting, and to demonstrate the problem that we're going to tackle in this chapter. Duplicate the vertices to define positions for a square right under the current one like this:



Use Z coordinates of `-0.5f` and add the appropriate indices for the extra square:

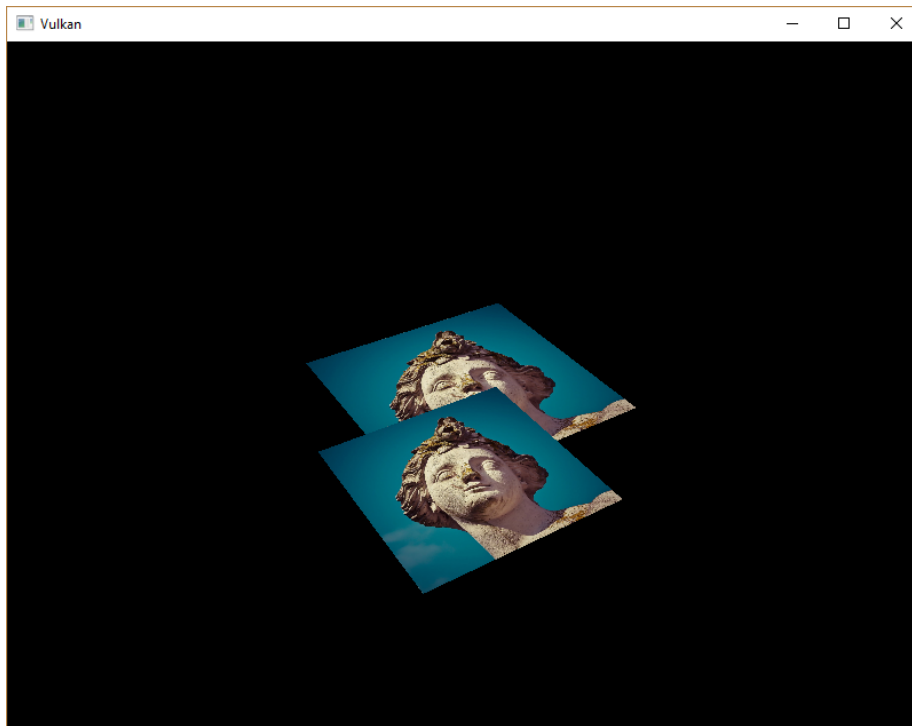
```
1 const std::vector<Vertex> vertices = {
2     {{-0.5f, -0.5f, 0.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 0.0f}},
```

```

3     {{0.5f, -0.5f, 0.0f}, {0.0f, 1.0f, 0.0f}, {1.0f, 0.0f}},
4     {{0.5f, 0.5f, 0.0f}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f}},
5     {{-0.5f, 0.5f, 0.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 1.0f}},
6
7     {{-0.5f, -0.5f, -0.5f}, {1.0f, 0.0f, 0.0f}, {0.0f, 0.0f}},
8     {{0.5f, -0.5f, -0.5f}, {0.0f, 1.0f, 0.0f}, {1.0f, 0.0f}},
9     {{0.5f, 0.5f, -0.5f}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f}},
10    {{-0.5f, 0.5f, -0.5f}, {1.0f, 1.0f, 1.0f}, {0.0f, 1.0f}}
11 };
12
13 const std::vector<uint16_t> indices = {
14     0, 1, 2, 2, 3, 0,
15     4, 5, 6, 6, 7, 4
16 };

```

Run your program now and you'll see something resembling an Escher illustration:



The problem is that the fragments of the lower square are drawn over the fragments of the upper square, simply because it comes later in the index array. There are two ways to solve this:

- Sort all of the draw calls by depth from back to front

- Use depth testing with a depth buffer

The first approach is commonly used for drawing transparent objects, because order-independent transparency is a difficult challenge to solve. However, the problem of ordering fragments by depth is much more commonly solved using a *depth buffer*. A depth buffer is an additional attachment that stores the depth for every position, just like the color attachment stores the color of every position. Every time the rasterizer produces a fragment, the depth test will check if the new fragment is closer than the previous one. If it isn't, then the new fragment is discarded. A fragment that passes the depth test writes its own depth to the depth buffer. It is possible to manipulate this value from the fragment shader, just like you can manipulate the color output.

```
1 #define GLM_FORCE_RADIANS
2 #define GLM_FORCE_DEPTH_ZERO_TO_ONE
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
```

The perspective projection matrix generated by GLM will use the OpenGL depth range of -1.0 to 1.0 by default. We need to configure it to use the Vulkan range of 0.0 to 1.0 using the `GLM_FORCE_DEPTH_ZERO_TO_ONE` definition.

Depth image and view

A depth attachment is based on an image, just like the color attachment. The difference is that the swap chain will not automatically create depth images for us. We only need a single depth image, because only one draw operation is running at once. The depth image will again require the trifecta of resources: image, memory and image view.

```
1 VkImage depthImage;
2 VkDeviceMemory depthImageMemory;
3 VkImageView depthImageView;
```

Create a new function `createDepthResources` to set up these resources:

```
1 void initVulkan() {
2     ...
3     createCommandPool();
4     createDepthResources();
5     createTextureImage();
6     ...
7 }
8
9 ...
10
11 void createDepthResources() {
```

```
12
13 }
```

Creating a depth image is fairly straightforward. It should have the same resolution as the color attachment, defined by the swap chain extent, an image usage appropriate for a depth attachment, optimal tiling and device local memory. The only question is: what is the right format for a depth image? The format must contain a depth component, indicated by `_D??_` in the `VK_FORMAT_`.

Unlike the texture image, we don't necessarily need a specific format, because we won't be directly accessing the texels from the program. It just needs to have a reasonable accuracy, at least 24 bits is common in real-world applications. There are several formats that fit this requirement:

- `VK_FORMAT_D32_SFLOAT`: 32-bit float for depth
- `VK_FORMAT_D32_SFLOAT_S8_UINT`: 32-bit signed float for depth and 8 bit stencil component
- `VK_FORMAT_D24_UNORM_S8_UINT`: 24-bit float for depth and 8 bit stencil component

The stencil component is used for stencil tests, which is an additional test that can be combined with depth testing. We'll look at this in a future chapter.

We could simply go for the `VK_FORMAT_D32_SFLOAT` format, because support for it is extremely common (see the hardware database), but it's nice to add some extra flexibility to our application where possible. We're going to write a function `findSupportedFormat` that takes a list of candidate formats in order from most desirable to least desirable, and checks which is the first one that is supported:

```
1 VkFormat findSupportedFormat(const std::vector<VkFormat>&
    candidates, VkImageTiling tiling, VkFormatFeatureFlags features)
    {
2
3 }
```

The support of a format depends on the tiling mode and usage, so we must also include these as parameters. The support of a format can be queried using the `vkGetPhysicalDeviceFormatProperties` function:

```
1 for (VkFormat format : candidates) {
2     VkFormatProperties props;
3     vkGetPhysicalDeviceFormatProperties(physicalDevice, format,
        &props);
4 }
```

The `VkFormatProperties` struct contains three fields:

- `linearTilingFeatures`: Use cases that are supported with linear tiling

- `optimalTilingFeatures`: Use cases that are supported with optimal tiling
- `bufferFeatures`: Use cases that are supported for buffers

Only the first two are relevant here, and the one we check depends on the `tiling` parameter of the function:

```
1 if (tiling == VK_IMAGE_TILING_LINEAR && (props.linearTilingFeatures
    & features) == features) {
2     return format;
3 } else if (tiling == VK_IMAGE_TILING_OPTIMAL &&
    (props.optimalTilingFeatures & features) == features) {
4     return format;
5 }
```

If none of the candidate formats support the desired usage, then we can either return a special value or simply throw an exception:

```
1 VkFormat findSupportedFormat(const std::vector<VkFormat>&
    candidates, VkImageTiling tiling, VkFormatFeatureFlags features)
    {
2     for (VkFormat format : candidates) {
3         VkFormatProperties props;
4         vkGetPhysicalDeviceFormatProperties(physicalDevice, format,
            &props);
5
6         if (tiling == VK_IMAGE_TILING_LINEAR &&
            (props.linearTilingFeatures & features) == features) {
7             return format;
8         } else if (tiling == VK_IMAGE_TILING_OPTIMAL &&
            (props.optimalTilingFeatures & features) == features) {
9             return format;
10        }
11    }
12
13    throw std::runtime_error("failed to find supported format!");
14 }
```

We'll use this function now to create a `findDepthFormat` helper function to select a format with a depth component that supports usage as depth attachment:

```
1 VkFormat findDepthFormat() {
2     return findSupportedFormat(
3         {VK_FORMAT_D32_SFLOAT, VK_FORMAT_D32_SFLOAT_S8_UINT,
4          VK_FORMAT_D24_UNORM_S8_UINT},
5         VK_IMAGE_TILING_OPTIMAL,
6         VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
7     );
8 }
```

```
7 }
```

Make sure to use the `VK_FORMAT_FEATURE_` flag instead of `VK_IMAGE_USAGE_` in this case. All of these candidate formats contain a depth component, but the latter two also contain a stencil component. We won't be using that yet, but we do need to take that into account when performing layout transitions on images with these formats. Add a simple helper function that tells us if the chosen depth format contains a stencil component:

```
1 bool hasStencilComponent(VkFormat format) {  
2     return format == VK_FORMAT_D32_SFLOAT_S8_UINT || format ==  
        VK_FORMAT_D24_UNORM_S8_UINT;  
3 }
```

Call the function to find a depth format from `createDepthResources`:

```
1 VkFormat depthFormat = findDepthFormat();
```

We now have all the required information to invoke our `createImage` and `createImageView` helper functions:

```
1 createImage(swapChainExtent.width, swapChainExtent.height,  
    depthFormat, VK_IMAGE_TILING_OPTIMAL,  
    VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,  
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, depthImage,  
    depthImageMemory);  
2 depthImageView = createImageView(depthImage, depthFormat);
```

However, the `createImageView` function currently assumes that the subresource is always the `VK_IMAGE_ASPECT_COLOR_BIT`, so we will need to turn that field into a parameter:

```
1 VkImageView createImageView(VkImage image, VkFormat format,  
    VkImageAspectFlags aspectFlags) {  
2     ...  
3     viewInfo.subresourceRange.aspectMask = aspectFlags;  
4     ...  
5 }
```

Update all calls to this function to use the right aspect:

```
1 swapChainImageViews[i] = createImageView(swapChainImages[i],  
    swapChainImageFormat, VK_IMAGE_ASPECT_COLOR_BIT);  
2 ...  
3 depthImageView = createImageView(depthImage, depthFormat,  
    VK_IMAGE_ASPECT_DEPTH_BIT);  
4 ...  
5 textureImageView = createImageView(textureImage,  
    VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_ASPECT_COLOR_BIT);
```

That's it for creating the depth image. We don't need to map it or copy another image to it, because we're going to clear it at the start of the render pass like the color attachment.

Explicitly transitioning the depth image

We don't need to explicitly transition the layout of the image to a depth attachment because we'll take care of this in the render pass. However, for completeness I'll still describe the process in this section. You may skip it if you like.

Make a call to `transitionImageLayout` at the end of the `createDepthResources` function like so:

```
1 transitionImageLayout(depthImage, depthFormat,  
    VK_IMAGE_LAYOUT_UNDEFINED,  
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL);
```

The undefined layout can be used as initial layout, because there are no existing depth image contents that matter. We need to update some of the logic in `transitionImageLayout` to use the right subresource aspect:

```
1 if (newLayout == VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL) {  
2     barrier.subresourceRange.aspectMask = VK_IMAGE_ASPECT_DEPTH_BIT;  
3  
4     if (hasStencilComponent(format)) {  
5         barrier.subresourceRange.aspectMask |=  
            VK_IMAGE_ASPECT_STENCIL_BIT;  
6     }  
7 } else {  
8     barrier.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;  
9 }
```

Although we're not using the stencil component, we do need to include it in the layout transitions of the depth image.

Finally, add the correct access masks and pipeline stages:

```
1 if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED && newLayout ==  
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL) {  
2     barrier.srcAccessMask = 0;  
3     barrier.dstAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;  
4  
5     sourceStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;  
6     destinationStage = VK_PIPELINE_STAGE_TRANSFER_BIT;  
7 } else if (oldLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL &&  
    newLayout == VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL) {  
8     barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
```

```

9     barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
10
11     sourceStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
12     destinationStage = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT;
13 } else if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED && newLayout ==
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL) {
14     barrier.srcAccessMask = 0;
15     barrier.dstAccessMask =
        VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
        VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;
16
17     sourceStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;
18     destinationStage = VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT;
19 } else {
20     throw std::invalid_argument("unsupported layout transition!");
21 }

```

The depth buffer will be read from to perform depth tests to see if a fragment is visible, and will be written to when a new fragment is drawn. The reading happens in the `VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT` stage and the writing in the `VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT`. You should pick the earliest pipeline stage that matches the specified operations, so that it is ready for usage as depth attachment when it needs to be.

Render pass

We're now going to modify `createRenderPass` to include a depth attachment. First specify the `VkAttachmentDescription`:

```

1 VkAttachmentDescription depthAttachment{};
2 depthAttachment.format = findDepthFormat();
3 depthAttachment.samples = VK_SAMPLE_COUNT_1_BIT;
4 depthAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;
5 depthAttachment.storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
6 depthAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;
7 depthAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
8 depthAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
9 depthAttachment.finalLayout =
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

```

The `format` should be the same as the depth image itself. This time we don't care about storing the depth data (`storeOp`), because it will not be used after drawing has finished. This may allow the hardware to perform additional optimizations. Just like the color buffer, we don't care about the previous depth contents, so we can use `VK_IMAGE_LAYOUT_UNDEFINED` as `initialLayout`.

```

1 VkAttachmentReference depthAttachmentRef{};
2 depthAttachmentRef.attachment = 1;
3 depthAttachmentRef.layout =
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

```

Add a reference to the attachment for the first (and only) subpass:

```

1 VkSubpassDescription subpass{};
2 subpass.pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS;
3 subpass.colorAttachmentCount = 1;
4 subpass.pColorAttachments = &colorAttachmentRef;
5 subpass.pDepthStencilAttachment = &depthAttachmentRef;

```

Unlike color attachments, a subpass can only use a single depth (+stencil) attachment. It wouldn't really make any sense to do depth tests on multiple buffers.

```

1 std::array<VkAttachmentDescription, 2> attachments =
    {colorAttachment, depthAttachment};
2 VkRenderPassCreateInfo renderPassInfo{};
3 renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO;
4 renderPassInfo.attachmentCount =
    static_cast<uint32_t>(attachments.size());
5 renderPassInfo.pAttachments = attachments.data();
6 renderPassInfo.subpassCount = 1;
7 renderPassInfo.pSubpasses = &subpass;
8 renderPassInfo.dependencyCount = 1;
9 renderPassInfo.pDependencies = &dependency;

```

Next, update the `VkSubpassDependency` struct to refer to both attachments.

```

1 dependency.srcStageMask =
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT |
    VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT;
2 dependency.srcAccessMask =
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;
3 dependency.dstStageMask =
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT |
    VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT;
4 dependency.dstAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT |
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;

```

Finally, we need to extend our subpass dependencies to make sure that there is no conflict between the transitioning of the depth image and it being cleared as part of its load operation. The depth image is first accessed in the early fragment test pipeline stage and because we have a load operation that *clears*, we should specify the access mask for writes.

Framebuffer

The next step is to modify the framebuffer creation to bind the depth image to the depth attachment. Go to `createFramebuffers` and specify the depth image view as second attachment:

```
1 std::array<VkImageView, 2> attachments = {
2     swapChainImageViews[i],
3     depthImageView
4 };
5
6 VkFramebufferCreateInfo framebufferInfo{};
7 framebufferInfo.sType = VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO;
8 framebufferInfo.renderPass = renderPass;
9 framebufferInfo.attachmentCount =
10     static_cast<uint32_t>(attachments.size());
11 framebufferInfo.pAttachments = attachments.data();
12 framebufferInfo.width = swapChainExtent.width;
13 framebufferInfo.height = swapChainExtent.height;
14 framebufferInfo.layers = 1;
```

The color attachment differs for every swap chain image, but the same depth image can be used by all of them because only a single subpass is running at the same time due to our semaphores.

You'll also need to move the call to `createFramebuffers` to make sure that it is called after the depth image view has actually been created:

```
1 void initVulkan() {
2     ...
3     createDepthResources();
4     createFramebuffers();
5     ...
6 }
```

Clear values

Because we now have multiple attachments with `VK_ATTACHMENT_LOAD_OP_CLEAR`, we also need to specify multiple clear values. Go to `recordCommandBuffer` and create an array of `VkClearColor` structs:

```
1 std::array<VkClearColor, 2> clearValues{};
2 clearValues[0].color = {0.0f, 0.0f, 0.0f, 1.0f};
3 clearValues[1].depthStencil = {1.0f, 0};
4
5 renderPassInfo.clearValueCount =
6     static_cast<uint32_t>(clearValues.size());
```

```
6 renderPassInfo.pClearValues = clearValues.data();
```

The range of depths in the depth buffer is 0.0 to 1.0 in Vulkan, where 1.0 lies at the far view plane and 0.0 at the near view plane. The initial value at each point in the depth buffer should be the furthest possible depth, which is 1.0.

Note that the order of `clearValues` should be identical to the order of your attachments.

Depth and stencil state

The depth attachment is ready to be used now, but depth testing still needs to be enabled in the graphics pipeline. It is configured through the `VkPipelineDepthStencilStateCreateInfo` struct:

```
1 VkPipelineDepthStencilStateCreateInfo depthStencil{};
2 depthStencil.sType =
    VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO;
3 depthStencil.depthTestEnable = VK_TRUE;
4 depthStencil.depthWriteEnable = VK_TRUE;
```

The `depthTestEnable` field specifies if the depth of new fragments should be compared to the depth buffer to see if they should be discarded. The `depthWriteEnable` field specifies if the new depth of fragments that pass the depth test should actually be written to the depth buffer.

```
1 depthStencil.depthCompareOp = VK_COMPARE_OP_LESS;
```

The `depthCompareOp` field specifies the comparison that is performed to keep or discard fragments. We're sticking to the convention of lower depth = closer, so the depth of new fragments should be *less*.

```
1 depthStencil.depthBoundsTestEnable = VK_FALSE;
2 depthStencil.minDepthBounds = 0.0f; // Optional
3 depthStencil.maxDepthBounds = 1.0f; // Optional
```

The `depthBoundsTestEnable`, `minDepthBounds` and `maxDepthBounds` fields are used for the optional depth bound test. Basically, this allows you to only keep fragments that fall within the specified depth range. We won't be using this functionality.

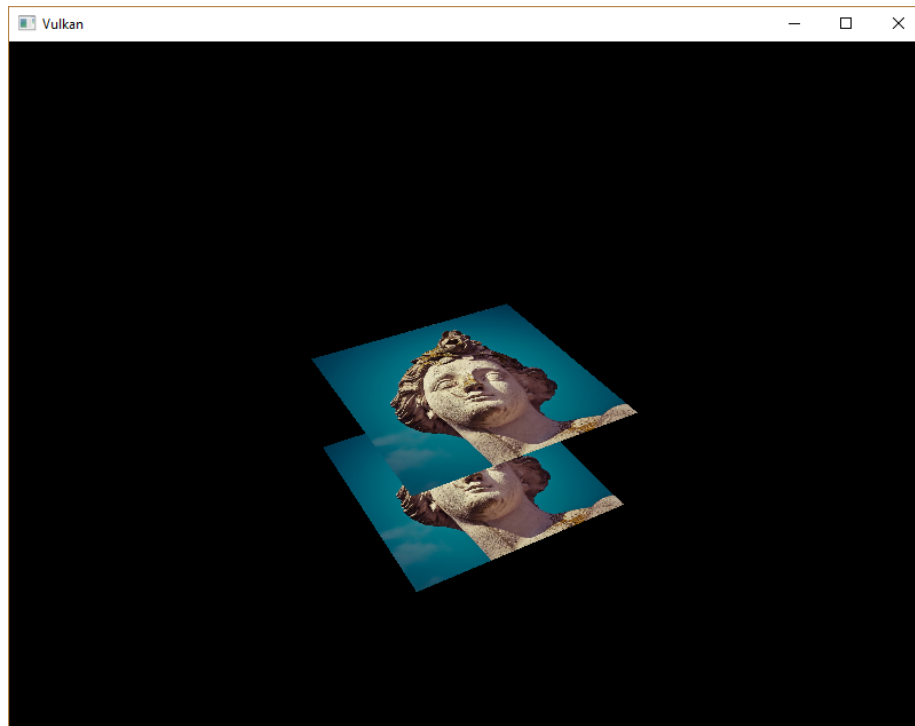
```
1 depthStencil.stencilTestEnable = VK_FALSE;
2 depthStencil.front = {}; // Optional
3 depthStencil.back = {}; // Optional
```

The last three fields configure stencil buffer operations, which we also won't be using in this tutorial. If you want to use these operations, then you will have to make sure that the format of the depth/stencil image contains a stencil component.

```
1 pipelineInfo.pDepthStencilState = &depthStencil;
```

Update the `VkGraphicsPipelineCreateInfo` struct to reference the depth stencil state we just filled in. A depth stencil state must always be specified if the render pass contains a depth stencil attachment.

If you run your program now, then you should see that the fragments of the geometry are now correctly ordered:



Handling window resize

The resolution of the depth buffer should change when the window is resized to match the new color attachment resolution. Extend the `recreateSwapChain` function to recreate the depth resources in that case:

```
1 void recreateSwapChain() {
2     int width = 0, height = 0;
3     while (width == 0 || height == 0) {
4         glfwGetFramebufferSize(window, &width, &height);
5         glfwWaitEvents();
6     }
7
8     vkDeviceWaitIdle(device);
```

```

9
10     cleanupSwapChain();
11
12     createSwapChain();
13     createImageViews();
14     createDepthResources();
15     createFramebuffers();
16 }

```

The cleanup operations should happen in the swap chain cleanup function:

```

1 void cleanupSwapChain() {
2     vkDestroyImageView(device, depthImageView, nullptr);
3     vkDestroyImage(device, depthImage, nullptr);
4     vkFreeMemory(device, depthImageMemory, nullptr);
5
6     ...
7 }

```

Congratulations, your application is now finally ready to render arbitrary 3D geometry and have it look right. We're going to try this out in the next chapter by drawing a textured model!

C++ code / Vertex shader / Fragment shader

Loading models

Introduction

Your program is now ready to render textured 3D meshes, but the current geometry in the `vertices` and `indices` arrays is not very interesting yet. In this chapter we're going to extend the program to load the vertices and indices from an actual model file to make the graphics card actually do some work.

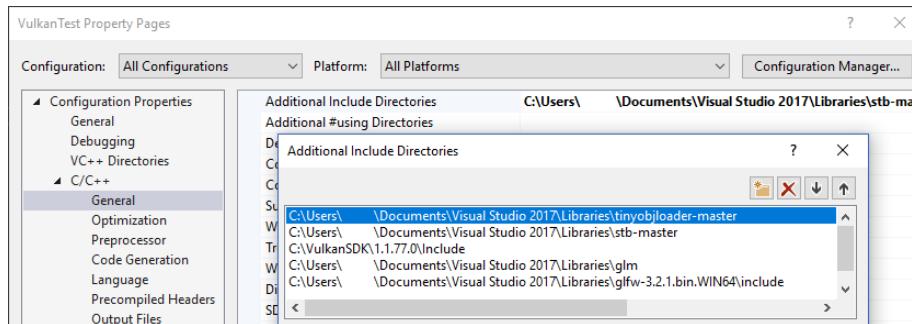
Many graphics API tutorials have the reader write their own OBJ loader in a chapter like this. The problem with this is that any remotely interesting 3D application will soon require features that are not supported by this file format, like skeletal animation. We *will* load mesh data from an OBJ model in this chapter, but we'll focus more on integrating the mesh data with the program itself rather than the details of loading it from a file.

Library

We will use the `tinyobjloader` library to load vertices and faces from an OBJ file. It's fast and it's easy to integrate because it's a single file library like `stb_image`. Go to the repository linked above and download the `tiny_obj_loader.h` file to a folder in your library directory.

Visual Studio

Add the directory with `tiny_obj_loader.h` in it to the `Additional Include Directories` paths.



Makefile

Add the directory with `tiny_obj_loader.h` to the include directories for GCC:

```
1 VULKAN_SDK_PATH = /home/user/VulkanSDK/x.x.x.x/x86_64
2 STB_INCLUDE_PATH = /home/user/libraries/stb
3 TINYOBJ_INCLUDE_PATH = /home/user/libraries/tinyobjloader
4
5 ...
6
7 CFLAGS = -std=c++17 -I$(VULKAN_SDK_PATH)/include
      -I$(STB_INCLUDE_PATH) -I$(TINYOBJ_INCLUDE_PATH)
```

Sample mesh

In this chapter we won't be enabling lighting yet, so it helps to use a sample model that has lighting baked into the texture. An easy way to find such models is to look for 3D scans on Sketchfab. Many of the models on that site are available in OBJ format with a permissive license.

For this tutorial I've decided to go with the Viking room model by nigelgoh (CC BY 4.0). I tweaked the size and orientation of the model to use it as a drop in replacement for the current geometry:

- `viking_room.obj`
- `viking_room.png`

Feel free to use your own model, but make sure that it only consists of one material and that it has dimensions of about 1.5 x 1.5 x 1.5 units. If it is larger than that, then you'll have to change the view matrix. Put the model file in a new `models` directory next to `shaders` and `textures`, and put the texture image in the `textures` directory.

Put two new configuration variables in your program to define the model and texture paths:

```
1 const uint32_t WIDTH = 800;
```

```

2 const uint32_t HEIGHT = 600;
3
4 const std::string MODEL_PATH = "models/viking_room.obj";
5 const std::string TEXTURE_PATH = "textures/viking_room.png";

```

And update `createTextureImage` to use this path variable:

```

1 stbi_uc* pixels = stbi_load(TEXTURE_PATH.c_str(), &texWidth,
    &texHeight, &texChannels, STBI_rgb_alpha);

```

Loading vertices and indices

We're going to load the vertices and indices from the model file now, so you should remove the global `vertices` and `indices` arrays now. Replace them with non-const containers as class members:

```

1 std::vector<Vertex> vertices;
2 std::vector<uint32_t> indices;
3 VkBuffer vertexBuffer;
4 VkDeviceMemory vertexBufferMemory;

```

You should change the type of the indices from `uint16_t` to `uint32_t`, because there are going to be a lot more vertices than 65535. Remember to also change the `vkCmdBindIndexBuffer` parameter:

```

1 vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0,
    VK_INDEX_TYPE_UINT32);

```

The `tinyobjloader` library is included in the same way as STB libraries. Include the `tiny_obj_loader.h` file and make sure to define `TINYOBJLOADER_IMPLEMENTATION` in one source file to include the function bodies and avoid linker errors:

```

1 #define TINYOBJLOADER_IMPLEMENTATION
2 #include <tiny_obj_loader.h>

```

We're now going to write a `loadModel` function that uses this library to populate the `vertices` and `indices` containers with the vertex data from the mesh. It should be called somewhere before the vertex and index buffers are created:

```

1 void initVulkan() {
2     ...
3     loadModel();
4     createVertexBuffer();
5     createIndexBuffer();
6     ...
7 }
8

```

```

9 ...
10
11 void loadModel() {
12
13 }

```

A model is loaded into the library's data structures by calling the `tinyobj::LoadObj` function:

```

1 void loadModel() {
2     tinyobj::attrib_t attrib;
3     std::vector<tinyobj::shape_t> shapes;
4     std::vector<tinyobj::material_t> materials;
5     std::string err;
6
7     if (!tinyobj::LoadObj(&attrib, &shapes, &materials, &err,
8         MODEL_PATH.c_str())) {
9         throw std::runtime_error(err);
10    }
11 }

```

An OBJ file consists of positions, normals, texture coordinates and faces. Faces consist of an arbitrary amount of vertices, where each vertex refers to a position, normal and/or texture coordinate by index. This makes it possible to not just reuse entire vertices, but also individual attributes.

The `attrib` container holds all of the positions, normals and texture coordinates in its `attrib.vertices`, `attrib.normals` and `attrib.texcoords` vectors. The `shapes` container contains all of the separate objects and their faces. Each face consists of an array of vertices, and each vertex contains the indices of the position, normal and texture coordinate attributes. OBJ models can also define a material and texture per face, but we will be ignoring those.

The `err` string contains errors and the `warn` string contains warnings that occurred while loading the file, like a missing material definition. Loading only really failed if the `LoadObj` function returns `false`. As mentioned above, faces in OBJ files can actually contain an arbitrary number of vertices, whereas our application can only render triangles. Luckily the `LoadObj` has an optional parameter to automatically triangulate such faces, which is enabled by default.

We're going to combine all of the faces in the file into a single model, so just iterate over all of the shapes:

```

1 for (const auto& shape : shapes) {
2
3 }

```

The triangulation feature has already made sure that there are three vertices per face, so we can now directly iterate over the vertices and dump them straight into our `vertices` vector:

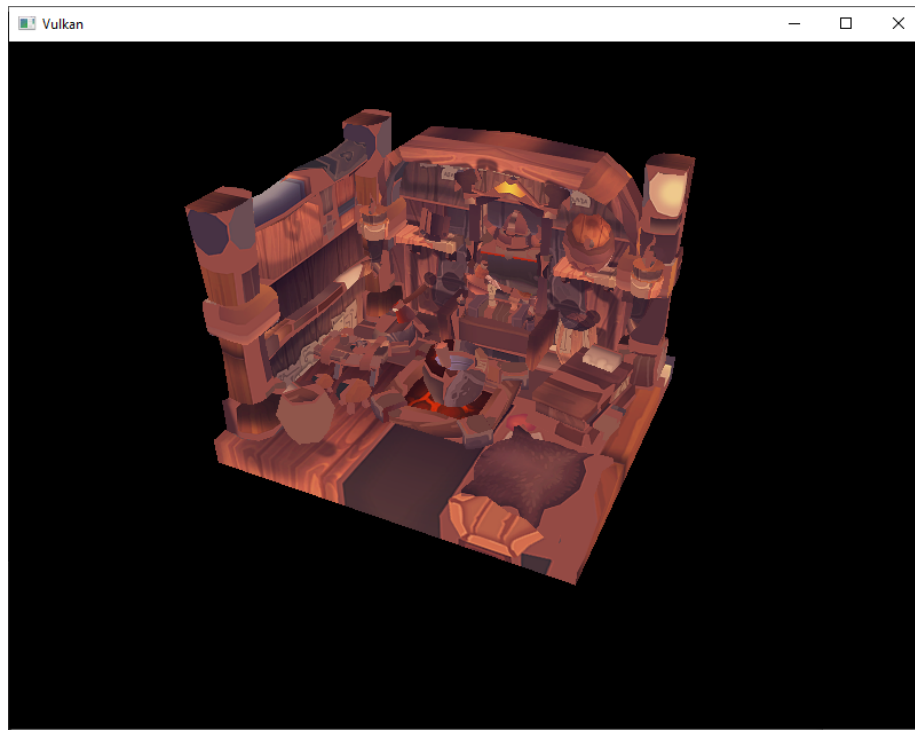
```
1 for (const auto& shape : shapes) {
2     for (const auto& index : shape.mesh.indices) {
3         Vertex vertex{};
4
5         vertices.push_back(vertex);
6         indices.push_back(indices.size());
7     }
8 }
```

For simplicity, we will assume that every vertex is unique for now, hence the simple auto-increment indices. The `index` variable is of type `tinyobj::index_t`, which contains the `vertex_index`, `normal_index` and `texcoord_index` members. We need to use these indices to look up the actual vertex attributes in the `attrib` arrays:

```
1 vertex.pos = {
2     attrib.vertices[3 * index.vertex_index + 0],
3     attrib.vertices[3 * index.vertex_index + 1],
4     attrib.vertices[3 * index.vertex_index + 2]
5 };
6
7 vertex.texCoord = {
8     attrib.texcoords[2 * index.texcoord_index + 0],
9     attrib.texcoords[2 * index.texcoord_index + 1]
10 };
11
12 vertex.color = {1.0f, 1.0f, 1.0f};
```

Unfortunately the `attrib.vertices` array is an array of `float` values instead of something like `glm::vec3`, so you need to multiply the index by 3. Similarly, there are two texture coordinate components per entry. The offsets of 0, 1 and 2 are used to access the X, Y and Z components, or the U and V components in the case of texture coordinates.

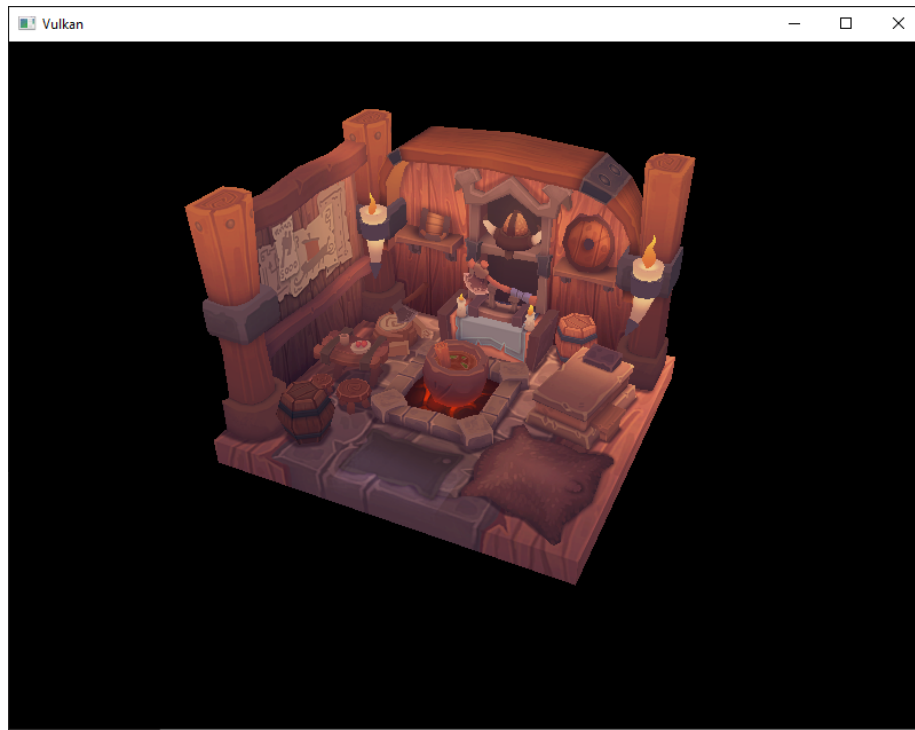
Run your program now with optimization enabled (e.g. **Release** mode in Visual Studio and with the `-O3` compiler flag for GCC). This is necessary, because otherwise loading the model will be very slow. You should see something like the following:



Great, the geometry looks correct, but what's going on with the texture? The OBJ format assumes a coordinate system where a vertical coordinate of 0 means the bottom of the image, however we've uploaded our image into Vulkan in a top to bottom orientation where 0 means the top of the image. Solve this by flipping the vertical component of the texture coordinates:

```
1 vertex.texCoord = {  
2     attrib.texcoords[2 * index.texcoord_index + 0],  
3     1.0f - attrib.texcoords[2 * index.texcoord_index + 1]  
4 };
```

When you run your program again, you should now see the correct result:



All that hard work is finally beginning to pay off with a demo like this!

As the model rotates you may notice that the rear (backside of the walls) looks a bit funny. This is normal and is simply because the model is not really designed to be viewed from that side.

Vertex deduplication

Unfortunately we're not really taking advantage of the index buffer yet. The `vertices` vector contains a lot of duplicated vertex data, because many vertices are included in multiple triangles. We should keep only the unique vertices and use the index buffer to reuse them whenever they come up. A straightforward way to implement this is to use a `map` or `unordered_map` to keep track of the unique vertices and respective indices:

```
1 #include <unordered_map>
2
3 ...
4
5 std::unordered_map<Vertex, uint32_t> uniqueVertices{};
6
7 for (const auto& shape : shapes) {
8     for (const auto& index : shape.mesh.indices) {
```

```

9     Vertex vertex{};
10
11     ...
12
13     if (uniqueVertices.count(vertex) == 0) {
14         uniqueVertices[vertex] =
15             static_cast<uint32_t>(vertices.size());
16         vertices.push_back(vertex);
17     }
18     indices.push_back(uniqueVertices[vertex]);
19 }
20 }

```

Every time we read a vertex from the OBJ file, we check if we've already seen a vertex with the exact same position and texture coordinates before. If not, we add it to `vertices` and store its index in the `uniqueVertices` container. After that we add the index of the new vertex to `indices`. If we've seen the exact same vertex before, then we look up its index in `uniqueVertices` and store that index in `indices`.

The program will fail to compile right now, because using a user-defined type like our `Vertex` struct as key in a hash table requires us to implement two functions: equality test and hash calculation. The former is easy to implement by overriding the `==` operator in the `Vertex` struct:

```

1 bool operator==(const Vertex& other) const {
2     return pos == other.pos && color == other.color && texCoord ==
3         other.texCoord;
4 }

```

A hash function for `Vertex` is implemented by specifying a template specialization for `std::hash<T>`. Hash functions are a complex topic, but [cppreference.com](http://en.cppreference.com) recommends the following approach combining the fields of a struct to create a decent quality hash function:

```

1 namespace std {
2     template<> struct hash<Vertex> {
3         size_t operator()(Vertex const& vertex) const {
4             return ((hash<glm::vec3>()(vertex.pos) ^
5                 (hash<glm::vec3>()(vertex.color) << 1)) >> 1) ^
6                 (hash<glm::vec2>()(vertex.texCoord) << 1);
7         }
8     };
9 }

```

This code should be placed outside the `Vertex` struct. The hash functions for the GLM types need to be included using the following header:

```
1 #define GLM_ENABLE_EXPERIMENTAL
2 #include <glm/gtx/hash.hpp>
```

The hash functions are defined in the `gtx` folder, which means that it is technically still an experimental extension to GLM. Therefore you need to define `GLM_ENABLE_EXPERIMENTAL` to use it. It means that the API could change with a new version of GLM in the future, but in practice the API is very stable.

You should now be able to successfully compile and run your program. If you check the size of `vertices`, then you'll see that it has shrunk down from 1,500,000 to 265,645! That means that each vertex is reused in an average number of ~6 triangles. This definitely saves us a lot of GPU memory.

C++ code / Vertex shader / Fragment shader

Generating Mipmaps

Introduction

Our program can now load and render 3D models. In this chapter, we will add one more feature, mipmap generation. Mipmaps are widely used in games and rendering software, and Vulkan gives us complete control over how they are created.

Mipmaps are precalculated, downscaled versions of an image. Each new image is half the width and height of the previous one. Mipmaps are used as a form of *Level of Detail* or *LOD*. Objects that are far away from the camera will sample their textures from the smaller mip images. Using smaller images increases the rendering speed and avoids artifacts such as Moiré patterns. An example of what mipmaps look like:

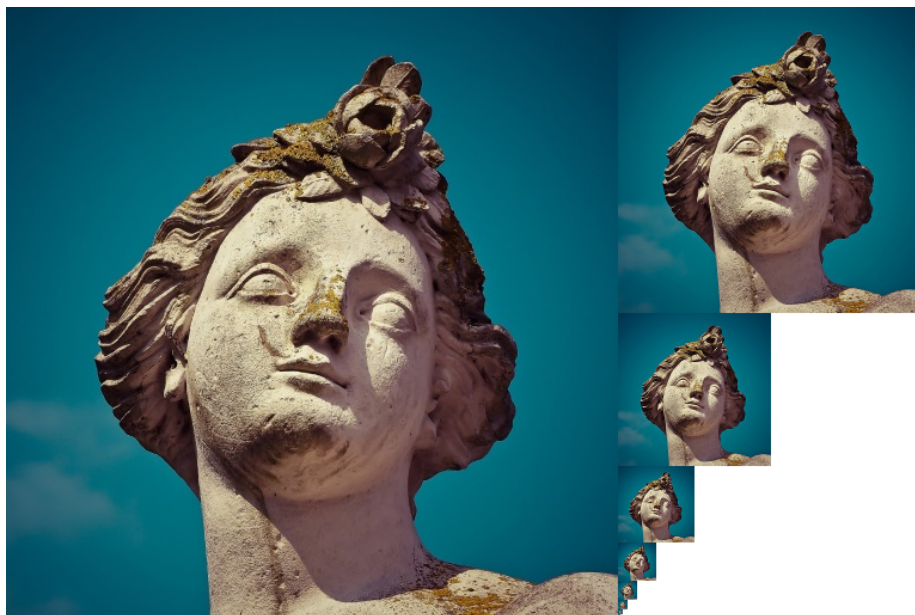


Image creation

In Vulkan, each of the mip images is stored in different *mip levels* of a `VkImage`. Mip level 0 is the original image, and the mip levels after level 0 are commonly referred to as the *mip chain*.

The number of mip levels is specified when the `VkImage` is created. Up until now, we have always set this value to one. We need to calculate the number of mip levels from the dimensions of the image. First, add a class member to store this number:

```
1 ...
2 uint32_t mipLevels;
3 VkImage textureImage;
4 ...
```

The value for `mipLevels` can be found once we've loaded the texture in `createTextureImage`:

```
1 int texWidth, texHeight, texChannels;
2 stbi_uc* pixels = stbi_load(TEXTURE_PATH.c_str(), &texWidth,
    &texHeight, &texChannels, STBI_rgb_alpha);
3 ...
4 mipLevels =
    static_cast<uint32_t>(std::floor(std::log2(std::max(texWidth,
    texHeight)))) + 1;
```

This calculates the number of levels in the mip chain. The `max` function selects the largest dimension. The `log2` function calculates how many times that dimension can be divided by 2. The `floor` function handles cases where the largest dimension is not a power of 2. 1 is added so that the original image has a mip level.

To use this value, we need to change the `createImage`, `createImageView`, and `transitionImageLayout` functions to allow us to specify the number of mip levels. Add a `mipLevels` parameter to the functions:

```
1 void createImage(uint32_t width, uint32_t height, uint32_t
    mipLevels, VkFormat format, VkImageTiling tiling,
    VkImageUsageFlags usage, VkMemoryPropertyFlags properties,
    VkImage& image, VkDeviceMemory& imageMemory) {
2     ...
3     imageInfo.mipLevels = mipLevels;
4     ...
5 }

1 VkImageView createImageView(VkImage image, VkFormat format,
    VkImageAspectFlags aspectFlags, uint32_t mipLevels) {
2     ...
```

```

3     viewInfo.subresourceRange.levelCount = mipLevels;
4     ...

1 void transitionImageLayout(VkImage image, VkFormat format,
    VkImageLayout oldLayout, VkImageLayout newLayout, uint32_t
    mipLevels) {
2     ...
3     barrier.subresourceRange.levelCount = mipLevels;
4     ...

```

Update all calls to these functions to use the right values:

```

1 createImage(swapChainExtent.width, swapChainExtent.height, 1,
    depthFormat, VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, depthImage,
    depthImageMemory);
2 ...
3 createImage(texWidth, texHeight, mipLevels, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_TILING_OPTIMAL, VK_IMAGE_USAGE_TRANSFER_DST_BIT |
    VK_IMAGE_USAGE_SAMPLED_BIT, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
    textureImage, textureImageMemory);

1 swapChainImageViews[i] = createImageView(swapChainImages[i],
    swapChainImageFormat, VK_IMAGE_ASPECT_COLOR_BIT, 1);
2 ...
3 depthImageView = createImageView(depthImage, depthFormat,
    VK_IMAGE_ASPECT_DEPTH_BIT, 1);
4 ...
5 textureImageView = createImageView(textureImage,
    VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_ASPECT_COLOR_BIT, mipLevels);

1 transitionImageLayout(depthImage, depthFormat,
    VK_IMAGE_LAYOUT_UNDEFINED,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL, 1);
2 ...
3 transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
    mipLevels);

```

Generating Mipmaps

Our texture image now has multiple mip levels, but the staging buffer can only be used to fill mip level 0. The other levels are still undefined. To fill these levels we need to generate the data from the single level that we have. We will

use the `vkCmdBlitImage` command. This command performs copying, scaling, and filtering operations. We will call this multiple times to *blit* data to each level of our texture image.

`vkCmdBlitImage` is considered a transfer operation, so we must inform Vulkan that we intend to use the texture image as both the source and destination of a transfer. Add `VK_IMAGE_USAGE_TRANSFER_SRC_BIT` to the texture image's usage flags in `createTextureImage`:

```
1 ...
2 createImage(texWidth, texHeight, mipLevels, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_TILING_OPTIMAL, VK_IMAGE_USAGE_TRANSFER_SRC_BIT |
    VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, textureImage,
    textureImageMemory);
3 ...
```

Like other image operations, `vkCmdBlitImage` depends on the layout of the image it operates on. We could transition the entire image to `VK_IMAGE_LAYOUT_GENERAL`, but this will most likely be slow. For optimal performance, the source image should be in `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` and the destination image should be in `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`. Vulkan allows us to transition each mip level of an image independently. Each blit will only deal with two mip levels at a time, so we can transition each level into the optimal layout between blits commands.

`transitionImageLayout` only performs layout transitions on the entire image, so we'll need to write a few more pipeline barrier commands. Remove the existing transition to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` in `createTextureImage`:

```
1 ...
2 transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
    mipLevels);
3 copyBufferToImage(stagingBuffer, textureImage,
    static_cast<uint32_t>(texWidth),
    static_cast<uint32_t>(texHeight));
4 //transitioned to VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL while
   generating mipmaps
5 ...
```

This will leave each level of the texture image in `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`. Each level will be transitioned to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` after the blit command reading from it is finished.

We're now going to write the function that generates the mipmaps:

```

1 void generateMipmaps(VkImage image, int32_t texWidth, int32_t
    texHeight, uint32_t mipLevels) {
2     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
3
4     VkImageMemoryBarrier barrier{};
5     barrier.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER;
6     barrier.image = image;
7     barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
8     barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
9     barrier.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
10    barrier.subresourceRange.baseArrayLayer = 0;
11    barrier.subresourceRange.layerCount = 1;
12    barrier.subresourceRange.levelCount = 1;
13
14    endSingleTimeCommands(commandBuffer);
15 }

```

We're going to make several transitions, so we'll reuse this `VkImageMemoryBarrier`. The fields set above will remain the same for all barriers. `subresourceRange.miplevel`, `oldLayout`, `newLayout`, `srcAccessMask`, and `dstAccessMask` will be changed for each transition.

```

1 int32_t mipWidth = texWidth;
2 int32_t mipHeight = texHeight;
3
4 for (uint32_t i = 1; i < mipLevels; i++) {
5
6 }

```

This loop will record each of the `VkCmdBlitImage` commands. Note that the loop variable starts at 1, not 0.

```

1 barrier.subresourceRange.baseMipLevel = i - 1;
2 barrier.oldLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL;
3 barrier.newLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL;
4 barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
5 barrier.dstAccessMask = VK_ACCESS_TRANSFER_READ_BIT;
6
7 vkCmdPipelineBarrier(commandBuffer,
8     VK_PIPELINE_STAGE_TRANSFER_BIT, VK_PIPELINE_STAGE_TRANSFER_BIT,
9     0,
10    0, nullptr,
11    0, nullptr,
12    1, &barrier);

```

First, we transition level `i - 1` to `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL`. This transition will wait for level `i - 1` to be filled, either from the previous

blit command, or from `vkCmdCopyBufferToImage`. The current blit command will wait on this transition.

```

1 VkImageBlit blit{};
2 blit.srcOffsets[0] = { 0, 0, 0 };
3 blit.srcOffsets[1] = { mipWidth, mipHeight, 1 };
4 blit.srcSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
5 blit.srcSubresource.mipLevel = i - 1;
6 blit.srcSubresource.baseArrayLayer = 0;
7 blit.srcSubresource.layerCount = 1;
8 blit.dstOffsets[0] = { 0, 0, 0 };
9 blit.dstOffsets[1] = { mipWidth > 1 ? mipWidth / 2 : 1, mipHeight >
    1 ? mipHeight / 2 : 1, 1 };
10 blit.dstSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
11 blit.dstSubresource.mipLevel = i;
12 blit.dstSubresource.baseArrayLayer = 0;
13 blit.dstSubresource.layerCount = 1;

```

Next, we specify the regions that will be used in the blit operation. The source mip level is `i - 1` and the destination mip level is `i`. The two elements of the `srcOffsets` array determine the 3D region that data will be blitted from. `dstOffsets` determines the region that data will be blitted to. The X and Y dimensions of the `dstOffsets[1]` are divided by two since each mip level is half the size of the previous level. The Z dimension of `srcOffsets[1]` and `dstOffsets[1]` must be 1, since a 2D image has a depth of 1.

```

1 vkCmdBlitImage(commandBuffer,
2     image, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
3     image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
4     1, &blit,
5     VK_FILTER_LINEAR);

```

Now, we record the blit command. Note that `textureImage` is used for both the `srcImage` and `dstImage` parameter. This is because we're blitting between different levels of the same image. The source mip level was just transitioned to `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` and the destination level is still in `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` from `createTextureImage`.

Beware if you are using a dedicated transfer queue (as suggested in Vertex buffers): `vkCmdBlitImage` must be submitted to a queue with graphics capability.

The last parameter allows us to specify a `VkFilter` to use in the blit. We have the same filtering options here that we had when making the `VkSampler`. We use the `VK_FILTER_LINEAR` to enable interpolation.

```

1 barrier.oldLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL;
2 barrier.newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;

```

```

3 barrier.srcAccessMask = VK_ACCESS_TRANSFER_READ_BIT;
4 barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
5
6 vkCmdPipelineBarrier(commandBuffer,
7     VK_PIPELINE_STAGE_TRANSFER_BIT,
8     VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT, 0,
9     0, nullptr,
10    0, nullptr,
11    1, &barrier);

```

This barrier transitions mip level $i - 1$ to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. This transition waits on the current blit command to finish. All sampling operations will wait on this transition to finish.

```

1     ...
2     if (mipWidth > 1) mipWidth /= 2;
3     if (mipHeight > 1) mipHeight /= 2;
4 }

```

At the end of the loop, we divide the current mip dimensions by two. We check each dimension before the division to ensure that dimension never becomes 0. This handles cases where the image is not square, since one of the mip dimensions would reach 1 before the other dimension. When this happens, that dimension should remain 1 for all remaining levels.

```

1     barrier.subresourceRange.baseMipLevel = mipLevels - 1;
2     barrier.oldLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL;
3     barrier.newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
4     barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
5     barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
6
7     vkCmdPipelineBarrier(commandBuffer,
8         VK_PIPELINE_STAGE_TRANSFER_BIT,
9         VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT, 0,
10        0, nullptr,
11        0, nullptr,
12        1, &barrier);
13
14     endSingleTimeCommands(commandBuffer);
15 }

```

Before we end the command buffer, we insert one more pipeline barrier. This barrier transitions the last mip level from `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. This wasn't handled by the loop, since the last mip level is never blitted from.

Finally, add the call to `generateMipmaps` in `createTextureImage`:

```

1 transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
    mipLevels);
2 copyBufferToImage(stagingBuffer, textureImage,
    static_cast<uint32_t>(texWidth),
    static_cast<uint32_t>(texHeight));
3 //transitioned to VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL while
    generating mipmaps
4 ...
5 generateMipmaps(textureImage, texWidth, texHeight, mipLevels);

```

Our texture image's mipmaps are now completely filled.

Linear filtering support

It is very convenient to use a built-in function like `vkCmdBlitImage` to generate all the mip levels, but unfortunately it is not guaranteed to be supported on all platforms. It requires the texture image format we use to support linear filtering, which can be checked with the `vkGetPhysicalDeviceFormatProperties` function. We will add a check to the `generateMipmaps` function for this.

First add an additional parameter that specifies the image format:

```

1 void createTextureImage() {
2     ...
3
4     generateMipmaps(textureImage, VK_FORMAT_R8G8B8A8_SRGB, texWidth,
        texHeight, mipLevels);
5 }
6
7 void generateMipmaps(VkImage image, VkFormat imageFormat, int32_t
    texWidth, int32_t texHeight, uint32_t mipLevels) {
8
9     ...
10 }

```

In the `generateMipmaps` function, use `vkGetPhysicalDeviceFormatProperties` to request the properties of the texture image format:

```

1 void generateMipmaps(VkImage image, VkFormat imageFormat, int32_t
    texWidth, int32_t texHeight, uint32_t mipLevels) {
2
3     // Check if image format supports linear blitting
4     VkFormatProperties formatProperties;
5     vkGetPhysicalDeviceFormatProperties(physicalDevice, imageFormat,
        &formatProperties);
6

```

7 ...

The `VkFormatProperties` struct has three fields named `linearTilingFeatures`, `optimalTilingFeatures` and `bufferFeatures` that each describe how the format can be used depending on the way it is used. We create a texture image with the optimal tiling format, so we need to check `optimalTilingFeatures`. Support for the linear filtering feature can be checked with the `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`:

```
1 if (!(formatProperties.optimalTilingFeatures &
      VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT)) {
2     throw std::runtime_error("texture image format does not support
      linear blitting!");
3 }
```

There are two alternatives in this case. You could implement a function that searches common texture image formats for one that *does* support linear blitting, or you could implement the mipmap generation in software with a library like `stb_image_resize`. Each mip level can then be loaded into the image in the same way that you loaded the original image.

It should be noted that it is uncommon in practice to generate the mipmap levels at runtime anyway. Usually they are pregenerated and stored in the texture file alongside the base level to improve loading speed. Implementing resizing in software and loading multiple levels from a file is left as an exercise to the reader.

Sampler

While the `VkImage` holds the mipmap data, `VkSampler` controls how that data is read while rendering. Vulkan allows us to specify `minLod`, `maxLod`, `mipLodBias`, and `mipmapMode` (“Lod” means “Level of Detail”). When a texture is sampled, the sampler selects a mip level according to the following pseudocode:

```
1 lod = getLodLevelFromScreenSize(); //smaller when the object is
      close, may be negative
2 lod = clamp(lod + mipLodBias, minLod, maxLod);
3
4 level = clamp(floor(lod), 0, texture.mipLevels - 1); //clamped to
      the number of mip levels in the texture
5
6 if (mipmapMode == VK_SAMPLER_MIPMAP_MODE_NEAREST) {
7     color = sample(level);
8 } else {
9     color = blend(sample(level), sample(level + 1));
10 }
```

If `samplerInfo.mipmapMode` is `VK_SAMPLER_MIPMAP_MODE_NEAREST`, `lod` selects the mip level to sample from. If the mipmap mode is `VK_SAMPLER_MIPMAP_MODE_LINEAR`, `lod` is used to select two mip levels to be sampled. Those levels are sampled and the results are linearly blended.

The sample operation is also affected by `lod`:

```
1 if (lod <= 0) {
2     color = readTexture(uv, magFilter);
3 } else {
4     color = readTexture(uv, minFilter);
5 }
```

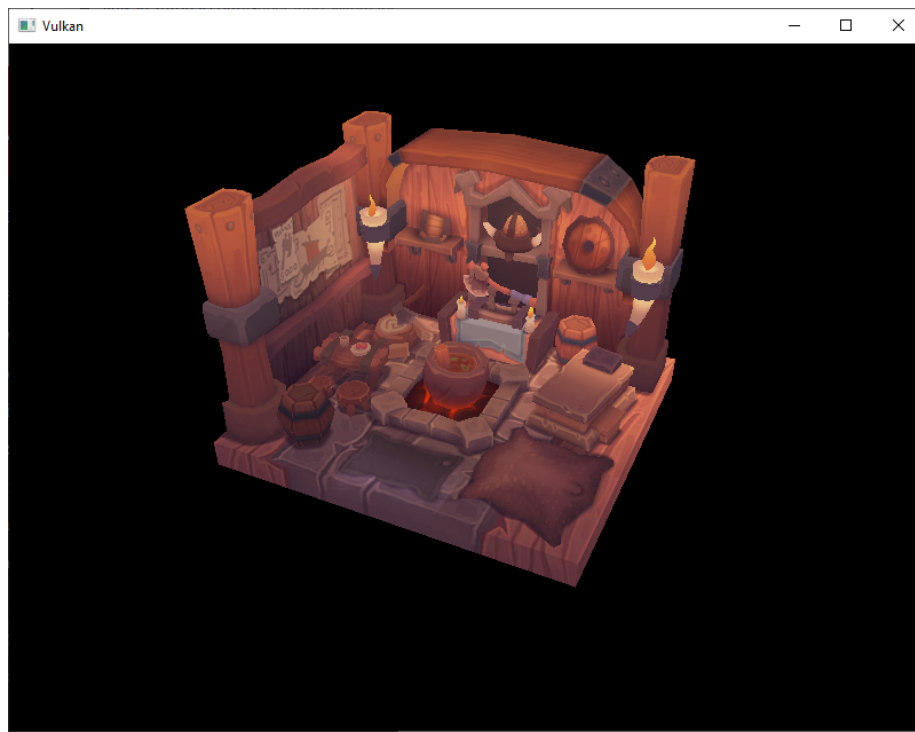
If the object is close to the camera, `magFilter` is used as the filter. If the object is further from the camera, `minFilter` is used. Normally, `lod` is non-negative, and is only 0 when close the camera. `mipLodBias` lets us force Vulkan to use lower `lod` and `level` than it would normally use.

To see the results of this chapter, we need to choose values for our `textureSampler`. We've already set the `minFilter` and `magFilter` to use `VK_FILTER_LINEAR`. We just need to choose values for `minLod`, `maxLod`, `mipLodBias`, and `mipmapMode`.

```
1 void createTextureSampler() {
2     ...
3     samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
4     samplerInfo.minLod = 0.0f; // Optional
5     samplerInfo.maxLod = VK_LOD_CLAMP_NONE;
6     samplerInfo.mipLodBias = 0.0f; // Optional
7     ...
8 }
```

To allow the full range of mip levels to be used, we set `minLod` to 0.0f, and `maxLod` to `VK_LOD_CLAMP_NONE`. This constant is equal to 1000.0f, which means that all available mipmap levels in the texture will be sampled. We have no reason to change the `lod` value, so we set `mipLodBias` to 0.0f.

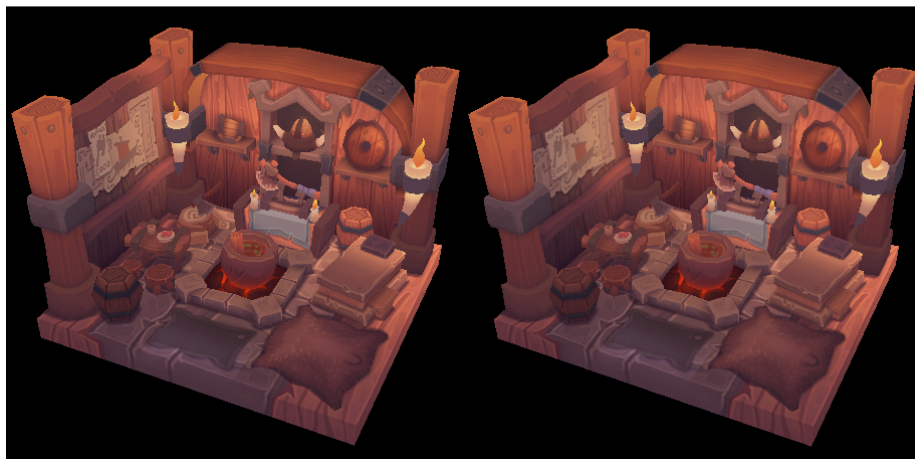
Now run your program and you should see the following:



It's not a dramatic difference, since our scene is so simple. There are subtle differences if you look closely.

Without mipmaps

With mipmaps



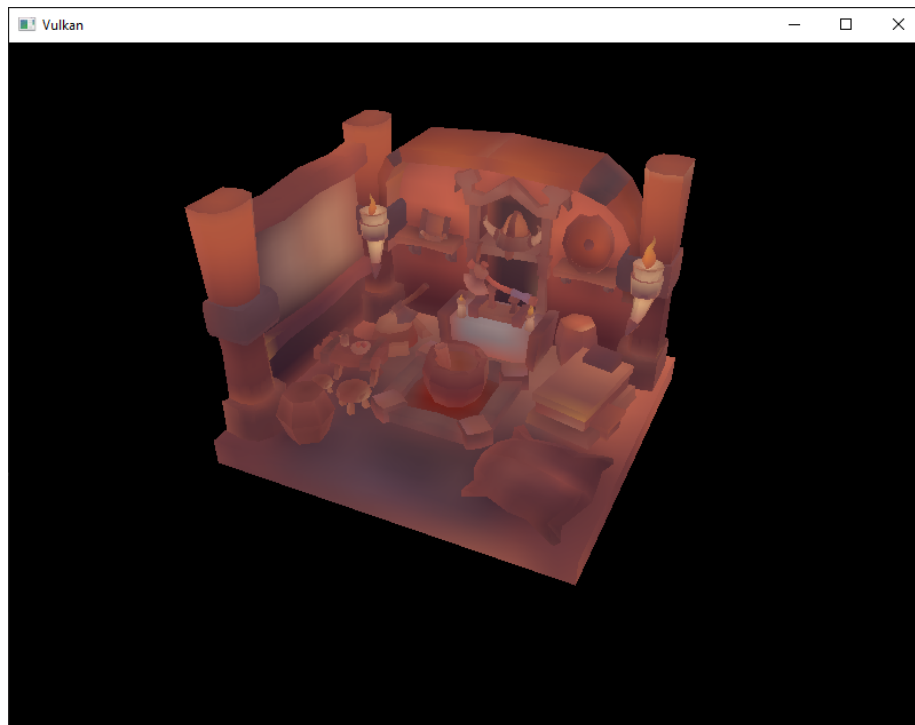
The most noticeable difference is the writing on the papers. With mipmaps, the writing has been smoothed. Without mipmaps, the writing has harsh edges and

gaps from Moiré artifacts.

You can play around with the sampler settings to see how they affect mipmapping. For example, by changing `minLod`, you can force the sampler to not use the lowest mip levels:

```
1 samplerInfo.minLod = static_cast<float>(mipLevels / 2);
```

These settings will produce this image:



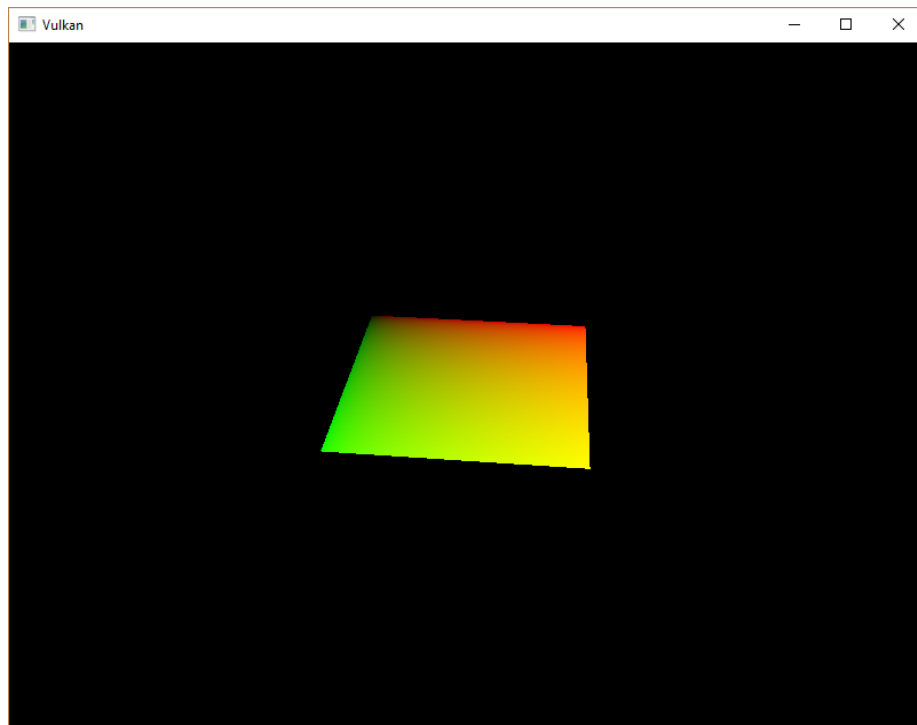
This is how higher mip levels will be used when objects are further away from the camera.

[C++ code](#) / [Vertex shader](#) / [Fragment shader](#)

Multisampling

Introduction

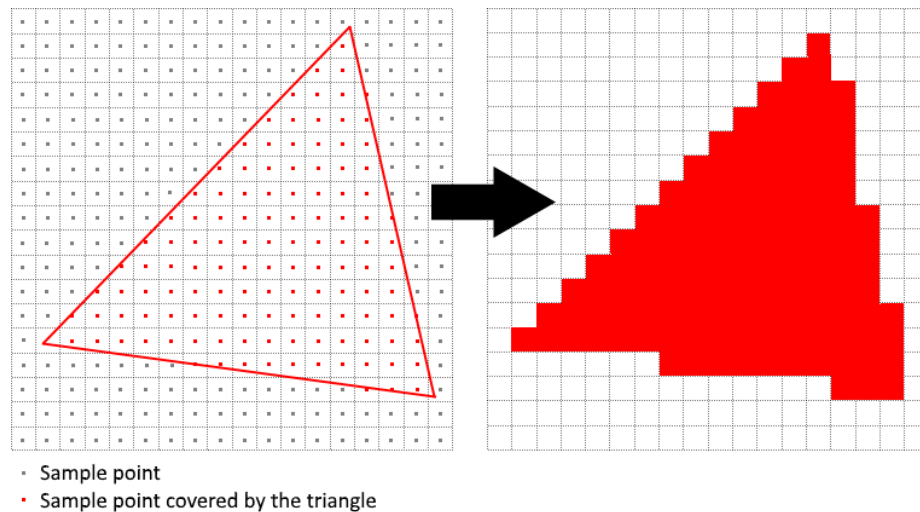
Our program can now load multiple levels of detail for textures which fixes artifacts when rendering objects far away from the viewer. The image is now a lot smoother, however on closer inspection you will notice jagged saw-like patterns along the edges of drawn geometric shapes. This is especially visible in one of our early programs when we rendered a quad:



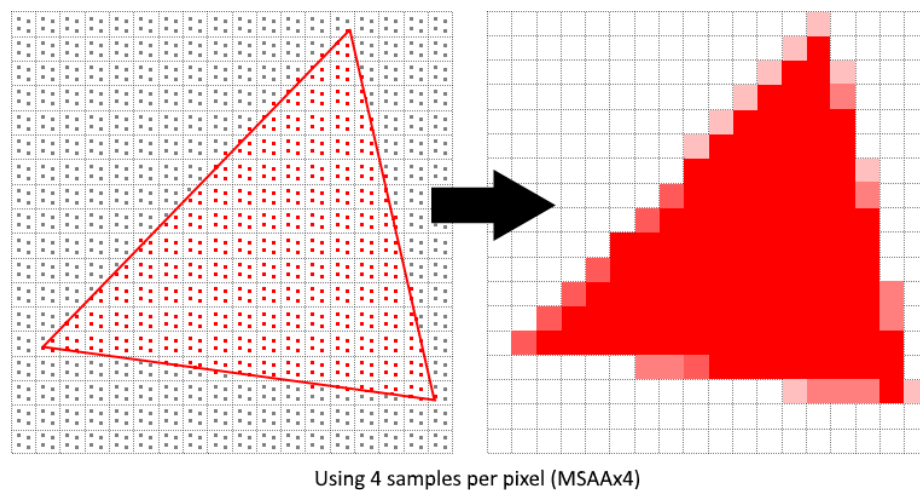
This undesired effect is called “aliasing” and it’s a result of a limited numbers of pixels that are available for rendering. Since there are no displays out there

with unlimited resolution, it will be always visible to some extent. There's a number of ways to fix this and in this chapter we'll focus on one of the more popular ones: Multisample anti-aliasing (MSAA).

In ordinary rendering, the pixel color is determined based on a single sample point which in most cases is the center of the target pixel on screen. If part of the drawn line passes through a certain pixel but doesn't cover the sample point, that pixel will be left blank, leading to the jagged "staircase" effect.



What MSAA does is it uses multiple sample points per pixel (hence the name) to determine its final color. As one might expect, more samples lead to better results, however it is also more computationally expensive.



In our implementation, we will focus on using the maximum available sample

count. Depending on your application this may not always be the best approach and it might be better to use less samples for the sake of higher performance if the final result meets your quality demands.

Getting available sample count

Let's start off by determining how many samples our hardware can use. Most modern GPUs support at least 8 samples but this number is not guaranteed to be the same everywhere. We'll keep track of it by adding a new class member:

```
1 ...
2 VkSampleCountFlagBits msaaSamples = VK_SAMPLE_COUNT_1_BIT;
3 ...
```

By default we'll be using only one sample per pixel which is equivalent to no multisampling, in which case the final image will remain unchanged. The exact maximum number of samples can be extracted from `VkPhysicalDeviceProperties` associated with our selected physical device. We're using a depth buffer, so we have to take into account the sample count for both color and depth. The highest sample count that is supported by both (&) will be the maximum we can support. Add a function that will fetch this information for us:

```
1 VkSampleCountFlagBits getMaxUsableSampleCount() {
2     VkPhysicalDeviceProperties physicalDeviceProperties;
3     vkGetPhysicalDeviceProperties(physicalDevice,
4         &physicalDeviceProperties);
5
6     VkSampleCountFlags counts =
7         physicalDeviceProperties.limits.framebufferColorSampleCounts
8         &
9         physicalDeviceProperties.limits.framebufferDepthSampleCounts;
10    if (counts & VK_SAMPLE_COUNT_64_BIT) { return
11        VK_SAMPLE_COUNT_64_BIT; }
12    if (counts & VK_SAMPLE_COUNT_32_BIT) { return
13        VK_SAMPLE_COUNT_32_BIT; }
14    if (counts & VK_SAMPLE_COUNT_16_BIT) { return
15        VK_SAMPLE_COUNT_16_BIT; }
16    if (counts & VK_SAMPLE_COUNT_8_BIT) { return
17        VK_SAMPLE_COUNT_8_BIT; }
18    if (counts & VK_SAMPLE_COUNT_4_BIT) { return
19        VK_SAMPLE_COUNT_4_BIT; }
20    if (counts & VK_SAMPLE_COUNT_2_BIT) { return
21        VK_SAMPLE_COUNT_2_BIT; }
22
23    return VK_SAMPLE_COUNT_1_BIT;
24 }
```

We will now use this function to set the `msaaSamples` variable during the physical device selection process. For this, we have to slightly modify the `pickPhysicalDevice` function:

```
1 void pickPhysicalDevice() {
2     ...
3     for (const auto& device : devices) {
4         if (isDeviceSuitable(device)) {
5             physicalDevice = device;
6             msaaSamples = getMaxUsableSampleCount();
7             break;
8         }
9     }
10    ...
11 }
```

Setting up a render target

In MSAA, each pixel is sampled in an offscreen buffer which is then rendered to the screen. This new buffer is slightly different from regular images we've been rendering to - they have to be able to store more than one sample per pixel. Once a multisampled buffer is created, it has to be resolved to the default framebuffer (which stores only a single sample per pixel). This is why we have to create an additional render target and modify our current drawing process. We only need one render target since only one drawing operation is active at a time, just like with the depth buffer. Add the following class members:

```
1 ...
2 VkImage colorImage;
3 VkDeviceMemory colorImageMemory;
4 VkImageView colorImageView;
5 ...
```

This new image will have to store the desired number of samples per pixel, so we need to pass this number to `VkImageCreateInfo` during the image creation process. Modify the `createImage` function by adding a `numSamples` parameter:

```
1 void createImage(uint32_t width, uint32_t height, uint32_t
    mipLevels, VkSampleCountFlagBits numSamples, VkFormat format,
    VkImageTiling tiling, VkImageUsageFlags usage,
    VkMemoryPropertyFlags properties, VkImage& image,
    VkDeviceMemory& imageMemory) {
2     ...
3     imageInfo.samples = numSamples;
4     ...
```

For now, update all calls to this function using `VK_SAMPLE_COUNT_1_BIT` - we will be replacing this with proper values as we progress with implementation:

```
1 createImage(swapChainExtent.width, swapChainExtent.height, 1,
    VK_SAMPLE_COUNT_1_BIT, depthFormat, VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, depthImage,
    depthImageMemory);
2 ...
3 createImage(texWidth, texHeight, mipLevels, VK_SAMPLE_COUNT_1_BIT,
    VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_TRANSFER_SRC_BIT |
    VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, textureImage,
    textureImageMemory);
```

We will now create a multisampled color buffer. Add a `createColorResources` function and note that we're using `msaaSamples` here as a function parameter to `createImage`. We're also using only one mip level, since this is enforced by the Vulkan specification in case of images with more than one sample per pixel. Also, this color buffer doesn't need mipmaps since it's not going to be used as a texture:

```
1 void createColorResources() {
2     VkFormat colorFormat = swapChainImageFormat;
3
4     createImage(swapChainExtent.width, swapChainExtent.height, 1,
        msaaSamples, colorFormat, VK_IMAGE_TILING_OPTIMAL,
        VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT |
        VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT,
        VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, colorImage,
        colorImageMemory);
5     colorImageView = createImageView(colorImage, colorFormat,
        VK_IMAGE_ASPECT_COLOR_BIT, 1);
6 }
```

For consistency, call the function right before `createDepthResources`:

```
1 void initVulkan() {
2     ...
3     createColorResources();
4     createDepthResources();
5     ...
6 }
```

Now that we have a multisampled color buffer in place it's time to take care of depth. Modify `createDepthResources` and update the number of samples used by the depth buffer:

```

1 void createDepthResources() {
2     ...
3     createImage(swapChainExtent.width, swapChainExtent.height, 1,
4                 msaaSamples, depthFormat, VK_IMAGE_TILING_OPTIMAL,
5                 VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
6                 VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, depthImage,
7                 depthImageMemory);
8     ...
9 }

```

We have now created a couple of new Vulkan resources, so let's not forget to release them when necessary:

```

1 void cleanupSwapChain() {
2     vkDestroyImageView(device, colorImageView, nullptr);
3     vkDestroyImage(device, colorImage, nullptr);
4     vkFreeMemory(device, colorImageMemory, nullptr);
5     ...
6 }

```

And update the `recreateSwapChain` so that the new color image can be recreated in the correct resolution when the window is resized:

```

1 void recreateSwapChain() {
2     ...
3     createImageViews();
4     createColorResources();
5     createDepthResources();
6     ...
7 }

```

We made it past the initial MSAA setup, now we need to start using this new resource in our graphics pipeline, framebuffer, render pass and see the results!

Adding new attachments

Let's take care of the render pass first. Modify `createRenderPass` and update color and depth attachment creation info structs:

```

1 void createRenderPass() {
2     ...
3     colorAttachment.samples = msaaSamples;
4     colorAttachment.finalLayout =
5         VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
6     ...
7     depthAttachment.samples = msaaSamples;
8     ...
9 }

```

You'll notice that we have changed the `finalLayout` from `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR` to `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`. That's because multisampled images cannot be presented directly. We first need to resolve them to a regular image. This requirement does not apply to the depth buffer, since it won't be presented at any point. Therefore we will have to add only one new attachment for color which is a so-called resolve attachment:

```

1   ...
2   VkAttachmentDescription colorAttachmentResolve{};
3   colorAttachmentResolve.format = swapChainImageFormat;
4   colorAttachmentResolve.samples = VK_SAMPLE_COUNT_1_BIT;
5   colorAttachmentResolve.loadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;
6   colorAttachmentResolve.storeOp = VK_ATTACHMENT_STORE_OP_STORE;
7   colorAttachmentResolve.stencilLoadOp =
8       VK_ATTACHMENT_LOAD_OP_DONT_CARE;
9   colorAttachmentResolve.stencilStoreOp =
10      VK_ATTACHMENT_STORE_OP_DONT_CARE;
11  colorAttachmentResolve.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
12  colorAttachmentResolve.finalLayout =
13      VK_IMAGE_LAYOUT_PRESENT_SRC_KHR;
14  ...

```

The render pass now has to be instructed to resolve multisampled color image into regular attachment. Create a new attachment reference that will point to the color buffer which will serve as the resolve target:

```

1   ...
2   VkAttachmentReference colorAttachmentResolveRef{};
3   colorAttachmentResolveRef.attachment = 2;
4   colorAttachmentResolveRef.layout =
5       VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
6   ...

```

Set the `pResolveAttachments` subpass struct member to point to the newly created attachment reference. This is enough to let the render pass define a multisample resolve operation which will let us render the image to screen:

```

1   ...
2   subpass.pResolveAttachments = &colorAttachmentResolveRef;
3   ...

```

Since we're reusing the multisampled color image, it's necessary to update the `srcAccessMask` of the `VkSubpassDependency`. This update ensures that any write operations to the color attachment are completed before subsequent ones begin, thus preventing write-after-write hazards that can lead to unstable rendering results:

```

1   ...

```

```

2     dependency.srcAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT
      | VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;
3     ...

```

Now update render pass info struct with the new color attachment:

```

1     ...
2     std::array<VkAttachmentDescription, 3> attachments =
      {colorAttachment, depthAttachment, colorAttachmentResolve};
3     ...

```

With the render pass in place, modify `createFramebuffers` and add the new image view to the list:

```

1 void createFramebuffers() {
2     ...
3     std::array<VkImageView, 3> attachments = {
4         colorImageView,
5         depthImageView,
6         swapChainImageViews[i]
7     };
8     ...
9 }

```

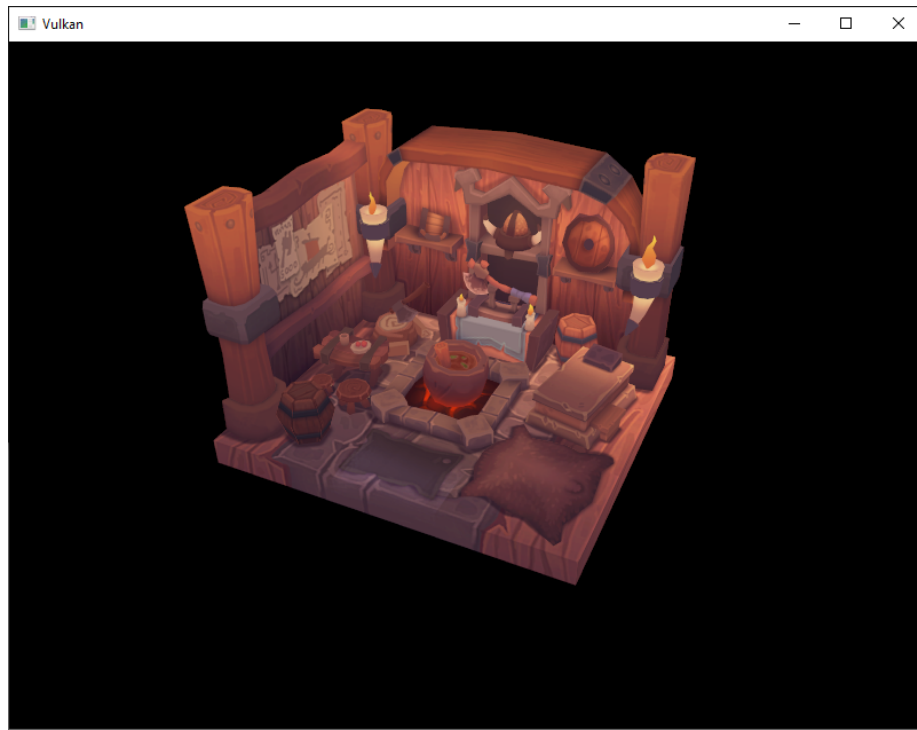
Finally, tell the newly created pipeline to use more than one sample by modifying `createGraphicsPipeline`:

```

1 void createGraphicsPipeline() {
2     ...
3     multisampling.rasterizationSamples = msaaSamples;
4     ...
5 }

```

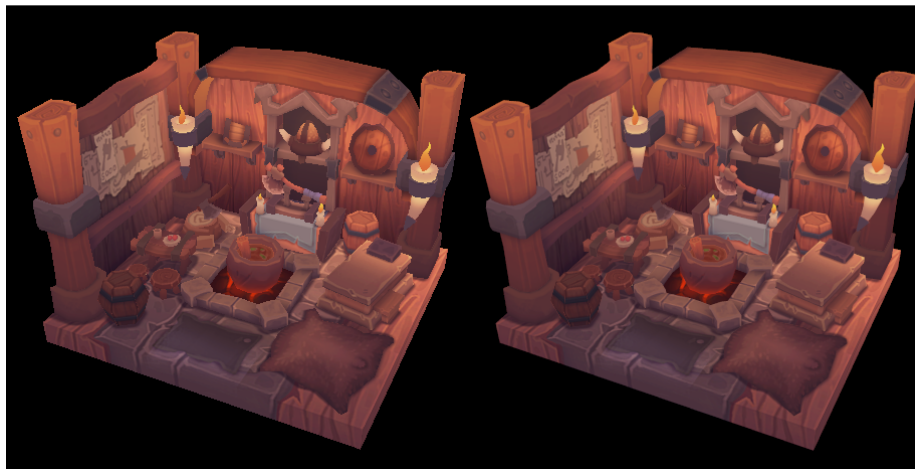
Now run your program and you should see the following:



Just like with mipmapping, the difference may not be apparent straight away. On a closer look you'll notice that the edges are not as jagged anymore and the whole image seems a bit smoother compared to the original.

Without multisampling

With multisampling (MSAAx8)



The difference is more noticable when looking up close at one of the edges:

Without multisampling

With multisampling (MSAAx8)



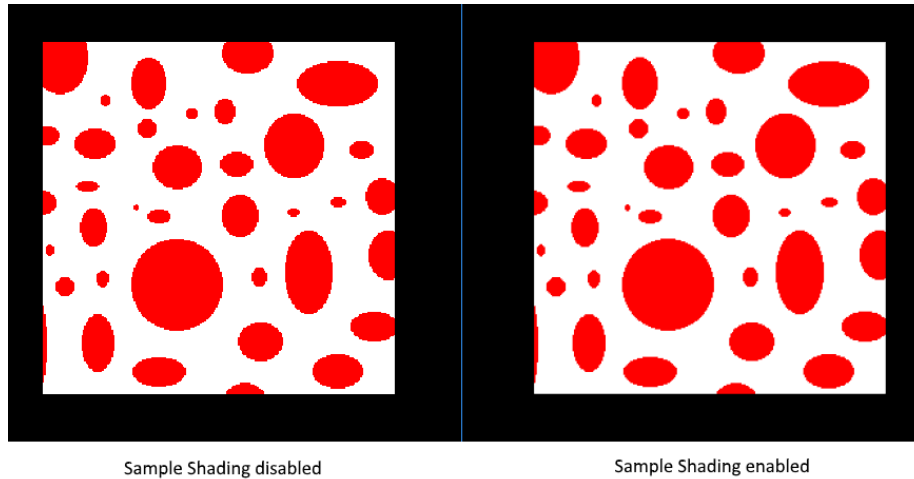
Quality improvements

There are certain limitations of our current MSAA implementation which may impact the quality of the output image in more detailed scenes. For example, we're currently not solving potential problems caused by shader aliasing, i.e. MSAA only smoothens out the edges of geometry but not the interior filling. This may lead to a situation when you get a smooth polygon rendered on screen but the applied texture will still look aliased if it contains high contrasting colors. One way to approach this problem is to enable Sample Shading which will improve the image quality even further, though at an additional performance cost:

```

1 void createLogicalDevice() {
2     ...
3     deviceFeatures.sampleRateShading = VK_TRUE; // enable sample
        shading feature for the device
4     ...
5 }
6
7 void createGraphicsPipeline() {
8     ...
9     multisampling.sampleShadingEnable = VK_TRUE; // enable sample
        shading in the pipeline
10    multisampling.minSampleShading = .2f; // min fraction for sample
        shading; closer to one is smoother
11    ...
12 }
```

In this example we'll leave sample shading disabled but in certain scenarios the quality improvement may be noticeable:



Conclusion

It has taken a lot of work to get to this point, but now you finally have a good base for a Vulkan program. The knowledge of the basic principles of Vulkan that you now possess should be sufficient to start exploring more of the features, like:

- Push constants
- Instanced rendering
- Dynamic uniforms
- Separate images and sampler descriptors
- Pipeline cache
- Multi-threaded command buffer generation
- Multiple subpasses
- Compute shaders

The current program can be extended in many ways, like adding Blinn-Phong lighting, post-processing effects and shadow mapping. You should be able to learn how these effects work from tutorials for other APIs, because despite Vulkan's explicitness, many concepts still work the same.

C++ code / Vertex shader / Fragment shader

Compute Shader

Introduction

In this bonus chapter we'll take a look at compute shaders. Up until now all previous chapters dealt with the traditional graphics part of the Vulkan pipeline. But unlike older APIs like OpenGL, compute shader support in Vulkan is mandatory. This means that you can use compute shaders on every Vulkan implementation available, no matter if it's a high-end desktop GPU or a low-powered embedded device.

This opens up the world of general purpose computing on graphics processor units (GPGPU), no matter where your application is running. GPGPU means that you can do general computations on your GPU, something that has traditionally been a domain of CPUs. But with GPUs having become more and more powerful and more flexible, many workloads that would require the general purpose capabilities of a CPU can now be done on the GPU in realtime.

A few examples of where the compute capabilities of a GPU can be used are image manipulation, visibility testing, post processing, advanced lighting calculations, animations, physics (e.g. for a particle system) and much more. And it's even possible to use compute for non-visual computational only work that does not require any graphics output, e.g. number crunching or AI related things. This is called “headless compute”.

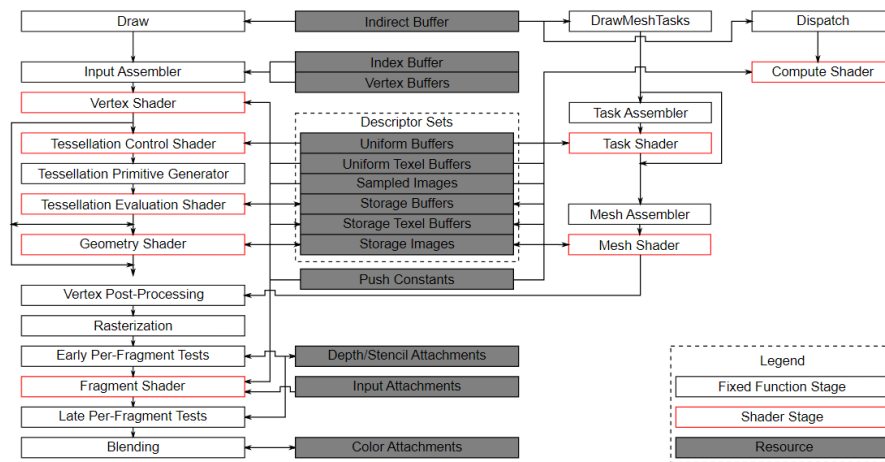
Advantages

Doing computationally expensive calculations on the GPU has several advantages. The most obvious one is offloading work from the CPU. Another one is not requiring moving data between the CPU's main memory and the GPU's memory. All of the data can stay on the GPU without having to wait for slow transfers from main memory.

Aside from these, GPUs are heavily parallelized with some of them having tens of thousands of small compute units. This often makes them a better fit for highly parallel workflows than a CPU with a few large compute units.

The Vulkan pipeline

It's important to know that compute is completely separated from the graphics part of the pipeline. This is visible in the following block diagram of the Vulkan pipeline from the official specification:



In this diagram we can see the traditional graphics part of the pipeline on the left, and several stages on the right that are not part of this graphics pipeline, including the compute shader (stage). With the compute shader stage being detached from the graphics pipeline we'll be able to use it anywhere we see fit. This is very different from e.g. the fragment shader which is always applied to the transformed output of the vertex shader.

The center of the diagram also shows that e.g. descriptor sets are also used by compute, so everything we learned about descriptors layouts, descriptor sets and descriptors also applies here.

An example

An easy to understand example that we will implement in this chapter is a GPU based particle system. Such systems are used in many games and often consist of thousands of particles that need to be updated at interactive frame rates. Rendering such a system requires 2 main components: vertices, passed as vertex buffers, and a way to update them based on some equation.

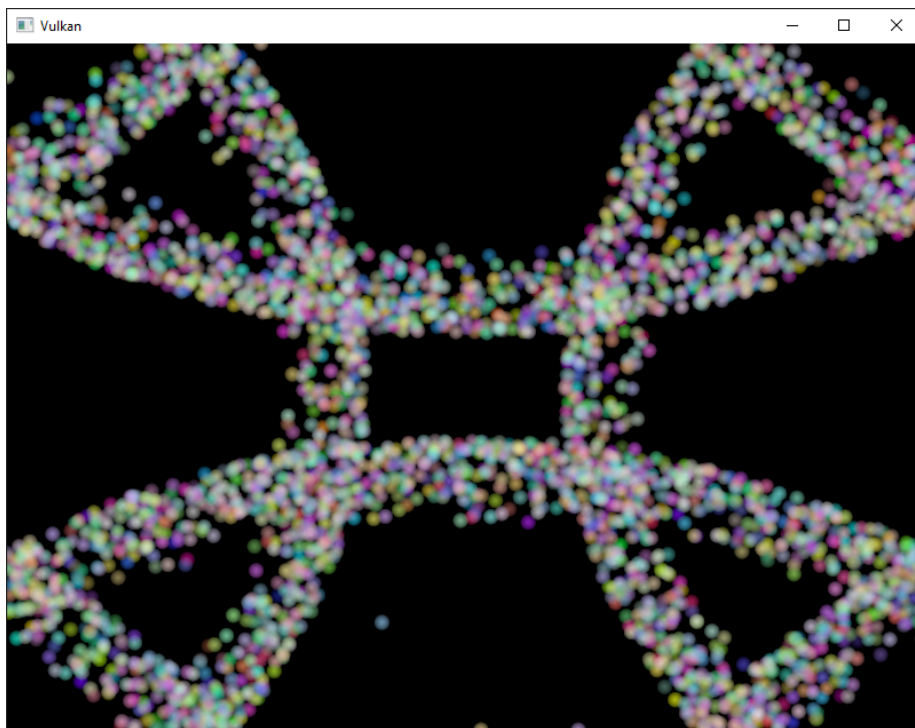
The “classical” CPU based particle system would store particle data in the system's main memory and then use the CPU to update them. After the update, the vertices need to be transferred to the GPU's memory again so it can display the updated particles in the next frame. The most straight-forward way would be recreating the vertex buffer with the new data for each frame. This is obviously very costly. Depending on your implementation, there are other

options like mapping GPU memory so it can be written by the CPU (called “resizable BAR” on desktop systems, or unified memory on integrated GPUs) or just using a host local buffer (which would be the slowest method due to PCI-E bandwidth). But no matter what buffer update method you choose, you always require a “round-trip” to the CPU to update the particles.

With a GPU based particle system, this round-trip is no longer required. Vertices are only uploaded to the GPU at the start and all updates are done in the GPU’s memory using compute shaders. One of the main reasons why this is faster is the much higher bandwidth between the GPU and it’s local memory. In a CPU based scenario, you’d be limited by main memory and PCI-express bandwidth, which is often just a fraction of the GPU’s memory bandwidth.

When doing this on a GPU with a dedicated compute queue, you can update particles in parallel to the rendering part of the graphics pipeline. This is called “async compute”, and is an advanced topic not covered in this tutorial.

Here is a screenshot from this chapter’s code. The particles shown here are updated by a compute shader directly on the GPU, without any CPU interaction:



Data manipulation

In this tutorial we already learned about different buffer types like vertex and index buffers for passing primitives and uniform buffers for passing data to a shader. And we also used images to do texture mapping. But up until now, we always wrote data using the CPU and only did reads on the GPU.

An important concept introduced with compute shaders is the ability to arbitrarily read from **and write to** buffers. For this, Vulkan offers two dedicated storage types.

Shader storage buffer objects (SSBO)

A shader storage buffer (SSBO) allows shaders to read from and write to a buffer. Using these is similar to using uniform buffer objects. The biggest differences are that you can alias other buffer types to SSBOs and that they can be arbitrarily large.

Going back to the GPU based particle system, you might now wonder how to deal with vertices being updated (written) by the compute shader and read (drawn) by the vertex shader, as both usages would seemingly require different buffer types.

But that's not the case. In Vulkan you can specify multiple usages for buffers and images. So for the particle vertex buffer to be used as a vertex buffer (in the graphics pass) and as a storage buffer (in the compute pass), you simply create the buffer with those two usage flags:

```
1 VkBufferCreateInfo bufferInfo{};
2 ...
3 bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT |
    VK_BUFFER_USAGE_STORAGE_BUFFER_BIT |
    VK_BUFFER_USAGE_TRANSFER_DST_BIT;
4 ...
5
6 if (vkCreateBuffer(device, &bufferInfo, nullptr,
    &shaderStorageBuffers[i]) != VK_SUCCESS) {
7     throw std::runtime_error("failed to create vertex buffer!");
8 }
```

The two flags `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT` and `VK_BUFFER_USAGE_STORAGE_BUFFER_BIT` set with `bufferInfo.usage` tell the implementation that we want to use this buffer for two different scenarios: as a vertex buffer in the vertex shader and as a store buffer. Note that we also added the `VK_BUFFER_USAGE_TRANSFER_DST_BIT` flag in here so we can transfer data from the host to the GPU. This is crucial as we want the shader storage buffer to stay in GPU memory only (`VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT`) we need to transfer data from the host to this buffer.

Here is the same code using using the `createBuffer` helper function:

```
1 createBuffer(bufferSize, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT |  
    VK_BUFFER_USAGE_VERTEX_BUFFER_BIT |  
    VK_BUFFER_USAGE_TRANSFER_DST_BIT,  
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, shaderStorageBuffers[i],  
    shaderStorageBuffersMemory[i]);
```

The GLSL shader declaration for accessing such a buffer looks like this:

```
1 struct Particle {  
2     vec2 position;  
3     vec2 velocity;  
4     vec4 color;  
5 };  
6  
7 layout(std140, binding = 1) readonly buffer ParticleSSBOIn {  
8     Particle particlesIn[ ];  
9 };  
10  
11 layout(std140, binding = 2) buffer ParticleSSBOOut {  
12     Particle particlesOut[ ];  
13 };
```

In this example we have a typed SSBO with each particle having a position and velocity value (see the `Particle` struct). The SSBO then contains an unbound number of particles as marked by the `[]`. Not having to specify the number of elements in an SSBO is one of the advantages over e.g. uniform buffers. `std140` is a memory layout qualifier that determines how the member elements of the shader storage buffer are aligned in memory. This gives us certain guarantees, required to map the buffers between the host and the GPU.

Writing to such a storage buffer object in the compute shader is straight-forward and similar to how you'd write to the buffer on the C++ side:

```
1 particlesOut[index].position = particlesIn[index].position +  
    particlesIn[index].velocity.xy * ubo.deltaTime;
```

Storage images

Note that we won't be doing image manipulation in this chapter. This paragraph is here to make readers aware that compute shaders can also be used for image manipulation.

A storage image allows you read from and write to an image. Typical use cases are applying image effects to textures, doing post processing (which in turn is very similar) or generating mip-maps.

This is similar for images:

```

1 VkImageCreateInfo imageInfo {};
2 ...
3 imageInfo.usage = VK_IMAGE_USAGE_SAMPLED_BIT |
    VK_IMAGE_USAGE_STORAGE_BIT;
4 ...
5
6 if (vkCreateImage(device, &imageInfo, nullptr, &textureImage) !=
    VK_SUCCESS) {
7     throw std::runtime_error("failed to create image!");
8 }

```

The two flags `VK_IMAGE_USAGE_SAMPLED_BIT` and `VK_IMAGE_USAGE_STORAGE_BIT` set with `imageInfo.usage` tell the implementation that we want to use this image for two different scenarios: as an image sampled in the fragment shader and as a storage image in the compute shader;

The GLSL shader declaration for storage image looks similar to sampled images used e.g. in the fragment shader:

```

1 layout (binding = 0, rgba8) uniform readonly image2D inputImage;
2 layout (binding = 1, rgba8) uniform writeonly image2D outputImage;

```

A few differences here are additional attributes like `rgba8` for the format of the image, the `readonly` and `writeonly` qualifiers, telling the implementation that we will only read from the input image and write to the output image. And last but not least we need to use the `image2D` type to declare a storage image.

Reading from and writing to storage images in the compute shader is then done using `imageLoad` and `imageStore`:

```

1 vec3 pixel = imageLoad(inputImage,
    ivec2(gl_GlobalInvocationID.xy)).rgb;
2 imageStore(outputImage, ivec2(gl_GlobalInvocationID.xy), pixel);

```

Compute queue families

In the physical device and queue families chapter we already learned about queue families and how to select a graphics queue family. Compute uses the queue family properties flag bit `VK_QUEUE_COMPUTE_BIT`. So if we want to do compute work, we need to get a queue from a queue family that supports compute.

Note that Vulkan requires an implementation which supports graphics operations to have at least one queue family that supports both graphics and compute operations, but it's also possible that implementations offer a dedicated compute queue. This dedicated compute queue (that does not have the graphics bit) hints at an asynchronous compute queue. To keep this tutorial beginner

friendly though, we'll use a queue that can do both graphics and compute operations. This will also save us from dealing with several advanced synchronization mechanisms.

For our compute sample we need to change the device creation code a bit:

```
1 uint32_t queueFamilyCount = 0;
2 vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount,
    nullptr);
3
4 std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
5 vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount,
    queueFamilies.data());
6
7 int i = 0;
8 for (const auto& queueFamily : queueFamilies) {
9     if ((queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT) &&
        (queueFamily.queueFlags & VK_QUEUE_COMPUTE_BIT)) {
10         indices.graphicsAndComputeFamily = i;
11     }
12
13     i++;
14 }
```

The changed queue family index selection code will now try to find a queue family that supports both graphics and compute.

We can then get a compute queue from this queue family in `createLogicalDevice`:

```
1 vkGetDeviceQueue(device, indices.graphicsAndComputeFamily.value(),
    0, &computeQueue);
```

The compute shader stage

In the graphics samples we have used different pipeline stages to load shaders and access descriptors. Compute shaders are accessed in a similar way by using the `VK_SHADER_STAGE_COMPUTE_BIT` pipeline. So loading a compute shader is just the same as loading a vertex shader, but with a different shader stage. We'll talk about this in detail in the next paragraphs. Compute also introduces a new binding point type for descriptors and pipelines named `VK_PIPELINE_BIND_POINT_COMPUTE` that we'll have to use later on.

Loading compute shaders

Loading compute shaders in our application is the same as loading any other shader. The only real difference is that we'll need to use the `VK_SHADER_STAGE_COMPUTE_BIT` mentioned above.

```

1 auto computeShaderCode = readFile("shaders/compute.spv");
2
3 VkShaderModule computeShaderModule =
    createShaderModule(computeShaderCode);
4
5 VkPipelineShaderStageCreateInfo computeShaderStageInfo{};
6 computeShaderStageInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
7 computeShaderStageInfo.stage = VK_SHADER_STAGE_COMPUTE_BIT;
8 computeShaderStageInfo.module = computeShaderModule;
9 computeShaderStageInfo.pName = "main";
10 ...

```

Preparing the shader storage buffers

Earlier on we learned that we can use shader storage buffers to pass arbitrary data to compute shaders. For this example we will upload an array of particles to the GPU, so we can manipulate it directly in the GPU's memory.

In the frames in flight chapter we talked about duplicating resources per frame in flight, so we can keep the CPU and the GPU busy. First we declare a vector for the buffer object and the device memory backing it up:

```

1 std::vector<VkBuffer> shaderStorageBuffers;
2 std::vector<VkDeviceMemory> shaderStorageBuffersMemory;

```

In the `createShaderStorageBuffers` we then resize those vectors to match the max. number of frames in flight:

```

1 shaderStorageBuffers.resize(MAX_FRAMES_IN_FLIGHT);
2 shaderStorageBuffersMemory.resize(MAX_FRAMES_IN_FLIGHT);

```

With this setup in place we can start to move the initial particle information to the GPU. We first initialize a vector of particles on the host side:

```

1 // Initialize particles
2 std::default_random_engine rndEngine((unsigned)time(nullptr));
3 std::uniform_real_distribution<float> rndDist(0.0f, 1.0f);
4
5 // Initial particle positions on a circle
6 std::vector<Particle> particles(PARTICLE_COUNT);
7 for (auto& particle : particles) {
8     float r = 0.25f * sqrt(rndDist(rndEngine));
9     float theta = rndDist(rndEngine) * 2 *
        3.14159265358979323846;
10    float x = r * cos(theta) * HEIGHT / WIDTH;
11    float y = r * sin(theta);

```

```

12     particle.position = glm::vec2(x, y);
13     particle.velocity = glm::normalize(glm::vec2(x,y)) *
        0.00025f;
14     particle.color = glm::vec4(rndDist(rndEngine),
        rndDist(rndEngine), rndDist(rndEngine), 1.0f);
15 }

```

We then create a staging buffer in the host's memory to hold the initial particle properties:

```

1     VkDeviceSize bufferSize = sizeof(Particle) * PARTICLE_COUNT;
2
3     VkBuffer stagingBuffer;
4     VkDeviceMemory stagingBufferMemory;
5     createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
        VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
        VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer,
        stagingBufferMemory);
6
7     void* data;
8     vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0,
        &data);
9     memcpy(data, particles.data(), (size_t)bufferSize);
10    vkUnmapMemory(device, stagingBufferMemory);

```

Using this staging buffer as a source we then create the per-frame shader storage buffers and copy the particle properties from the staging buffer to each of these:

```

1     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
2         createBuffer(bufferSize, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT
            | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT |
            VK_BUFFER_USAGE_TRANSFER_DST_BIT,
            VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
            shaderStorageBuffers[i], shaderStorageBuffersMemory[i]);
3         // Copy data from the staging buffer (host) to the shader
            storage buffer (GPU)
4         copyBuffer(stagingBuffer, shaderStorageBuffers[i],
            bufferSize);
5     }
6 }

```

Descriptors

Setting up descriptors for compute is almost identical to graphics. The only difference is that descriptors need to have the `VK_SHADER_STAGE_COMPUTE_BIT` set to make them accessible by the compute stage:

```

1 std::array<VkDescriptorSetLayoutBinding, 3> layoutBindings{};
2 layoutBindings[0].binding = 0;
3 layoutBindings[0].descriptorCount = 1;
4 layoutBindings[0].descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
5 layoutBindings[0].pImmutableSamplers = nullptr;
6 layoutBindings[0].stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
7 ...

```

Note that you can combine shader stages here, so if you want the descriptor to be accessible from the vertex and compute stage, e.g. for a uniform buffer with parameters shared across them, you simply set the bits for both stages:

```

1 layoutBindings[0].stageFlags = VK_SHADER_STAGE_VERTEX_BIT |
    VK_SHADER_STAGE_COMPUTE_BIT;

```

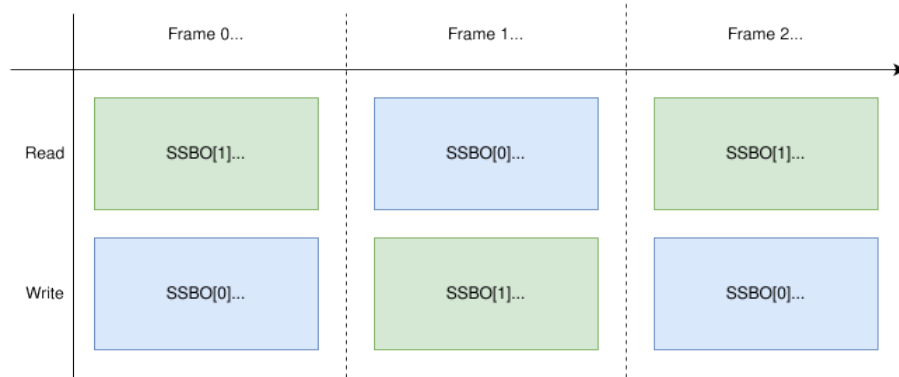
Here is the descriptor setup for our sample. The layout looks like this:

```

1 std::array<VkDescriptorSetLayoutBinding, 3> layoutBindings{};
2 layoutBindings[0].binding = 0;
3 layoutBindings[0].descriptorCount = 1;
4 layoutBindings[0].descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
5 layoutBindings[0].pImmutableSamplers = nullptr;
6 layoutBindings[0].stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
7
8 layoutBindings[1].binding = 1;
9 layoutBindings[1].descriptorCount = 1;
10 layoutBindings[1].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
11 layoutBindings[1].pImmutableSamplers = nullptr;
12 layoutBindings[1].stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
13
14 layoutBindings[2].binding = 2;
15 layoutBindings[2].descriptorCount = 1;
16 layoutBindings[2].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
17 layoutBindings[2].pImmutableSamplers = nullptr;
18 layoutBindings[2].stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
19
20 VkDescriptorSetLayoutCreateInfo layoutInfo{};
21 layoutInfo.sType =
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
22 layoutInfo.bindingCount = 3;
23 layoutInfo.pBindings = layoutBindings.data();
24
25 if (vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr,
    &computeDescriptorSetLayout) != VK_SUCCESS) {
26     throw std::runtime_error("failed to create compute descriptor
        set layout!");
27 }

```

Looking at this setup, you might wonder why we have two layout bindings for shader storage buffer objects, even though we'll only render a single particle system. This is because the particle positions are updated frame by frame based on a delta time. This means that each frame needs to know about the last frames' particle positions, so it can update them with a new delta time and write them to it's own SSBO:



For that, the compute shader needs to have access to the last and current frame's SSBOs. This is done by passing both to the compute shader in our descriptor setup. See the `storageBufferInfoLastFrame` and `storageBufferInfoCurrentFrame`:

```

1 for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
2     VkDescriptorBufferInfo uniformBufferInfo{};
3     uniformBufferInfo.buffer = uniformBuffers[i];
4     uniformBufferInfo.offset = 0;
5     uniformBufferInfo.range = sizeof(UniformBufferObject);
6
7     std::array<VkWriteDescriptorSet, 3> descriptorWrites{};
8     ...
9
10    VkDescriptorBufferInfo storageBufferInfoLastFrame{};
11    storageBufferInfoLastFrame.buffer = shaderStorageBuffers[(i - 1)
12        % MAX_FRAMES_IN_FLIGHT];
13    storageBufferInfoLastFrame.offset = 0;
14    storageBufferInfoLastFrame.range = sizeof(Particle) *
15        PARTICLE_COUNT;
16
17    descriptorWrites[1].sType =
18        VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
19    descriptorWrites[1].dstSet = computeDescriptorSets[i];
20    descriptorWrites[1].dstBinding = 1;
21    descriptorWrites[1].dstArrayElement = 0;
22    descriptorWrites[1].descriptorType =

```

```

    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
20 descriptorWrites[1].descriptorCount = 1;
21 descriptorWrites[1].pBufferInfo = &storageBufferInfoLastFrame;
22
23 VkDescriptorBufferInfo storageBufferInfoCurrentFrame{};
24 storageBufferInfoCurrentFrame.buffer = shaderStorageBuffers[i];
25 storageBufferInfoCurrentFrame.offset = 0;
26 storageBufferInfoCurrentFrame.range = sizeof(Particle) *
    PARTICLE_COUNT;
27
28 descriptorWrites[2].sType =
    VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
29 descriptorWrites[2].dstSet = computeDescriptorSets[i];
30 descriptorWrites[2].dstBinding = 2;
31 descriptorWrites[2].dstArrayElement = 0;
32 descriptorWrites[2].descriptorType =
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
33 descriptorWrites[2].descriptorCount = 1;
34 descriptorWrites[2].pBufferInfo = &storageBufferInfoCurrentFrame;
35
36 vkUpdateDescriptorSets(device, 3, descriptorWrites.data(), 0,
    nullptr);
37 }

```

Remember that we also have to request the descriptor types for the SSBOs from our descriptor pool:

```

1 std::array<VkDescriptorPoolSize, 2> poolSizes{};
2 ...
3
4 poolSizes[1].type = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
5 poolSizes[1].descriptorCount =
    static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT) * 2;

```

We need to double the number of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` types requested from the pool by two because our sets reference the SSBOs of the last and current frame.

Compute pipelines

As compute is not a part of the graphics pipeline, we can't use `vkCreateGraphicsPipelines`. Instead we need to create a dedicated compute pipeline with `vkCreateComputePipelines` for running our compute commands. Since a compute pipeline does not touch any of the rasterization state, it has a lot less state than a graphics pipeline:

```

1 VkComputePipelineCreateInfo pipelineInfo{};

```

```

2 pipelineInfo.sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO;
3 pipelineInfo.layout = computePipelineLayout;
4 pipelineInfo.stage = computeShaderStageInfo;
5
6 if (vkCreateComputePipelines(device, VK_NULL_HANDLE, 1,
    &pipelineInfo, nullptr, &computePipeline) != VK_SUCCESS) {
7     throw std::runtime_error("failed to create compute pipeline!");
8 }

```

The setup is a lot simpler, as we only require one shader stage and a pipeline layout. The pipeline layout works the same as with the graphics pipeline:

```

1 VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
2 pipelineLayoutInfo.sType =
    VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
3 pipelineLayoutInfo.setLayoutCount = 1;
4 pipelineLayoutInfo.pSetLayouts = &computeDescriptorSetLayout;
5
6 if (vkCreatePipelineLayout(device, &pipelineLayoutInfo, nullptr,
    &computePipelineLayout) != VK_SUCCESS) {
7     throw std::runtime_error("failed to create compute pipeline
        layout!");
8 }

```

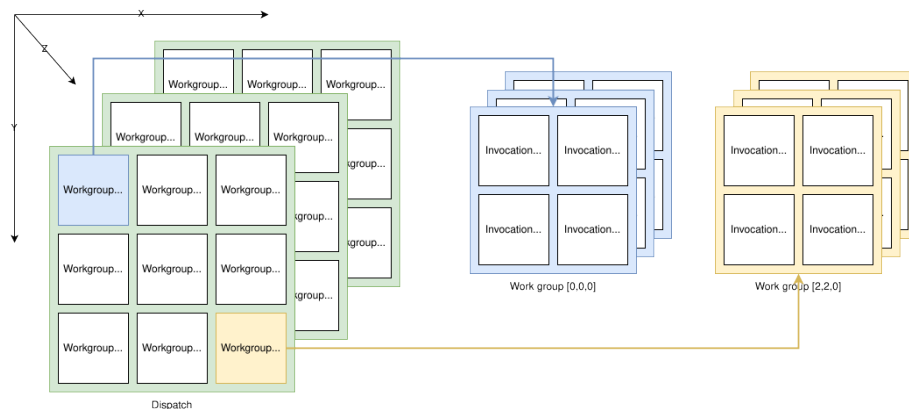
Compute space

Before we get into how a compute shader works and how we submit compute workloads to the GPU, we need to talk about two important compute concepts: **work groups** and **invocations**. They define an abstract execution model for how compute workloads are processed by the compute hardware of the GPU in three dimensions (x, y, and z).

Work groups define how the compute workloads are formed and processed by the the compute hardware of the GPU. You can think of them as work items the GPU has to work through. Work group dimensions are set by the application at command buffer time using a dispatch command.

And each work group then is a collection of **invocations** that execute the same compute shader. Invocations can potentially run in parallel and their dimensions are set in the compute shader. Invocations within a single workgroup have access to shared memory.

This image shows the relation between these two in three dimensions:



The number of dimensions for work groups (defined by `vkCmdDispatch`) and invocations depends (defined by the local sizes in the compute shader) on how input data is structured. If you e.g. work on a one-dimensional array, like we do in this chapter, you only have to specify the x dimension for both.

As an example: If we dispatch a work group count of `[64, 1, 1]` with a compute shader local size of `[32, 32, 1]`, our compute shader will be invoked $64 \times 32 \times 32 = 65,536$ times.

Note that the maximum count for work groups and local sizes differs from implementation to implementation, so you should always check the compute related `maxComputeWorkGroupCount`, `maxComputeWorkGroupInvocations` and `maxComputeWorkGroupSize` limits in `VkPhysicalDeviceLimits`.

Compute shaders

Now that we have learned about all the parts required to setup a compute shader pipeline, it's time to take a look at compute shaders. All of the things we learned about using GLSL shaders e.g. for vertex and fragment shaders also applies to compute shaders. The syntax is the same, and many concepts like passing data between the application and the shader are the same. But there are some important differences.

A very basic compute shader for updating a linear array of particles may look like this:

```
1 #version 450
2
3 layout (binding = 0) uniform ParameterUBO {
4     float deltaTime;
5 } ubo;
6
7 struct Particle {
8     vec2 position;
```

```

9     vec2 velocity;
10    vec4 color;
11 };
12
13 layout(std140, binding = 1) readonly buffer ParticleSSBOIn {
14     Particle particlesIn[ ];
15 };
16
17 layout(std140, binding = 2) buffer ParticleSSBOOut {
18     Particle particlesOut[ ];
19 };
20
21 layout (local_size_x = 256, local_size_y = 1, local_size_z = 1) in;
22
23 void main()
24 {
25     uint index = gl_GlobalInvocationID.x;
26
27     Particle particleIn = particlesIn[index];
28
29     particlesOut[index].position = particleIn.position +
        particleIn.velocity.xy * ubo.deltaTime;
30     particlesOut[index].velocity = particleIn.velocity;
31     ...
32 }

```

The top part of the shader contains the declarations for the shader's input. First is a uniform buffer object at binding 0, something we already learned about in this tutorial. Below we declare our Particle structure that matches the declaration in the C++ code. Binding 1 then refers to the shader storage buffer object with the particle data from the last frame (see the descriptor setup), and binding 2 points to the SSBO for the current frame, which is the one we'll be updating with this shader.

An interesting thing is this compute-only declaration related to the compute space:

```

1 layout (local_size_x = 256, local_size_y = 1, local_size_z = 1) in;

```

This defines the number invocations of this compute shader in the current work group. As noted earlier, this is the local part of the compute space. Hence the `local_` prefix. As we work on a linear 1D array of particles we only need to specify a number for x dimension in `local_size_x`.

The `main` function then reads from the last frame's SSBO and writes the updated particle position to the SSBO for the current frame. Similar to other shader types, compute shaders have their own set of builtin input variables. Built-ins

are always prefixed with `gl_`. One such built-in is `gl_GlobalInvocationID`, a variable that uniquely identifies the current compute shader invocation across the current dispatch. We use this to index into our particle array.

Running compute commands

Dispatch

Now it's time to actually tell the GPU to do some compute. This is done by calling `vkCmdDispatch` inside a command buffer. While not perfectly true, a dispatch is for compute as a draw call like `vkCmdDraw` is for graphics. This dispatches a given number of compute work items in at max. three dimensions.

```
1 VkCommandBufferBeginInfo beginInfo{};
2 beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
3
4 if (vkBeginCommandBuffer(commandBuffer, &beginInfo) != VK_SUCCESS) {
5     throw std::runtime_error("failed to begin recording command
6         buffer!");
7 }
8 ...
9
10 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_COMPUTE,
11     computePipeline);
12 vkCmdBindDescriptorSets(commandBuffer,
13     VK_PIPELINE_BIND_POINT_COMPUTE, computePipelineLayout, 0, 1,
14     &computeDescriptorSets[i], 0, 0);
15
16 vkCmdDispatch(commandBuffer, PARTICLE_COUNT / 256, 1, 1);
17
18 ...
19
20 if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
21     throw std::runtime_error("failed to record command buffer!");
22 }
```

The `vkCmdDispatch` will dispatch `PARTICLE_COUNT / 256` local work groups in the x dimension. As our particles array is linear, we leave the other two dimensions at one, resulting in a one-dimensional dispatch. But why do we divide the number of particles (in our array) by 256? That's because in the previous paragraph we defined that every compute shader in a work group will do 256 invocations. So if we were to have 4096 particles, we would dispatch 16 work groups, with each work group running 256 compute shader invocations. Getting the two numbers right usually takes some tinkering and profiling, depending on your workload and the hardware you're running on. If your particle size

would be dynamic and can't always be divided by e.g. 256, you can always use `gl_GlobalInvocationID` at the start of your compute shader and return from it if the global invocation index is greater than the number of your particles.

And just as was the case for the compute pipeline, a compute command buffer contains a lot less state than a graphics command buffer. There's no need to start a render pass or set a viewport.

Submitting work

As our sample does both compute and graphics operations, we'll be doing two submits to both the graphics and compute queue per frame (see the `drawFrame` function):

```
1 ...
2 if (vkQueueSubmit(computeQueue, 1, &submitInfo, nullptr) !=
   VK_SUCCESS) {
3     throw std::runtime_error("failed to submit compute command
       buffer!");
4 };
5 ...
6 if (vkQueueSubmit(graphicsQueue, 1, &submitInfo,
   inFlightFences[currentFrame]) != VK_SUCCESS) {
7     throw std::runtime_error("failed to submit draw command
       buffer!");
8 }
```

The first submit to the compute queue updates the particle positions using the compute shader, and the second submit will then use that updated data to draw the particle system.

Synchronizing graphics and compute

Synchronization is an important part of Vulkan, even more so when doing compute in conjunction with graphics. Wrong or lacking synchronization may result in the vertex stage starting to draw (=read) particles while the compute shader hasn't finished updating (=write) them (read-after-write hazard), or the compute shader could start updating particles that are still in use by the vertex part of the pipeline (write-after-read hazard).

So we must make sure that those cases don't happen by properly synchronizing the graphics and the compute load. There are different ways of doing so, depending on how you submit your compute workload but in our case with two separate submits, we'll be using semaphores and fences to ensure that the vertex shader won't start fetching vertices until the compute shader has finished updating them.

This is necessary as even though the two submits are ordered one-after-another, there is no guarantee that they execute on the GPU in this order. Adding in wait and signal semaphores ensures this execution order.

So we first add a new set of synchronization primitives for the compute work in `createSyncObjects`. The compute fences, just like the graphics fences, are created in the signaled state because otherwise, the first draw would time out while waiting for the fences to be signaled as detailed here:

```

1 std::vector<VkFence> computeInFlightFences;
2 std::vector<VkSemaphore> computeFinishedSemaphores;
3 ...
4 computeInFlightFences.resize(MAX_FRAMES_IN_FLIGHT);
5 computeFinishedSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
6
7 VkSemaphoreCreateInfo semaphoreInfo{};
8 semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;
9
10 VkFenceCreateInfo fenceInfo{};
11 fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
12 fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;
13
14 for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
15     ...
16     if (vkCreateSemaphore(device, &semaphoreInfo, nullptr,
17         &computeFinishedSemaphores[i]) != VK_SUCCESS ||
18         vkCreateFence(device, &fenceInfo, nullptr,
19             &computeInFlightFences[i]) != VK_SUCCESS) {
20         throw std::runtime_error("failed to create compute
21             synchronization objects for a frame!");
22     }
23 }

```

We then use these to synchronize the compute buffer submission with the graphics submission:

```

1 // Compute submission
2 vkWaitForFences(device, 1, &computeInFlightFences[currentFrame],
3     VK_TRUE, UINT64_MAX);
4
5 updateUniformBuffer(currentFrame);
6
7 vkResetFences(device, 1, &computeInFlightFences[currentFrame]);
8
9 vkResetCommandBuffer(computeCommandBuffers[currentFrame],
10     /*VkCommandBufferResetFlagBits*/ 0);
11 recordComputeCommandBuffer(computeCommandBuffers[currentFrame]);

```

```

10
11 submitInfo.commandBufferCount = 1;
12 submitInfo.pCommandBuffers = &computeCommandBuffers[currentFrame];
13 submitInfo.signalSemaphoreCount = 1;
14 submitInfo.pSignalSemaphores =
    &computeFinishedSemaphores[currentFrame];
15
16 if (vkQueueSubmit(computeQueue, 1, &submitInfo,
    computeInFlightFences[currentFrame]) != VK_SUCCESS) {
17     throw std::runtime_error("failed to submit compute command
        buffer!");
18 };
19
20 // Graphics submission
21 vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE,
    UINT64_MAX);
22
23 ...
24
25 vkResetFences(device, 1, &inFlightFences[currentFrame]);
26
27 vkResetCommandBuffer(commandBuffers[currentFrame],
    /*VkCommandBufferResetFlagBits*/ 0);
28 recordCommandBuffer(commandBuffers[currentFrame], imageIndex);
29
30 VkSemaphore waitSemaphores[] = {
    computeFinishedSemaphores[currentFrame],
    imageAvailableSemaphores[currentFrame] };
31 VkPipelineStageFlags waitStages[] = {
    VK_PIPELINE_STAGE_VERTEX_INPUT_BIT,
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT };
32 submitInfo = {};
33 submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
34
35 submitInfo.waitSemaphoreCount = 2;
36 submitInfo.pWaitSemaphores = waitSemaphores;
37 submitInfo.pWaitDstStageMask = waitStages;
38 submitInfo.commandBufferCount = 1;
39 submitInfo.pCommandBuffers = &commandBuffers[currentFrame];
40 submitInfo.signalSemaphoreCount = 1;
41 submitInfo.pSignalSemaphores =
    &renderFinishedSemaphores[currentFrame];
42
43 if (vkQueueSubmit(graphicsQueue, 1, &submitInfo,
    inFlightFences[currentFrame]) != VK_SUCCESS) {
44     throw std::runtime_error("failed to submit draw command

```

```

    buffer!");
45 }

```

Similar to the sample in the semaphores chapter, this setup will immediately run the compute shader as we haven't specified any wait semaphores. This is fine, as we are waiting for the compute command buffer of the current frame to finish execution before the compute submission with the `vkWaitForFences` command.

The graphics submission on the other hand needs to wait for the compute work to finish so it doesn't start fetching vertices while the compute buffer is still updating them. So we wait on the `computeFinishedSemaphores` for the current frame and have the graphics submission wait on the `VK_PIPELINE_STAGE_VERTEX_INPUT_BIT` stage, where vertices are consumed.

But it also needs to wait for presentation so the fragment shader won't output to the color attachments until the image has been presented. So we also wait on the `imageAvailableSemaphores` on the current frame at the `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT` stage.

Drawing the particle system

Earlier on, we learned that buffers in Vulkan can have multiple use-cases and so we created the shader storage buffer that contains our particles with both the shader storage buffer bit and the vertex buffer bit. This means that we can use the shader storage buffer for drawing just as we used “pure” vertex buffers in the previous chapters.

We first setup the vertex input state to match our particle structure:

```

1 struct Particle {
2     ...
3
4     static std::array<VkVertexInputAttributeDescription, 2>
        getAttributeDescriptions() {
5         std::array<VkVertexInputAttributeDescription, 2>
            attributeDescriptions{};
6
7         attributeDescriptions[0].binding = 0;
8         attributeDescriptions[0].location = 0;
9         attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
10        attributeDescriptions[0].offset = offsetof(Particle,
            position);
11
12        attributeDescriptions[1].binding = 0;
13        attributeDescriptions[1].location = 1;

```

```

14     attributeDescriptions[1].format =
        VK_FORMAT_R32G32B32A32_SFLOAT;
15     attributeDescriptions[1].offset = offsetof(Particle, color);
16
17     return attributeDescriptions;
18 }
19 };

```

Note that we don't add `velocity` to the vertex input attributes, as this is only used by the compute shader.

We then bind and draw it like we would with any vertex buffer:

```

1 vkCmdBindVertexBuffers(commandBuffer, 0, 1,
    &shaderStorageBuffer[currentFrame], offsets);
2
3 vkCmdDraw(commandBuffer, PARTICLE_COUNT, 1, 0, 0);

```

Conclusion

In this chapter, we learned how to use compute shaders to offload work from the CPU to the GPU. Without compute shaders, many effects in modern games and applications would either not be possible or would run a lot slower. But even more than graphics, compute has a lot of use-cases, and this chapter only gives you a glimpse of what's possible. So now that you know how to use compute shaders, you may want to take look at some advanced compute topics like:

- Shared memory
- Asynchronous compute
- Atomic operations
- Subgroups

You can find some advanced compute samples in the official Khronos Vulkan Samples repository.

C++ code / Vertex shader / Fragment shader / Compute shader

FAQ

This page lists solutions to common problems that you may encounter while developing Vulkan applications.

I get an access violation error in the core validation layer

Make sure that MSI Afterburner / RivaTuner Statistics Server is not running, because it has some compatibility problems with Vulkan.

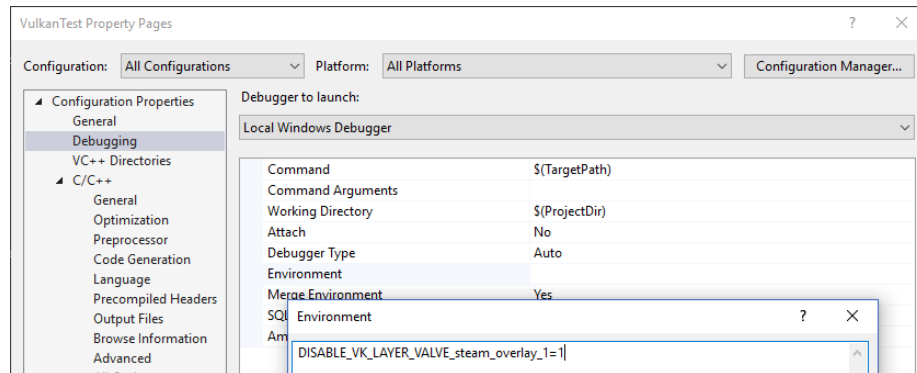
I don't see any messages from the validation layers / Validation layers are not available

First make sure that the validation layers get a chance to print errors by keeping the terminal open after your program exits. You can do this from Visual Studio by running your program with Ctrl-F5 instead of F5, and on Linux by executing your program from a terminal window. If there are still no messages and you are sure that validation layers are turned on, then you should ensure that your Vulkan SDK is correctly installed by following the “Verify the Installation” instructions on this page. Also ensure that your SDK version is at least 1.1.106.0 to support the `VK_LAYER_KHRONOS_validation` layer.

vkCreateSwapchainKHR triggers an error in SteamOverlayVulkanLayer64.dll

This appears to be a compatibility problem in the Steam client beta. There are a few possible workarounds: * Opt out of the Steam beta program. * Set the `DISABLE_VK_LAYER_VALVE_steam_overlay_1` environment variable to 1 * Delete the Steam overlay Vulkan layer entry in the registry under `HKEY_LOCAL_MACHINE\SOFTWARE\Khronos\Vulkan\ImplicitLayers`

Example:



vkCreateInstance fails with VK_ERROR_INCOMPATIBLE_DRIVER

If you are using MacOS with the latest MoltenVK SDK then `vkCreateInstance` may return the `VK_ERROR_INCOMPATIBLE_DRIVER` error. This is because Vulkan SDK version 1.3.216 or newer requires you to enable the `VK_KHR_PORTABILITY_subset` extension to use MoltenVK, because it is currently not fully conformant.

You have to add the `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR` flag to your `VkInstanceCreateInfo` and add `VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME` to your instance extension list.

Code example:

```

1 ...
2
3 std::vector<const char*> requiredExtensions;
4
5 for(uint32_t i = 0; i < glfwExtensionCount; i++) {
6     requiredExtensions.emplace_back(glfwExtensions[i]);
7 }
8
9 requiredExtensions.emplace_back(VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME);
10
11 createInfo.flags |= VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR;
12
13 createInfo.enabledExtensionCount = (uint32_t)
14     requiredExtensions.size();
15 createInfo.ppEnabledExtensionNames = requiredExtensions.data();
16
17 if (vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS)
18 {
19     throw std::runtime_error("failed to create instance!");
20 }

```